This document is for the sole use of Kun Wang of Seas

SystemVerilog Accelerated Verification Using UVM

Engineer Explorer Series

Course Version 1.2.6

Lecture Manual **Revision 1.0**

cadence®

This document is for the sole use of Kun Wang of Seas

© 1990-2023 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

This document is for the sole use of Kun Wang of Seas

Table of Contents

SystemVerilog Accelerated Verification Using UVM

Module 1 **About This Course** **2**

Module 2 **Introduction to UVM Methodology** **8**

Module 3 **Overview of DUT and Project** **29**

Module 4 **Data Modeling** **38**
Lab 1 Creating a Stimulus Model

Module 5 **Simulation Phases** **60**

Module 6 **Test and Testbench Classes** **72**
Lab 2 Creating Test and Testbench Components

Module 7 **Creating an Interface UVC** **93**
Lab 3 Creating a Simple UVC

Module 8 **Configuration** **113**

Module 9 **Type Overrides and the Factory** **127**
Lab 4 Using Factories

Module 10 **Sequences** **145**
Lab 5 Generating UVM Sequences

Module 11 **Connecting to a DUT** **173**
Lab 6 Connecting to a DUT Using Virtual Interfaces

Module 12 **Multiple UVCs** **189**
Lab 7 Integrating Multiple UVCs

This document is for the sole use of Kun Wang of Seas

Module 13 Multichannel (Virtual) Sequences 208
Lab 8 Writing Multichannel Sequences and System-Level Tests

Module 14 Building a Scoreboard 224
Lab 9A Creating a Scoreboard using TLM
Lab 9B Router Module UVC

Module 15 Transaction Level Modelling (TLM) 246
Lab 9C Using TLM Export Connectors (Optional)
Lab 9D Using TLM Analysis FIFOs (Optional)

Module 16 Functional Coverage Modelling (Optional) 272
Lab 10 Creating a Simple Functional Coverage Model (Optional)

Module 17 Register Modeling in UVM 290
Lab 11 Register Modelling in UVM
Lab11a Generation
Lab11b Integration
Lab11c Simulation

Module 18 Top Five Things that Break in UVM-IEEE (And How to Fix Them).....324

Module 19 Next Steps.....336

This document is for the sole use of Kun Wang of Seas



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 1

About This Course

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Prerequisites

This course assumes you have the following knowledge:

- SystemVerilog language for design and verification

The following knowledge is useful but not essential

- An appreciation of object-oriented design
- Some familiarity with the Cadence® Xcelium® or Incisive® simulator

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Objectives

The focus of this course is on features of the Universal Verification Methodology (UVM) that target complex design verification.

In this course, you will:

- Define the advantages of a standard verification methodology
- Explore the UVM Verification Component (UVC) architecture
- Control the generation of stimuli
- Create reusable UVCs
- Control environmental behaviors
- Generate stimulus sequences and check responses
- Implement a scoreboard to help check the behavior of a design
- Connect the scoreboard using Transaction-Level Modeling (TLM)
- Generate, integrate, and simulate a Register Model to help verify DUT register and memory behavior

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Your Expectations

This training will not cover the following topics:

- SystemVerilog language
- Verification planning
- Verification plan annotation to vManager™
- Coverage model design principles
- Coverage model design and implementation
- Regression management and analysis
- Test launching, failure analysis, coverage hole analysis, ranking

These are very important aspects of a verification solution but are addressed in different courses.

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Course Agenda

1. About this course
2. Introduction to UVM Methodology
3. Overview of DUT and Project
4. Data Modeling
   - Lab 1: Creating a Stimulus Model
5. Simulation Phases
6. Test and Testbench Classes
   - Lab 2: Creating Test and Testbench Components
7. Creating an Interface UVC
   - Lab 3: Creating a Simple UVC
8. Configuration
9. Type Overrides using the Factory
   - Lab 4: Using Factories
10. UVM Sequences
    - Lab 5: Creating UVM Sequences
11. Connecting to a DUT
    - Lab 6: Connecting to the DUT Using Virtual Interfaces
12. Multiple UVCs
    - Lab 7: Integrating Multiple UVCs
13. Multichannel Sequences
    - Lab 8: Writing Multichannel Sequences and System-Level Tests
14. Building a Scoreboard
    - Lab 9a: Creating a Scoreboard Using TLM
    - Lab 9b: Router Module UVC
15. Transaction Level Modelling
    - Lab 9c: Using TLM Export Connectors (Optional)
    - Lab 9d: Using TLM Analysis FIFOs (Optional)
16. Functional Coverage (Optional)
    - Lab 10: Creating a Simple Functional Coverage Model (Optional)
17. Register Modeling in UVM
    - Lab 11a: Generation
    - Lab 11b: Integration
    - Lab 11c: Simulation
18. What's new in UVM-IEEE

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Become Cadence Certified by Earning a Digital Badge

Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
  - Expand career opportunities
- Professional credibility
  - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.



How do I register to take the exam?

- Log in to our Learning Management System to locate the exam in your transcript.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- Credly validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 2

Introduction to UVM Methodology

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Introduction to UVM Methodology Objectives

In this module, you

- Discuss the benefits of the Universal Verification Methodology (UVM)
- Define the architecture of a UVM Verification Component (UVC)
- Explain the differences between interface and module UVCs
- Recognize the structure of the UVM library

© Cadence Design Systems, Inc. All rights reserved.

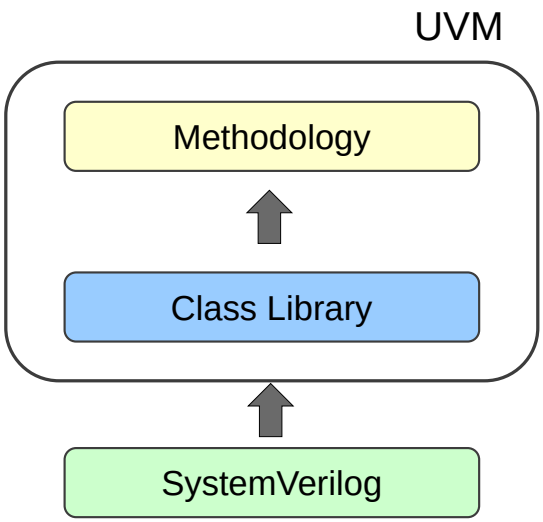
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is UVM?



- A class library of verification building blocks
  - Written in standard IEEE1800 SystemVerilog
- A proven verification methodology
  - Defines how to use the class library
  - Scalable from block-level to system-level verification
- IEEE standard 1800.2
  - Documented and maintained by IEEE
  - Open source reference library from Accellera
  - Apache license
  - Does not lock users into a single vendor solution

 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM is a library of SystemVerilog classes which implement verification building blocks along with support features such as phasing, messaging, factory etc. The library provides additional class automation, such as print, copy, comparison, cloning etc., which are not available in SystemVerilog.

The library API is defined as an IEEE standard (1800.1) and Accellera (the industry standard body) provides a reference implementation of the library. By extending our testbench environment from the UVM library, we can exploit the built-in automation and support features.

However, UVM is also a methodology for using the building blocks and features of the library. By following the methodology, you can create scalable verification components which can be used from block-level to system-level verification.

By following the methodology and the architecture it dictates, we can create standardized, reusable verification blocks.

UVM has an Apache license, which allows commercial IP to be built and sold on top of UVM.

All this means that companies can create internal, external and commercial verification IP conforming to UVM, and hence allowing the greatest possible reuse.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Library

Library classes provide verification functionality and building blocks. For example:

- Structural components
  - Verification blocks supporting a standard architecture
- Automation
  - Automatic implementation of print, copy, compare, etc. methods
- Simulation control
- Reporting

Benefits:

- Productivity
  - How much manual work can be saved?
- Predictability
  - Will I meet my deadlines on time?
- Quality
  - Will my testbench be free of bugs?

© Cadence Design Systems, Inc. All rights reserved.

cadence

UVM is a library of SystemVerilog classes which implement verification building blocks along with support features such as phasing, messaging, etc.

Automation refers to the automatic generation of common class operations such as copy, print and compare, which are not available in SystemVerilog. By generating these for us, UVM saves us from having to write these ourselves. In general, UVM provides the building blocks for testbench generation, saving us considerable time.

Developing a testbench using UVM building blocks allows us to leverage tried and tested code and therefore minimizes bugs and errors.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Methodology

Methodology defines how to use library classes.

Benefits:

- Proven solution for verification success
- Captures best-known practices with minimal improvisation
- Provides consistency and uniformity in the way verification components are developed and used
  - Internal and commercial reuse can dramatically speed up delivery and improve quality
- Accommodates unanticipated requirements
  - Supports easy configuration, customization, and modification
- Ease of use
  - How easily can a Test Writer develop tests for an existing verification environment?
- Supports key verification methodologies such as Metric-Driven Verification (MDV)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Although the first version of UVM was only established in 2009, it leverages the verification methodology developed by Cadence's Specman® tool which was released in 1996. So UVM has a proven success record and leverages decades of experience from prior tools.

A good methodology, based on a standard verification component architecture, increases productivity and predictability, as it saves much time otherwise taken in planning and partitioning the testbench design.

UVM recognizes that a reusable block or third-party verification IP may never completely match the requirements of a user, and so provides powerful configuration and type-override capabilities to allow the user to customize and modify verification blocks.

A test writer is a verification engineer who develops tests, according to the verification plan, for an existing verification environment. UVM is designed to allow a test writer to have maximum control without requiring an in-depth knowledge of the environment.

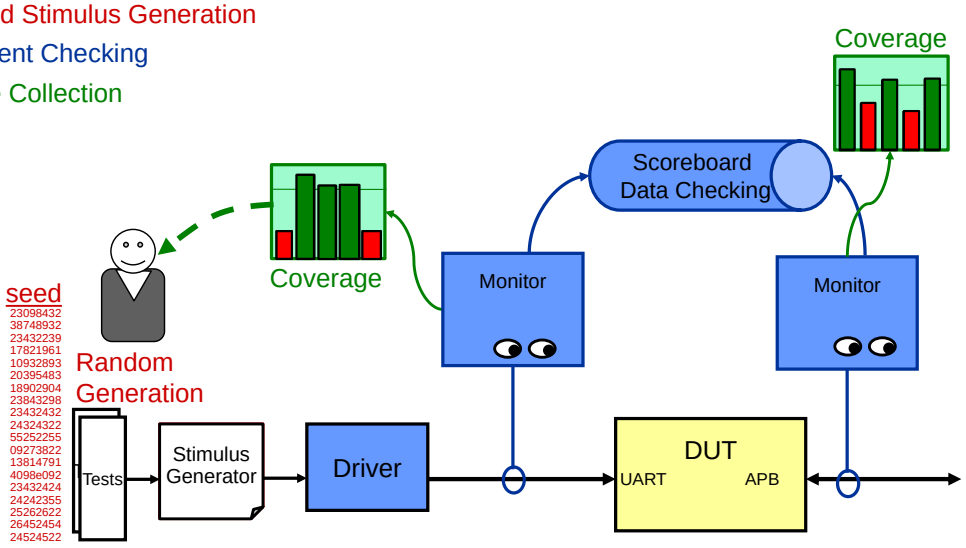
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Metric-Driven Verification Review

Components of a MDV environment

- Automated Stimulus Generation
- Independent Checking
- Coverage Collection



13 © Cadence Design Systems, Inc. All rights reserved.

A Metric-Driven Verification methodology is composed of three main features:

- Random stimulus to test the widest possible design functionality with the smallest stimulus code.
- Assertions and independent checking to define behavior which should or should not happen, and so find bugs in the design.
- Coverage so we can determine what has and has not been tested. Coverage feeds back to the test writer, who can then modify the stimulus to close coverage holes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM and Acceleration

Acceleration can improve simulation performance by 300x or more

Several forms of acceleration:

- Processor emulation, FPGA rapid prototyping, Multi-threaded software

Simple partitioning of accelerated DUT and simulator testbench

- Signal Based Acceleration (SBA)
- Can produce results in the 3x-20x range

Best acceleration efficiency comes from optimizing partitioning

- For example, parts of the testbench interacting with DUT signals are moved to the acceleration partition to reduce synchronization overhead
- Transaction-Based Acceleration (TBA)
- Can produce results in the 20x-300x range

TBA results in a slightly different methodology for UVM

TB: Future-Proof Your UVM Environments With Acceleration Optimization

 © Cadence Design Systems, Inc. All rights reserved.

cadence

When users migrate from simulation to hardware acceleration, the first step is usually to partition the UVM testbench to the simulator, move the DUT to the hardware side and verify the same test runs in both simulation and hardware acceleration.

This simple, signal-based acceleration solution takes less time to implement and can produce results in the 3x to 20x range.

With transaction-based acceleration, parts of the testbench is also moved to the hardware accelerator For example, task calls which drive the DUT ports are moved from UVM UVC classes into SystemVerilog interfaces where they can be accelerated and also where they will reduce hardware/software synchronization.

Accelerating more of the testbench and reducing the number of synchronizations and can produce results in the 20x-300x range.

This performance boost is desirable but requires that any portion of the testbench that goes to the hardware side is synthesizable.

Obviously developing your UVM testbench environment for acceleration compatibility from the beginning saves a huge amount of time and effort.

For more information, search for the following Training Byte on support.cadence.com.

Future-Proof Your UVM Environments With Acceleration Optimization

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating Reusable Environments

Having a standard architecture for verification components is key for sharing code

- Easier to learn and use components
- Easier for joint debug

Some of the requirements of component architecture include the following:

- Quick adoption and readability
- Modular and well-structured topology
  - Leads both naïve and sophisticated users to the right solution
- Seamless support for both system-level and block-level
- Conform to industry best practices

Let's walk through the process of creating a reusable verification component.

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

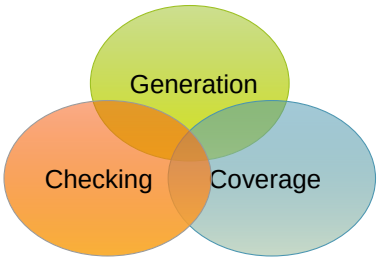
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Verification Component (UVC)

Definition

- Verification IP block
- Reusable, pre-verified, configurable, plug-and-play
  - Architected and packaged in a consistent way
- Complete: provides all the logic required for verification
  - Needs to be re-used and configured as a whole



Two types of UVC

- Interface UVCs
  - Bus-based UVCs (for example, PCI and AHB)
  - Communication UVCs (for example, Ethernet, MAC, Datalink)
- Module UVCs (for example, UART core, ALU module)
  - Scoreboard
  - Register model

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

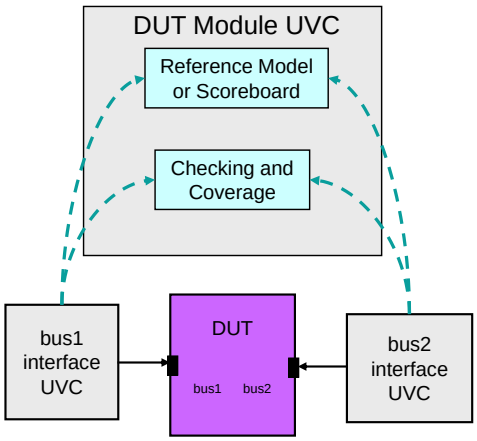
Module and Interface UVCs

Interface UVC

- Specific to a protocol, bus, or set of signals on the DUT
- Generates stimulus for the interface
- Reusable everywhere interface is used
- Set architecture and configuration

Module UVC

- Specific to DUT or flavor of DUT
- Provides checks and coverage across interface UVCs
- Reusable everywhere DUT is used
- No set architecture



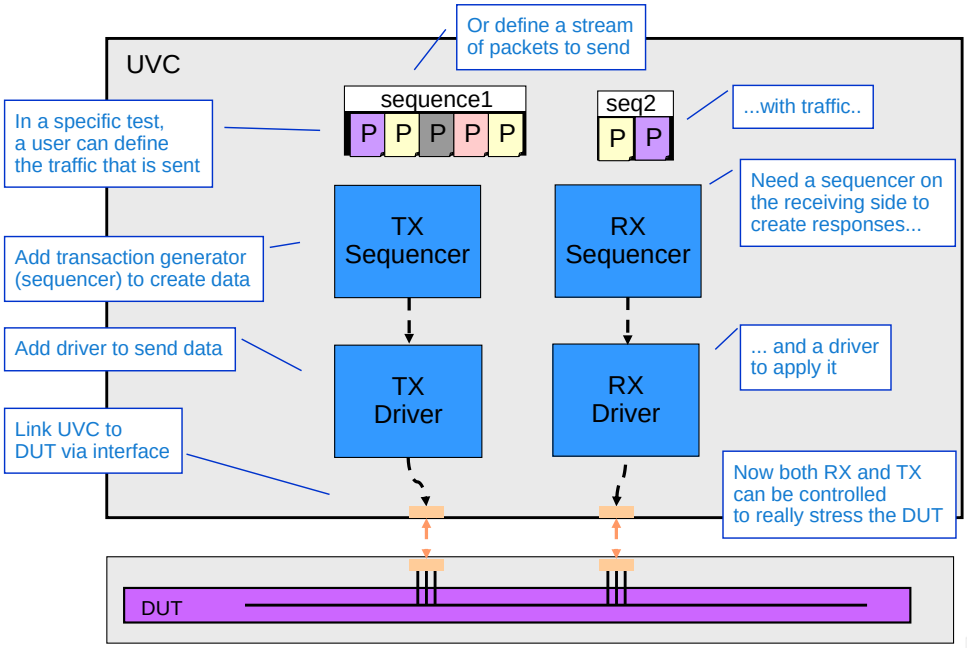
© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating an Interface UVC



18 © Cadence Design Systems, Inc. All rights reserved.

The TX (transmitter) side sends data into the DUT. The RX (receiver) side receives data from the DUT.

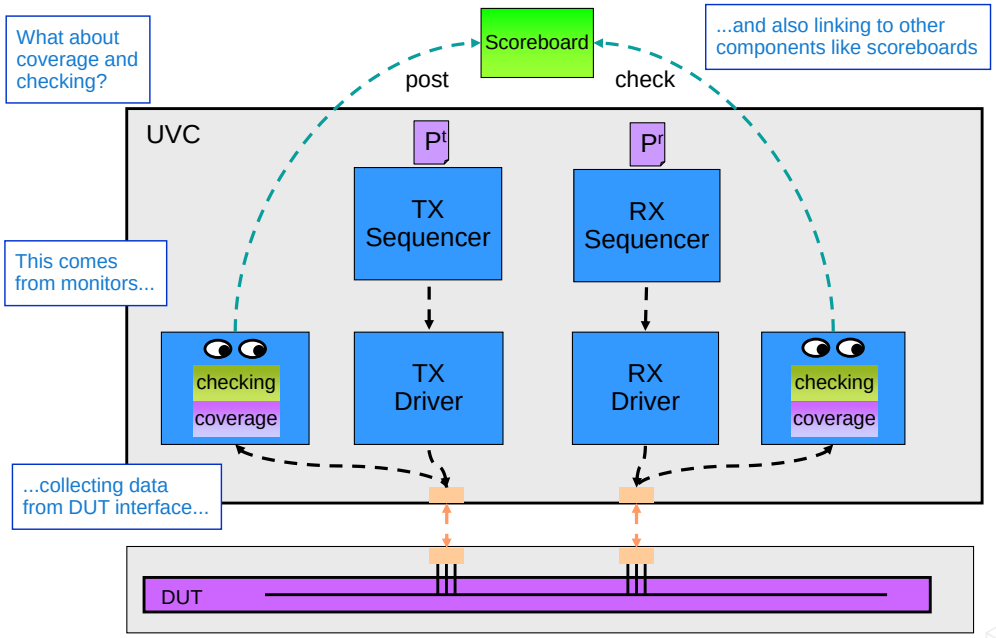
The implication here is that TX/RX components are related, for example, they might send and receive functionality on a single port of a multi-port switch network. If the TX/RX components had different protocols then they would be in different UVCs. If the TX/RX components had the same protocol, but were part of different interfaces to the DUT, they would again be in different UVCs.

For example, a four-port switch with the same protocol for each port, and TX/RX functionality in each port, would have four separate instances of the same UVC, one for each port. The first instance would implement the TX/RX functionality for port 1, the second for port 2, etc.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Monitors, Checking and Coverage



 © Cadence Design Systems, Inc. All rights reserved.

The sequencer and driver allows us to send stimulus into and capture responses out of the DUT. However we still need to check the outputs are correct for the given inputs and to use coverage to measure what we have and have not tested.

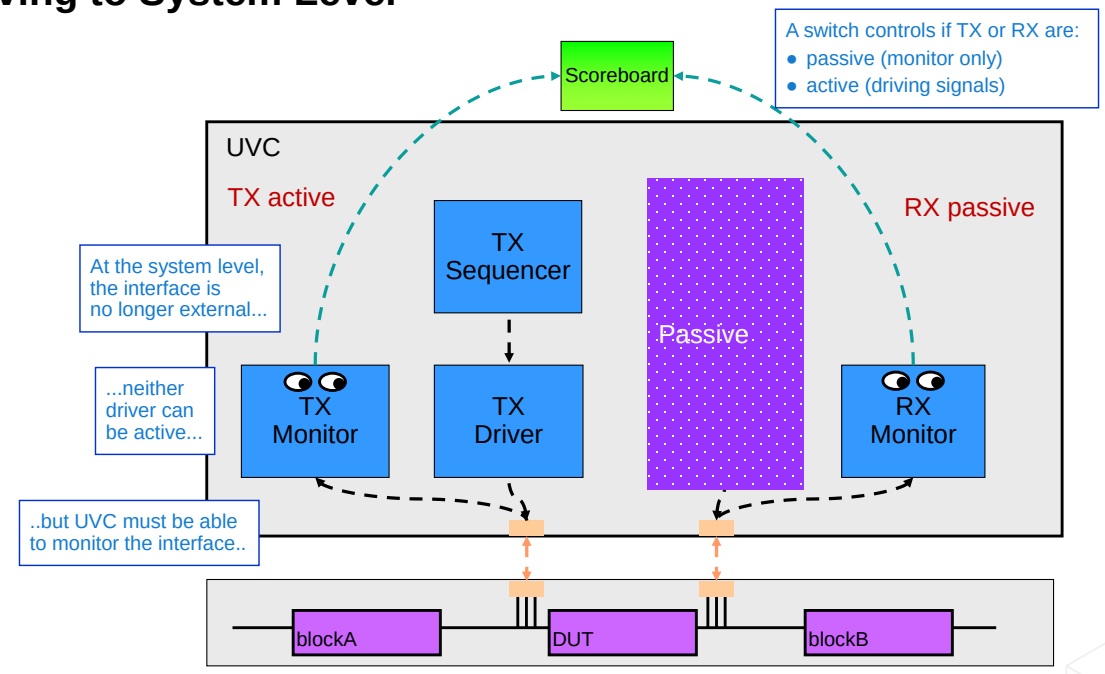
Both coverage and checking should come from monitors that collect information directly from the DUT.

The scoreboard checks the operation of the DUT by collecting input and output data, and checking that the outputs received are compatible with the inputs sent.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Moving to System Level



20 © Cadence Design Systems, Inc. All rights reserved.

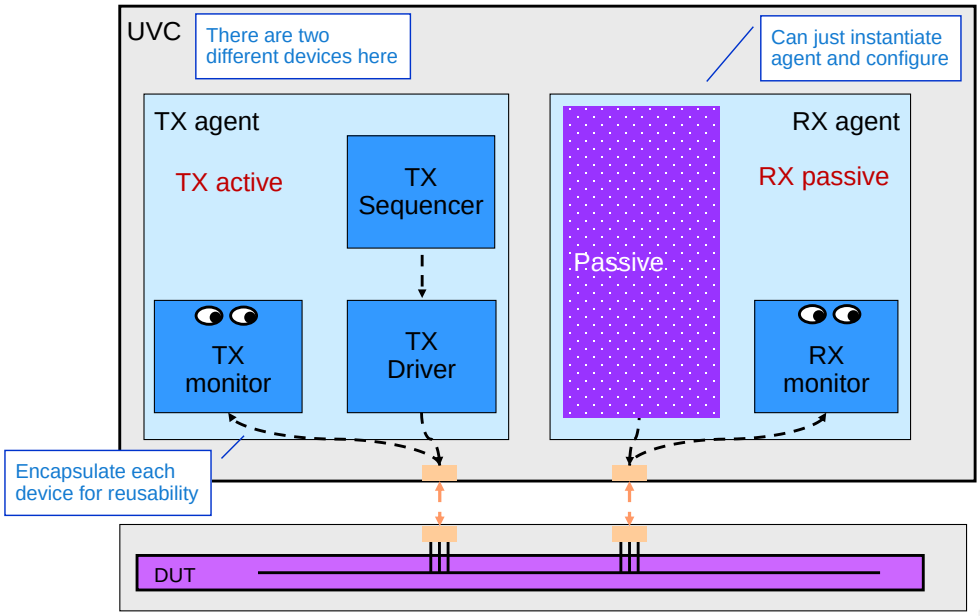
At the system level, the interface to the design under test (DUT) is no longer an external interface. Another RTL block drives it, but we still want to have passive checking and coverage on the interface.

We need the monitor component to be always be there, whether we drive the DUT signals or another RTL block drives the signals.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface UVC Architecture



21 © Cadence Design Systems, Inc. All rights reserved.
cadence

There are two different devices here. Each has a sequencer, driver and monitor, internal communication between them, some configuration, etc.

The device internals are not structured, which might make them harder to reuse and debug. We may also want to instantiate more than two devices (e.g., AHB bus).

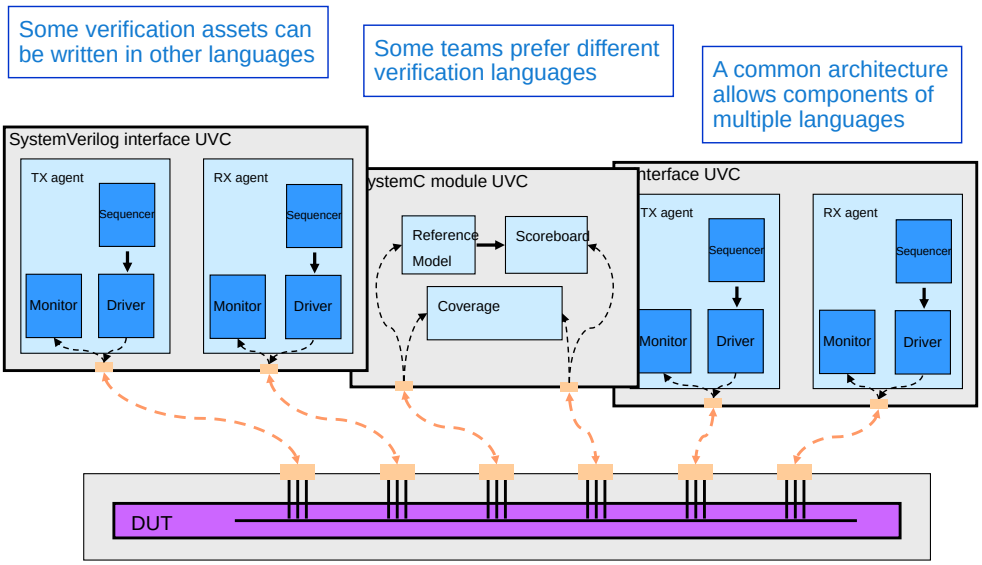
Encapsulating each device within a single container will simplify the usage.

Users of the environment do not need to worry about hookup of sub-components. They can just instantiate an agent and configure it for the right task.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Common Interface



22 © Cadence Design Systems, Inc. All rights reserved.

Verification teams may have existing assets written in other languages. Some teams may wish to create assets in other verification languages. We need a common interface to allow the use and development of components in multiple languages.

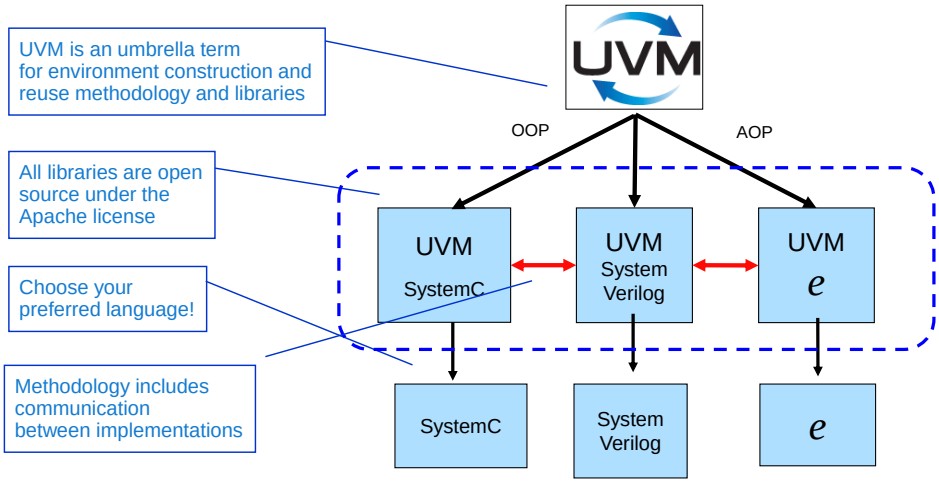
By using a common methodology around different languages (SV, e, SC) productivity gains can be seen in the area of reuse:

- Verification IP can dramatically speed-up delivery and improve quality.
- There is no need to throw away existing verification assets.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Vision: Multi-Language UVM



- Proven methodology independent of language
- Built-in language interoperability
- Cadence® supports the UVM-ML Open Architecture

23 © Cadence Design Systems, Inc. All rights reserved.

cadence

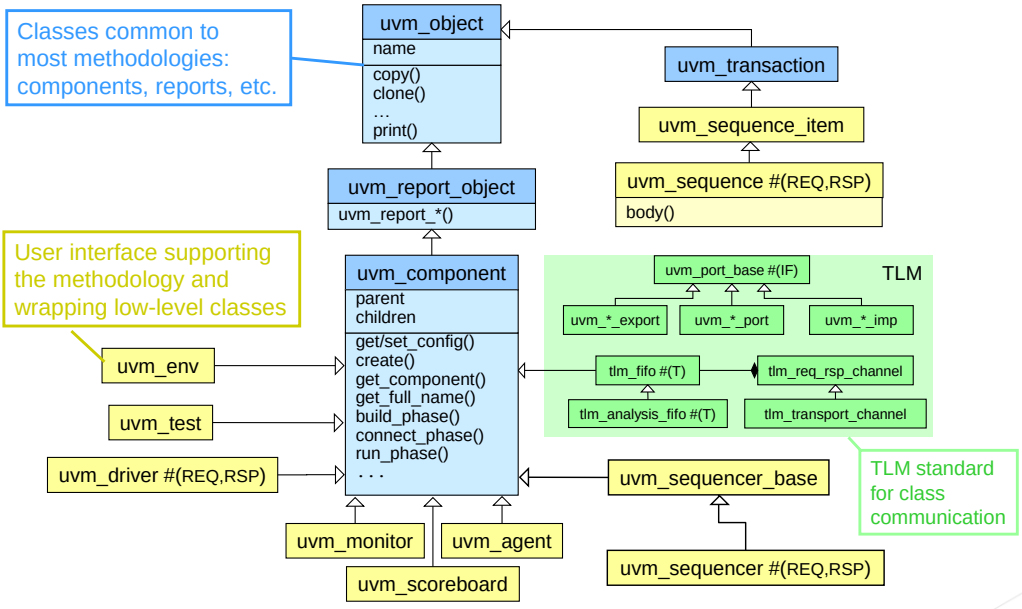
UVM is not restricted to SystemVerilog. It is a language-independent verification methodology. It allows you to integrate verification components from SystemVerilog, SystemC, Specman e and indeed any other language which conforms to the methodology.

It is the aim of Accellera to have a standard solution to multi-language UVM, which will include a standard simulation backplane for transferring data between different language domains. However as of UVM IEEE1800.1 2017, there is no standard as yet, and the multi-language solutions are vendor-specific. Cadence supports the UVM-ML Open Architecture.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simplified UVM Class Hierarchy



24 © Cadence Design Systems, Inc. All rights reserved.

Although the UVM class library looks complicated (and this diagram is a deliberate simplification), most UVM verification environments are written by extending the "leaf" classes only, e.g., `uvm_env`, `uvm_test`, `uvm_monitor`, etc.

There are three main layers to the UVM library:

- A SystemVerilog implementation of the OSCI TLM standard. This defines how class components can communicate with each other.
- The structural elements layer, which provide the features common to most verification methodologies, such as component construction, messaging, and phasing.
- The methodology layer that provides classes to specifically support the methodology and architecture for creating UVM VIP.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Where Are the Files?

UVM library is installed with the Cadence simulator

- `` `ncroot`/tools/methodology/UVM/CDNS-1.x/sv ``
- Can also be downloaded from the accellera.org website

Library includes:

- Documentation (up to UVM1.2)
  - ./docs
    - PDF reference and user guides
    - HTML documentation
- Examples (up to UVM1.2)
  - ./examples/
- Source files
  - ./src/

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary of UVM Technical Advantages

Interoperable, IEEE standard verification library and methodology

- Supported by all key EDA vendors

Open standard

- Based on IEEE 1800 SystemVerilog standard
- Apache license

Multi-language capability

- Enables industry-wide VIP plug and play

Advanced, proven methodology

- Built-in automation and testbench capabilities
- Defines fully encapsulated, configurable verification components
- Supports scalable module-to-system and project-to-project reuse

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Introduction to UVM Methodology Quiz

1. What is the architecture of an interface UVC?
2. What is the difference between an interface and a module UVC?
3. Where can you find the UVM PDF and HTML documentation?

Solutions at the end of the module

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Introduction to UVM Methodology Quiz

1. What is the architecture of an interface UVC?
   - A UVC instantiates one or more agent components which each instantiate a sequencer, driver, and monitor.

2. What is the difference between an interface and a module UVC?
   - An interface UVC is specific to and creates a stimulus for a bus, protocol, or set of signals. It has a set architecture and is reusable everywhere the interface is used.
   - A module UVC is specific to a DUT or a DUT variant. It is reusable everywhere the DUT variant is used and has no set architecture.

3. Where can you find the UVM PDF and HTML documentation?
   - In the docs directory of the UVM library.

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 3

Overview of DUT and Project

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Project Objectives

In this module, you

- Understand the specification of the YAPP router design
- Define the UVM verification environment for testing the YAPP router

YAPP stands for Yet Another Packet Protocol

© Cadence Design Systems, Inc. All rights reserved.

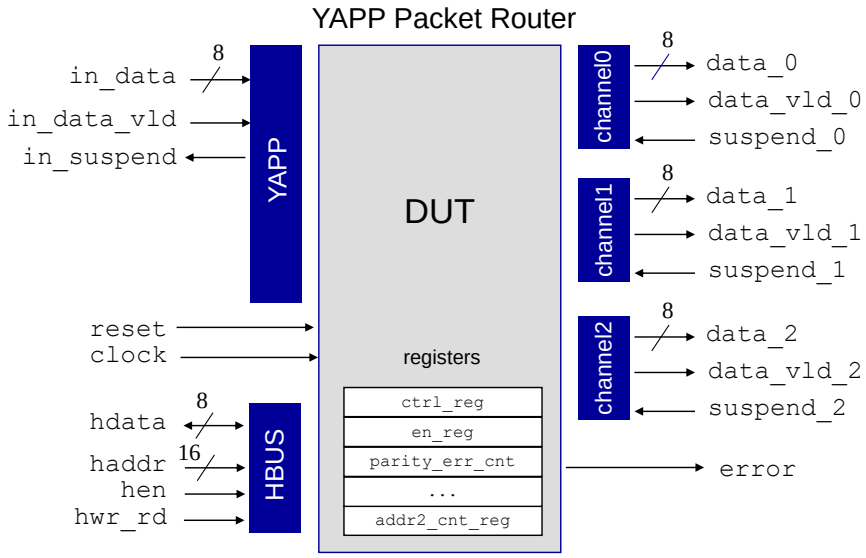
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

DUT: YAPP Router



31 © Cadence Design Systems, Inc. All rights reserved.

We will be using the YAPP router as the DUT for both lab exercises and lecture examples throughout this course.

The packet router accepts data packets on a single input YAPP port and routes the packets to one of three output channels: Channel0, Channel1, or Channel2. It has a HBUS interface for programming registers that are described shortly...

`error` is set when packet is found to have incorrect parity.

A more detailed description of the YAPP router specification is given in your lab book.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Router DUT Packet Specification

A packet is a sequence of bytes:

- First byte is a header
- Next variable set of bytes is the payload
- Last byte is the parity

Header is a 2-bit address field and a 6-bit length field

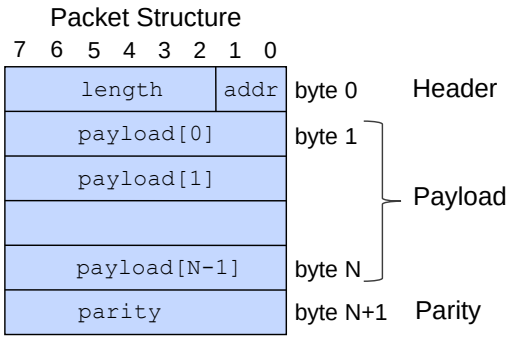
- Address determines to which output channel the packet is routed
  - Address “3” is illegal
- Length specifies the number of bytes in the payload

Payload has a minimum size of 1 byte and a maximum size of 63 bytes

Parity is a full byte of even parity

- Calculated by XORing the header and every byte of the payload

Router raises error output on receiving bad parity



32 © Cadence Design Systems, Inc. All rights reserved.

A packet is a sequence of bytes.

The first byte of the packet is the header. It has two fields:

- 2-bit address field representing the output channel to which the packet should be routed. Valid addresses are 0-2. Address 3 is illegal.
- 6-bit length field representing the number of bytes in the payload (excluding the header and parity).

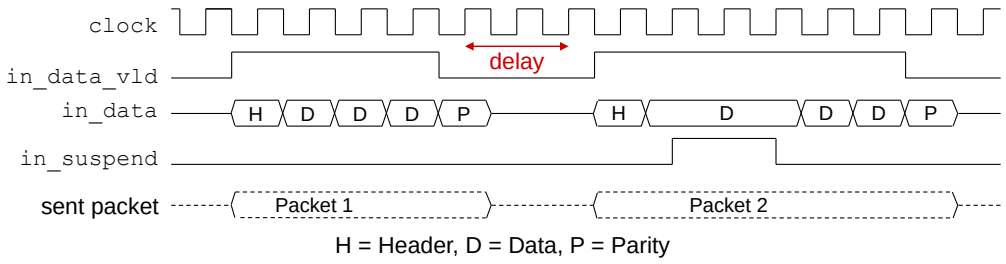
Bytes 1 to N of the packet is the data payload. Minimum length of the payload is 1 byte. Maximum length is 63. Therefore N and the length field of the header are equal and in the range 1 to 63.

Byte N+1 is the parity byte. Parity is even and calculated over the header and payload bytes. Parity can be calculated by simply XOR'ing the header with every byte of the payload.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

YAPP Input Port Protocol (Reference)



H = Header, D = Data, P = Parity

- All *input* signals are active high and are to be driven on the *falling* edge of the clock.
- `in_data_vld` must be asserted on the same clock when the first byte of a packet (the header byte) is driven onto the `in_data` bus. Because the header byte contains the address, this tells the router to which output channel the packet should be routed.
- Each subsequent data byte will be driven on the `in_data` bus with each new falling clock.
- After the last payload byte has been driven, on the next falling `clock`, the `in_data_vld` signal must be de-asserted, and the packet parity byte should be driven.
- The input data cannot change while the `in_suspend` signal is active (indicating FIFO full).

© Cadence Design Systems, Inc. All rights reserved.

This protocol description is for reference only. At the appropriate point in the lab exercises, the code to process this protocol will be provided for you, as this class is focussed on creating UVM verification environments rather than SystemVerilog stimulus tasks.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Packet Router DUT Register and Memory Specification

Address	Register	Reset	Field	Field Name	Policy	Description
0x1000	ctrl_reg	0xff	5:0	maxpktsize	RW	Maximum packet length
			7:6		RW	Unused
0x1001	en_reg	0x01	0	router_en	RW	Router enable
			1	parity_err_cnt_en	RW	Parity error count enable
			2	oversized_pkt_cnt_en	RW	Oversized packet count enable
			3	[reserved]	RW	Not implemented
			4	addr0_cnt_en	RW	Address 0 packet count enable
			5	addr1_cnt_en	RW	Address 1 packet count enable
			6	addr2_cnt_en	RW	Address 2 packet count enable
			7	addr3_cnt_en	RW	Illegal address 3 packet count enable
0x1004	parity_err_cnt_reg	0x00	7:0		RO	Packet parity error count
0x1005	oversized_pkt_cnt_reg	0x00	7:0		RO	Oversized (dropped) packet count
0x1006	addr3_cnt_reg	0x00	7:0		RO	Illegal address packet count
0x1009	addr0_cnt_reg	0x00	7:0		RO	Address 0 packet count
0x100a	addr1_cnt_reg	0x00	7:0		RO	Address 1 packet count
0x100b	addr2_cnt_reg	0x00	7:0		RO	Address 2 packet count
0x100d	mem_size_reg	0x00	7:0		RO	Length of last packet received

If the packet length is greater than the maxpktsize, packet is dropped

Packets dropped if router is disabled

Start Address	Memory Name	Size	Policy	Description
0x1010	yapp_pkt_mem	0:63	RO	Stores last packet received
0x1100	yapp_mem	0:255	RW	"Scratch" memory

34 © Cadence Design Systems, Inc. All rights reserved.

cadence

These are the registers for the extended YAPP router design.

Maximum packet size refers to the payload length and does not include header or parity bytes. The router checks the packet size by reading the length field of the header. If this exceeds `maxpktsize`, the whole packet is dropped, i.e., the router suspends packet processing until the next packet is received.

The router and router counters are enabled by individual bits in `en_reg`. The router specification says that if these bits are changed while the router is processing a packet, then the router behavior is undefined.

`parity_err_cnt_reg` – incremented when a bad parity packet is received

`oversized_pkt_cnt_reg` – incremented on packet with length greater than `maxpktsize`

`addr3_cnt_reg` – incremented when a packet with an illegal address (3) is received

`addr0_cnt_reg` – incremented when a packet with address 0 is received

`addr1_cnt_reg` – incremented when a packet with address 1 is received

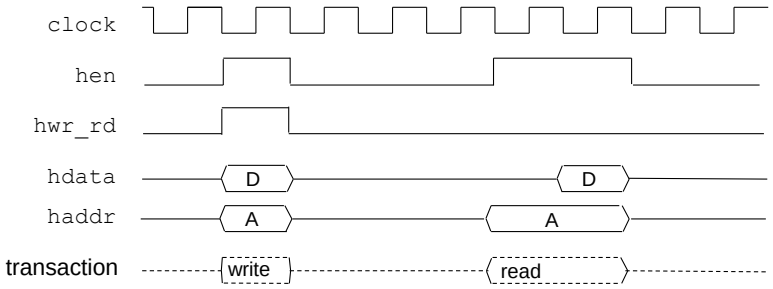
`addr2_cnt_reg` – incremented when a packet with address 2 is received

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

HBUS Protocol (Reference)

The HBUS port provides synchronous read/write access to program the router.



A = Address, D = Data

- A write transaction takes one clock cycle as follows:
  - `hwr_rd` and `hen` need to be 1. On the next rising edge of `clock`, data on `hdata` is read into the register given by `haddr`. `hen` and can be driven to 0 in the next cycle.
- A read transaction takes two clock cycles as follows:
  - `hwr_rd` needs to be 0, and `hen` needs to be 1 in the first cycle. Address on `haddr` is read on the rising edge of the `clock` and `hdata` is driven by the DUT in cycle 2. `hen` can be driven low after cycle 2 ends. This will cause the DUT to tri-state `hdata` bus.

© Cadence Design Systems, Inc. All rights reserved.

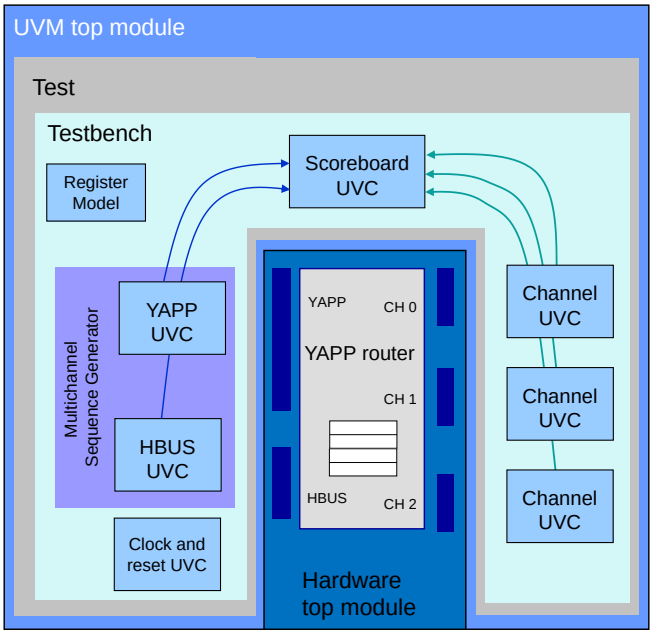
cadence

This protocol description is for reference only. At the appropriate point in the lab exercises, a complete HBUS UVC will be provided for you.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Will We Verify the YAPP?



- Create a UVC for each interface and clock/reset.
- Testbench instantiates and configures UVCs.
- Test instantiates testbench and controls stimulus.
  - Many different tests can be defined for one testbench.
- Add a mechanism to coordinate stimulus from multiple UVCs.
- Add scoreboard for end-to-end checking.
- Add a Register Model to help verify DUT register behavior.
- Hardware top module instantiates DUT and interfaces.
- UVM top module instantiates test and connects UVC interfaces.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Don’t worry about all these components yet… we will work through them and eventually build each piece.

This slide describes the architecture for a UVM verification environment compatible with hardware acceleration. Specifically the creation of separate top level modules for the software side (UVM) and the hardware side.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab Overview

The lab exercises are to create a complete UVM verification environment for the YAPP.

- Following a methodology for a real-life project

In the labs, you will:

- Create stimulus data
- Build one UVM Verification Component (UVC)
- Integrate other pre-built UVCs
- Add a multichannel sequencer
- Add a scoreboard
- Consider a Register Model

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 4

Data Modeling

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Modeling Objectives

In this module, you

- Model transaction data items for the inputs and outputs of the DUT
- Define default constraints and switches to control randomization
- Add the necessary constructs to use data items in a UVM environment
- Add UVM automation
  - Using the UVM “opt-in” system to allow printing, copying, comparing, etc.

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Modeling: Requirements

What we are going to do:

- Model basic building blocks for data representation
  - For example, packets, CPU transactions, register operations

What do we need:

- Generation of data through randomization or direct specification
  - With default random behavior creating legal data
- Sequence-level control
  - If the first five packets don't have errors, then inject three parity errors in succession
- Abstraction from protocol details
  - Signal-level details of sending a packet are modeled in the driver layer
- Data visualization through automation
  - Printing both control and physical fields

© Cadence Design Systems, Inc. All rights reserved.

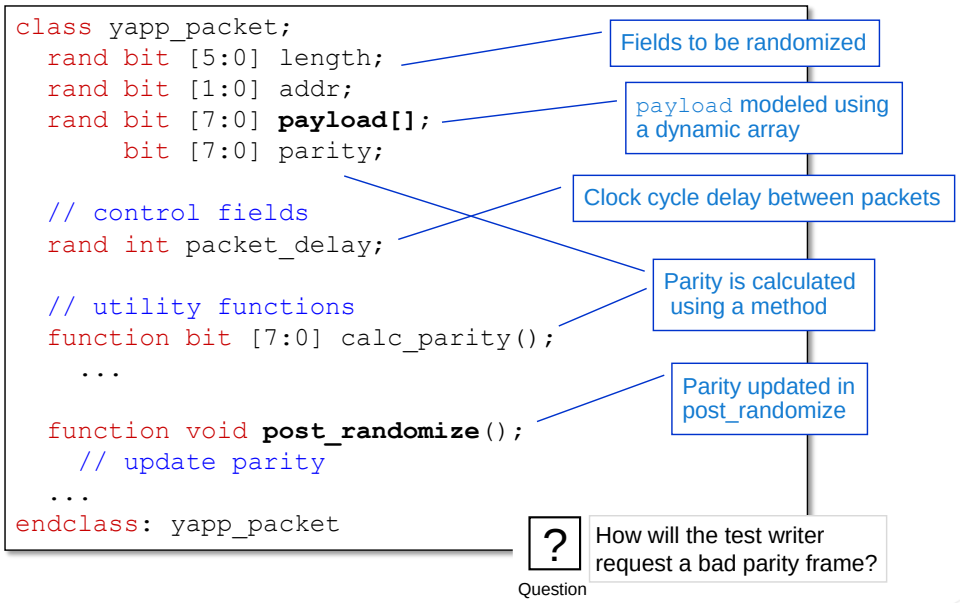
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Item Example: YAPP Packet



41 © Cadence Design Systems, Inc. All rights reserved.

The packet payload is defined using a dynamic array of bytes.

The length, address and payload fields will be randomized and so are defined using `rand` modifiers. When a dynamic array (payload) is randomized, the simulator randomizes a natural number (greater or equal to zero); creates the dynamic array of size given by that number, and then randomizes the data in each elements of the array. Obviously we will need constraints to control the randomization (see later).

`packet_delay` is the number of clock cycles to wait between packets. This information will be used by the UVC driver and gives the user timing control over the injection pf packets. Obviously we will need a constraint to control delay randomization (see later).

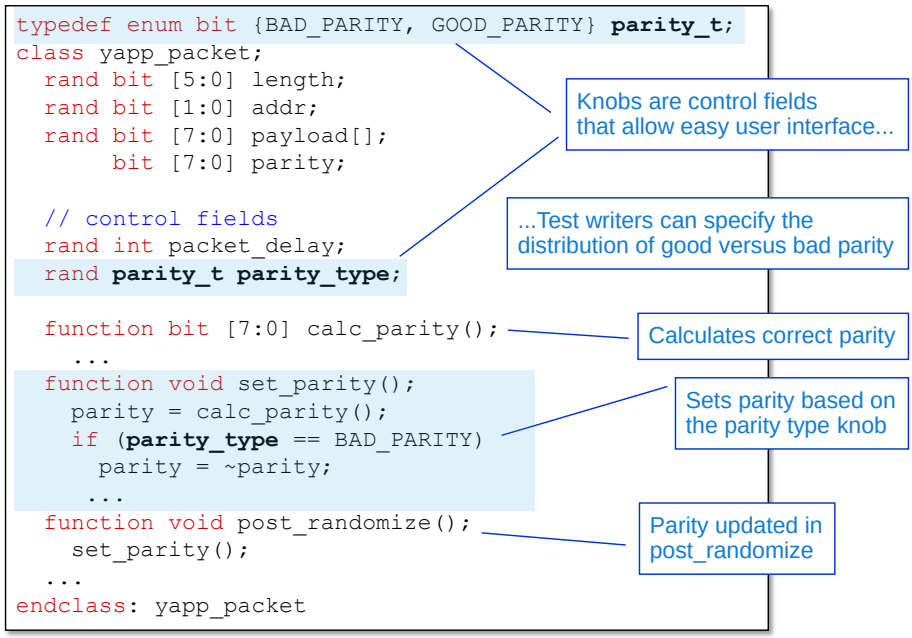
The parity field should not be randomized. For a normal packet, this must be calculated from the randomized values of the header (length and address) and every byte of the payload. Therefore we will need a separate method (`calc_parity`) to calculate correct parity from the header and payload fields. We will call this method in a `post_randomize()` method which his automatically called after a successful randomize.

However we still need the ability to create packets with both good and bad parity. How can we do this?

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Design for Test Writing Using Knobs



42 © Cadence Design Systems, Inc. All rights reserved.

We add a policy or control knob to allow the test writer to create good or bad parity packets.

We define a control field (`parity_type`) of an enumerate type (`parity_t`) with the values `BAD_PARITY` or `GOOD_PARITY`. Using an enumerate type makes debugging parity errors easier.

`parity_type` will be randomized and we can then update the packet with correct or incorrect parity according to its value.

For greatest efficiency we will use two separate methods to update parity. The `calc_parity()` method always calculates correct parity. This will be essential for checking packets for good or bad parity in the output channels and scoreboards. The `set_parity()` method actually assigns the parity field according to the value of `parity_type`. If `parity_type` is good, then the correct parity is assigned. Otherwise we break the parity (e.g., by adding 1 to the correct parity).

Recommendation: Think of the desired scenarios and make sure you have the required control knobs to achieve them.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Basic Constraints

```
typedef enum bit {BAD_PARITY, GOOD_PARITY} parity_t;
class yapp_packet;
  rand bit [5:0] length;
  rand bit [1:0] addr;
  rand bit [7:0] payload[];
       bit [7:0] parity;

  // control fields
  rand int packet_delay;
  rand parity_t parity_type;

 // Default Constraints
  constraint payload_size   { length == payload.size(); }
  constraint default_length { length > 0; length < 64; }
  constraint pkt_delay { packet_delay > 1; packet_delay < 20; }

  // utility functions
  function bit [7:0] calc_parity();
  ...
endclass: yapp_packet
```

Dependency constraint (length and payload consistent)

Payload length is legal

By default we don't want a huge delay

? Question

Can you think of any other required constraints?
Hint: In typical traffic, most frames are legal.

43 © Cadence Design Systems, Inc. All rights reserved.

cadence

The YAPP packet specification defines a number of rules for a legal packet. Many of the fields in the YAPP packet definition are randomized, so we need to add some basic constraints to make the randomized YAPP packet legal. These constraints can be overridden or disabled to test the router response to illegal packets.

The `length` field of the packet header must match the number of bytes in the payload. The payload is defined as a SystemVerilog dynamic array. Randomizing a dynamic array randomizes both array length and contents, therefore we need a dependency constraint, `payload_size`, which defines that the `length` field and payload size are equal.

The payload also has limitations on length. The minimum payload is one byte, and maximum is 63, so we define a basic constraint for this. Note that the upper payload limit constraint is implied by the `payload_size` constraint, given that `length` is 6 bits.

`packet_delay` is a field to introduce clock cycle delays between adjacent packets. As the field is randomized and defined as an `int`, we need a constraint to limit the delay.

What other constraints are required? Address? Parity?

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

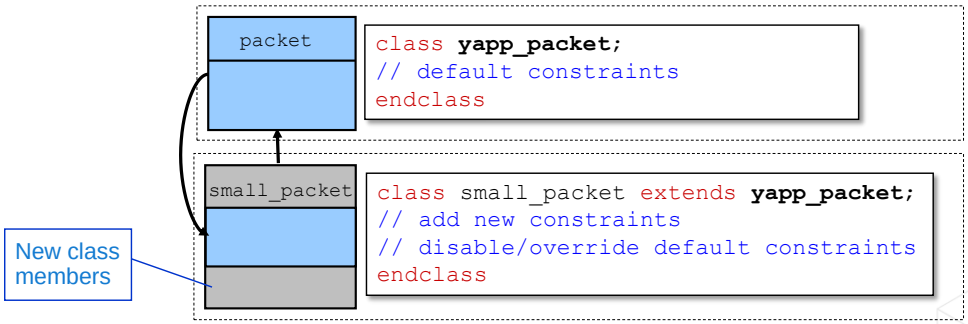
Layering Constraints for Testing

In a specific test, you might want to:

- Adjust the randomness
- Disable existing constraints
- Further change the generation using new constraints

This is achieved using class inheritance

- Create a new class that inherits logic from the parent class



 © Cadence Design Systems, Inc. All rights reserved.

The basic constraints in the yapp_packet declaration will generate randomized, legal packets over the entire payload range. However for specific test, we may want to:

- Change the constraints, for example to create more bad parity packets or short packets with payload length less than 20 bytes.
- Disable constraints, for example to generate illegal packets to address 3.
- Add new constraints, for example to send packets only to address 0.

We will use inheritance to achieve this. We can create subclasses of yapp_packet which can disable, or override existing constraints and add new constraints.

UVM gives us a simple mechanism for switching between parent and subclass instances as we will see.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Example: Layering Constraints

```
class yapp_packet;
  rand int packet_delay;
  constraint medium_pkt_delay {packet_delay >  1;
                               packet_delay < 20;}
  ...
endclass: yapp_packet
```

```
class short_delay_packet extends yapp_packet;

 // further constrain the delay values
 constraint medium_pkt_delay {packet_delay > 1;
                              packet_delay < 26;}
...
endclass: short_delay_packet
```

These constraint blocks will be resolved together

? Question

What is the legal range of `packet_delay` in `short_delay_packet`?
How can you get a maximum `packet_delay` of 25?

45 © Cadence Design Systems, Inc. All rights reserved.
cadence

How do you get a `packet_delay` of 25?

You can redefine the `pkt_delay` constraint block or disable the constraint by setting its `constraint_mode` to zero.

Which one is best?

The constraint mode can only be changed for a specific instance of a class, therefore this option is only good for individual packets.

A subclass which redefines a constraint can be used for all instances of a class, therefore this options is best for all modifying all packets of a particular test.

You can remove a parent class constraint in a subclass by re-declaring the constraint using empty brackets:

```
constraint c1 {}
```

When overriding constraints, it may be more readable to remove the existing constraint and define a new constraint with a more meaningful name.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Class Methods

```
class yapp_packet;
  rand bit [5:0] length;
  rand bit [1:0] addr;
  rand bit [7:0] payload[];
       bit [7:0] parity;
  ...
endclass: yapp_packet
```

What kind of operations may be required for such a data item?

- Randomization
- Printing
- Comparing
- Copying
- Packing
- Waveform dumping

Randomization is defined in SystemVerilog

UVM class library provides the rest

46 © Cadence Design Systems, Inc. All rights reserved.

There are a standard set of operations that you would want to carry out on a class. As a minimum, you would want to randomize a class instance and print the property values. You would also want to compare different instances of the same class and copy class properties from one instance to another. You may want to pack all (or selected) class properties into a single vector or data structure for transmission to the DUT. You may also wish to modify the representation of an instance in a waveform viewer.

SystemVerilog provides very little infrastructure for classes – only the randomization method is built-in to the language. Normally the user would have to create custom implementations of the required methods, but UVM can provide these for us.

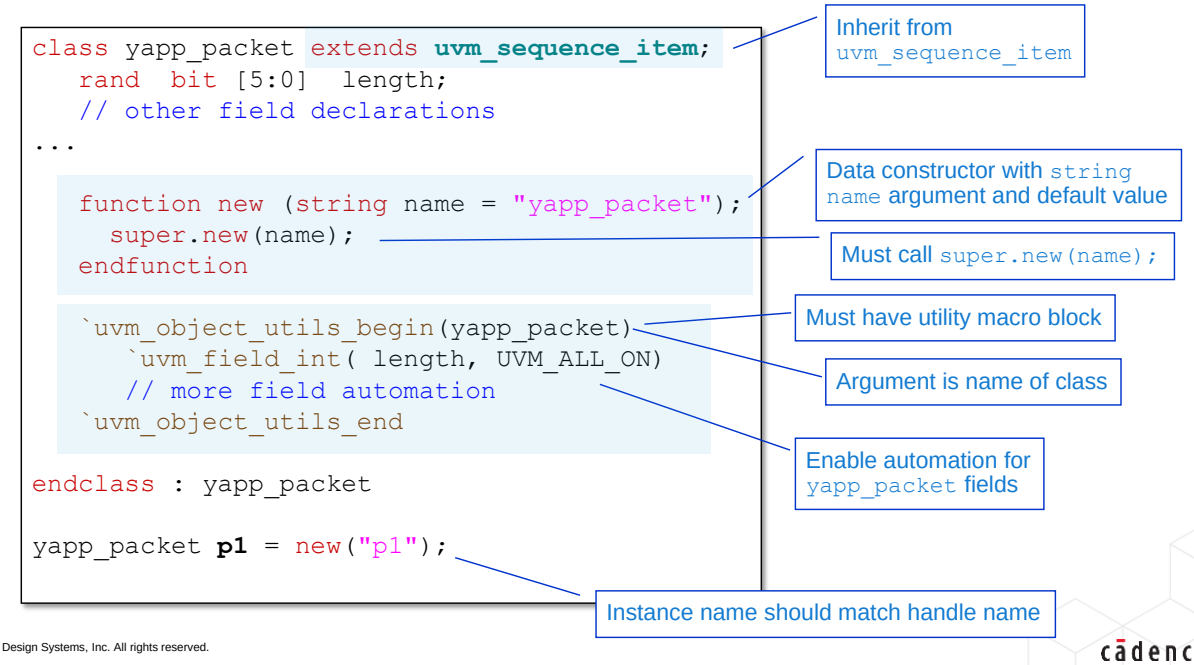
Standard UVM class operations are:

- Copy – copies a field from one instance to another. Also used in clone.
- Compare – compares a field from one instance with another.
- Print – prints the field to the standard output. Also sprint which creates the print string, but does not send it to the output.
- Pack/unpack – concatenates the fields into a single integral array.
- Record – enables vendor-specific transaction recording in a waveform viewer.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Modeling with UVM



To enable a data class for UVM, we have to add 3 elements as a minimum:

1. Extend from the UVM subclass `uvm_sequence_item` to inherit UVM functionality.
2. Add a UVM data constructor with a single argument `name` of type `string` with a default value (by convention we use the data class name). This is the instance name of the class. The constructor must call `super.new(name)` as the first line to pass the instance name argument up into the `uvm_sequence_item` class.

   Failure to use a default value for `name` gives the following compilation error:

   - `*E,FAABP1 task, function, or assertion instance does not specify all required formal arguments`
3. Add a field automation block using `uvm_object_utils` macros.

   `` `uvm_object_utils `` macros are used to enable common operations declared in `uvm_object` (`copy`, `compare`, `print`, etc.)

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Field Macros

Use field macros to enable automation methods for class properties

- `copy`, `compare`, `pack`, `record`, `print`

Syntax: `` `uvm_field_*(<field_name>, <flags>) ``

- `<field_name>` must be an existing property identifier
- `<flags>` define what is supported for that field

Non-array types use these macros:

```
`uvm_field_int( <field_name>, <flags> )                 // integral types
`uvm_field_object( <field_name>, <flags> )              // class handles
`uvm_field_string( <field_name>, <flags> )              // strings
`uvm_field_event( <field_name>, <flags> )               // events
`uvm_field_real( <field_name), <flags> )                // reals
`uvm_field_enum( <enum_type>, <field_name>,  <flags>)   // enums
```

Macros for enumeration types have an additional argument for the type name

48 © Cadence Design Systems, Inc. All rights reserved.

cadence

You usually have a field macro for every field (property) of the class. These can only be declared inside the object utilities block (between `` `uvm_object_utils_begin `` and `` `uvm_object_util_end ``).

If you do not declare a field macro for a property, then the property is excluded from all automation methods, i.e., it will not be printed, copied, compared, recorded or packed.

The name of the macro depends on the type of the property. The macro has two or more arguments. The first argument is the name of the field. The field type must be compatible with the macro name. The second argument is a flag list which defines which automation methods are enabled for the field.

The macros shown here are for properties of integral, class, string, event and enumerate types. Remember an integral type in Verilog is any packed one-dimensional array of `bit` or `logic`, including `integer` and `int` types. For example, the following are all integral types:

```
bit [7:0] parity;
logic [2:0] enab;
int packet_delay;
```

Field macros for enumerate fields have an additional, initial macro which must be the name of the enumerate type, implying that a `typedef` should be used to name enumerate types for class properties. Fields declared of an antonymous enumerate type can not be automated.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Array Type Field Macros

unpacked 1-D dynamic arrays

- `uvm_field_**array_int**()
- `uvm_field_**array_object**()
- `uvm_field_**array_string**()
- `uvm_field_**array_enum**()

unpacked 1-D static arrays

- `uvm_field_**sarray_int**()
- `uvm_field_**sarray_object**()
- `uvm_field_**sarray_string**()
- `uvm_field_**sarray_enum**()

queues

- `uvm_field_**queue_int**()
- `uvm_field_**queue_object**()
- `uvm_field_**queue_string**()
- `uvm_field_**queue_enum**()

associative arrays

- `uvm_field_**aa**_<d_type>_<ix_type>
- `uvm_field_**aa**_int_string()
- `uvm_field_**aa**_object_string()
- `uvm_field_**aa**_string_string()
- `uvm_field_**aa**_int_int()
- `uvm_field_**aa**_int_integer()
- `uvm_field_**aa**_int_int_unsigned()
- `uvm_field_**aa**_string_int()
- `uvm_field_**aa**_object_int()
- ...

- Array macros depend on the array type

49 © Cadence Design Systems, Inc. All rights reserved.

cadence

Simple logic and bit arrays use the `uvm_field_int macro on the previous page. These array macros are for unpacked one-dimensional static arrays, one-dimensional dynamic arrays and associative arrays.

For a dynamic array, the choice of macro depends on the data type of the array, so for:

```
bit [7:0] payload[];
```

The field automation macro is:

```
`uvm_field_array_int(payload, <flags>)
```

For a unpacked static array, the choice of macro depends on the data type of the array, so for:

```
string strarr [7:0];
```

The field automation macro is:

- `uvm_field_sarray_string(strarr, <flags>)

For associative arrays, d_type is the data type and ix_type is the index type. So for:

```
logic [7:0] assoc [string];
```

The field automation macro is:

```
`uvm_field_aa_int_string(assoc, <flags>)
```

There are many other associative array macros beyond those listed here.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Flag Arguments for Field Macros

Flags are numeric and can be combined using a bitwise OR '|' or AND '+' operator.

Operation Flag	Description
UVM_ALL_ON UVM_DEFAULT	All field operations enabled (copy, compare, print, record, and pack) – default setting.
UVM_NOCOPY	Excludes the field for copy and clone.
UVM_NOCOMPARE	Excludes the field for compare.
UVM_NOPRINT	Excludes the field in print and sprint.
UVM_NOPACK	Excludes the field in pack and unpack.
UVM_NORECORD	Excludes the field in vendor-specific transaction recording.

Additional Print/Record Radix Flags for Integral Types				
UVM_BIN	UVM_DEC	UVM_UNSIGNED	UVM_OCT	UVM_HEX
Binary %b	Signed decimal %d	Unsigned decimal %u	Octal %o	Hex %h (default)

50 © Cadence Design Systems, Inc. All rights reserved.

Field macro flag arguments define which class operations are implemented for the specified field.

UVM_ALL_ON or UVM_DEFAULT enable all operations. The UVM_NO* flag options only exclude specific operations but implicitly allow everything else.

Standard UVM class operations are:

- Copy – copies a field from one instance to another. Also used in clone.
- Compare – compares a field from one instance with another.
- Print – prints the field to the standard output. Also used in sprint which creates the print string, but does not send it to the output.
- Pack/unpack – concatenates the fields into a single integral array and back.
- Record – enables vendor-specific transaction recording in a waveform viewer.

Flags are implemented as a vector with separate bits for every operation. Therefore flags can be combined with | and + operations.

Additional print flags to control how integral fields are displayed in print or record. The default is UVM_HEX. UVM_DEC displays values as signed (2's complement) whereas UVM_UNSIGNED displays them as unsigned (binary).

The print flags are for scalar or arrayed integral fields only. Field macros of non-integral types, such as uvm_field_string, sets the print option automatically to the correct type. Setting a print flag in a non-integral field macro has no effect.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Field Macro Example

Every data item property should have a field automation macro.

```
typedef enum bit {GOOD_PARITY, BAD_PARITY} parity_t;
class yapp_packet extends uvm_sequence_item;
    ...
    rand  bit [7:0] payload[];
          bit [7:0] parity;
    rand  parity_t parity_type;
    rand  int packet_delay;

    // field declaration automation flags
   `uvm_object_utils_begin(yapp_packet)
     ...
     `uvm_field_array_int( payload,  UVM_ALL_ON )
     `uvm_field_int( parity, UVM_ALL_ON + UVM_BIN )
     `uvm_field_enum( parity_t, parity_type,  UVM_ALL_ON )
     `uvm_field_int( packet_delay, UVM_ALL_ON + UVM_NOCOMPARE )
   `uvm_object_utils_end

endclass: yapp_packet
```

Enables automation for `yapp_packet` fields

Print radix

`UVM_NOCOMPARE` overrides `UVM_COMPARE` set by `UVM_ALL_ON`

51 © Cadence Design Systems, Inc. All rights reserved.

cadence

Usually everything is enabled and print radix options are added:

e.g., enable everything and print in unsigned binary:

- `` `uvm_field_int(data, UVM_ALL_ON + UVM_UNSIGNED); ``

To disable specific operations, it is usually more readable to turn everything on (with either `UVM_ALL_ON` or `UVM_DEFAULT`), and then add exceptions and print radix options.

e.g., enable everything except compare and print in binary:

```
`uvm_field_int(data, UVM_ALL_ON + UVM_NOCOMPARE + UVM_BIN);
```

However flag options to disable specific operations implicitly allow all other operations, therefore the above could be written as|:

- `` `uvm_field_int(data, UVM_NOCOMPARE + UVM_BIN); ``

Multiple disable flag options must be added separately:

e.g., disable copy and compare only and print in Hex (default)

```
o`uvm_field_int(data, UVM_ALL_ON + UVM_NOCOPY + UVM_NOCOMPARE);
o`uvm_field_int(data, UVM_NOCOPY + UVM_NOCOMPARE);
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Copy and Clone Automation Methods

Copy method

- `function void copy(uvm_object rhs);`
- Copies all fields from the `rhs` argument to the calling object
  - Except instance name

Clone method

- `function uvm_object clone();`
  - Creates a new instance and copies all fields
    - Including instance name
  - As clone returns `uvm_object`, you will need to use `$cast`

```
yapp_packet  p1, p2, p3;       // data item handles
int ok;

initial begin
  p1 = new("p1");              // create p1 instance
  p2 = new("p2");              // create p2 instance
  ok = p1.randomize();

  p2.copy(p1);                 // copy p1 to p2
  $cast (p3, p1.clone());      // create instance p3 and copy
  ...                          // all fields from p1 to p3
```

? Question

What is the difference between copy and clone?

52 © Cadence Design Systems, Inc. All rights reserved.

cadence

`copy` Syntax

```
function void copy(uvm_object rhs);
```

UVM also defines a function `do_copy` which, if defined in a data class, will be called before the copy operation. This allows custom copying for fields, e.g., conditional copying of one field depending on the value of another. In such an example, the customized field must be excluded from standard copying using field flags (`UVM_NOCOPY`).

`clone` Syntax

```
function uvm_object clone();
```

Question: What is the difference between `clone()` and `copy()`?

Answer: `clone()` creates a new object and then copies all the fields across (clone = construct + copy)

Another important difference between `copy` and `clone` is in the handling of the instance name for the object. When copying from `p1` to `p2`, the instance name is not copied. Therefore `p2` keeps its original instance name `p2`. When cloning `p1` to `p3`, `p3` is given the instance name from `p1`. There is a UVM method to update the instance name:

```
$cast(p3, p1.clone());
p3.set_name("p3");
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Compare Automation Method

Compare method

- `function bit compare(uvm_object rhs, uvm_comparer comparer=null);`
- Compares the contents of the `rhs` object with the calling object
- Takes an optional `comparer` argument (described in the reference manual)
- Returns a 1 if all compared fields match, otherwise returns 0, and prints report

```
yapp_packet  p1,p2;
int ok;

initial begin
  p1 = new("p1");
  p2 = new("p2");
  ok = p1.randomize();
  ok = p2.randomize();
  if (!p2.compare(p1))
    ...
```

Compare `p1` and `p2`

Report for miscompare (see later for details)

```
UVM_INFO @ 0: reporter [MISCMP] Miscompare for p2.addr: lhs = 'h2 : rhs = 'h3
UVM_INFO @ 0: reporter [MISCMP] 3 Miscompare(s) (1 shown) for object p1@2614 vs. p2@2578
```

Question

Where would you use the compare method?

How can we exclude fields from comparison?

© Cadence Design Systems, Inc. All rights reserved.

cadence

`compare` Syntax

```
function bit compare(uvm_object rhs, uvm_comparer comparer=null);
```

Note that by default, only the first property mismatch is shown, and miscompares are info report (see later for report severity). This default behavior can be changed via a third argument to the `compare` of type `uvm_comparer`.

`uvm_comparer` is a built in UVM class containing settings for default comparison. By creating an instance of this class and updating the settings we can modify the default comparison behavior.

`uvm_comparer` also defines a function `do_compare` which, if defined in a data class, will be called before the compare operation. This allows custom comparison for the object fields, e.g., conditional comparison of one field depending on the value of another. In such an example, the customized field must be excluded from standard comparison using field flags (`UVM_NOCOMPARE`).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Print Automation Method

```
yapp_packet  p1;
int ok;
string str;

initial begin
  p1 = new("p1");
  ok = p1.randomize();

 // default uvm_default_table_printer
  p1.print();
  // change print policy
  p1.print(uvm_default_tree_printer);

  str = p1.sprint();
...
```

Creates string but doesn't print

Table view: (default)

```
----------------------------------------
Name          Type          Size    Value
----------------------------------------
p1            yapp_packet   -       @1150
  length      integral      6        'd14
  addr        integral      2        ' d2
  parity      integral      8     'b110001
  payload     da(integral)  5        -
    [0]       integral      8       'h2d
    [1]       integral      8       'h95
    ...
----------------------------------------
```

Tree view

```
p1: (yapp_packet@1150) {
  length: 'd14
  addr: 'd2
  parity: 'b110001
  payload:  {
    [0]: 'h2d
    [1]: 'h95
    ...
  }
}
```

Automation flags determine print radix

- Customized print views can also be defined

54 © Cadence Design Systems, Inc. All rights reserved.

The `print()` method converts the packet values into a string, and writes the string to the simulator output. There is also a sub-method called `sprint()`. This creates the print string but does not write it to the output. We use `sprint()` in UVM messaging (see later).

Syntax

```
function void print ( uvm_printer  printer  =  null );

function string sprint ( uvm_printer  printer  =  null );
```

`uvm_printer` is a built in UVM policy class containing settings and formatting for default printing. By extending this class and modifying the settings we can redefine the default print operation to create new custom print policies.

Three print policies are pre-defined:

- `uvm_default_table_printer` (default)
- `uvm_default_line_printer`
- `uvm_default_tree_printer`

UVM also defines a function `do_print` which, if defined in a data class, will be called before the print operation. This allows custom printing for object fields, e.g., conditional printing of one field depending on the value of another. In such an example, the customized field must be excluded from standard printing using field flags (`UVM_NOPRINT`).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating a UVM Package

Most user-defined UVM classes are placed in separate files.

Classes can only be compiled from a module or package scope.

Recommendation:

- Include related class files into a package.
- Import the package into modules where required.

The UVM library is supplied in the package `uvm_pkg`.

- Import package to access the library.
- The macro file `uvm_macros.svh` must be included separately.

yapp_packet.sv

```
class yapp_packet extends uvm_sequence_item;
  ...
endclass: yapp_packet
```

yapp_pkg.sv

```
package yapp_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"

  `include "yapp_packet.sv"
   ...
endpackage
```

55 © Cadence Design Systems, Inc. All rights reserved.

Two declarations are required to access the UVM library.

First, you need to import the UVM package

```
import uvm_pkg::*;
```

Second, in order to resolve the macros used in the UVM library, which are expanded as a pre-compilation step, you directly include the UVM macros definition file:

```
`include "uvm_macros.svh"
```

For maintenance and portability, it is a convention that most UVM classes are declared in separate files. However, the class files cannot be compiled directly, but can only be compiled from a module or package scope.

The recommendation is to include related UVM class files into a package, and import the package into the module (or package) where it is required. Remember that the package file must be separately compiled on the command line.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Compiling UVM Code

yapp_pkg.sv

```
package yapp_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  `include "yapp_packet.sv"

endpackage : yapp_pkg
```

top.sv

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  // import yapp package
  import yapp_pkg::*;

  initial begin
    ...
  end
endmodule : top
```

To compile UVM code:

- Set `$UVMHOME` to install the directory of the required UVM library.
- Use the simulator option `-uvmhome` to reference `$UVMHOME`.
- Use the `incdir` to reference any include file directories.

Use a run file for simplicity.

run.f

```
-uvmhome $UVMHOME
-incdir ../sv

../sv/yapp_pkg.sv
top.sv
```

```
%xrun -f run.f
```

56 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM1.2 is supported by IUS versions 14.1 or later.

Several versions of UVM may be provided with the simulator, depending on the release.

The following steps should allow you to compile a UVM test environment with the required UVM library version:

1. Create an environment variable `$UVMHOME` to point to the root install directory of the UVM library version you require (containing `src`, `examples`, and `doc` directories).
2. Use the simulator option `-uvmhome` to reference `$UVMHOME`.
3. Set an `incdir` reference to any include file directories (e.g., UVC `sv`).
4. Remember to compile any user-defined UVM packages as well as the top-most module.

Hint: Create a run file containing these steps as shown in the `run.f` file on the slide.

You can then compile and simulate using the run file via the following simulator option:

- Xcelium™

```
xrun -f run.f
```

- Incisive®

```
irun -f run.f
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Modeling Quiz

```
-----------------------------------------------
Name             Type           Size  Value
-----------------------------------------------
packet           yapp_packet    -     @2592
  length         integral       6     'h4
  payload        da(integral)   4     -
    [0]          integral       8     'h6b
    [1]          integral       8     'h58
    [2]          integral       8     'haa
    [3]          integral       8     'he4
  parity         integral       8     'h0
  parity_type    parity_t       1     BAD_PARITY
  packet_delay   integral       32    -549139852
-----------------------------------------------
-----------------------------------------------
Name             Type           Size  Value
-----------------------------------------------
packet           yapp_packet    -     @2679
  length         integral       6     'h6
  payload        da(integral)   6     -
    [0]          integral       8     'h5f
    [1]          integral       8     'he5
    [2]          integral       8     'h3
    [3]          integral       8     'h25
    [4]          integral       8     'h1
    [5]          integral       8     'he3
  parity         integral       8     'h0
  parity_type    parity_t       1     GOOD_PARITY
  packet_delay   integral       32    -199888430
-----------------------------------------------
```

- Here is a print of two randomized YAPP packets
- Identify three issues with these packets and suggest the cause of the problem

Solutions at the end of the module

57 © Cadence Design Systems, Inc. All rights reserved.

cadence

A simple print can help us find and debug issues with our data stimulus.

As a minimum we should check that every property is printed and the value of each property is correct.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab1 Creating a Stimulus Model

- Model the packet for the YAPP DUT
- Build a simple packet using the UVM sequence Item, include a method to calculate parity, and call the method in `post_randomize`
- Generate a list of packets
- Explore automation: `copy`, `clone`, and `print` (try different print formats)

Please follow the naming guidelines for class properties carefully – this will save time later.

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Data Modeling Quiz

```
-----------------------------------------------
Name              Type           Size  Value
-----------------------------------------------
packet            yapp_packet    -     @2592
  length          integral       6     'h4
  payload         da(integral)   4     -
    [0]           integral       8     'h6b
    [1]           integral       8     'h58
    [2]           integral       8     'haa
    [3]           integral       8     'he4
  parity          integral       8     'h0
  parity_type     parity_t       1     BAD_PARITY
  packet_delay    integral       32    -549139852
-----------------------------------------------
-----------------------------------------------
Name              Type           Size  Value
-----------------------------------------------
packet            yapp_packet    -     @2679
  length          integral       6     'h6
  payload         da(integral)   6     -
    [0]           integral       8     'h5f
    [1]           integral       8     'he5
    [2]           integral       8     'h3
    [3]           integral       8     'h25
    [4]           integral       8     'h1
    [5]           integral       8     'he3
  parity          integral       8     'h0
  parity_type     parity_t       1     GOOD_PARITY
  packet_delay    integral       32    -199888430
-----------------------------------------------
```

addr not printed

- Here is a print of two randomized YAPP packets.
- Identify three issues with these packets and suggest the cause of the problem.

1. The address property is not printed due to a missing field automation macro.
2. Parity is `'h0` for both packets. So there is a problem in the parity calculation.
3. Packet delay has large, negative values in both packets due to an incorrect or missing constraint.

© Cadence Design Systems, Inc. All rights reserved.

cadence

A simple print can help us find and debug issues with our data stimulus.

As a minimum we should check that every property is printed and the value of each property is correct.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 5

Simulation Phases

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation Phases Objectives

In this module, you will:

- Discuss the need for simulation phases
- Describe the UVM simulation phase implementation
- Create hierarchy using build phases

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation Phases for Components

Design verification is divided into phases:

- Testbench creation, configuration, run, check, report results, etc.

Most verification components perform set tasks in each simulation phase.

- Build and connect hierarchy during the “creation” phase.
- Stimulus generation during the “run” phase.

Some components only need to perform tasks in specific phases.

- Scoreboard examining results in the “check” phase.

Phases must run in order.

- Next phase cannot begin until the current phase is complete
  - For example, don't check results until the run is complete.

Having a set of standard pre-defined phases allows easy integration of multiple components.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Verification of a design in simulation can be divided up into a series phases. For example:

- Creating and connecting the testbench components
- Configuring the testbench and design
- Running stimulus
- Checking scoreboards and reporting results

For some components, their main functionality is executed in specific phases, for example, a scoreboard examining results in the check. Other components may have actions in every phase.

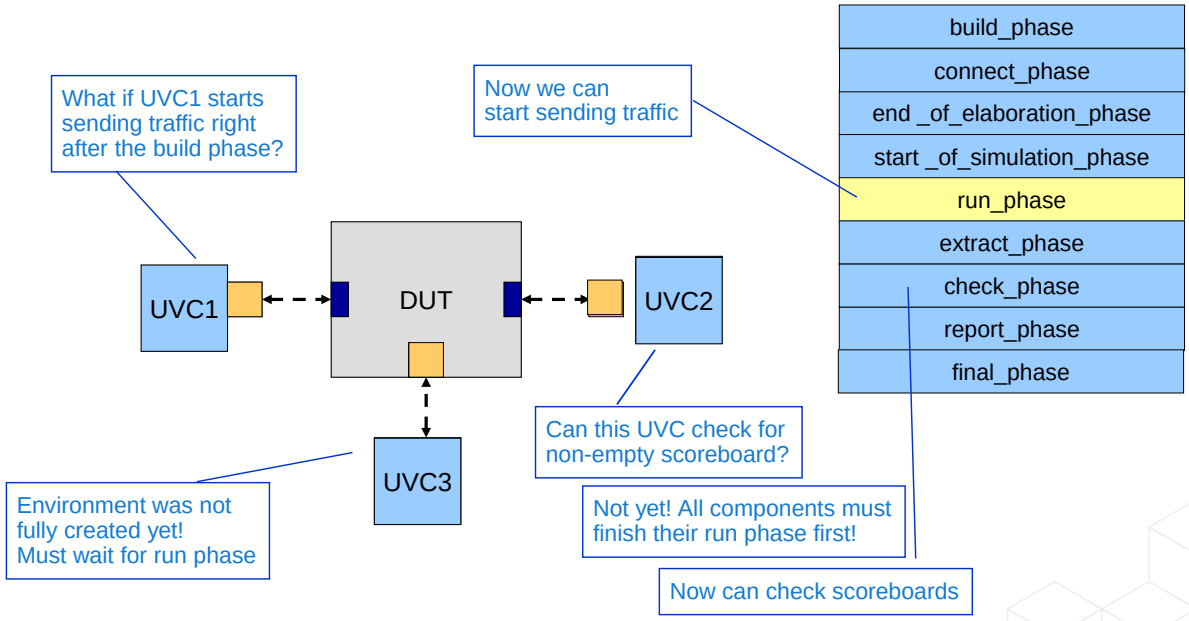
For clean simulation, the phases must run in order, and the next phase should not begin until the current phase is completely finished. For example, the scoreboard cannot check results until every UVC has finished sending stimulus.

UVM defines a set of standard phases for easy integration of multiple testbench components.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation Phases and Coordination



63 © Cadence Design Systems, Inc. All rights reserved.

In this example, all three UVCs want to send data into the DUT.

If we build the DUT first, followed by UVC1, then UVC1 can't start sending stimulus into the design until every UVC has been built and connected. Therefore we need a series of phases to build, connect and carry out initial configuration of the testbench before traffic is sent into the design. UVM defines the build to start of simulation phases for this purpose.

Once these phases are complete, UVCs can start sending stimulus in the run phase.

If UVC2 has finished sending data, and needs to check the scoreboard, it must wait until all other UVCs have finished and the run phase is complete.

Therefore after the run phase, we need a series of phases to check and report results. UVM defines the extract to final phases for this.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Simulation Phase Names and Descriptions

All phases except `run_phase()` execute in zero time as functions

build_phase	Build top-level testbench topology
connect_phase	Connect environment topology
end_of_elaboration_phase	Post-elaboration activity (e.g., print topology)
start_of_simulation_phase	Prepare for simulation
run_phase	Task – run-time execution of the test
extract_phase	Gathers details on the final DUT state
check_phase	Processes and checks the simulation results
report_phase	Simulation results analysis and reporting
final_phase	Tie up loose ends, close files

- User-defined phases can also be added and there are additional run-time phases

64 © Cadence Design Systems, Inc. All rights reserved.
cadence

These are the names of the pre-defined phases in UVM.

`build_phase` – for building the testbench component hierarchy.

`connect_phase` – for making connections between components in the hierarchy.

`end_of_elaboration_phase`, `start_of_simulation_phase` – these two phases are for debug (printing the component hierarchy); initial configuration and any other pre-run activity.

`run_phase` – simulation of the design.

`extract_phase`, `check_phase`, `report_phase` – these phases are for extracting and checking results, and generating reports on the simulation status and success.

`final_phase` – any other activity before end of simulation.

All phases execute in zero time, and are implemented using functions, except `run_phase`, which can consume time and must be implemented as a task.

You can also add user-defined phases, although this can break reusability. There are also additional sub-phases of the run phase – called runtime phases. These are described on the next slide.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Phase Method Reference

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  ...
function void connect_phase(uvm_phase phase);
  ...
function void end_of_elaboration_phase(uvm_phase phase);
  ...
function void start_of_simulation_phase(uvm_phase phase);
  ...
task run_phase(uvm_phase phase);
  ...
function void extract_phase(uvm_phase phase);
...
function void check_phase(uvm_phase phase);
...
function void report_phase(uvm_phase phase);
...
```

Build phase must contain super.build_phase(phase);

No other phase needs to contain a super call

Run phase is a task, all other phase methods are functions

Add these methods to your components to execute code in the required phase.

- Automatically called by the simulator.

65 © Cadence Design Systems, Inc. All rights reserved.

cadence

All phase methods have a single input argument named `phase` of type `uvm_phase`. We will see uses for this argument later in the course.

All phases methods except the run phases are implemented with `void` functions.

The run phase is implemented with a task.

The build phase method must contain as call to the parent class build phase, i.e.,

```
super.build_phase(phase);
```

This is to pass essential configuration information up into the UVM library (more information in the Configuration module later in the class).

It is a common mistake to mistype the name or arguments of a phase method.

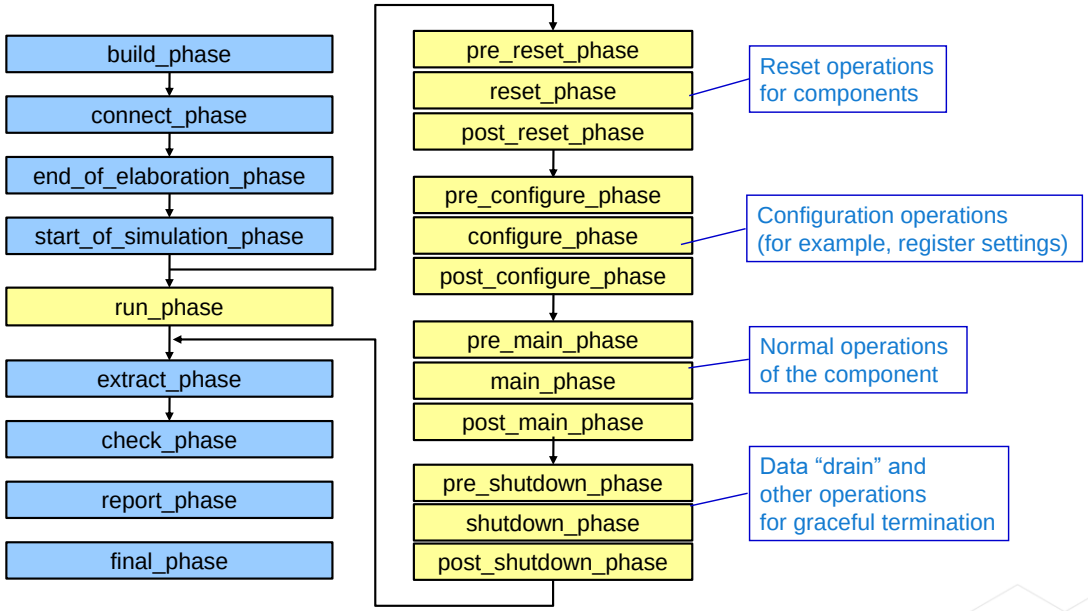
Mistyping the arguments will give compilation errors as the implementation arguments no longer match the UVM parent class implementation or prototype.

Mistyping the method name will not generate errors, but the implementation will not be executed in the appropriate phase. Debugging method name errors can be difficult, so you need to be careful when writing phase methods.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Run-Time Sub-Phases



66 © Cadence Design Systems, Inc. All rights reserved.

cadence

Run-time sub-phases were added in UVM1.0, but their implementation is not yet complete.

The intention is to define separate phases for the reset, configuration, normal and shutdown operations of a UVC during the run phase. A UVC could then generate different stimulus during the run phase depending on which sub-phase was executing.

The sub-phases run in parallel with `run_phase`. For example, if UVC `One` generates stimulus in `reset_phase`, UVC `Two` in `main_phase` and UVC `Three` in `run_phase`, then at the beginning of the `run_phase`, both `One` and `Three` would be generating stimulus. As soon as `One` has finished, the phasing would move onto `main_phase` where `Two` would generate stimulus, potentially at the same time as `Three` if that has not yet finished.

This part of the implementation is stable.

However the specification also allows for run time sub-phases to jump back, for example if a reset is received during `main_phase`, the user could write code to detect this and stop `Two` from generating stimulus. The phasing could then jump back to `reset_phase` and `One` could generate stimulus, followed by `Two` when the `reset_phase` ends. Note that `Three` would not be affected by any sub-phase jumps.

This part of the implementation is not yet stable.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Using Run-Time Sub-Phases

Using these phases is optional!

Run-time sub-phases are intended to help control UVCs.

- UVCs can generate different stimuli in different phases
- UVCs can "jump back" to a previous run-time phase
  - For example, when a reset is received during the main phase
- Related UVCs can be grouped into domains
  - For example, UVCs with a common reset

However, the implementation of run-time phases is not yet fixed (UVM 1.2).

- Phase-specific stimulus can be used
- Domains may be used to synchronize stimulus between different UVCs
- Phase jumping and other features should be avoided


TB: UVM Run-Time Phasing

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Our recommendations:

- Use `run_phase()` to define stimulus for interface UVCs (using the default sequence property).
- It is possible to use specific run-time sub-phases for separating out stimulus which has to run at specific times during the run phase (e.g., reset). However we recommend you avoid this unless you have no other alternative.
- Avoid the advanced features of run-time phases (domains, jumping forward, backwards, multi-reset, etc.).

There is a Training Byte on UVM Run-Time Phasing with more information.

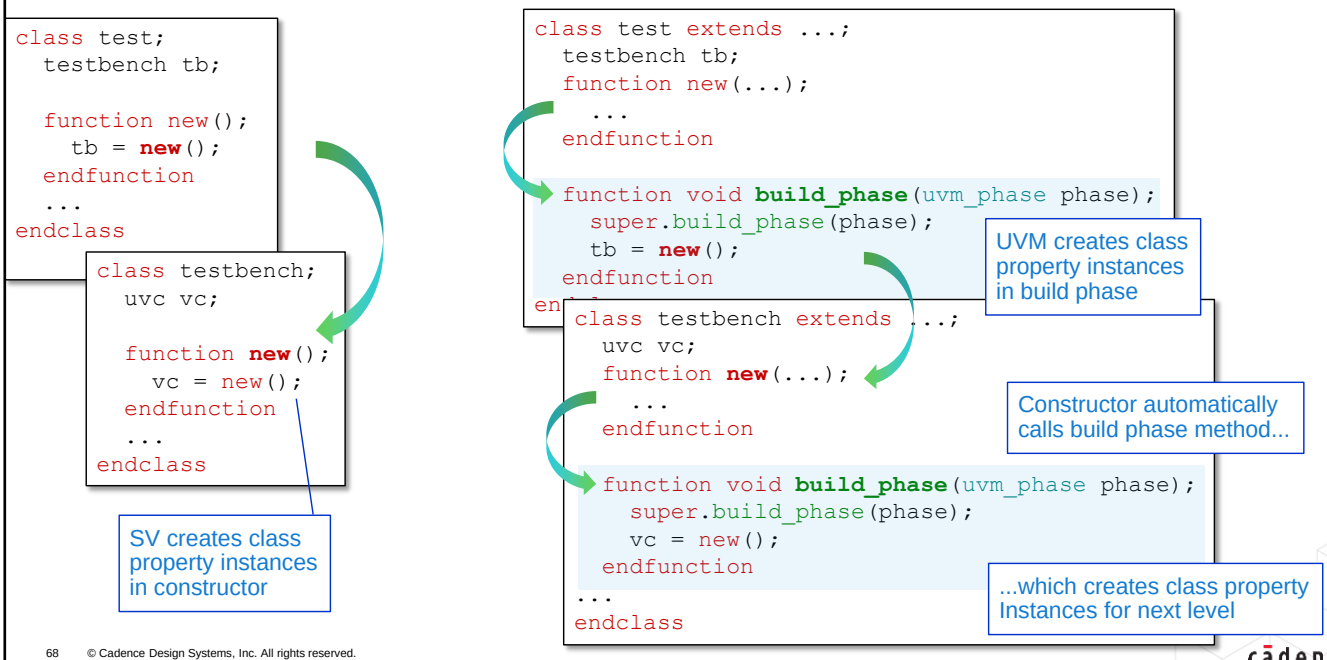
- See `support.cadence.com/TrainingBytes/UVM`

Refer also to the *UVM Class Reference* manual in the library documentation for more details on phases.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Build Phases for Aggregate Classes



68 © Cadence Design Systems, Inc. All rights reserved.

The component hierarchy for a UVM verification environment is created using properties which are instances of other classes (i.e., using aggregate or composite classes).

In SystemVerilog, we create class property instances in the constructor. So the constructor of the `test` class contains a call to the constructor of the `testbench` class to create the `tb` component instance. Likewise the `testbench` constructor creates the next level of hierarchy by calling the `uvc` constructor to create the `vc` instance.

However in UVM we use the build phase to create the component instances. So the build phase of `test` contains a call to the `testbench` constructor to create the `tb` instance. In UVM, executing the testbench constructor automatically calls the testbench build phase (if it is defined) to create the next level of hierarchy – the `vc` instance of class `uvc`.

The build phase is a design pattern – a standard solution to a common object-oriented design problem. We'll see why we use build phases in the configuration module later.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Are Phases Activated?

Execute the global task `run_test()` from a module scope to start phasing.

Execution:

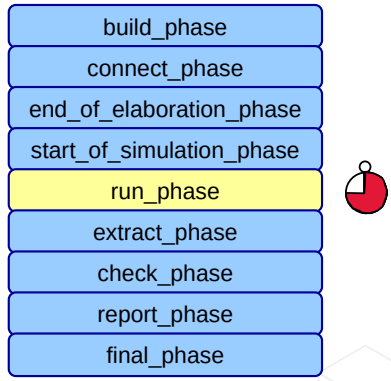
- Build phase functions execute top-down
- Run phase tasks execute in parallel
- All other phases execute bottom up
- All components finish the previous phase before the next phase starts

Timing:

- Run phase and all the run-time phases are tasks and can consume time
- All other phases are defined with functions and execute in zero time

To use any phase, define the phase method, and it will be executed during that phase.

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  ...
  initial begin
    run_test();
  ...
```



69 © Cadence Design Systems, Inc. All rights reserved.

cadence

Phases are activated by executing the calling the UVM task `run_test()`. This task is not a class method and has global visibility, so it can be called from the top level module.

`run_test` executes the build phase functions from the top level class instance downwards. When there are no more build phase methods to execute, phasing moves on to the connect phase. Connect phase functions are executed bottom up – i.e. simulation the lowest leaf instances for connect phase methods first, then navigates up the hierarchy, executing connect phase functions as they are found.

End of elaboration and start of simulation are also executed bottom up.

The run phase can consume simulation time and so is implemented with `run_phase` tasks rather than functions. Run phase tasks execute in parallel.

When the run phase is complete, the extract, check, report and final phases are executed. All these phases execute bottom up, in zero time using functions.

To use any phase, you simply declare a phase method and add the required functionality. The phase method is automatically executed by the simulator during that phase.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation Phases Quiz

1. How do you activate the phasing mechanism?
2. Which phases(s) are tasks?
3. Which phase must contain a super call?

Solutions at the end of the module

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Simulation Phases Quiz

1. How do you activate the phasing mechanism?
   - Execute the global task `run_test()` from a top-level module

2. Which phases(s) are tasks?
   - `run_phase()` and all twelve run-time sub-phases (`pre_reset_phase()` to `post_shutdown_phase()`)

3. Which phase must contain a super call?
   - `build_phase()` must contain a call to `super.build_phase()`

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 6

Test and Testbench Classes

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Test and Testbench Classes Objectives

In this module, you

- Create and use testbench classes
- Define the requirement and implementation of UVM test classes
- Select specific test classes to execute
- Debug hierarchies using `print_topology()`
- Use UVM reports for debug and error messaging

© Cadence Design Systems, Inc. All rights reserved.

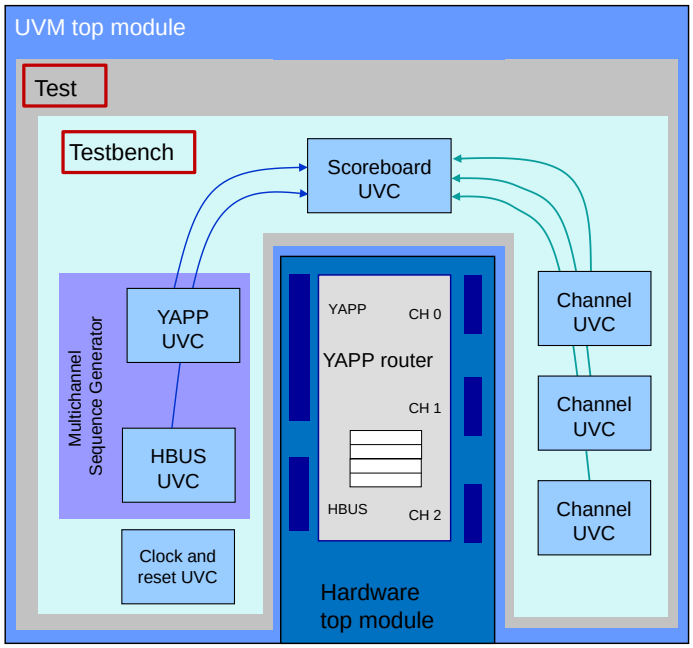
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building the Router Verification Environment



In this module, we add the test and testbench class layers.

Tests are the top level of the UVM environment:

- All UVM hierarchy is under a test class

Test classes:

- Control the stimulus generated by UVCs
- Instantiate the testbench class

The testbench:

- Instantiates and configures UVCs

74 © Cadence Design Systems, Inc. All rights reserved.

The topmost component in a UVM environment is the test class. Its key roles are:

- Instantiation of the testbench component
- Configuration of UVC properties and policies
- Selection of stimulus for interface UVCs
- Controlling generation of data stimulus using type overrides

The testbench:

- Instantiates UVCs (both interface and module)
- Instantiates other components such as multichannel sequencers, register models etc.
- Configures the UVCs

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

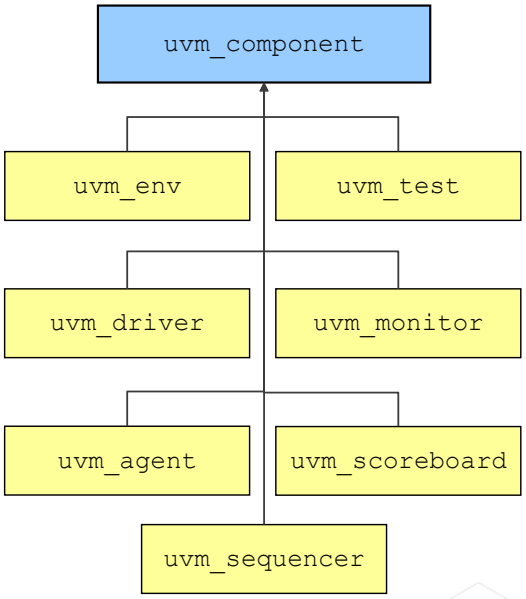
Components Versus Objects

Data items:

- Define stimulus
- Extend from `uvm_sequence_item`
- Have **object** utility macros and constructors

Components:

- Form the verification environment using an aggregate class hierarchy
- Are derived from `uvm_component`
- UVM provides sub-classes for your test, testbench, UVC, agent, sequencer, etc.
  - You derive your components from these subclasses and add your implementation.
- Have **component** utility macros and constructors



75 © Cadence Design Systems, Inc. All rights reserved.

cadence

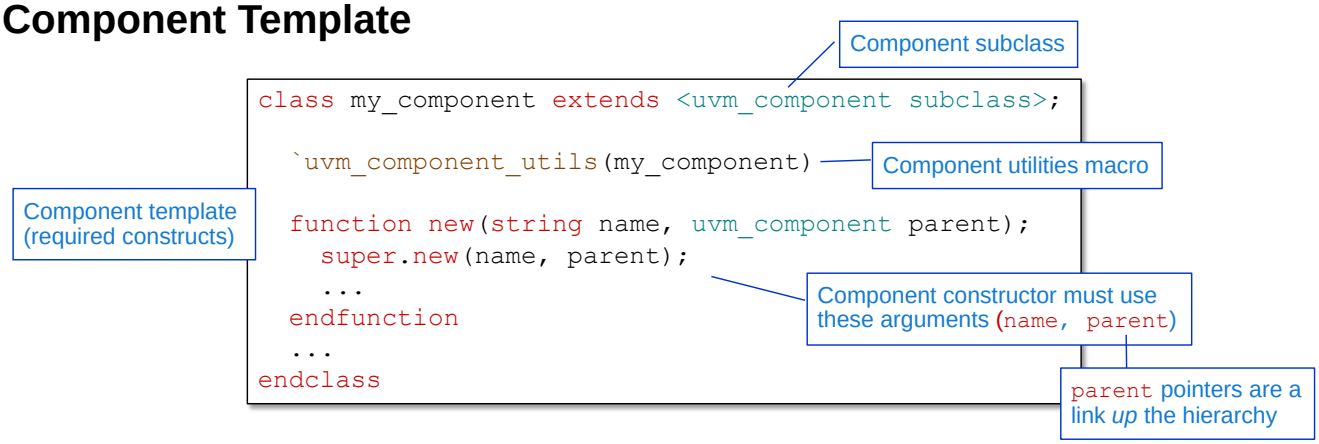
We have already seen how UVM provides a parent class for data items (`uvm_sequence_item`) as a means of accessing the UVM functionality for data.

UVM also defines parent classes for components. `uvm_component` is the root class for all component classes and we could extend all our user-defined components directly from this class. However, UVM defines specific subclasses of `uvm_component` for every UVC subcomponent, for example `uvm_sequencer`, `uvm_driver`, `uvm_monitor`, `uvm_agent`. These component-specific subclasses define essential functionality for the component implementations and should be used in preference to `uvm_component`.

© Cadence Design Systems, Inc. All rights reserved.

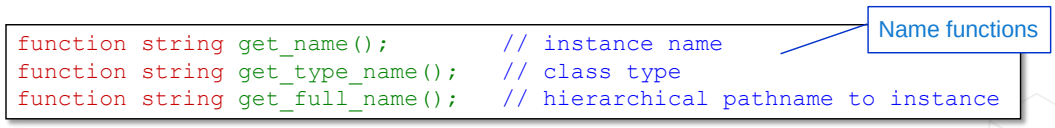
This document is for the sole use of Kun Wang of Seas

Component Template



Every component must have a component utility macro and component constructor.

- Many compilation errors are due to a missing or incorrect constructor.
- The constructor arguments allow access to useful name functions.



76 © Cadence Design Systems, Inc. All rights reserved.
cadence

Just like a data class, component classes need a UVM utility macro and a specific constructor. However component classes have a different macros and constructors than data classes.

Every component needs a *component utility macro* to enable automation for units. The utility macro comes in two forms:

- `` `uvm_component_utils_begin(<type name>) ``
- `// field macros`
- `` `uvm_component_utils_end ``

or

- `` `uvm_component_utils(<type name>) ``

If your component class declares local data properties then we use the begin/end utility which allows us to apply field automation to the data properties.

If your component does not declare local data properties, we can use the second macro form.

Every component class requires a *component constructor* with an instance name argument of type string and a parent pointer argument of type `uvm_component`. The constructor must call `super.new(name,parent)` as the first line to pass the arguments up into the UVM library.

The constructor arguments allow several useful methods, for example `get_full_name()` which returns the hierarchical pathname for the component as a string.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Is a Testbench?

A testbench encapsulates all verification components for a specific DUT

Instantiates and configures:

- Interface UVCs:
  - YAPP, Channel, HBUS
- Module UVCs:
  - Scoreboard
  - Reference model
- Other components:
  - Multichannel (virtual) sequencer
  - Register model

Could have multiple testbenches for one DUT

- For example, verifying multiple flavors of parameterized IP
- We'll cover multiple testbenches later...

77 © Cadence Design Systems, Inc. All rights reserved.

cadence

The testbench must carry out the following:

- Declare handles for all UVCs.
- Configure the UVCs in the build_phase method.

Configuration of the UVCs is vitally important. Complex UVCs may require many configuration statements for correct operation.

Typical UVC configuration tasks include:

- Setting the `default_sequence` property of a UVC sequencer.
- Enabling transaction recording by globally setting `recording_detail` properties.
- Setting UVCs to be passive (if required).
- Defining the number and type of agents in a UVC, e.g., transmit or receive.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Testbench Shell

router_tb.sv

```
class router_tb extends uvm_env;

  `uvm_component_utils(router_tb)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
   // configure and create uvcs
    ...
  endfunction
...
```

No testbench subclass (could use `uvm_component`)

Component utilities macro

Component constructor

We'll add these later...

- We will add UVC instances to the testbench later

78 © Cadence Design Systems, Inc. All rights reserved.

cadence

At the moment, the testbench is just a shell without any UVCs or other components. We will add these later to the build phase of the testbench.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Test Requirements

Test classes control UVC stimulus

There are usually multiple test classes:

- Constrained random tests tend to be much shorter
- Many test classes are declared in the same file
  - Test library
- Allow easier test reuse
  - Use inheritance to avoid describing hierarchy and settings for every test

We want to:

- Reduce knowledge of the environment for writing tests
  - Provide a standard test-writer interface
  - Minimal knowledge and effort in creating or enhancing tests
  - Ability to write hierarchy-independent tests
- Reduce compilation time when launching a specific test
  - Compile multiple tests at the same time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

There are usually many test classes for each DUT, applying different stimulus or configurations to achieve different verification goals. In UVM, it only takes a few lines of code to set stimulus or configuration, therefore each test class is quite small. For this reason, and contrary to what we have seen so far, we declare many test component classes in a single file. The file is called a test library.

A test component must meet the following requirements:

Compilation efficiency. There are typically many different test components in one or more test library files. We don't want to recompile the entire testbench and DUT when switching from one test to another. Therefore we need to be able to compile all tests at the same time, and then easily switch between different tests using the same simulation kernel.

Reuse. The instantiation of the next level of the component hierarchy can be declared in a user-defined "base" test class, which is then extended by test subclasses. The subclasses can define individual stimulus and configuration without the need to declare the component hierarchy.

The test class provides a test writer interface for defining verification of the DUT. This allows test writers to define stimulus and configuration to achieve specific verification goals, with minimal knowledge of the verification hierarchy or design protocols, and with minimal effort.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simple Test Example

router_test_lib.sv

```
// test classes in one test library file

class base_test extends uvm_test;

  `uvm_component_utils(base_test)

  function new (string name, uvm_component parent);
    super.new(name, parent);
  endfunction: new

  router_tb  tb;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    tb = new("tb", this);
  endfunction : build_phase

endclass: base_test

class mytest extends base_test;
  ...
```

Extend uvm_test

Component utility macro

Component constructor

Handle on testbench

Create tb in build_phase

Multiple test classes in one test library file

Additional test classes extend from base to inherit tb instance

80 © Cadence Design Systems, Inc. All rights reserved.

The test class contains:

- A component utilities macro
- A component constructor
- An instance of the testbench component created in a `build_phase()` method

As test classes tend to small in size, several test classes may be placed into one test library file.

The test library file is not reusable code, therefore it is placed in a `tb` (testbench) directory, not the `sv` directory.

This is a simple example just to instantiate the testbench component. We will refine our test as we progress through the course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selecting a Test: Option 1 – `run_test`

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  import yapp_pkg::*;

  `include "router_tb.sv"
  `include "router_test_lib.sv"

   ...
   initial begin
     run_test("mytest");
   end

endmodule
```

Import YAPP UVC

Include testbench

Include test library

Test class is explicitly named
This test is *always* run during simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can select which test class to execute by passing the test type name as a string argument to the `run_test` method.

Notice there is no test class handle declared, or test class instance created. The `run_test` task, when executed from an `initial` block in your topmost SystemVerilog module, implicitly creates a declaration and instance of the test class type which is passed to it.

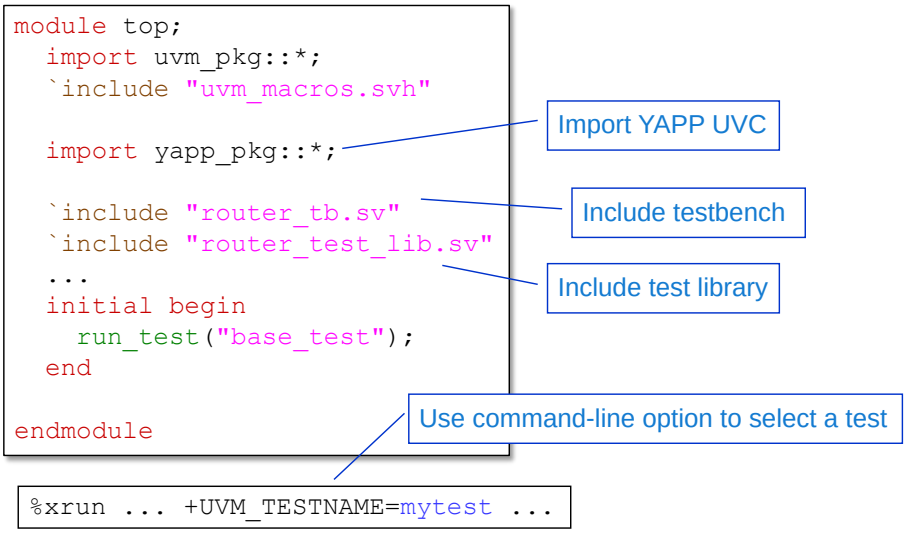
If the test class cannot be found or is undefined, a fatal runtime error is issued.

This option hardwires the selection of test class into the module, but it can be overridden by the second option (next slide). This option is good for defining a default test class.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selecting a Test: Option 2 – UVM_TESTNAME



- `+UVM_TESTNAME` takes precedence over the `run_test()` argument

82 © Cadence Design Systems, Inc. All rights reserved.

Passing the test class name as an option to the command line is a much more flexible method to select a test. This allows us to create multiple test classes, compile them all, and select a specific test from the command line each time the simulation is run.

Changing the test class does not require recompilation, as long as the chosen test class has already been compiled.

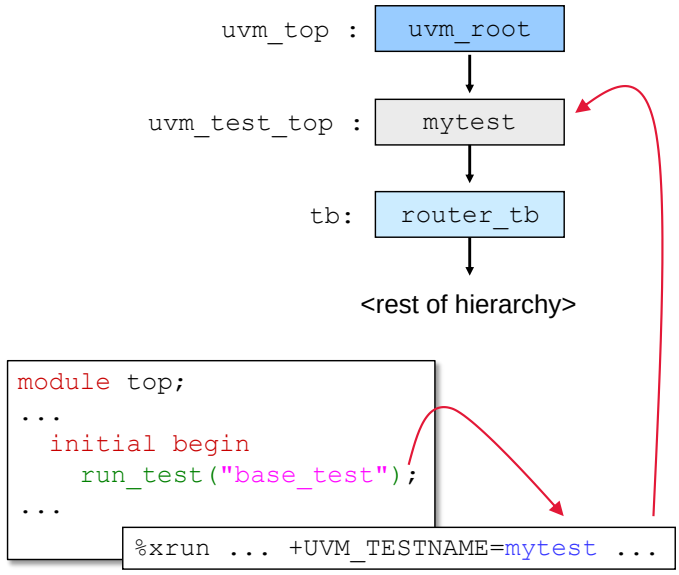
`+UVM_TESTNAME` takes precedence over an argument to `run_test()`.

Multiple `+UVM_TESTNAME` arguments are allowed, but the first compiled argument takes precedence.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Hierarchy



The topmost component is an instance of `uvm_root` named `uvm_top`.

- Top level for phasing, searching, configuration, debugging, etc.
- Gives access to useful methods:
  - `run_test()`
  - `print_topology()`

What does `run_test` do?

- Creates an instance of the defined test in the handle `uvm_test_top`.
  - With a parent of `null`
- Initiates simulation phases which build the rest of the test hierarchy.

83 © Cadence Design Systems, Inc. All rights reserved.

The topmost component is an instance of `uvm_root` named `uvm_top`, which is created automatically. `uvm_top` is a singleton design pattern – i.e., only one instance of `uvm_root` can be created. The instance is globally visible throughout the UVM environment, and so is useful for any search, configuration and debug calls which need to be applied from the top of the hierarchy.

The `uvm_top` handle allows access to various useful methods, for example `run_test` is a globally visible method of `uvm_root`.

The `run_test` call:

1. Declares a handle of the selected test component named `uvm_test_top`.
2. Creates the test instance with a parent of `null`.
3. Initiates the phasing.

The phasing will execute the `build_phase` method of the test component, which then creates the rest of the verification hierarchy.

The `uvm_top` handle also allows access to `print_topology` which prints the entire component hierarchy. This is very useful for debugging hierarchy issues. It should be invoked once all the hierarchy has been built and connected, e.g., in the `end_of_elaboration` phase.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Topology Report

```
class base_test extends uvm_test;
  ...
  function void end_of_elaboration_phase(uvm_phase phase);
    uvm_top.print_topology();
  endfunction
  ...
endclass: base_test
```

Print testbench hierarchy

irun.log

```
...
UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:
------------------------------------------------------------------------
Name                        Type                        Size  Value
------------------------------------------------------------------------
uvm_test_top                base_test                   -     @4680
  tb                        router_tb                   -     @2652
    ...
    ...
------------------------------------------------------------------------
    ...
```

Hierarchy

UVCs will appear under testbench

Test executed

Handle references

84 © Cadence Design Systems, Inc. All rights reserved.

A topology report provides a huge amount of useful information about your verification environment.

In Cadence's IUS simulator, the simulation output, including the topology report, is saved to the file `irun.log`.

The topology report can tell you:

- What test is running (type of `uvm_test_top`). Is this correct?
- The hierarchy of your test environment. Is every UVC present and is every UVC structure as expected?
- Starting value of key properties (e.g., `is_active`). Is every property printed? If not, you may have forgotten to automate the property to enable printing.

`uvm_top.print_topology()` is a useful debug construct for an UVM testbench. It allows the designer to check the hierarchy structure and property values of the testbench, and is key in understanding which test is running; which agents are active and which sequences are running. Always include this construct somewhere in your testbench.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reports for Debug and Error Messages

Debugging activity inside a large environment with many UVCs is critical.

Need to report:

- Errors
- Debug
- Progress

Reports need to be categorized via severity:

- Fatal, Error, Warning, Info

Need to link actions with reports

- Exit simulation on fatal
- Stop simulation after a set number of errors

Need a different mechanism than simulator messages to avoid filtering effects.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Once we have one of more UVCs inside a large verification environment, we need to able to report what is happening during simulation.

We want to display:

- Errors, both in simulation results and in creating the verification environment.
- Debug messages to help understand when things go wrong.
- Progress reports to show the simulation is doing what it should.

Some messages (e.g., errors) will be more important than others (e.g., progress reports). Therefore we need to be able to categorize messages by severity:

- Fatal – the simulation has failed and cannot continue.
- Error – we have a problem, but simulation can continue.
- Warning – there is a possible problem or anticipated issue.
- Info – for progress reports.

Severity allows us to link specific actions with messages, for example to stop the simulation on a fatal message or after a set number of error messages. A summary of the messages of each severity at the end of the simulation will enable us to understand the simulation results at a glance.

We need a standalone messaging mechanism for UVM to allow separate control of simulation and simulator reports and to ensure portability.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

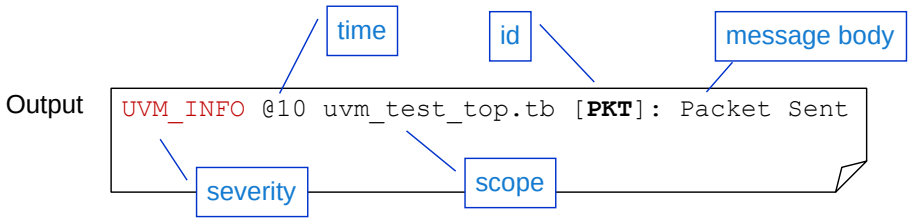
Report Macros: Info

Syntax:

```
`uvm_info(string id, string message, int verbosity)
```

- Other macros for severity `` `uvm_fatal ``, `` `uvm_error ``, `` `uvm_warning ``

```
`uvm_info("PKT", "Packet Sent", UVM_LOW)
```



Tips:

- *Could* use `get_type_name()` to give an `id` of the class type
- Use `$sformatf()` to create message string using `$display` syntax

86 © Cadence Design Systems, Inc. All rights reserved.

The macro is a call to a built-in report method. The macros are more efficient than calling the method directly.

Full Method Syntax

```
function void uvm_report_* (string id, string message,
    int verbosity_level=0, string filename="", int line=0);
```

Where

- `*` is one of `fatal`, `error`, `warning` or `info`.
- `id` is an identifier for the message. You can configure report output and actions using the ID. Use a `get_type_name()` call to set id argument to the current class type name.
- `message` is the output text. It must be a string, so `$display` cannot be used directly. The `$sformatf()` system function creates a string using `$display` syntax.

The following arguments are optional:

- `verbosity` is an integer which allows further filtering of messages.
- `filename` and `line` can be used to specify the location of the message by name of file and line number of message

The report macros add the `filename` and `line` arguments automatically.

The fatal, error and warning report macros also set the `verbosity` argument to ensure these messages are always displayed.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Verbosity

```
`uvm_info(get_type_name(), $sformatf("Packet:\n%s", pkt.sprint()), UVM_LOW)
```

Like print but only creates string

Verbosity is a predefined int value that can be used to filter reports.

- UVM_NONE, UVM_LOW, UVM_MEDIUM (default), UVM_HIGH, UVM_FULL, UVM_DEBUG

By changing the verbosity maximum, reports can be filtered.

- Reports with a verbosity ***greater than*** the maximum are **not** processed

Several ways to change verbosity, for example:

- Simulator arguments
  - Global +UVM_VERBOSITY=verbosity
  - Hierarchical +uvm_set_verbosity=*comp*,*id*,*verbosity*,*phase*
- TCL (runtime)
  - uvm_message -hierarchy *<hier>* -verbosity *verbosity*

87 © Cadence Design Systems, Inc. All rights reserved.

TB: UVM Reports: Getting the Message Out

cadence

`uvm_info("PKT",$sformatf("Packet:n%s",pkt.sprint()),UVM_LOW)

Raises a report of severity level `info`, of verbosity `UVM_LOW`, with id `PKT` and a message string which prints the contents of `pkt` (assuming `pkt` is a handle of a UVM data item). `sprint()` is a sub-method of `print()` – it creates the print string but does not write it to the output.

Reports with a verbosity *greater* than the maximum are ignored. For example, if the verbosity is set to `UVM_LOW`, all reports coded with `UVM_MEDIUM` and higher will *not* print.

There are several ways to change verbosity.

On the command line, sets the verbosity of all components (default `UVM_MEDIUM`)

```
+UVM_VERBOSITY=verbosity
```

Or on the command line, set the verbosity for a specific hierarchy, specific id and specific phase

```
+uvm_set_verbosity=<comp>,<id>,<verbosity>,<phase>
```

TCL (run-time):

```
uvm_message -hierarchy <hier> -verbosity verbosity
```

For more details on verbosity control and reports in general, please search for the Training Byte series "UVM Reports : Getting the Message Out" on `support.cadence.com`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Full Range of Report Macros

```
`uvm_info(string id, string message, int verbosity)
`uvm_warning(string id, string message)
`uvm_error(string id, string message)
`uvm_fatal(string id, string message)
```

- `` `uvm_info() `` conditionally checks the verbosity level before performing any string manipulation to increase performance
- The verbosity of warning, error, and fatal are set to `UVM_NONE` so these reports cannot be hidden by the verbosity setting
- Fatal reports end simulation by default
- A count of occurrences of each report type is printed at the end of simulation
- Report behavior can be customized using a report catcher


TB: Modify Any UVM Report Using A Report Catcher

 © Cadence Design Systems, Inc. All rights reserved.

There are four UVM report macros, one for each severity. The macros call UVM report functions. However using the macro call is more efficient, as this checks the verbosity level before creating the string message. If the verbosity setting prevents the report from displaying, the message is not created.

Report macros for warning, fatal and error severities do not have verbosity arguments. The verbosity of these reports is automatically set to `UVM_NONE` so that they will always display, regardless of the verbosity threshold.

By default, only a Fatal severity will stop the simulation, although this behavior can be changed (see next slide).

A t the end of simulation, a report summary is printed reporting a count of the reports displayed by severity and also by ID.

By default, the output of a UVM report is standardized. The format and order and amount of information can be customized by adding a report catcher to the report generator. Report catchers can also modify the severity of a message if, for example, different UVCs are not using severity consistently.

Report catchers are described in a Training Byte entitled "Modify Any UVM Report Using A Report Catcher".

- See `support.cadence.com/TrainingBytes/UVM`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Report Actions

Default actions are based on severity.

- Actions can be changed for severity, id, or hierarchy

Severity	Default Action
uvm_info	UVM_DISPLAY
uvm_warning	UVM_DISPLAY
uvm_error	UVM_DISPLAY + UVM_COUNT
uvm_fatal	UVM_DISPLAY+ UVM_EXIT

All Actions	
UVM_NO_ACTION	Do nothing
UVM_DISPLAY	Send report to standard output
UVM_LOG	Write report to file
UVM_COUNT	Increment quit counter*
UVM_EXIT	Terminate simulation
UVM_CALL_HOOK	Execute hook methods
UVM_STOP	Execute $stop to enter simulation interactive mode

*A maximum number of UVM_COUNT actions can be set, after which the simulation finishes:

- Default max_quit_count is 0, indicating no maximum

```
xrun ... +UVM_MAX_QUIT_COUNT=3
```
Command line

TB: UVM Reports: Getting the Message Out

© Cadence Design Systems, Inc. All rights reserved.

UVM defines default actions for reports by severity. For example, `uvm_fatal displays the report and exits the simulator.

UVM provides commands to hierarchically change default actions by severity or report id. So we can override the default actions for a report in a specific hierarchical path, with a specific ID and of a specific severity.

The UVM_COUNT action increments a counter each time a report with the action is raised. A maximum value for the count can be defined, after which the simulation exits. Default value is 0 indicating no maximum value. Easiest and most flexible way to set the maximum is with the command line option +UVM_MAX_QUIT_COUNT, allowing the maximum to be changed for every simulation run.

Method calls can also set the quit count. In UVM1.1, a set_report_max_quit_count method can be called at any time, but in UVM1.2, multiple, customized report servers are allowed so the uvm_report_server method set_max_quit_count must be used.

The UVM_LOG action writes the report to a file. By default, no file is specified, even if UVM_LOG action is defined, although a report with the UVM_DISPLAY action will still be written to the simulator transcript window and hence to a simulation log file.

For more details on report actions and reports in general, please search for the Training Byte series "UVM Reports : Getting the Message Out" on support.cadence.com.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Test and Testbench Classes Quiz

1. What is the minimum constructor code for a component item?

2. What are the two ways of selecting a test?

3. Write an `info` report macro, with verbosity `UVM_MEDIUM`, which prints the data handle `packet`. The message `id` should be `PKT`.

Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 2 Creating Test and Testbench Components

- Create a testbench component
- Use a test class to instantiate the testbench
- Launch different tests

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Tests and Testbenches Quiz

1. What is the minimum constructor code for a component item?

```
function new (string name, uvm_component parent);
  super.new(name, parent);
endfunction
```

2. What are the two ways of selecting a test?
   a) Via the argument of the `run_test` task
   b) Using the `+UVM_TESTNAME` simulator option

3. Write an `info` report macro, with verbosity `UVM_MEDIUM`, which prints the data handle `packet`. The message id should be `PKT`.

```
`uvm_info("PKT",
          $sformatf("Packet is:\n%s", packet.sprint()),
          UVM_MEDIUM)
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 7

Creating an Interface UVC

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating an Interface UVC Objectives

In this module, you

- Create the basics of a simple interface UVC
- Define the recommended directory structure for UVC files

 © Cadence Design Systems, Inc. All rights reserved.

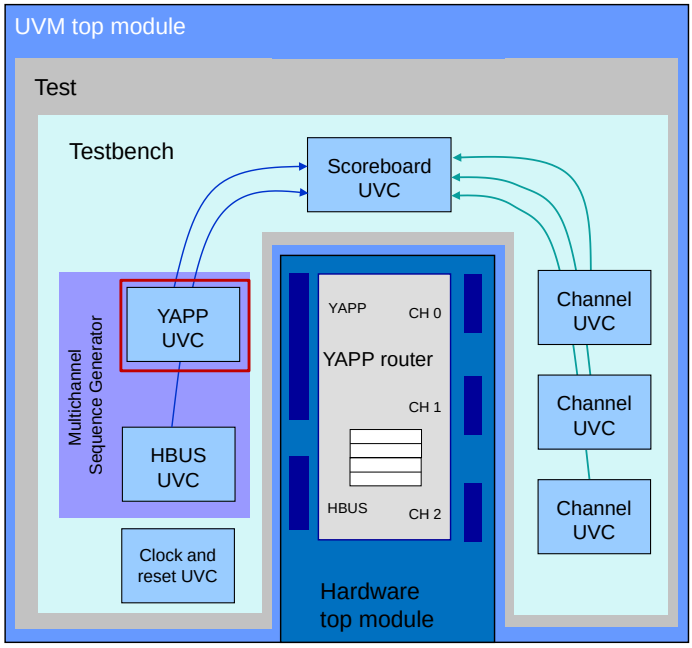
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building the Router Verification Environment



The YAPP UVC will drive YAPP packets into the DUT.

In this module, we start to create the YAPP UVC:

- Front end only
- Back-end connection to DUT is later

95 © Cadence Design Systems, Inc. All rights reserved.

The test environment for the YAPP router will be built up over many modules and labs in this course.

For the next few modules and the next lab, we will concentrate on developing the basics of the YAPP UVC and the UVM top module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface UVC Hierarchy Review

UVC

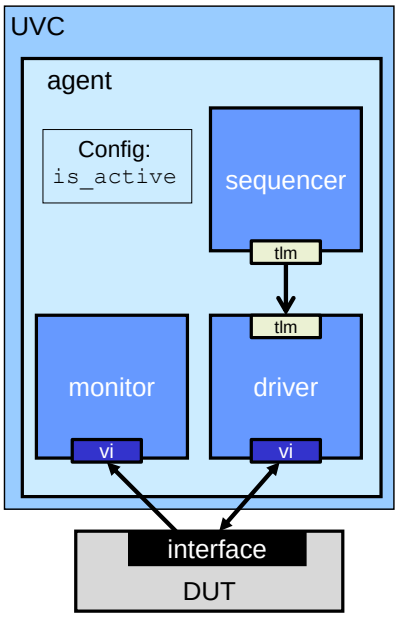
- Top level
- Contains one or more agent instances

Agent

- Contains instances of the sequencer, driver and monitor
- Configurable – `is_active` flag indicates whether the agent is active or passive
- If agent is active – driver, sequencer and monitor are constructed
- If agent is passive – only the monitor is constructed

Monitor

- Collects information from the DUT
- Connects to the DUT via a virtual interface



Sequencer

- Controls generation of the stimulus
- Points to list of data items – sequence
- Connects to the driver via a TLM interface
- Generates next sequence item on driver request

Driver

- Connects to the sequencer via a TLM interface
- Gets the next data item from the sequencer
- Drives data on the DUT interface with the correct timing
- Indicates to the sequencer that the item is done
- Connects to the DUT via a SystemVerilog virtual interface

© Cadence Design Systems, Inc. All rights reserved.

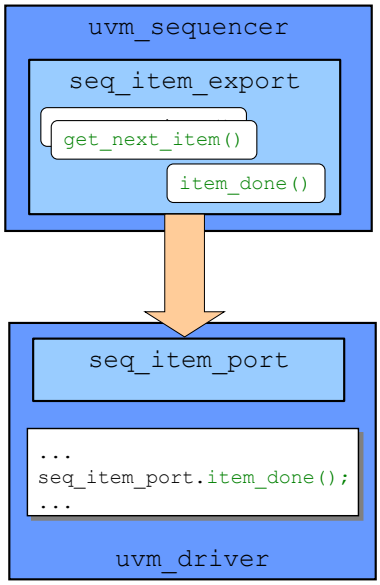
cadence

This is a reminder of the structure and functionality of an interface UVC.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequencer-Driver TLM Connection



uvm_sequencer contains a built-in TLM connection – **seq_item_export**.

- This defines communication methods:
  - get_next_item(), item_done() etc.

uvm_driver contains a built-in TLM connection – **seq_item_port**

The driver **seq_item_port** is connected to the sequencer **seq_item_export**.

The driver can then execute the sequencer communication methods.

- This allows the driver to "pull-down" data from the sequencer.

By extending from **uvm_sequencer** and **uvm_driver**, we can use this connection.

97 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM provides a built-in connection for sequencer and driver, defined as part of the `uvm_sequencer` and `uvm_driver` classes.

`uvm_driver` has a port connection object `seq_item_port`. `uvm_sequencer` has an export connection object `seq_item_export`. These two objects must be explicitly connected together by the agent of the UVC.

The sequencer connection object defines a number of communication methods to handle the synchronization and transfer of a data item (e.g., `yapp_packet`) between driver and sequencer.

The user simply has to call predefined communication methods off the driver connection object.

`seq_item_*` is a specialized TLM connection. TLM allows communication between components and comes in many different forms.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequencer-Driver Operation

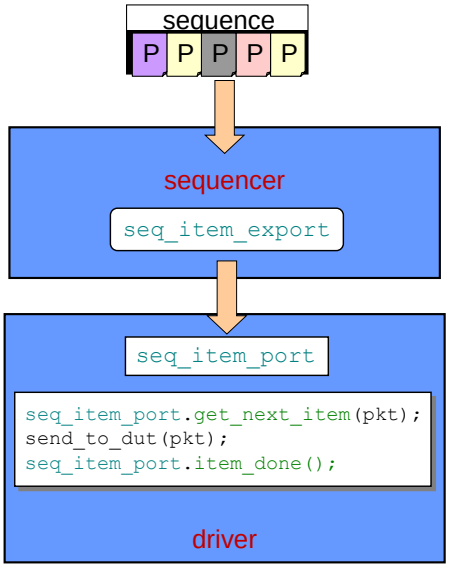
Sequencer is linked to a sequence

- A pattern of individual data items
  - More details later in class

Sequencer-driver operation

- Driver calls `get_next_item()`
- Sequencer generates next data item from sequence and sends to driver
  - Returned as output argument to `get_next_item()`
- Driver sends data item to DUT by driving interface signals
- Driver indicates the item is finished by calling `item_done()` from port

Operation continues until all sequence items sent



98 © Cadence Design Systems, Inc. All rights reserved.
cadence

A sequencer is linked to a sequence which defines stimulus in terms of a pattern of individual data items (like `yapp_packet`). The sequencer will feed these items, one at a time, to the driver on demand.

The driver calls the communication methods of the sequencer from its `seq_item_port` to pull-down the individual data items and send them to the DUT.

The most common procedure is as follows:

- Driver requests an item by calling `get_next_item()` from the `seq_item_port`. The simulator executes the method in the sequencer to which the driver is connected. The sequencer finds the next data item from the sequence; creates and generates the item (usually through randomization) and return a pointer to the item as the return value of the `get_next_item()` method.
- The driver can then process the item and send it to the DUT (usually through another method).
- The driver then acknowledges the item has been sent by calling `item_done()` from the `seq_item_port`.
- This procedure can continue (in a loop) until all items from the sequence have been sent.

There are other operation modes of the driver/sequencer but the "pull-down" procedure described above is by far the most common.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Driver

```
class yapp_driver extends uvm_driver #(yapp_packet);

  // yapp_packet req;

  `uvm_component_utils(yapp_driver)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item( req );
      send_to_dut( req );
      seq_item_port.item_done();
    end
  endtask

  virtual task send_to_dut(yapp_packet packet);
    ... // protocol-specific syntax to drive DUT

endclass : yapp_driver
```

Extend `uvm_driver` with type parameter

Parameterization defines built-in `yapp_packet` handle `req`

Component utilities macro

Component constructor

Run phase method

`get_next_item` returns `yapp_packet` instance

`send_to_dut` implements protocol

99 © Cadence Design Systems, Inc. All rights reserved.

Every component must have a component utility macro (`uvm_component_utils`) and a component constructor with two arguments - `name` and `parent`.

Using a data type parameter on the `uvm_driver` class gives several advantages:

- A built-in handle of the data type, named `req`, is automatically created.
- The `get_next_item()` method call off the `seq_item_port` returns an instance of the data item directly.
- Better compile-time error checking.

Only the active path of a UVC can have type parameters, i.e., only driver and sequencer components, and sequence data items.

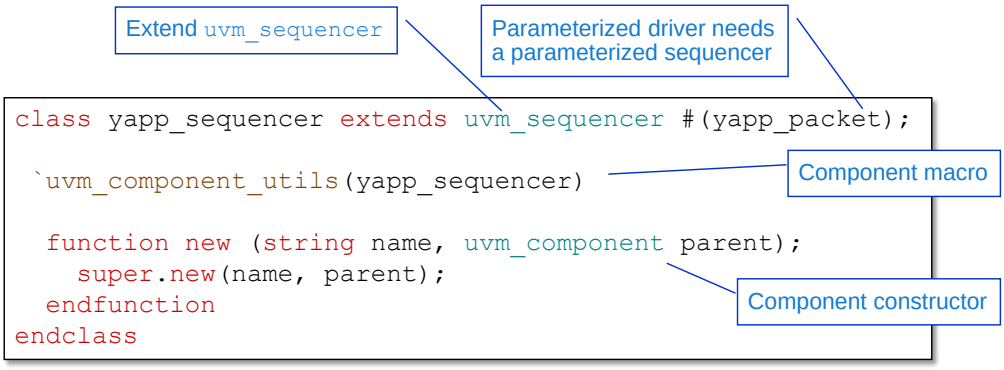
Getting and processing data items is carried out in a `run_phase()` task:

- The driver calls the `get_next_item()` task off `seq_item_port` to retrieve the next data item. The call execution blocks indefinitely until an item is provided.
- The driver then send the data item to the RTL design using a `send_to_dut()` task (details later in the course).
- Finally the driver calls the `item_done()` function to indicate to the sequencer that the driver is done processing the item. The call can optionally take a return data item as an argument.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequencer



By default, a sequencer does not generate a stimulus

- A sequence data item defines transactions for a sequencer
- Many sequences can be defined for a given sequencer
- You must execute a sequence on the sequencer to apply its stimulus

100 © Cadence Design Systems, Inc. All rights reserved.

Using the TLM sequencer/driver connection and sequences to define stimulus makes our sequencer declaration very small and simple.

Note the communication methods (`get_next_item`, `item_done`) are defined in the `uvm_sequencer` subclass.

A sequencer does not create any transactions by default. A separate sequence data item defines the transaction traffic for a sequencer. There are typically many different sequences defined for a given sequencer, reflecting all the different transactions a UVC can create.

A simple configuration setting can be used to tell a sequencer which sequence to execute in `run_phase()` or a specific run sub-phase. Alternatively a sequence can be executed on a sequencer from within a test class.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating the Agent: Declarations

```
class yapp_agent extends uvm_agent;

  //uvm_active_passive_enum is_active = UVM_ACTIVE;

  yapp_driver     driver;
  yapp_sequencer  sequencer;
  yapp_monitor    monitor;

  `uvm_component_utils_begin(yapp_agent)
   `uvm_field_enum(uvm_active_passive_enum, is_active, UVM_ALL_ON)
  `uvm_component_utils_end

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  // build_phase():
  //   create instances

  // connect_phase():
  //   make connections
endclass
```

Extend `uvm_agent` - **no** type parameter

`uvm_agent` defines built-in `is_active` switch

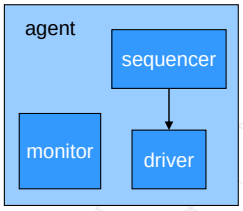
Handles for subcomponents

Component macro

Automate built-in field for debug

Component constructor

See next slide...



101 © Cadence Design Systems, Inc. All rights reserved.

cadence

Agents and monitors cannot have type parameters.

By extended from `uvm_agent`, we inherit the `is_active` property to control the active/passive mode of the agent. `is_active` is of type `uvm_active_passive_enum` which is a built-in UVM enumerated type with values `UVM_ACTIVE` or `UVM_PASSIVE`. As `is_active` is not automated inside `uvm_agent`, we should add local automation within the begin/end form of our agent `uvm_component_utils` macro.

The `is_active` property is explicitly given a default value of `UVM_ACTIVE`. We will show you how to change the `is_active` value later in the Configurations module of the course.

The agent must contain the driver, sequencer and monitor components, for which we need handles. Instances for the handles will be created in the `build_phase()` method (not the constructor), and connected in the `connect_phase()` method. This is so hierarchical construction can be controlled by the UVM simulation phases.

Only the `is_active` property requires automation. As component instances, the `sequencer`, `driver` and `monitor` handles are automatically automated.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating the Agent: Build and Connect

```
class yapp_agent extends uvm_agent;
  //uvm_active_passive_enum is_active = UVM_ACTIVE;

  // HANDLES, MACRO and CONSTRUCTOR declarations as previous slide

 virtual function void build_phase (uvm_phase phase);
   super.build_phase(phase);
   monitor = new("monitor", this);
   if (is_active == UVM_ACTIVE) begin
     sequencer = new("sequencer", this);
     driver    = new("driver", this);
   end
 endfunction

 virtual function void connect_phase(uvm_phase phase);
   if (is_active == UVM_ACTIVE)
     driver.seq_item_port.connect(sequencer.seq_item_export);
 endfunction

endclass
```

Inherited is_active switch

super.build_phase is essential!

Agent sub-components created in build (sequencer & driver only if agent is active)

Connect sequencer and driver if agent is active

Connect method is called from the port and takes the export as an argument

102 © Cadence Design Systems, Inc. All rights reserved.

cadence

The sub-components of the agent must be created in a `build_phase()` function. The monitor component is always created, but the sequencer and driver care only created if the agent is active, i.e. the `is_active` property of the agent has the value of `UVM_ACTIVE`. The constructor calls for the components must have two arguments:

- A string matching the handle identifier for the `name` argument.
- The keyword `this` for the `parent` argument, indicating these components were created in this instance of the agent class.

In a `connect_phase()` function, we connect the driver `seq_item_port` to the sequencer `seq_item_export`, but only if the agent is active. The `connect` method is always called from the port object and takes the export as an argument.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating and Instantiating the UVC (env) Top Level

The UVC top level uses the `uvm_env` (environment or env) subclass.

You should have the knowledge to create this...

```
class yapp_env extends uvm_env;
  // agent handle
  // component utilities macro
  // component constructor:

  // build_phase():
     // instantiate agent
endclass
```

Extend `uvm_env`
no type parameter

... and instantiate it in the testbench

```
class router_tb extends uvm_env;
  ...
  // yapp_env handle
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // instantiate yapp uvc
  endfunction
...
```

Create YAPP UVC instance
in testbench build phase

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The top level of the UVC is defined using the `uvm_env` class. env is short for environment, and this term is frequently used for describing the top level of the UVC. However, the env and environment terms are also used for other layers in a verification design (e.g., the testbench) so you need to be careful with its use.

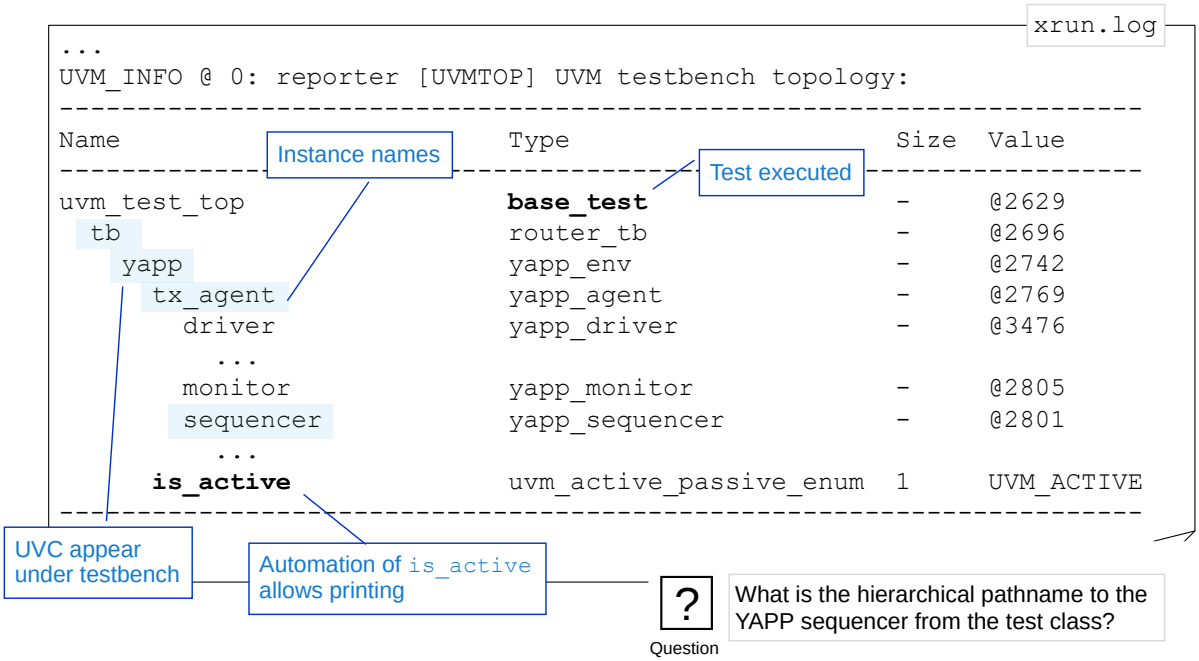
The UVC is a component, so don't forget to include a component utilities macro and component constructor.

The UVC must declare an agent handle, and construct an instance of the agent in the build phase.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Topology with YAPP UVC Instantiation



104 © Cadence Design Systems, Inc. All rights reserved.

A topology report provides a huge amount of useful information about your verification environment.

In Cadence's IUS simulator, the simulation output, including the topology report, is saved to the file `xrun.log`.

The topology report can tell you:

- What test is running (type of `uvm_test_top`). Is this correct?
- The hierarchy of your test environment. Is every UVC present and is every UVC structure as expected?
- Starting value of key properties (e.g., `is_active`). Is every property printed? If not, you may have forgotten to automate the property to enable printing.

`uvm_top.print_topology()` is a useful debug construct for an UVM testbench. It allows the designer to check the hierarchy structure and property values of the testbench, and is key in understanding which test is running; which agents are active and which sequences are running. Always include this construct somewhere in your testbench.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Stimulus for the UVC

```
class yapp_5_packets extends uvm_sequence#(yapp_packet);

 `uvm_object_utils(yapp_5_packets)

  function new(string name=" yapp_5_packets ");
    super.new(name);
  endfunction

  virtual task body();
    repeat (5)
      `uvm_do(req)
  endtask

endclass : yapp_5_packets
```

Type parameter

Data constructor and macro

Transactions defined in `body()`

Sequences, default sequences, and configurations will be covered in detail later

From test...

... the sequencer at this pathname...

... in its run phase...

```
// in test build phase
uvm_config_wrapper::set( this,
                         "tb.yapp.tx_agent.sequencer.run_phase",
                         "default_sequence",
                         yapp_5_packets::get_type());
```

... executes this sequence

105 © Cadence Design Systems, Inc. All rights reserved.

A sequence data item defines the transaction traffic for a specific sequencer.

A configuration setting (here using `uvm_config_wrapper`) tells a specific sequencer to execute a given sequence in the `run_phase` or a run sub-phase.

Each sequencer has a `default_sequence` pointer for the `run_phase` and for every run sub-phase. By default the pointer is null. By setting the pointer using a configuration `set` call, the sequencer can be set to execute the required sequence in the required phase.

This slide is here to explain how a sequencer generates stimulus data. This information may be useful to help understand the pre-supplied sequence we will use to initially test your YAPP UVC. We will explore sequences and configurations is much more detail later in the class.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Reference: Type Parameterization

```
class yapp_driver extends uvm_driver #(yapp_packet);
  ...
class yapp_sequencer extends uvm_sequencer #(yapp_packet);
  ...
class yapp_5_packets extends uvm_sequence #(yapp_packet);
  ...
endclass : yapp_5_packets
```

```
class yapp_monitor extends uvm_monitor;
  ...
class yapp_agent extends uvm_agent;
  ...
class yapp_env extends uvm_env;
  ...
endclass : yapp_env
```

Only the active path of a UVC has built-in type parameters

- Driver, sequencer, sequence

Other UVC classes do not

- Monitor, agent, uvc

Type parameters allow:

- Better compile-time checking
- Built-in `req` handle

106 © Cadence Design Systems, Inc. All rights reserved.

Only the active path of a UVC can have type parameters, i.e., only driver and sequencer components, and sequence data items. If you have a type parameter on any one of these classes, then each one must be type parameterized, i.e. a typed sequencer can only be connected to a typed driver and can only execute typed sequences.

You do not have to use type parameters, although the benefits are considerable, and so this course does not show non-parameterized UVCs.

For example, a data type parameter on the `uvm_driver` class gives several advantages:

- A built-in handle of the data type, named `req`, is automatically created.
- The `get_next_item()` method call off the `seq_item_port` returns an instance of the data item directly.
- Better compile-time error checking.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Directory Structure

Each UVC should have its own directory structure:

- To create modularity
- To enable multiple coders
- To facilitate reuse

Each UVC will have two kinds of code:

- Reusable code
  - Core functionality of the component
  - Resides in an `sv` subdirectory
- Non-reusable code
  - Shows how to use the component and how to make a complete example
    - Support code, scripts, tests, etc.
  - Reside in any number of subdirectories
    - `tb`, `examples`, etc.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

As you begin to build verification components, it is important to structure and package UVC code to make it easier to reuse. One important step is to give each UVC its own directory structure.

Each component will have two kinds of code:

- Reusable code (the core functionality of the component)
- Non-reusable code (support code, scripts, tests, etc.)

The core, reusable code will reside in the `sv` subdirectory. The rest of the code will reside in any number of subdirectories (`tb`, `dut`, etc.).

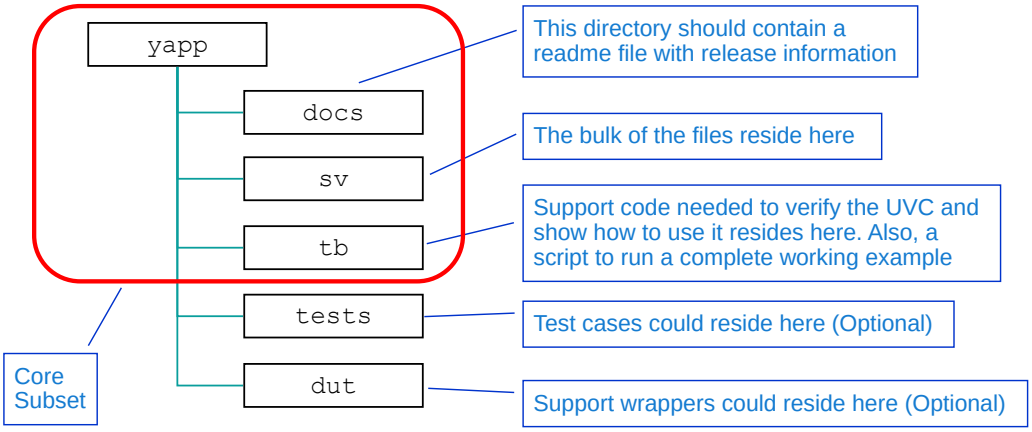
The purpose of the non-reusable code is to demonstrate how to use the component and to make a complete example including a run script.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVC Directory Structure

The UVC name is used for the “root” directory of the UVC files.



108 © Cadence Design Systems, Inc. All rights reserved.

When building a UVC, you define a UVC name and create a directory using this name. Our example uses `yapp` for the UVC name.

Non-reusable files should be located in a `<uvc>/testbench` or `<uvc>/tb` directory. Simulations are also run in this directory or another work/regression area.

Reusable files are located in the `<uvc>/sv` directory.

There is some freedom to create your own subdirectories, and some directories are optional, but the key is to have a UVC name directory, and the subdirectories: `sv`, `tb`, and `docs`.

© Cadence Design Systems, Inc. All rights reserved.

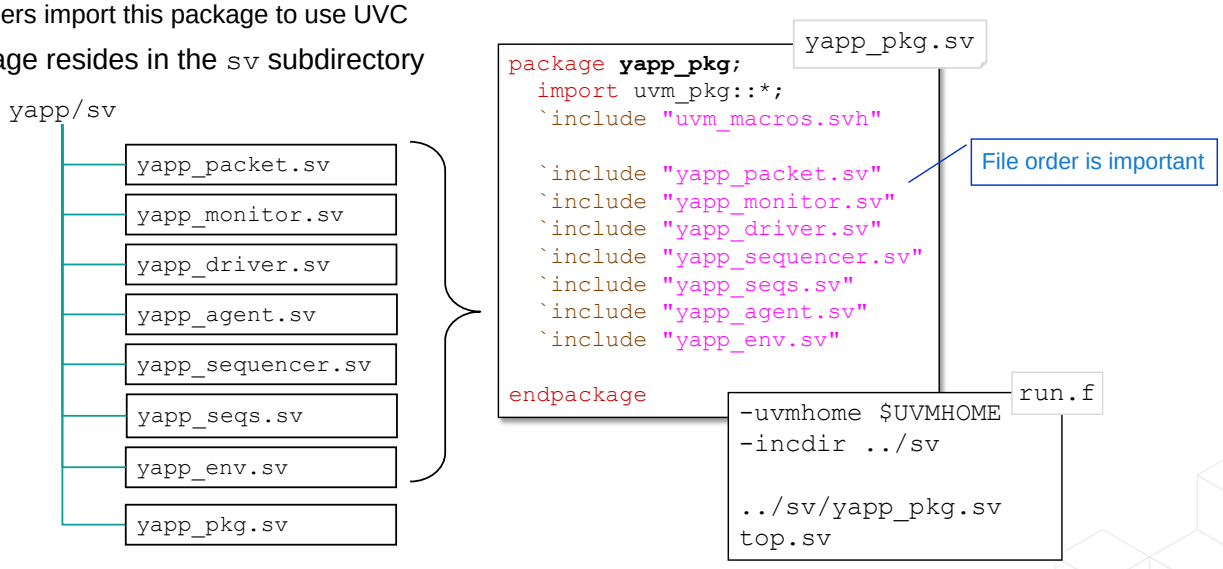
This document is for the sole use of Kun Wang of Seas

UVC Package and Include Files

Each UVC has a single package that includes all the UVC base files.

- Users import this package to use UVC

Package resides in the `sv` subdirectory



109 © Cadence Design Systems, Inc. All rights reserved.

All the sv files required by the UVC are included into a SystemVerilog package. As packages are compiled individually, the package must also contain an import of the UVM package and an include for the UVM macros header file.

The order of files includes is important and must be listed from bottom-up. This is due to declaration dependencies in the UVC hierarchy. You must include the data item first. You must include the monitor, driver and sequencer before the agent can be included. You must include the agent before the UVC env can be included.

Any other code not directly related to the UVC should be declared in the tb directory and be accessed separately, not as part of the UVC package.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Creating an Interface UVC Quiz

1. Which UVC components can have type parameters?

2. What is the name and default value of the property which controls whether an agent creates a sequencer and driver instance?

3. What is the difference between the `sv` and `tb` directories in a UVC directory structure?

Solutions at the end of the module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 3 Creating a Simple UVC

- Create a driver, simple sequencer and simple monitor
- Create an agent to build the driver, sequencer and monitor
- Encapsulate all the components into a UVC top level
- Use build and connect methods to create and join hierarchy
- Instantiate the UVC in the testbench
- Generate packets and view in the log file

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Creating an Interface UVC Quiz

1. Which UVC components can have type parameters?

   Driver and sequencer components only have type parameters

2. What is the name and default value of the property which controls whether an agent creates a sequencer and driver instance?

   The `is_active` property with a default value of `UVM_ACTIVE`

3. What is the difference between the `sv` and `tb` directories in a UVC directory structure?

   The `sv` directory is for reusable UVC code which is instantiated by the user. The `tb` directory is for script-driven test code to verify the UVC and provide examples of instantiation and configuration.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 8

Configuration

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Configuration Objectives

In this module, you

- Use the UVM configuration mechanism to:
  - Control topology during the build, for example:
    - Set UVC agent active or passive
    - Set the number of UVC master and slave agents
  - Assign instance-specific properties for UVCs, for example:
    - Specifying illegal address ranges for stimulus

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Configuring Topology

Topology can be configured according to property values

- For example, **`is_active`** property of the agent determines whether the sequencer and driver are created
- **`is_active`** is called a *config property*

Need to control config properties from a higher level

- Typically, in a high-level testbench or test

Properties must be set before components are created

- For example, set agent passive before it is constructed

We do *not* want to pass property values from the top via arguments to **`new()`** or **`build_phase()`**



Question

What would be the problem in passing property values via method arguments?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The testbench hierarchy can be configured by component properties. For example, in a UVC, every agent should have an `is_active` property which determines whether the agent is active or passive. When the agent is built the sequencer and driver components are created only if `is_active` has the value `UVM_ACTIVE`. Properties such as `is_active` are known as config, or configuration properties.

UVC configuration is usually controlled by higher-level components such as a testbench or test. A testbench will configure a UVC when it is instantiated. A test may reconfigure the hierarchy for a specific test.

In both cases, the configuration must be set up before the UVC components are created. If we set `is_active` to `UVM_PASSIVE` after the agent was built, it would be too late – the sequencer and driver would already be created.

We could pass down config property values via arguments to the constructor or build phase methods. However if we have five UVCs, each with three config properties, then that is a large number of arguments to manage. Also if we swapped one UVC for another with different properties, the constructor or build phase method arguments would need to be rewritten throughout the hierarchy.

UVM uses a different mechanism to set config properties.

This is a standard problem in Object Oriented Design, and configuration used with build phases is a standard solution (called a design pattern).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Review – is_active Agent Config Property

```
class yapp_agent extends uvm_agent;
  //uvm_active_passive_enum is_active = UVM_ACTIVE;

  `uvm_component_utils_begin(yapp_agent)
    `uvm_field_enum(uvm_active_passive_enum, is_active, UVM_ALL_ON)
  `uvm_component_utils_end

  yapp_sequencer  sequencer;   // sequencer handle
  yapp_driver     driver;      // driver handle
...
 virtual function void build_phase(uvm_phase phase);
   super.build_phase(phase);
   if (is_active == UVM_ACTIVE) begin
     sequencer = new("sequencer", this);
     driver    = new("driver", this);
   end
   ...
  virtual function void connect_phase(uvm_phase phase);
    if (is_active == UVM_ACTIVE)
      driver.seq_item_port.connect(sequencer.seq_item_export);
...
```

Inherited config property with default value

Automation (essential)

Call `super.build_phase` **before** checking property

Build phase conditionally creates sequencer and driver

Conditionally connect sequencer and driver

? How do we set `is_active`?

Question

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This is the same agent code as we showed previously in the module on Creating a Simple UVC, except the monitor code is not shown.

The config property `is_active` is used to conditionally build the sequencer and driver instances, and to conditionally connect driver and sequencer. The default value of the property is set to `UVM_ACTIVE` in the declaration, so how can we change the value of `is_active` to set the agent to passive? We shall see how to do this in the following slides.

There are two important points to note when using config properties:

1. The property is automated, i.e., it has a field automation macro.
2. In the build phase method, `super.build_phase()` is called first before we test the value of `is_active`.

The following slides will explain the significance of these points in more detail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting Config Properties

UVM provides a configuration mechanism to set property values from higher levels:

```
uvm_config_db#(<type>)::set(<context>, <instance>, <field>, <value>);
```

Where:

- `value` is the desired property value
- `field` is the config property name (string)
- `instance` is the hierarchy of instance names to the property component (string)
- `context` is the scope for resolving `instance` (usually `this`)
- `set` method is static
- `type` is the config property type

For simple config settings, all you have to do is call `set`

- The config is automatically resolved in the build phase

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Config property values are assigned using the `set` method of the configuration database.

Syntax for configuration database `set` method:

```
static function void set(uvm_component cntxt, string inst_name,
                         string field_name, T value )
```

where

- `cntxt` is a context (usually assigned to this) from where the relative pathname of `inst_name` is resolved.
- `inst_name` is a string concatenation of component instance names representing a relative pathname to a specific component instance.
- `field_name` is the string name of a config property in the specific component instance.
- `value` is a literal of a compatible type for the config property.

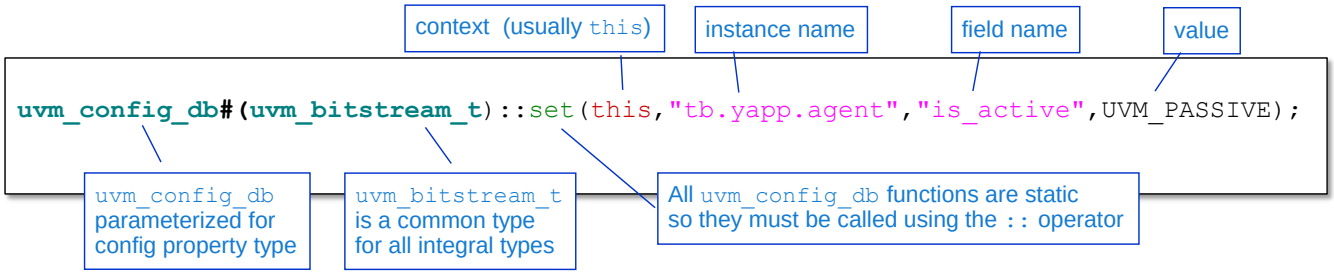
`set` is a static method called from `uvm_config_db` which must be type parameterized with a type name compatible with the type of the property `field_name` and value.

If constructed correctly, config settings are automatically resolved in the build phase.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Config Property Setting Example



- Database must have a type parameter
  - Each unique type parameter creates a separate database specialization
  - Allows compile time checking and debug
  - Allows configuration for user-defined types such as virtual interfaces
  - UVM has convenience types for common `uvm_config_db` parameterizations

    `uvm_config_int = uvm_config_db#(uvm_bitstream_t)`
- Property pathname is a concatenation of context, instance, and field

118 © Cadence Design Systems, Inc. All rights reserved.

Here is an example of setting a UVC agent to passive by assigning the value `UVM_PASSIVE` to the `is_active` property of the component at the instance pathname `yapp.agent`.

The configuration database must be parameterized for the type of the config property. This effectively creates a separate configuration database for that type. Type parameters allow config databases to be created for any standard or user-defined type and allows better compile time checking. However the type used in the database `set` must match the type of the property, the type of the value and the type of the operation to assign the config property.

Type matching can be a particular issue for integral types (e.g. `int`, `integer`, `byte`, `logic`, `bit`) as separate config databases are created for each named type. Therefore UVM defines a common integral type, `uvm_bitstream_t`, for all integral configuration accesses.

```
typedef logic signed [4095:0] uvm_bitstream_t;
```

There are several typedefs declared for common `uvm_config_db` parameterizations:

```
typedef uvm_config_db#(uvm_bitstream_t) uvm_config_int;
typedef uvm_config_db#(string) uvm_config_string;
typedef uvm_config_db#(uvm_object) uvm_config_object;
```

The property pathname is constructed by concatenating the `context`, `instance` and `field` arguments into an absolute hierarchical pathname.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Configuration Works

```
uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
```

Context	Instance	Field	Value
this	"tb.yapp.agent"	"is_active"	UVM_PASSIVE

Set method registers the desired setting in the configuration database

- Method must be called *before* the property component is built
  - This is why instance and field names are strings

Setting applied by `apply_config_settings` method in the build phase

- Called automatically by `super.build_phase()`
  - Must be called *before* checking the config property value
- The config database is searched for a matching pathname and field name
- Settings are applied *only* when match is found
  - Errors in instance and field names will leave properties un-set!

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `set` method writes the desired configuration setting into a configuration database. You must call the method before the component containing the affected configuration property is built, so that the setting can be propagated when the component is created. If the `set` method is called after the component is created, the setting will not take effect. The requirement to make the setting before the component is built explains why both the instance and field name arguments of set config methods are strings. Neither the instance pathname nor the property field name exist when the methods must be called.

Settings are applied by the method `apply_config_settings()`. This is called automatically by a call to `super.build_phase()` in the build phase method of any component. `apply_config_settings` checks the path name of the current component against every entry in the configuration database. If a pathname match is found, it looks for a field name match, and if one is found, the property is assigned the value from the database.

The user can also (and sometimes has to) explicitly get configurations using a method call.

Obviously the `super.build_phase()` call must be made before a config property value is used.

If no match is found in the instance name or field name, then the configuration setting is not applied. There is no warning or error if a setting does not match, therefore errors in either the instance or field name will leave config properties unset and these can be difficult to debug.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting YAPP Agent to Passive from Test

```
class base_test extends uvm_test;
  router_tb tb;
  ...
  function void build_phase(uvm_phase phase);
    uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
    super.build_phase(phase);
    tb = new("tb", this);
    ...
  endfunction
endclass
```

Property pathname is a string of instance names relative to current scope

```
class router_tb extends uvm_env;
  yapp_env yapp;
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    yapp = new("yapp", this);
    ...
```

```
class yapp_env extends uvm_env;
  yapp_agent agent;
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent = new("agent", this);
  ...
endclass
```

```
class yapp_agent extends uvm_agent;
  // inherited config property
  //uvm_active_passive_enum is_active = UVM_ACTIVE;
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (is_active == UVM_ACTIVE) begin
      ...
```

Match found, is_active set here

 © Cadence Design Systems, Inc. All rights reserved.

cadence

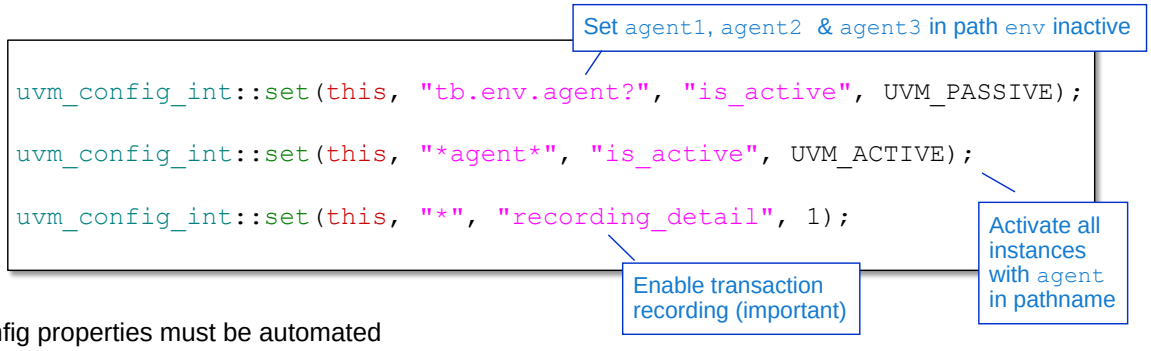
We call `set` from the test class with a context argument of `this`. Therefore the instance argument is a relative pathname from the test class. The instance argument is a string concatenation of instance names representing a hierarchical pathname to the property component. Remember instance names are assigned in the first argument of a component constructor. The instance name does not need to match the handle name, but UVM is easier if both handle and instance names are identical.

When we call `super.build_phase()` in the `yapp_agent`, this calls `apply_config_settings()` which checks the config database for a pathname and field name match. If we have both correct, then `is_active` is set to `UVM_PASSIVE` by the super call. The super call must be made before we check the `is_active` value.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Config Setting Rules



- Config properties must be automated
- Instance and field arguments must match hierarchy
- Instance *only* can include * or ? wildcard characters, or regular expressions
  - Allow one setting for multiple properties in same or different components
- Config settings in higher scope take precedence over lower scopes
  - For example, test class set overrides a testbench set
- Config settings in the same scope conform to “last one in wins”

121 © Cadence Design Systems, Inc. All rights reserved.

If you do not have a field automation macro for your config property, then configuration fails with no warning or error. Remember if a property is automated, it should be printed in the topology report. Always check your config properties appear in the report.

Wildcard or regular expressions can be used in the instance argument only. Note that wildcard symbol * represents any number of characters, and ? represents exactly one character. This simplifies the instance argument and allows a single config set to affect multiple properties.

Config settings conform to "last one in wins" for multiple settings in the same scope, and "highest scope wins" for multiple settings in different scopes.

The type parameter used in the set and the get (explicit or implicit) must match and must be compatible with the property type.

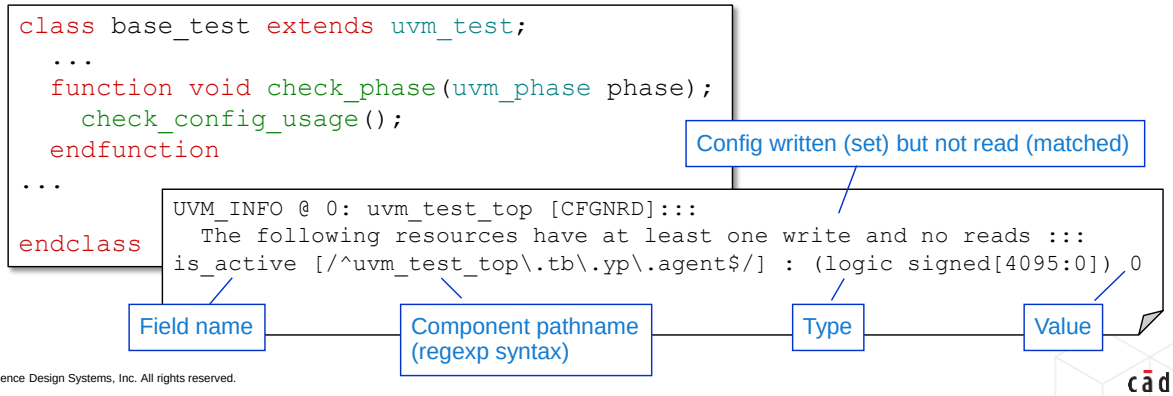
As well as user-defined config properties, there are a few pre-defined UVM properties which can be set in a test class. The most important one is recording_detail. By default, this is set to 0 and no transactions are recorded. By enabling the recording detail for every component using a wildcard, transactions can be viewed in the waveform window.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Debugging Configuration Settings

- Configuration can fail
  - Typically mismatches in instance or field arguments
- By default, unmatched configurations are not reported
- Unmatched settings can be printed at end of simulation
  - Prints arguments of config settings which are unmatched
  - Compare pathname in report (used in `set`) with pathname from topology report



122 © Cadence Design Systems, Inc. All rights reserved.

The most common config setting problems are:

- Forgetting to automate properties to which config settings are applied.
- Mismatches in instance or field strings due to typos or missing path references.
  - Remember the wildcard symbol `*` represents *one* or more characters, and `?` represents exactly 1 character.
- Type mismatches between method calls and properties.

In many cases config settings will not be applied and no errors or warnings are generated.

Configurations can be checked by calling `check_config_usage();`

```
function void check_config_usage (bit recurse = 1);
```

This reports settings in the configuration table which have been written (set) but not read (matched).

However, `check_config_usage()` does NOT generate mismatch errors for un-automated config properties, even though these still fail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selected Configuration Database Methods

Method	Description	Syntax
`set`	Writes an entry setting `field_name` in the path `cntxt.inst_name` to `value`.	`static function void set (uvm_component cntxt, string inst_name, string field_name, T value);`
`get`	Reads the entry for `field_name` in the path `cntxt.inst_name`. Assigns `value` and returns 1 if the entry exists, 0 otherwise.	`static function bit get (uvm_component cntxt, string inst_name, string field_name, inout T value);`
`exists`	Returns 1 if a config setting exists for `field_name` in the path `cntxt.inst_name`, 0 otherwise. Set `spell_chk` to 1 to enable spell checking if the field should exist.	`static function bit exists (uvm_component cntxt, string inst_name, string field_name, bit spell_chk = 0);`
`wait_modified`	Blocks until a new configuration setting is applied for `field_name` in the path `cntxt.inst_name`.	`static task wait_modified (uvm_component cntxt, string inst_name, string field_name);`

An explicit manual `get` may be required when:

- `set` is called from the top-level module or outside the build phase
- Property type does not have an automation macro

See Connecting to a DUT module

123 © Cadence Design Systems, Inc. All rights reserved.

cadence

`set` – sets value of configuration property (`field_name`) at pathname given by pre-pending `cntxt` argument to `inst_name`.

`get` – the build phase automatically calls `apply_config_settings()` to retrieve config settings. However if `set` is executed from the top level module, or `set` is called outside of the build phase, then a `get` must be explicitly called to retrieve a config setting. `get` returns 0 if no entry is found for the given field in an instance under the given context.

`exists` – checks if a config entry for a field in an instance under a given context exists. Returns 1 if it does, 0 otherwise.

`wait_modified` – blocks execution until the field in an instance under the given context is updated by a `set` call. Useful for synchronizing run-time config settings. Only indicates that the field configuration has changed – the field value must be updated via a separate `get`.

Argument descriptions are as follows:

- `cntxt` – component context from which the set is being done. Usually mapped to `this`. `get_full_name`() of the context is implicitly added to the instance name.
- `inst_name` – pathname to the target component instance from the context. This name may contain wildcards or regular expression syntax .
- `field_name` – target field in the target instance. From UVM1.2 the field cannot contain wildcards.
- `value` – configuration value for the field name at this instance.

There are also `file` and `line` arguments which are debug options.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

set_config Methods

There is a simpler alternative for setting integral or string config properties.

- Based on `set_config_*` method

```
virtual function void set_config_* (string inst_name,
                                    string field,
                                    bitstream_t value);
```

  - Where `*` is `int`, `string` or `object`, depending on type of config property
- Has same arguments as uvm_config_db `set` except for context
  - Context is set to the scope where `set_config` is called
- However, `set_config` calls are deprecated in UVM1.2
  - Will be removed from a future release
  - Should be avoided in new UVM code
  - Consider converting any use in legacy environments to database calls

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Syntax for `set_config_int` method:

```
virtual function void set_config_int (string inst_name,
                                      string field,
                                      bitstream_t value)
```

The arguments are identical to the uvm_config_db call, except that the simple methods do not have a separate context argument. The context is automatically set to the scope where the method call is made.

`uvm_config_db` has better debugging and compile-time checking than the simple `set_config` method call. However it is more complex to use, and the compile time checking comes at the expense of strong type sensitivity, which can cause problems in use.

For simple string and integral settings, `set_config` is easy and straightforward to use.

However in UVM1.2, the set_config method calls are marked as deprecated, which means they will be removed from a future version of UVM. Therefore although you may see these calls in legacy code, you should avoid using them in new designs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Configuration Quiz

Given this topology, what is wrong with the following *test* class config settings?

```
uvm_test_top                set_config_test          -     @2756
  tb                        router_tb                -     @2780
    yapp                    yapp_env                 -     @2843
      agent                 yapp_tx_agent            -     @2893
        driver              yapp_tx_driver           -     @3716
        monitor             yapp_tx_monitor          -     @2942
        sequencer           yapp_tx_sequencer        -     @2991
        is_active           uvm_active_passive_enum  1     UVM_ACTIVE
```

```
uvm_config_int::set(this, "tb.yapp_env.agent", "is_active",  UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "isactive", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "is_*", UVM_PASSIVE);

uvm_config_int::set(this, ".tb.yapp.agent", "is_active", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.?", "is_active", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
```

Solutions at the end of the module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Configuration Quiz

Given this topology, what is wrong with the following test class config settings?

```
uvm_test_top                set_config_test           -     @2756
  tb                        router_tb                 -     @2780
    yapp                    yapp_env                  -     @2843
      agent                 yapp_tx_agent             -     @2893
        driver              yapp_tx_driver            -     @3716
        monitor             yapp_tx_monitor           -     @2942
        sequencer           yapp_tx_sequencer         -     @2991
        is_active           uvm_active_passive_enum   1     UVM_ACTIVE
```

```
uvm_config_int::set(this, "tb.yapp_env.agent", "is_active",  UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "isactive", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "is_*", UVM_PASSIVE);

uvm_config_int::set(this, ".tb.yapp.agent", "is_active", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.?", "is_active", UVM_PASSIVE);

uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
```

126 © Cadence Design Systems, Inc. All rights reserved.

cadence

1. Incorrect instance path – the class type name **`yapp_env`** is used instead of the instance name **`yapp`**.
2. Incorrect field name – field name should be **`is_active`**.
3. Incorrect field name – wildcards cannot be used in field names, only in instance paths.
4. Incorrect instance path – the extra **`.`** at the start of the path breaks the instance match.
5. Incorrect instance path – the wildcard **`?`** at the end of the path matches a single character only, which will not match the agent instance name.
6. The final example is correct.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 9

Type Overrides and the Factory

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Type Overrides and the Factory Objective

In this module, you

- Change the default behavior of any data item or component using the UVM factory and override methods

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Adding Different Behavior to Existing Objects

Need to modify behavior for many scenarios:

- Supporting test requirements
  - For example, changing default data randomization:
    - Replace, remove or layer constraints
- Reuse of Verification IP
  - For example, changing procedural behavior:
    - Customizing driver protocol for specific tests
    - Modifying monitor checking algorithm for error recovery
- Module-to-system reuse
  - For example, adding more properties to a class
    - Extracting more information from a scoreboard

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

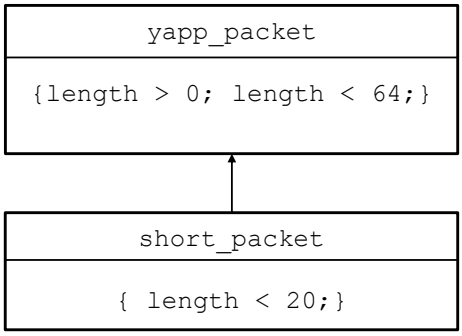
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Constraint Layering

- We use randomization to explore unanticipated areas
- In a specific test, you might want to constrain random data to hit a corner case
- Object-oriented programming allows for deriving a new subclass from the parent class and adding new constraints
- But how do we use these sub-classes?

Let's look at an example …



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Constraint Layering Problem

```
class yapp_packet extends uvm_sequence_item;
  rand bit [5:0] length;
  `uvm_object_utils_begin(yapp_packet)
  ...
  constraint length_const { length > 0; length < 64; }
  ...
endclass
```

```
class short_packet extends yapp_packet;
  `uvm_object_utils(short_packet)
     // field macros inherited from yapp_packet
   // constructor
  constraint short_length_const {length < 20;}
endclass
```

```
class yapp_driver extends uvm_driver #(yapp_packet);
  // yapp_packet req;
...
```

```
...
yapp_packet collect_packet = new("collect_packet");
...
```

In multiple locations...

...will need to change `yapp_packet` to `short_packet`

131 © Cadence Design Systems, Inc. All rights reserved.

cadence

Both the YAPP packet base class and short packet subclass must contain an object utilities macro. Both must also contain an object constructor.

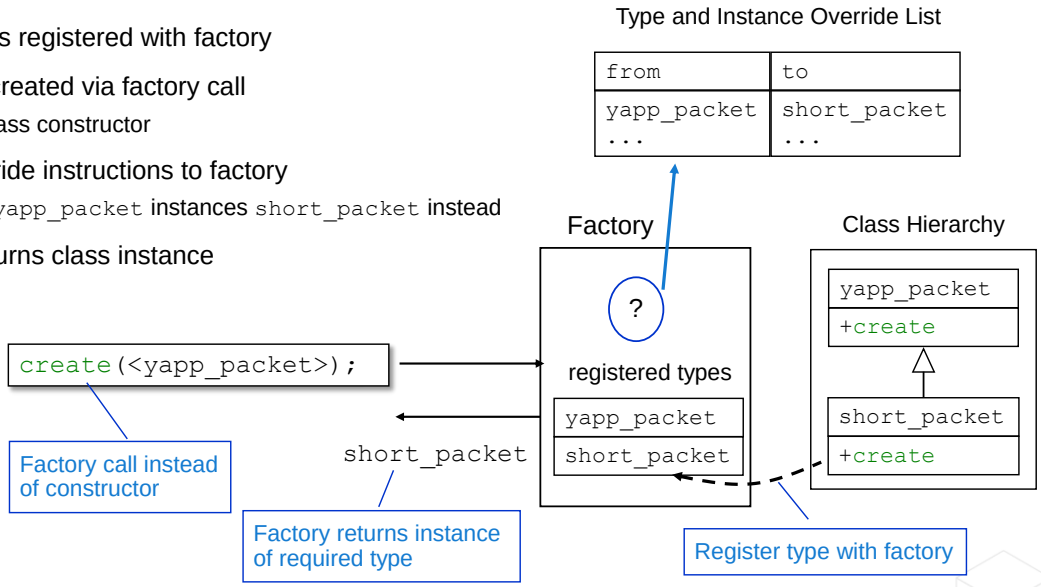
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solution: Factory

Central location to create class instances

- Each type is registered with factory
- Instances created via factory call
  - *Not* via class constructor
- Apply override instructions to factory
  - Make all `yapp_packet` instances `short_packet` instead
- Factory returns class instance



132 © Cadence Design Systems, Inc. All rights reserved.

Creating objects of the correct type in a reusable, flexible fashion is a standard problem in object-oriented design, with a standard solution called a factory. A factory is a class which creates object instances of the required type from a family or hierarchy of related classes.

Calls to a class constructor are replaced with calls to a factory creator method. The factory creates the required class instance and returns a handle to the instance. The handle is of the base type for that class hierarchy, allowing the factory to return any subclass of the base type.

As further subclasses of a type are developed, they can be easily added to the factory for creation. In some implementations, the creation methods are included in the subclass declarations, and the factory is a standard “interface” between the constructor code and the subclass creation methods.

Factories allow greater flexibility for obtaining an object instance of a particular type. For example, instead of simply calling a constructor to create a new object, the factory could select the object from an existing pool of resources; apply complex configuration on the object, etc.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Factory

Operation of the factory requires:

1. Registration of all UVM types with the factory
   - Utility macros register types with the factory
     - `` `uvm_object_utils(class_name) ``
     - `` `uvm_component_utils(class_name) ``
2. Using a factory `create` call for instances
   - Utility macros define `create()` as static method of class
   - Make sure all data and component class instances use `create()` rather than `new()`
     - Some exceptions (e.g., TLM connection objects – see later)
3. Global and instance-specific type override instructions for factory

 © Cadence Design Systems, Inc. All rights reserved.

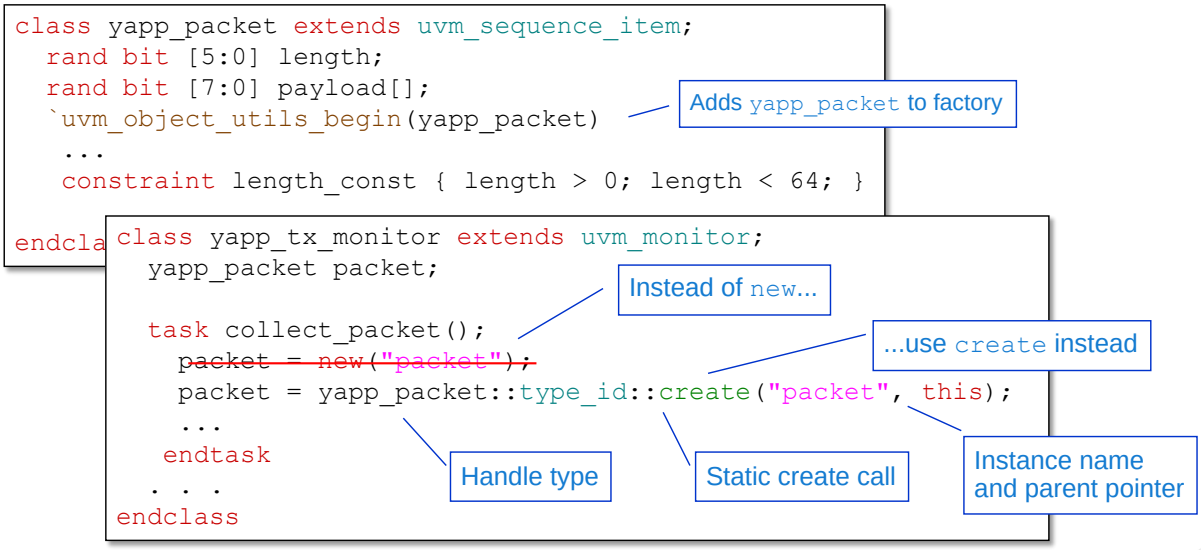
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Object Factory Use: create



Almost *all* instances should be made by calling factory `create` instead of constructor `new`

134 © Cadence Design Systems, Inc. All rights reserved.

Syntax for the create call is:

```
<handle> = <class name>::type_id::create("<name>", this);
```

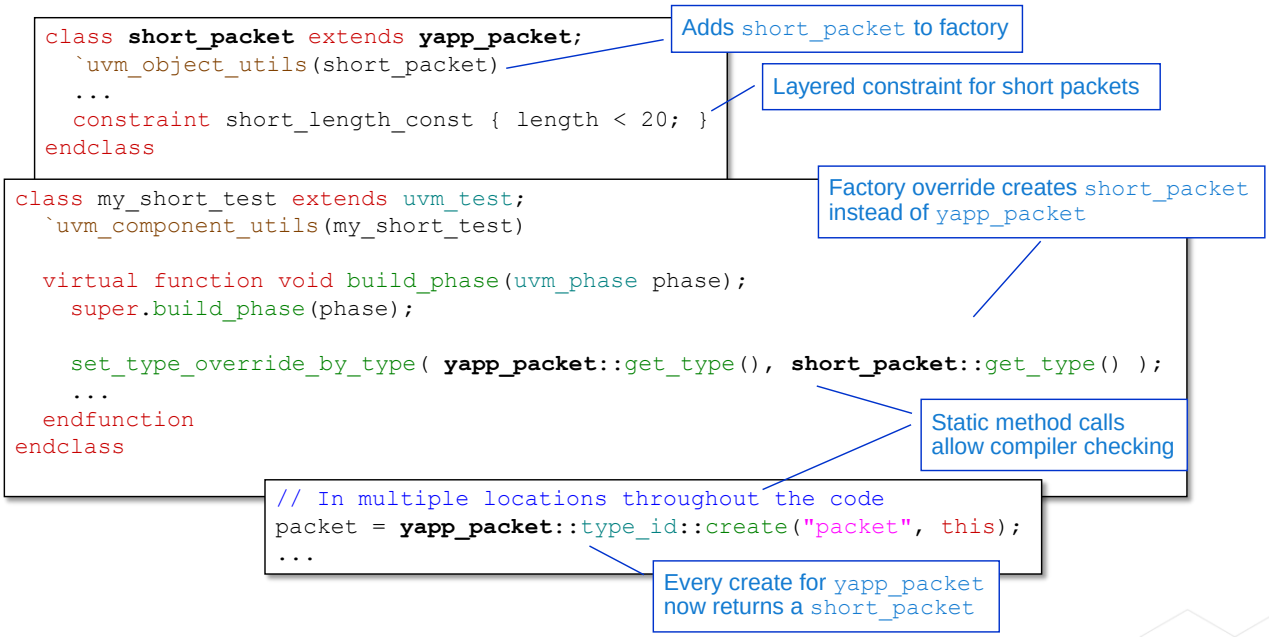
If the class does not have a utility macro, then the create call will flag a compilation error.

If we do not use the factory create call for a class instance, then factory type overrides cannot be used which means the class cannot be modified through the use of a subclass declaration. In most cases we want to allow this, so almost all instances should be made with create calls. There are exceptions to this rule we will see later (for example, TLM connector objects which are introduced in the Building a Scoreboard module).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Type Overrides with Factory



 © Cadence Design Systems, Inc. All rights reserved.

`set_type_override_by_type` uses types for the original and override type identifiers. The type identifiers can be obtained using `get_type()` method calls, as in the example above.

It is possible to register a type with the factory under the wrong name, by having a typo or using a different identifier than the class type name in the argument to the utility macro. The factory cannot check the validity of the registration name given in the utility macro argument. An incorrect registration name can cause compilation errors; run-time warnings or failed type overrides.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Changing Procedural Behavior

```
class yapp_driver extends uvm_driver #(yapp_packet);
  `uvm_component_utils(yapp_driver)

  virtual task send_to_dut();
    // protocol for driving packets into DUT
  endtask
  ...
```

Developer implementation code

```
class config1_driver extends yapp_driver;
  `uvm_component_utils(config1_driver)
  ...
  virtual task send_to_dut();
    // modified protocol
  endtask
  ...
```

Modified protocol

```
// in testbench file
set_type_override_by_type(yapp_driver::get_type(), config1_driver::get_type());

yapp = yapp_env::type_id::create("yapp", this);
```

Use modified protocol

Set override before creating UVC

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Overriding Specific Instances

A class object may be used in several parts of an environment

Only some of these instances may need to be overridden:

- `set_type_override_by_type` applies to every instance of a class
- `set_inst_override_by_type` overrides a specific class instance

```
class short_packet extends yapp_packet;
  `uvm_object_utils(short_packet)
   ...
  constraint short_length_const { length < 20; }
endclass
```

```
class my_short_test extends uvm_test;
  `uvm_component_utils(my_short_test)
  virtual function void build_phase(uvm_phase phase);
    ...
    set_inst_override_by_type("*agent0*", yapp_packet::get_type(), short_packet::get_type());
  endfunction
endclass
```

Only override packets in the `agent0` path

137 © Cadence Design Systems, Inc. All rights reserved.

cadence

Instance overrides allow you to apply overrides to instances under a specific hierarchy or a specific instance in a hierarchy.

The instance override method call takes an additional argument which is a hierarchical pathname to the instance(s) you wish to override. The pathname is a string within which you can use wildcard characters.

Explicit pathname which do not use wildcards must be complete. For example, to apply an override to the `yp` instance of yapp_packet in the `monitor` of `tx_agent` under the `yapp` env, the explicit pathname for the instance override would be:

`yapp.tx_agent.monitor.yp`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Type Versus Instance Overrides

Set type overrides replaces ALL instances:

- For a specific test, replace all `data_p` packets with `long_p` packets

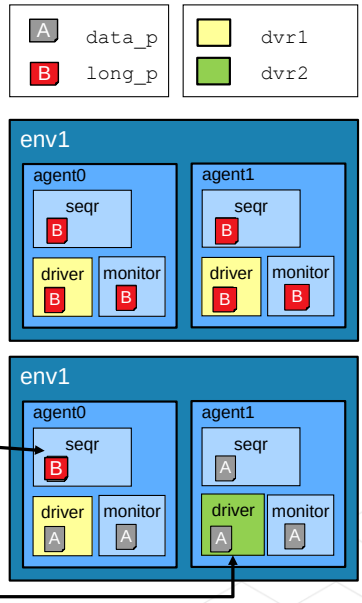
```
set_type_override_by_type(data_p::get_type(),
                          long_p::get_type());
```

Set instance overrides replace specific instances:

- Replace `data_p` instances in `agent0` sequencer
- Replace `agent1` driver `dvr1` with `dvr2`

```
set_inst_override_by_type("env1.agent0.seqr",
  data_p::get_type(), long_p::get_type());

set_inst_override_by_type("env1.agent1.driver",
  dvr1::get_type(), dvr2::get_type());
```



138 © Cadence Design Systems, Inc. All rights reserved.

Type override replaces all instances:

Syntax

```
set_type_override_by_type(orig_type, override_type);
```

Instance override replaces specific instances:

Syntax

```
set_inst_override_by_type ("hierarchical_path", orig_type,
  override_type);
```

© Cadence Design Systems, Inc. All rights reserved.

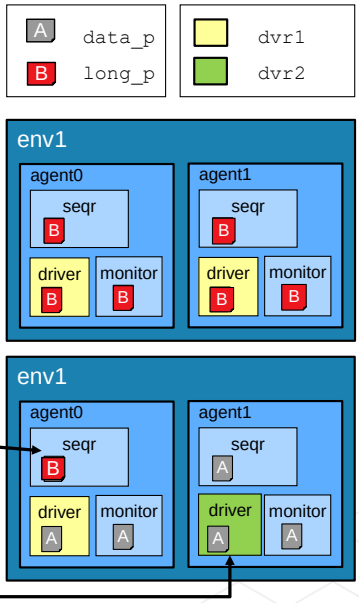
This document is for the sole use of Kun Wang of Seas

Alternative Syntax for Type/Instance Overrides

Type override replaces ALL instances:

- For a specific test, replace all `data_p` packets with `long_p` packets

```
data_p::type_id::set_type_override(long_p::get_type());
```



Instance override replaces specific instances:

- Replace `data_p` instances in `agent0` sequencer
- Replace `agent1` driver `dvr1` with `dvr2`

```
data_p::type_id::set_inst_override
  (long_p::get_type(), "env1.agent0.seqr");

dvr1::type_id::set_inst_override
  (dvr2::get_type(), "env1.agent1.driver");
```

139 © Cadence Design Systems, Inc. All rights reserved.

Alternative syntax which works exactly like the syntax on the previous slide.

Type override replaces all instances:

Syntax

```
orig_type::type_id::set_type_override( override_type::get_type());
```

Instance override replaces specific instances under the relative path:

Syntax

```
orig_type::type_id::set_inst_override( override_type::get_type(),
  "hierarchical_path");
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Override Rules

```
// set override under UVC env1 of testbench tb
set_inst_override_by_type("tb.env1.*", frame::get_type(), longframe::get_type());

// set override under UVCs env2, env3 etc. of testbench tb, but not env1 (matched above)
set_inst_override_by_type("tb.env?.*", frame::get_type(), shortframe::get_type());

// remove (reset) override on type frame (from UVM1.2 only)
set_type_override_by_type(frame::get_type(), frame::get_type());
```

First match only applies

UVM1.2 ONLY – this is an error in previous versions

Instance overrides: `set_inst_override_by_type`

- Path can include `*` and `?` wildcard characters
- Take precedence over type overrides
- Only the first matches applies:
  - Override specific instances before overriding groups of instances

Both overrides chain:

- Overriding type `A` to `B` and also `B` to `C`, means type `A` becomes `C`, not `B`

From UVM1.2, type overrides can be removed (reset)

Factory Override List

from	to
A	B
B	C
...	...

Request for `A` will return `C`

Overrides are recursively searched

140 © Cadence Design Systems, Inc. All rights reserved.

cadence

The most common type override problems are caused by mismatches in instance strings due to typos or missing path references. Override settings will not be applied and no errors or warnings are generated.

Instance overrides take precedence over type overrides.

For instances with multiple overrides the first match applies. For example, in an environment of UVCs `env1`, `env2` and `env3` under a testbench `tb`, then the first instance override applies to the specific instance `env1` and locks overrides to `longframe`. The second instance override applies to all instances of `env?` under `tb`, but because `env1` has previously been matched, this second set does not apply to the `env1` instance.

By default, a new type override overwrites all previous overrides for that type. Setting an optional third argument of the type override methods to 0 prevents overwriting of existing overrides.

Both type and instance overrides chain.

If an override converts all `typeA` instances to `typeB`, and a second override converts all `typeB` instances to `typeC`, then creating a `typeA` instance results in a `typeC` instance. An override affects class instances which are directly created of that specific type, but also class instances which become that type as a result of another override.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Notes About the Factory

Overrides can be called at any time

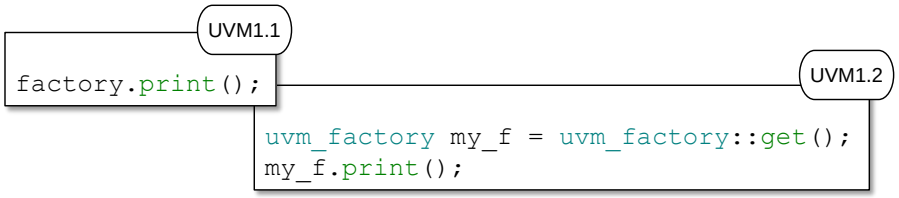
- Overrides apply for all subsequent calls to the factory

You don't have to instantiate the factory

- It is auto instantiated on first registration of an object

For debug, you can print the factory contents:

- Prints registered types and overrides



UVM1.2 allows customized factories

- For example, for extra debug or trace

 © Cadence Design Systems, Inc. All rights reserved.

cadence

```
function void print (int all_types=1)
```

Prints the state of the `uvm_factory`, depending on the value of the argument `all_types`:

- When 0, only type and instance overrides are displayed.
- When 1 (default), all registered user-defined types are printed as well.
- When 2, the UVM types (prefixed with `uvm_`) are included in the list of registered types.

Up to and including UVM1.1d, there is only one factory (named factory) and the debug print method is called from this.

From UVM1.2, customized factories can be declared to allow, for example, extra debug tracing if you wish to log every type override applied to a simulation run. You can still only have one factory at a time, but you can switch between different implementations, dynamically if you wish. Therefore from UVM1.2, you must retrieve the factory in use via the static method `uvm_factory::get()` before calling the print method.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Type Overrides and Factory Quiz

1. What is the code to create an instance of `driver` using the factory?

```
yapp_tx_driver driver;
```

2. What has the highest precedence – type overrides or instance overrides?

3. Given the following overrides in UVM1.2, what actual type is an instance of `A` created by the factory in the path `test.yapp.driver`?

```
set_type_override_by_type(A::get_type(), B::get_type());
set_inst_override_by_type("*yapp*", B::get_type(), C::get_type());
set_type_override_by_type(A::get_type(), A::get_type());
```

Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 4 Using Factories

- Update your lab files to enable factory use
- Create multiple tests
  - Use a base test class and derive tests from it
  - Use type override to dynamically control packet generation
- Use configuration to control UVC topology

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Type Overrides and Factory Quiz

1. What is the code to create an instance of driver using the factory?

```
yapp_tx_driver driver;
```

```
driver = yapp_tx_driver::type_id::create("driver",this);
```

2. What has the highest precedence – type overrides or instance overrides?

   Instance overrides

3. Given the following overrides in UVM1.2, what actual type is an instance of `A` created by the factory in the path `test.yapp.driver`?

```
set_type_override_by_type(A::get_type(), B::get_type());
set_inst_override_by_type("*yapp*", B::get_type(), C::get_type());
set_type_override_by_type(A::get_type(), A::get_type());
```

The actual type is `A` (see notes).

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Question 3:

The first type override replaces all instances of `A` with `B` globally. So now all instances of `A` are created as instances of `B`.

The second instance override replaces all instances of `B` with `C` in any instance path containing `yapp`. So now in the path `test.yapp.driver`, all instances of `A` are created as instance of `C`.

The third type override (UVM1.2 only) resets (removes) the override on `A`, so all instances of `A` are created as instances of `A`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 10

Sequences

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequences Objectives

In this module, you

- Use UVM macros and methods to create sequences to implement common stimulus requirements:
  - Simple randomized data
  - Directed tests
  - Stimulus controlled by properties
  - Hierarchical stimulus
- Define which sequence is executed by the sequencer
- Use the UVM objection mechanism to control the end of simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Are Sequences?

A sequence is a set of transactions that accomplish a defined task for the DUT.

- Typically, tests are described at the transaction level
  - Easier to write, maintain, and control
  - Examples: instructions, packets
- Single items cannot capture the high-level intention
  - Ordered stream of transactions is required
  - Examples: program generation, connection-based protocols

Some sequences should be provided as part of a reusable component...

- Capture important scenarios that should be exercised
- Provide examples for a test writer to extend from or reuse

...however, a test writer will frequently have to create additional sequences

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Sequence Requirements

- Easy to write and use
  - No need to learn complex mechanism
  - Minimize the protocol/environment knowledge requirements
- Combine declarative and procedural constraints
- Can be generated on-the-fly or at time zero
- Can nest sequences
- Can be timed as well as procedural
  - For acceleration, delays should be mapped to interface methods (see later)
- Ability to create a system-level sequence – drive multiple interfaces from a single sequence (described later)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM sequences need to be easy to use without complex syntax or methodology overheads. They need to be easy to use so that test writer, with only a minimal knowledge of the test environment and the stimulus protocol, can use and create sequences .

Sequences need to be able to combine the default declarative constraints which apply to every data item (e.g., declared in the `yapp_packet` class) with inline procedural constraints to modify individual data items.

Sequences should support the generation of data items as required during the run phase, perhaps in response to data from the DUT, or the generation and storage of thousands of data items at the beginning of the simulation, which is a technique for hardware acceleration and emulation.

Sequences must support nesting to allow complex sequences to be built out of simple sequence building blocks.

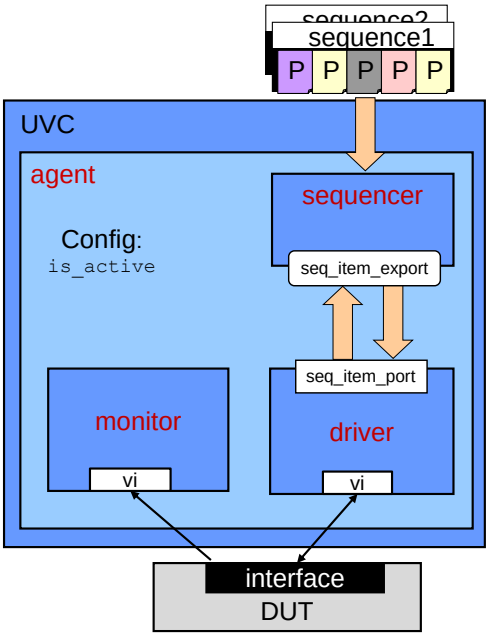
Sequences must support timed behavior, such as delays, event expressions and parallelism (through fork-join blocks) as well as procedural behavior such as looping, conditional branching and iteration. However for acceleration efficiency, explicit delays and event expressions should be executed in the hardware partition only. A sequence should trigger a driver to execute interface methods to, for example, count clocks or wait for DUT events.

Finally we need to create sequences which control more than more UVC, to synchronize stimulus and behavior for all the inputs to the DUT. System-level (or virtual) sequences will be covered later in the class.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequences and UVCs Review



Sequence – **data item**

- Defines stimulus pattern of individual items (YAPP packets)
- Can contain declarations, procedural code, timing, constraint blocks, etc.
- Can reuse other predefined sequences
- Provides a standard test writer interface

Sequencer – **component**

- Provides randomization of sequences and items
- Provides TLM connection to a driver
- Executes sequence by supplying individual items (YAPP packets) to driver on demand

Driver – **component**

- Provides TLM connection to a sequencer
- Request the next item (YAPP packet) from sequencer
- Implements the protocol for driving the item on the interface

 © Cadence Design Systems, Inc. All rights reserved.

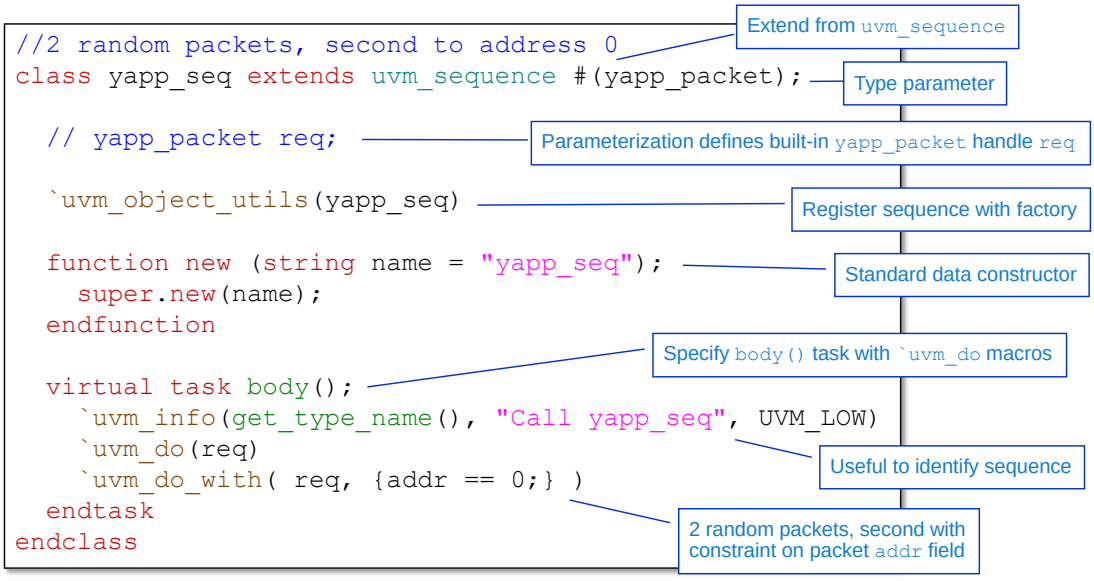
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Sequences Example



- Sequences are data items with a data constructor and object utility macro

150 © Cadence Design Systems, Inc. All rights reserved.

The sequence has a type parameter to match the type parameter on its associated sequencer and connected driver. Type parameterization defines a built-in handle on the type named `req`.

The type parameter (`yapp_packet`) must be of a type which derives from `uvm_sequence_item` in order to use `uvm_do` macros.

Remember a sequence is a data item. Therefore it has a data constructor with single `name` argument. A sequence uses the standard object utilities macro to register the sequence with the factory.

The stimulus provided by the sequence is defined in a `body()` task. Inside the task we typically use UVM `do` macro variants to define the stimulus. This sequence creates one random packet with the default `yapp_packet` constraints (equivalent to `req.randomize()`), followed by a second randomized packet with a constraint specifying the `addr` field of the packet equals zero (equivalent to `req.randomize() with {addr == 0;}`)
However the `do` macros carry out much more functionality than just randomizing the packet, as we shall see on the following slides.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

The do Operation

```
...
task body ();    // sequence behavior
 `uvm_do_with(req, {addr == 0;} )
endtask
```

Translates into:

- Allocate using the factory
- Wait till the item is needed
- Randomize (with constraints or not)
- Put on the consumer interface
- Block code execution till `item_done()`

Factory support out-of-the-box

Reactive generation

Combines procedural (inline) and declarative (layered) constraints for extra flexibility

Compatible with built-in TLM connection

Allows modeling timing, for example, scenario that calls for a frame, waits for FIFO full, and sends an illegal frame

Naive and expert users alike can easily exploit key randomization concepts.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `uvm_do` macros allow you to easily combine declarative (layered) constraints from the data item descriptions with procedural (inline) constraints in the macro argument list. In this way, the declarative constraints which affect every data item instance can be combined with procedural constraints which affect individual instances, to create the stimulus you need.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

`uvm_do Macros: What Do They Do?

Step	Summary	Description	Methods
1	creation	Create instance, set parent and link to sequencer	`create`
2	synchronize	Wait for sequencer `get_next_item()` request	`start_item`
3	pre_do hook	Executes the `pre_do()` task of the sequence	
4	randomization	Randomize the instance. If randomization fails, a warning is issued	`randomize`
5	mid_do hook	Executes the `mid_do()` function of the sequence	`finish_item`
6	send item and wait until done	For items: send item to sequencer, and wait until item done For subsequences: execute the sequence `body()` method	
7	post_do hook	Execute the `post_do()` function of the sequence	

152 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `uvm_do` macro performs seven steps to create and randomize an item, and synchronize with the sequencer to pass the item to the driver under the control of the get_next_item/item_done method calls.

There are various ways to customize the behavior of the `uvm_do` macro. One (less common) option is to use hook methods (similar to pre/post-randomise). If you define the hook method for the sequence, it is automatically executed at a defined point in the macro execution.

```
virtual task pre_do(bit is_item);
```

- Called after the sequencer has selected this sequence, and before the item is randomized. Although a task, consuming simulation cycles may result in unexpected behavior on the driver.

```
virtual function void mid_do(uvm_sequence_item this_item);
```

- Called after the item has been randomized, and before the item is sent to the driver.

```
virtual function void post_do(uvm_sequence_item this_item);
```

- Called after the sequencer has received the `item_done()` call from the driver.

A more common customization option is to replace `uvm_do` with other macro variants (see next slide) or to use method calls instead (see later). These perform the same actions as `uvm_do`, but allow us to insert custom code between various steps in the flow.

A warning (`` `uvm_warning ``) is issued if randomization fails due to constraint errors, *but* the simulation continues with a non-randomized data item.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Alternative to `uvm_do` Macros: Explicit Flow with Methods

```
//Single packet to address 0 - explicit sequence
class yapp_explicit extends uvm_sequence#(yapp_packet);
  int ok;
  `uvm_object_utils(yapp_explicit)

  function new (string name = "yapp_explicit");
    super.new(name);
  endfunction

  virtual task body();
    req = yapp_packet::type_id::create("req");
    start_item(req);
    ok = req.randomize() with {addr == 0;};
    finish_item(req);
    ...
  endtask
endclass : yapp_explicit
```

Explicit creation

`create` call in data item, not component, so no `this`

`start_item` is equivalent to steps 2-3 of `` `uvm_do ``

Explicit randomization

`finish_item` is equivalent to steps 5-7 of `` `uvm_do ``

153 © Cadence Design Systems, Inc. All rights reserved.

cadence

Some people prefer to use the explicit flow instead of variations of the macros. UVM provides several ways to accomplish the same result and give full control of data creation using the sequence interface.

The explicit flow requires explicit creation of the data item (YAPP packet) instance which should be via a factory `create` call. However, the `create` call cannot use `this` as the second parent argument. Parent arguments are only applicable in component classes, where they are used to locate the instance in the component hierarchy. As we are calling create in a data item (sequence `yapp_explicit`) there is no component hierarchy, so the argument is not passed (it defaults to `null`).

If you do include the `this` argument in the `create` call of a data item you get a type compatibility compile error.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Additional `uvm_do Macros

For full flexibility, additional macros can be used for item and sequence execution.

Syntax

- `` `uvm_*(variable) ``
- `` `uvm_*_with(variable, { constraint-block} ) ``

`` `uvm_do ``	Performs all seven steps from previous slide (creation to `post_do`)
`` `uvm_do_with ``	Same as `` `uvm_do ``, but adds in-line constraints during randomization (`randomize with {}`)

`` `uvm_create ``	Only performs step 1 (creation)
`` `uvm_send ``	Performs steps 2-7 (synchronize to `post_do`), but skips step 4 (randomization)
`` `uvm_rand_send ``	Performs steps 2-7 (synchronise to `post_do`)
`` `uvm_rand_send_with ``	Performs steps 2-7 (synchronize to `post_do`) and adds additional constraints for randomization (step 4)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Syntax

- `` `uvm_do(variable) ``
- `` `uvm_do_with(variable, { constraint-block} ) ``
- `` `uvm_create(variable) ``
- `` `uvm_send(variable) ``
- `` `uvm_rand_send(variable) ``
- `` `uvm_rand_send_with(variable, { constraint-block} ) ``

The variable must be derived from `uvm_sequence_item` or `uvm_sequence`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Directed Stimulus with Macros

```
//send illegal packet to address 3
class directed extends uvm_sequence #(yapp_packet);
  int ok;
  `uvm_object_utils(directed)
   // constructor
   ...
  virtual task body();
    `uvm_info(get_type_name(), "Call directed", UVM_LOW)
    `uvm_create(req)
    ok = req.randomize();
    req.addr = 3;
    req.set_parity();
    `uvm_send(req)
  endtask
endclass
```

Create instance

Randomize instance to create legal packet

Overwrite and updates fields as required

Send without further randomization

- Create and send macros are ideal for defining directed or semi-directed stimulus

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Create and send macro combinations can be useful for directed or semi-directed stimulus.

For example:

- Create a data instance using `` `uvm_create() ``.
- Optionally randomize the instance to create legal data (here make sure the YAPP packet length and payload are consistent).
- Overwrite the required fields of the instance (here to set the address).
- Perform any housekeeping on the data (here update `parity`).
- Send the instance without further randomization using `` `uvm_send() ``.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequence Properties

```
// 2 yapp packets to consecutive addresses
class yapp_seq_addr extends uvm_sequence #(yapp_packet);

  `uvm_object_utils(yapp_seq_addr)

  function new (string name = "yapp_seq_addr");
        super.new(name);
  endfunction

  rand bit[1:0] seqadd;
  constraint c1 { seqadd <= 2'b01; }

 task body();
   `uvm_info(get_type_name(), "Call yapp_seq_addr", UVM_LOW)
   `uvm_do_with( req, {addr == seqadd;} )
   `uvm_do_with( req, {addr == seqadd+1;} )
 endtask

endclass
```

`rand` sequence property randomized each time sequence called

Constraint for property randomization

Property used as constraints for packet generation

156 © Cadence Design Systems, Inc. All rights reserved.

Note that a sequence can have properties (e.g., `seqadd`). Properties can be used in the definition of the sequence behaviour. If the sequence property is declared as `rand`, then the property is randomized when the sequence is executed. The property can then be used anywhere in the `body()` task to generate stimulus, for example:

- In a repeat loop to create a variable number of packets.
- In `do_with` constraint blocks to define randomization constraints.

Constraints can be declared either in the sequence or in-line with the sequence call, to control the randomization of the sequence property. Here for example, we add a constraint to prevent `seqaddr` being 2 or 3, which would give us a sequence containing the illegal packet address 3.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Nesting Sequences

```
// call yapp_seq_addr sequence wrapped with random packets
class nested extends uvm_sequence #(yapp_packet);

  `uvm_object_utils(nested)
   // constructor

  yapp_seq_addr ysa;

  task body (); // sequence behavior
    `uvm_info(get_type_name(), "Call nested", UVM_LOW)

    `uvm_do(req)
    `uvm_do_with(ysa, {seqadd == 0;} ) // sub-sequence
    `uvm_do(req)

   endtask
endclass
```

Handle for previously defined sequence
Choose meaningful name for debug!

Constraint on `yapp_seq_addr` sequence `rand` property

157 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequences can be nested, for example to build up more complex stimulus from basic sequences.

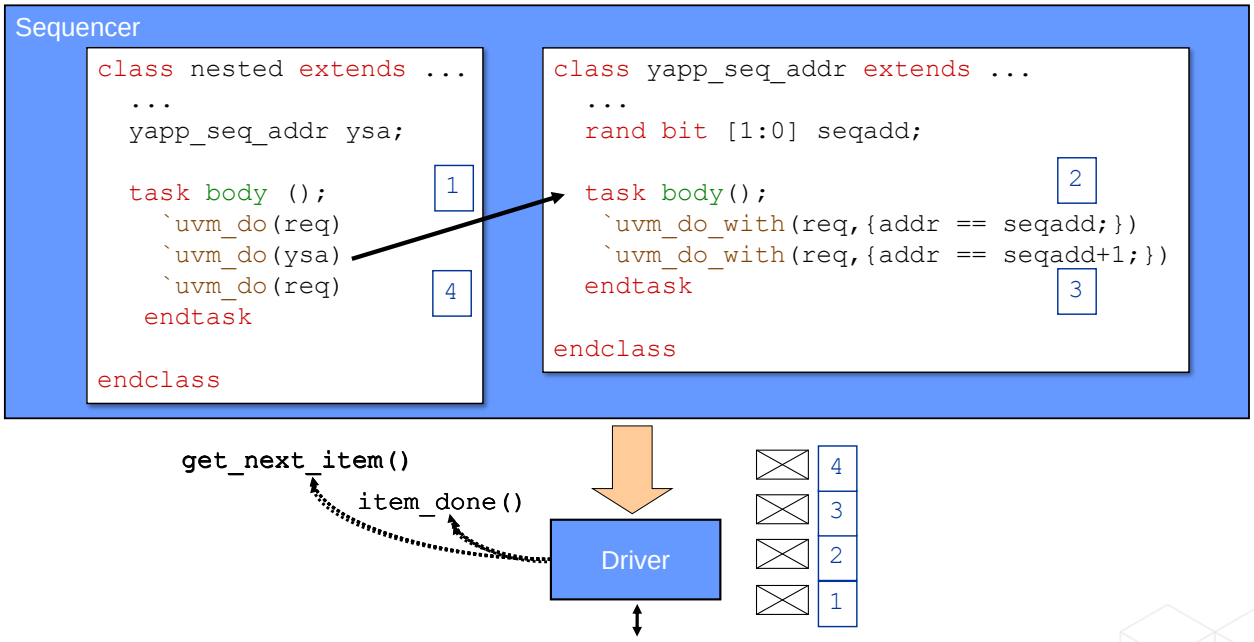
A handle must be defined for the nested sequence, must be previously declared. It's a good idea to choose a meaningful name for the handle, related to the nested sequence, as this can help in debug (where only the handle name may be shown, not the nested sequence type).

A `uvm_do` macro can then be called on the sequence handle to execute it as a subsequence. The `uvm_do_with` macro variant can be used to pass inline constraints to the properties of the subsequence.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequence Execution During Simulation Run Time



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Executing Sequences

How do we control which sequences run on a UVC sequencer and when?

Several choices:

1. `run_phase()` (and every run sub-phase) has a `default_sequence` property
   - Set `default_sequence` to a sequence to execute it in that phase
2. Execute sequences directly on a UVC sequencer from a test class
3. Sequences can be grouped together into a sequence library
   - Not covered in this course – see the Training Byte


TB: UVM Sequence Libraries

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequence libraries allow you to group selected sequences into a library collection, from where you can execute a random number of randomly selected sequences.

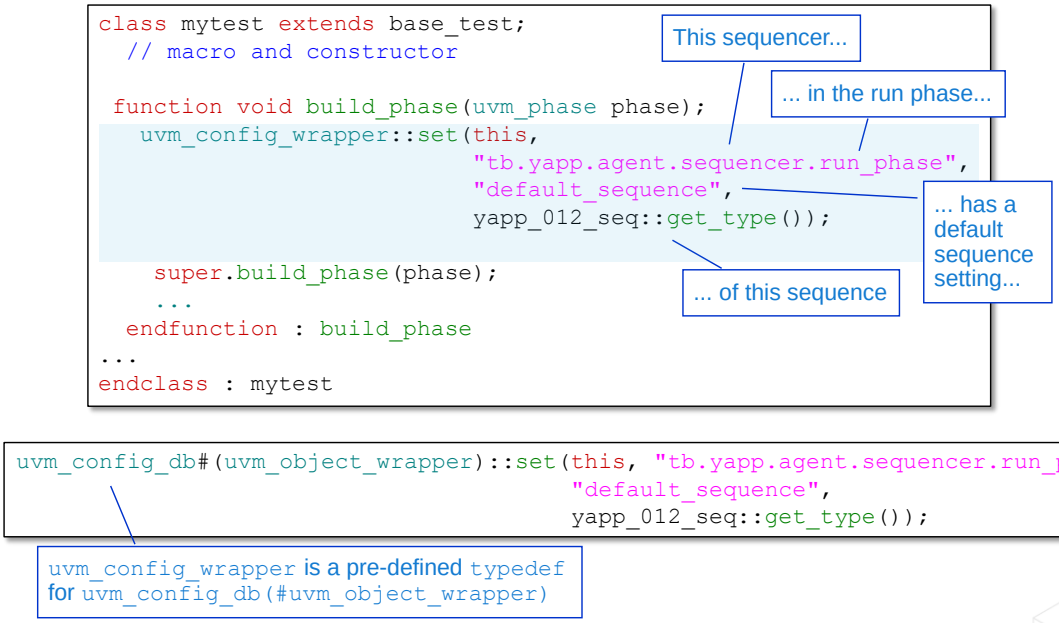
There is a Training Byte on "UVM Sequence Libraries" with more information.

- See `support.cadence.com/TrainingBytes/UVM`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Run Phase Default Sequence



Every UVC has a `default_sequence` property for the `run_phase` and for every run sub-phase. By default the pointer is null.

A standard `uvm_config_db` call can be used to set the property to a specific sequence. That sequence is then executed by the UVC sequencer in the defined phase.

The type parameter for the `uvm_config_db` call is `uvm_object_wrapper`. This is a factory type declaration representing an abstract interface for all `uvm_object-based` and `uvm_component-based` objects which are registered with `uvm_factory`.

For convenience, a predefined `typedef`, `uvm_config_wrapper`, is declared for a `uvm_config_db` type-parameterized for `uvm_object_wrapper`.

Configuration review:

- The first argument is the context (mapped to `this`).
- The second argument is the pathname to the UVC sequencer, appended with the run phase in which the sequence should execute. The phase, if omitted, defaults to `run_phase`().
- Third argument is the field name `default_sequence`.
- Last argument is the sequence type name, using `get_type()` to enable compile-time type checking.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Executing UVC Sequence in Test

```
class mytest extends base_test;
  // macro and constructor
  yapp_sequencer yapp_seqr;

  yapp_seq_addr yapp_addr;

 function void build_phase(uvm_phase phase);
    yapp_addr = yapp_seq_addr::type_id::create("yapp_addr",this);
    super.build_phase(phase);
    ...
  endfunction : build_phase

  function void connect_phase(uvm_phase phase);
    yapp_seqr = tb.yapp.agent.sequencer;
  endfunction : connect_phase

  task run_phase(uvm_phase phase);
    ok = yapp_addr.randomize();
    yapp_addr.start(yapp_seqr);
    ...
  endtask : run_phase
...
endclass : mytest
```

Handle for every sequencer to be controlled

Handle for every sequence to be executed

Create sequence instance

Assign sequencer handles to UVC sequencer instances

Sequence with `rand` properties must be explicitly randomized

Run `yapp_addr` sequence instance on `yapp_seqr` sequencer handle

161 © Cadence Design Systems, Inc. All rights reserved.

As an alternative to setting the default sequence of a sequencer, a test class may execute sequences directly on a sequencer.

The procedure is as follows:

- Declare a handle for the sequence(s) to be executed and create all these handles in the build phase of the test.
- Declare a handle for the UVC sequencer(s) on which the sequences will be executed. These handles are not created, but will be linked directly to the UVC sequencer instances.
- In the connect phase of the test, assign the path name of the UVC sequencer instance to the sequencer handle. The pathname must relative to the test instance and use the handle names (not the instance names) of the components in the path. Wildcards are not allowed in the pathname – we are making a direct reference copy of the sequencer instance, not making a configuration setting.
- If the sequence instance contains rand properties, then the sequence must be explicitly randomized to give values to these properties. If you do not randomize the sequence, rand properties are left at their default value. Randomization could be done once in the build_phase, after the instance creation, or many times in the run_phase, as required.
- In the run phase of the test, execute the sequence instance on the sequencer reference using start.

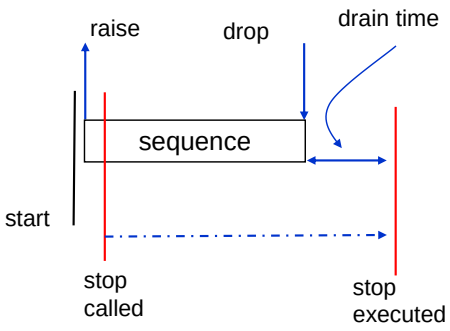
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Ending Simulation

An objection (consensus) mechanism is used to elegantly end the simulation.

- Simulation immediately tries to end the run phase
- A sequence must raise an objection when it starts to hold off the phase end
- A sequence must drop the objection on finishing to allow the phase to end
- After a certain time (drain time) with no further objections raised, the phase ends



Every run (sub)phase has a built-in objection mechanism.

The run phase ends if no objections were raised or when all objections are removed.

- Expandable, reusable, and flexible

Not all sequences should raise objections

- Not appropriate for background traffic, response generators, or subsequences

162 © Cadence Design Systems, Inc. All rights reserved.

The objection mechanism manages the termination of `run_phase` and hence simulation end.

The `run_phase` has a built-in objection mechanism. In addition, every run sub-phase also has its own objection handling.

At the start of every run phase, the objection mechanism tries to end the phase immediately. If a sequence executes in that phase, it must raise an objection to the phase end. Raised objections prevent the phase ending until dropped. Once the sequence has ended, it must drop the objection.

If no objections are raised for a phase, or when the objection count returns to zero, then the phase waits for a certain period of time (called the drain time) in anticipation of another objection being raised. If no new objections are raised during the drain time, the phase ends. The default drain time is zero, i.e., the phase will end immediately when the objection count reaches zero or if no objections are raised.

When all run sub-phases have ended, and the `run_phase` itself has ended, then the simulation progresses to `extract_phase`, `check_phase` and `report_phase`.

Only `run_phase` and it's sub-phases can rise objections as these are the only phases which consume time.

For efficiency and robustness, you should raise the minimum number of objections to maintain the simulation. Therefore not all sequences should handle objections. For example background, response, or nested sequences.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Objection Handling

There are three ways to execute a sequence and objection handling is different for each:

1. From the test class
   - Using `start()`
   - Called a root sequence
   - Test handles objections
2. Set as the default sequence of a sequencer
   - Including multichannel (virtual) sequencer
   - Also called a root sequence
   - Sequence handles objections
3. Called from within another sequence
   - Nested or subsequence
   - Should not handle objections

test sequence

```
class mytest extends base_test;
  ...
  yapp_012_seq y012;

  task run_phase(uvm_phase phase);
    ...
    y012.start(yapp_seqr);
    ...
```

default sequence

```
class mytest extends uvm_test;
...
 function void build_phase(uvm_phase phase);
   uvm_config_wrapper::set(this,
       "env.tx_agent.sequencer.run_phase",
       "default_sequence",
       yapp_012_seq::get_type());
  ...
```

```
class nested extends uvm_base_seq;

  yapp_012_seq y012;

  task body ();
    `uvm_do(y012)
...
```

subsequence

163 © Cadence Design Systems, Inc. All rights reserved.

A sequence can be executed in one of three ways:

- A sequence can be called from a test class using the `start()` method. Such sequences are called root sequences. The test class handles the objections.
- A sequence can be set as the default sequence of a sequencer using the configuration database. This is also a root sequence. The sequence itself handles the objections.
- A sequence can be called from within another sequence. This is a nested or subsequence. These sequences should not handle objections as it is assumed the topmost sequence in the hierarchy will handle the objections. Raising and dropping additional objections in the subsequence is inefficient and increases the likelihood of objection errors.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Test Objections

Objections are raised and dropped by methods

```
raise_objection(<object>, <description>, <count>);
drop_objection(<object>, <description>, <count>);
```

- Raise or drop the number of objections for `object` by `count` (default 1)
- `description` is a string used for tracing and debug

In a test, objection methods are called in the run phase (or subphase)

- Using the `phase` method argument

```
class mytest extends uvm_test;
...
  task run_phase(uvm_phase phase);
    phase.raise_objection(this, "mytest");

    yapp012.start(yapp_seqr);
    ...
    phase.drop_objection(this, "mytest");
  endtask : run_phase
...
```

Raise an objection for this component for run phase

Drop objection after stimulus sent

164 © Cadence Design Systems, Inc. All rights reserved.

cadence

Objections are raised and dropped via methods:

```
virtual function void raise_objection (uvm_object obj = null,
string description = "", int count = 1)
```

Raises the objection count for the nominated object (use `this` to specify the calling object) by `count`. The `description` string argument is used for debug.

```
virtual function void drop_objection (uvm_object obj = null,
string description = "", int count = 1)
```

Drops the objection count for the nominated object (use `this` to specify the calling object) by `count`. The `description` string argument is used for debug. Dropping the objection count below 0 is an error.

In a test class, objections are raised and dropped at the beginning and end of the run phase method and called from the `phase` argument of the method.

Objections propagate up the hierarchy to `uvm_top` so that the simulation can keep track of objections raised by every UVC. Once the `uvm_top` objection count for a phase returns to zero (or if no objections are raised for a phase) the phase ends.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequence Objections

For a sequence, objection methods are called in `body()` from `starting_phase`

- Automatically set to the run phase/subphase where the sequence is started

If the sequence is executed from a test, `starting_phase` is `null`

- In which case the sequence does not handle objections (leaves it to the test)

Also, subsequences should not handle objections

- How do we know if a sequence is a subsequence?

```
class yapp_012_seq extends uvm_sequence#(yapp_packet);
  ...
  task body();
    if (starting_phase != null)
      starting_phase.raise_objection( this, get_type_name() );
    ...
    `uvm_do_with(req, {addr == 0;})
     ...
    if (starting_phase != null)
      starting_phase.drop_objection( this, get_type_name() );

   endtask
..
```

Raises objection for default or subsequence only...

...not efficient for subsequences to raise objections

165 © Cadence Design Systems, Inc. All rights reserved.

Inside sequences, a built-in handle starting_phase is automatically set to the phase where the sequence is started. This will be run_phase or one of the run time subphases We must raise and drop objections off this handle.

This handles objections when the sequence is set as the default sequence of a sequencer.

However if the sequence is executed from a test class using start(), starting_phase is null and so cannot be used to handle objections. This is good, as if starting_phase is null we assume this sequence is executed in a test class and we don't need to handle objections in the sequence – the test handles them.

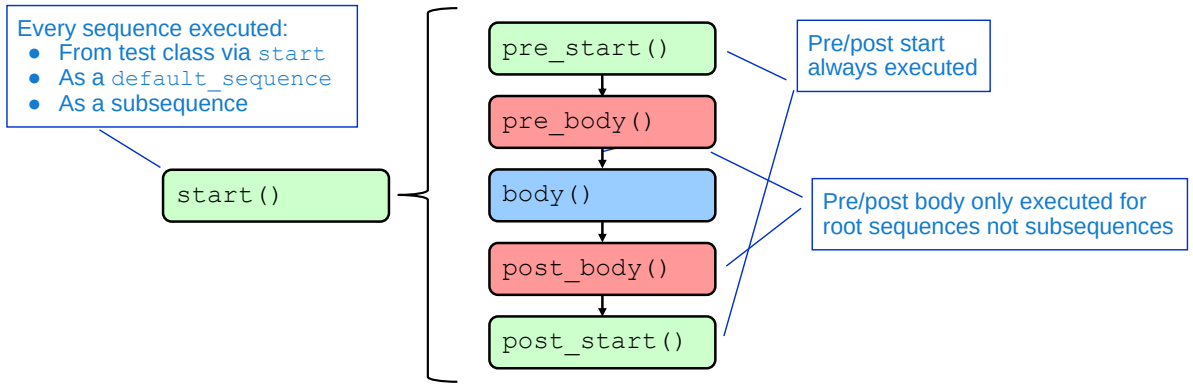
The final option is a sequence executed as a nested or subsequence. In this case we don't want the sequence to handle objections. We assume the root sequence which eventually calls the subsequence is handling the objections, and raising/dropping further objections is inefficient.

But how do we detect when a sequence is being run as a subsequence?

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequence Callbacks



Several layers of callbacks for sequences:

- Pre and post start methods are always executed
- Pre and post body methods only executed for root sequences:
  - Not subsequences

166 © Cadence Design Systems, Inc. All rights reserved.

There are several layers of callbacks which can be used in sequences. Remember callback methods are executed automatically, they should never be called by the user.

Pre/post start methods are executed for every sequence. These can be declared in a base sequence class, from which all sequences are inherited, to define functionality common to every sequence.

Pre/post body methods are only executed for root sequences. Therefore, if we place objection handling in pre/post body methods, objections will not be executed for subsequences. We can declare these methods in a base sequence, extend all our user sequences from the base, and not have to worry about objections.

Prototypes for the callback methods are as follows:

```
virtual task pre_start();
virtual task pre_body();
virtual task post_body();
virtual task post_start();
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Efficient Sequence Objections

Objections are handled in pre/post body declared in a base sequence class

This is efficient for all sequence execution options:

- Default sequences use body objections
- Test sequences use test objections
- Subsequences use objections of the root sequence which calls them

```
class yapp_base_seq extends uvm_sequence#(yapp_packet);
  ...
  task pre_body();
    if (starting_phase != null)
      starting_phase.raise_objection( this, get_type_name() );
  endtask

  task post_body();
    if (starting_phase != null)
      starting_phase.drop_objection( this, get_type_name() );
   endtask
...
class yapp012 extends yapp_base_seq;
 ...
```

null for test sequence

Pre/post body only executed for root sequence

Extend from base to inherit objection handling

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can define a base sequence class from which all our sequences will be inherited. By placing objection handling in pre/post body methods of the base sequence, we can efficiently handle objections under all situations:

Set as default sequence of sequencer.

- A default sequence is a root sequence and so the pre/post body methods are executed and objections are raised/dropped there.

Test sequence.

- A test sequence is a root sequence and so the pre/post body methods are executed. However for a test sequence, starting_phase is null and so the objections are not handled in the sequence. The test must raise and drop objections.

Subsequence.

- Not a root sequence so pre/post body methods are not executed. The root sequence which ultimately called the subsequence handles the objections, using one of the two options above.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Objection Changes in UVM1.2

```
task pre_body();
  uvm_phase sp = get_starting_phase();
  if (sp!= null)
    sp.raise_objection(this, get_type_name());
endtask
```

Raising or dropping objections directly from `starting_phase` is deprecated

- Must use `get_starting_phase()` method
- Prevents modification of phase during sequence

```
class yapp_base_seq extends uvm_sequence#(yapp_packet);
...
  function new(string name="yapp_base_seq");
    super.new(name);
    set_automatic_phase_objection(1);
  endfunction
```

Sequence objection handling can be automatic:

- Raised at when sequence started
- Dropped when sequence ends

168 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM1.2 makes two changes to objection handling in sequences:

1. Calling objection method directly from `starting_phase` is deprecated. As a safety feature, `starting_phase` is protected in UVM1.2 to prevent the phase being modified while an objection is raised, which would corrupt the phase objection count. You need to use the method, `get_starting_phase()` to read the current phase. This locks the phase until the objection is dropped.

2. In most cases, a sequence raises an objection when it begins, and drops the objection when it ends. UVM1.2 allows this to be done automatically by setting the argument of the method `set_automatic_phase_objection()` to 1. This effectively implements the `starting_phase` objection calls for you (including use of `get_starting_phase()`). This does not automatically handle objections for sequences executed from test classes. This must still done manually.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Drain Time

- Once all raised objections are dropped, the global stop can be executed
- A drain time allows propagation of outstanding items (e.g., DUT response)
  - If no drain time is set, the test will stop immediately
- Drain time can be set as:
  - Absolute delay by calling `set_drain_time` for a specific phase
    - Usually from a test class

```
task run_phase(uvm_phase phase);
  uvm_objection obj = phase.get_objection();
  obj.set_drain_time(this, 200ns);
endtask
```



TB: UVM Static and Dynamic Drain Times

  - Event delay by defining method `all_dropped` in env or other component
    - Automatically called when objection count for component returns to 0

```
task all_dropped (uvm_objection objection, uvm_object source_obj,
                  string description, int count)
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Absolute drain time delays can be set by using the method `set_drain_time`:

```
function void set_drain_time (uvm_object obj, time drain);
```

By using `this` as the `obj` argument, the method sets the drain time for the component where the method is called.

The drain time method must be called off an objection object, and this can be obtained for the current phase by calling `get_objection()` from the `phase` argument of the run phase task.

```
virtual task all_dropped (uvm_objection objection,
          uvm_object source_obj, string description, int count)
```

Drain time can be synchronized to a signal event by defining the task `all_dropped` in a component. When the objection count drops to zero, the component hierarchy is searched upwards from the dropped objection point. Any found `all_dropped` tasks must be completed before the next level up is searched. When the search reaches the test component, the global stop request is executed. The arguments define the type of objection (allows user-defined objections for purposes other than end of simulation); the source of the final objection drop; a description for debug and the number of objections in the final drop.

Both `all_dropped()` and `set_drain_time()` can be used, in which case `all_dropped()` is executed at the end of any specified drain time.

The behaviour of `all_dropped()` is not intuitive. Please see the Training Byte "UVM Static and Dynamic Drain Times" on `support.cadence.com`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Sequences Quiz

1. What utility macro does a sequence use?

2. What is the difference between the two following body methods:

```
task body();
  `uvm_do(req)
endtask
```

```
task body();
  `uvm_create(req)
  `uvm_send(req)
endtask
```

3. How do you call objection methods for sequences executed:
   a. As a default sequence
   b. In a test class

Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 5 Generating UVM Sequences

- Declare YAPP sequences for several stimulus goals:
  - Simple, nested, property-controlled and directed
- Execute selected sequences on YAPP sequencer
- Create an exhaustive sequence to execute all your declared sequences
- Find, debug and fix randomization failures

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Sequences Quiz

1. What utility macro does a sequence use?
   - A sequence is a data item, so it uses `uvm_object_utils`.
2. What is the difference between the two following body methods:

```
task body();
  `uvm_do(req)
endtask
```

```
task body();
  `uvm_create(req)
  `uvm_send(req)
endtask
```

`uvm_do randomizes req, whereas `uvm_create/`uvm_send does not include any randomization.

3. How do you call objection methods for sequences executed:
   - As a default sequence
     - Called from `starting_phase`
   - In a test class
     - Called from the phase argument of the `run_phase()` method

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 11

Connecting to a DUT

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Connecting to a DUT Objectives

In this module, you

- Connect the driver and monitor to the RTL ports of a DUT
  - Using a virtual interface
- Complete the driver and monitor components

This module uses a methodology compatible with acceleration (hardware or software)

- For more information, see the following Training Byte on support.cadence.com



TB: Future-Proof Your UVM Environments With Acceleration Optimization

 © Cadence Design Systems, Inc. All rights reserved.

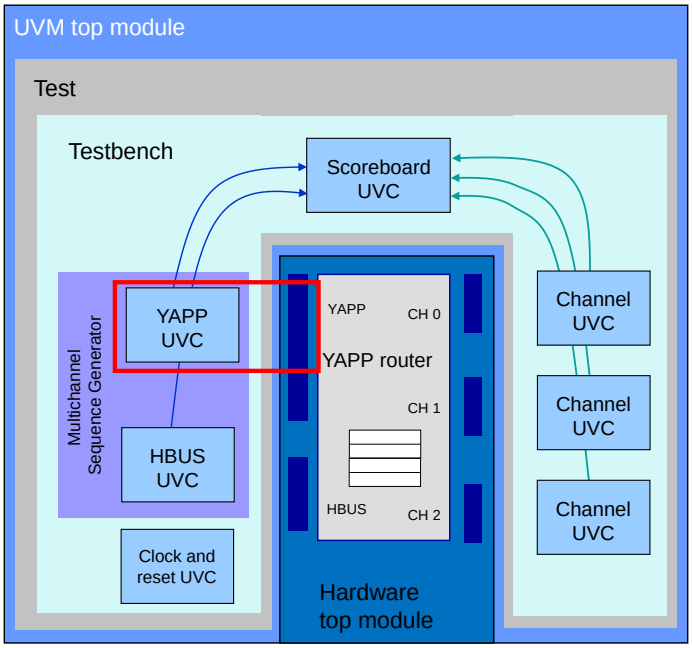
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building the Router Verification Environment



In this module we complete the YAPP UVC

- Protocol to drive YAPP packets onto RTL signals
- Back end connection to the DUT
  - Using an interface

175 © Cadence Design Systems, Inc. All rights reserved.

cadence

In this module, we'll be describing the testbench layer in the environment topology, and showing how to connect the YAPP UVC to the RTL ports of the router DUT using SystemVerilog interfaces and virtual interfaces.

Specifically we will show how to complete the YAPP UVC for compatibility with hardware acceleration.

Don’t worry about all the other components yet. We will work through them and eventually build each piece.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Review of SystemVerilog Interfaces

yapp_if.sv

```
interface yapp_if (input clock, reset);
  logic [7:0] in_data;
  logic       in_data_valid;
  logic       in_suspend;
endinterface : yapp_if
```

run.f

```
...
-incdir ../sv
../sv/yapp_pkg.sv
../sv/yapp_if.sv
...
```

Compile interface in `run.f`

An interface is a named bundle of nets or variables

Interfaces are instantiated inside modules and can be accessed through a port as a single item

The nets or variables of an interface can be referenced where needed

- Via the “dot” operator: `interface.signal_name`.

An interface can contain parameters, variables, functions and tasks, assertions and clocking blocks

- Similar to a module

An interface must be compiled separately like a module (for example, in `run.f`)

- Cannot `` `include `` inside a package or other module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

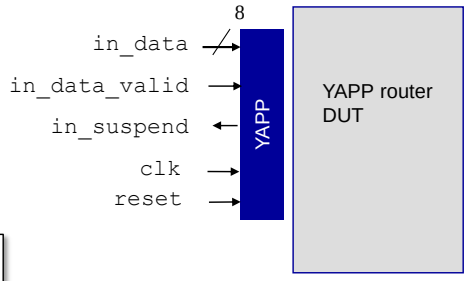
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface Example (Module-Based Verification)

```
interface yapp_if (input clock, reset);
  logic [7:0] in_data;
  logic       in_data_valid;
  logic       in_suspend;
endinterface : yapp_if
```




```
module top;
  yapp_if  in0      (clk, reset);
  yapp_dut router0 (in0, ...);

  initial begin
    //drive data bus
    in0.in_data = 8`hFA;
    //sample packet valid signal
    if (in0.in_data_valid == 1'b1)
      ...
  end
endmodule
```

Instantiate interface

Connect instance to DUT interface port

In module-based verification, access interface signals via instance name

177 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the above, the interface is connected to the DUT instantiation as a single interface port.

Alternatively, interface signals could be connected directly to separate DUT ports using positional (or preferably named) mapping:

```
yapp_dut router0 (clk, reset, in0.data, in0.packet_valid,
  in0.suspend_data_in);
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface Connection to UVM Driver and Monitor

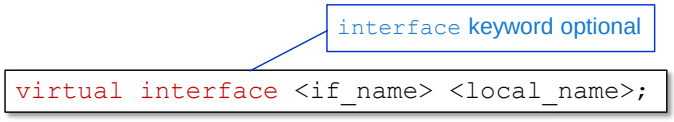
UVM components cannot be directly connected to interface instances

- Breaks reusability
- Interface instances are static

Solution is to use a SystemVerilog virtual interface:

- An interface variable that can be connected to an interface instance
- Can be declared as a class property
- Access interface signals using virtual interface as a prefix
- Needs to be connected to an actual interface

Syntax



Virtual interfaces provide a mechanism for separating abstract models from the actual signals.

178 © Cadence Design Systems, Inc. All rights reserved.

When declaring a SystemVerilog virtual interface, the keyword `interface` is optional.

However for clarity, and as the keyword virtual is already used within classes for virtual methods and virtual classes, it is recommended to use `interface`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Virtual Interface Example

```
class driver extends uvm_driver#(yapp_packet);

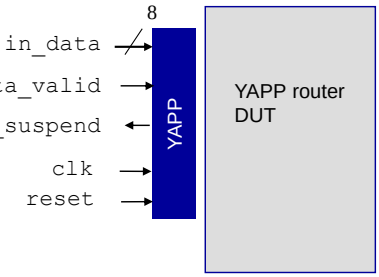
  virtual interface yapp_if vif;

  task get_and_drive();
    @(negedge vif.reset);
    forever begin
      seq_item_port.get_next_item(req);
      vif.send_to_dut(req.length, req.addr ...);
      ...
```

Virtual interface

For acceleration, task is in the interface

Driver accesses interface signals and methods via virtual interface




```
interface yapp_if (input clock, reset);
  logic [7:0] in_data;
  logic       in_data_valid;
  logic       in_suspend;

  task send_to_dut(input bit[5:0] length, bit[1:0] addr, ...);
    repeat(packet_delay)
      @(negedge clock);
    @(negedge clock iff (!in_suspend));
    in_data_vld <= 1'b1;
    in_data <= { length, addr };
    ...
```

Interface must be synthesisable for acceleration so packet data passed via individual arguments

179 © Cadence Design Systems, Inc. All rights reserved.

Once the driver has declared a virtual interface (here called `vif`) it can then access signals of the interface via the virtual interface or pass data to interface methods which access the interface signals.

For the best results in hardware acceleration, it is key to reduce the interaction between the UVM testbench and the interface signals, as the interface is implemented in the accelerated hardware partition. Therefore although direct access from the driver to infrequently changing interface signals (such as `reset`) is fine, all other signals (such as `clock`, data and enable inputs) should be accessed via interface methods to reduce interaction.

In the example, every time a YAPP packet is to be driven into the DUT, the packet data is passed to the interface method `send_to_dut()` which accesses the signals in the hardware partition.

Note that for acceleration, the interface method must be synthesisable. Therefore we cannot pass the packet data to `send_to_dut()` via an object handle as this is non-synthesisable. The data must be passed through individual arguments. For packet classes with large numbers of properties, the packet data could be mapped to a structure first (via a method) before passing to the interface. Hardware acceleration can support some abstract data items such as dynamic arrays (`payload`).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Transaction Recording

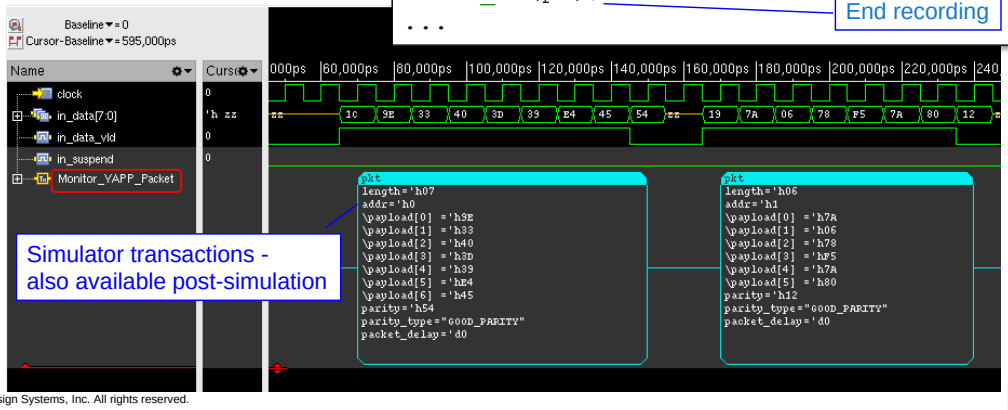
Use transaction recording methods in driver and monitor

- Create named transactions
- Implementation is vendor specific

```
yapp_packet p1;
int ok;
...
  ok = begin_tr(p1, "Monitor_YAPP_Packet" ));
  // Packet protocol implementation
  end_tr(p1);
...
```

Start recording

End recording



Simulator transactions - also available post-simulation

 © Cadence Design Systems, Inc. All rights reserved.

The `UVM_RECORD` field automation flag enables fields for transaction recording. Transaction recording is a simulator feature which can display signal transitions in a more intuitive manner, for example, showing a YAPP packet as one item rather than separate signals.

UVM defines a standard set of method calls to enable transaction recording, for example `begin_tr` and `end_tr`. These are used in the driver and monitor components around the code to read and write DUT RTL signals.

The implementation of the method calls is vendor specific. Therefore vendors must provide additional code beyond the standard UVM library for these implementations.

In Cadence's simulator, transactions are created using the Simulation Database Interface (SDI). Further information on SDI can be found in Cadence documentation.

If you use the UVM library directly from Accellera, it will not contain the transaction recording implementation. However from IUS14.1, the simulator will automatically compile the transaction extensions for the Accellera library.

If you use the UVM library provided in the Cadence simulator installation, the implementation is included.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM and Hardware Top Modules

Software: UVM (not accelerated)

```
module tb_top;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import yapp_pkg::*;

  `include "router_tb.sv"
  // other include files

  initial begin
    //connect interfaces
    ...
    run_test();
...
```

Import UVC packages

Connect virtual interfaces to instances

UVM top module (`tb_top`):

- Imports and includes UVM and UVC files
- Instantiates UVM hierarchy (`run_test`)
- Connects virtual interfaces...

Hardware (accelerated)

```
module hw_top;
  ...

  yapp_if in0(clock, reset);

  clkgen clkgen (...);

  yapp_router dut(
    .reset, .clock, .error,
    .in_data(in0.in_data),
    .in_data_vld(in0.in_data_vld),
    .in_suspend(in0.in_suspend),
...
```

Interfaces

Other modules

DUT

Map interface signals to DUT ports

Hardware top module (`hw_top`):

- Instantiates UVC interfaces
- Instantiates DUT and makes connections
- Instantiates other modules
  - e.g., clock generator

181 © Cadence Design Systems, Inc. All rights reserved.

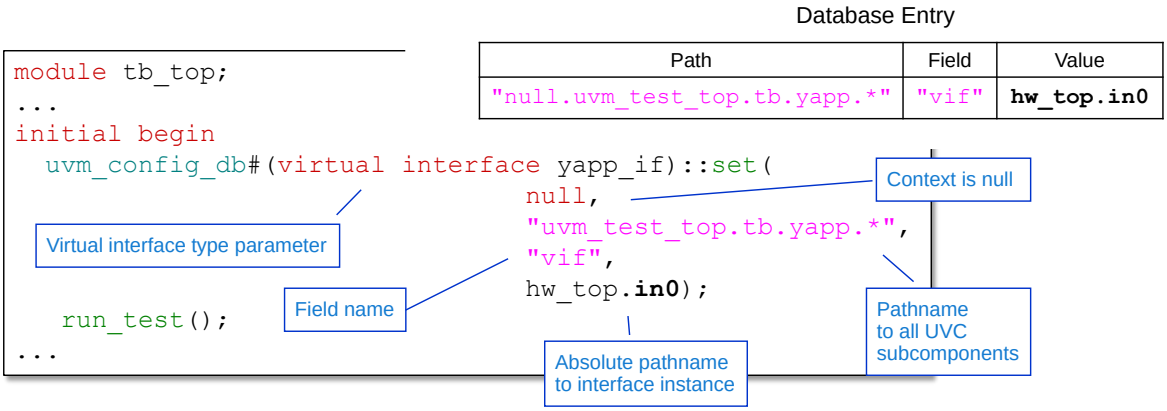
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting the Virtual Interface



Use the config database to set the virtual interface from the `tb_top` module

- `uvm_config_db` is a type parameterized for the interface type
- Context and instance name are concatenated to create config database pathname
- The interface `set` requires an explicit `get`...

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Standard `uvm_config_db set` calls are used to set the virtual interfaces of the UVC.

The set should be called from the top level module, where the interfaces are instantiated. Setting in the top removes hierarchy dependencies in the testbench and allows consistency.

The `uvm_config_db` must be type parameterized for the interface type. Since we are assigning a virtual interface, you also need the keyword `virtual` (and optionally `interface`) in the type parameter.

The value argument of the `set` call is the instantiation name of the interface.

Setting the config from the top level module has some consequences:

- The context argument of the set must be `null`, as this is the top level of the testbench. You cannot use `this` because we are calling set from a module, not a class instance.
- A config `set` made in the top level module is not automatically updated, you must explicitly call a config `get` to extract the value from the config database (see next slide).

If you use pathname wildcards and a consistent virtual interface field name, then you can update both driver and monitor virtual interfaces in the same `set` call.

The config pathname uses a wildcard (`*`) after the UVC instance name (`yapp`) to access all subcomponents of the UVC. Although not an issue for the YAPP UVC (which only has a single agent component) this is a more robust approach for UVCs with conditional or multiple agent subhierarchy (e.g., the Channel and HBUS UVCs).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Getting the Virtual Interface

```
class yapp_tx_driver extends uvm_driver#(yapp_packet);
  ...
  virtual interface yapp_if vif;

 function void connect_phase( uvm_phase phase);

   if ( ! uvm_config_db#(virtual interface yapp_if)::get(
                          this,     // context
                          "",       // instance name (empty string)
                          "vif",    // field name
                          vif )     // value
        )
     `uvm_error("NOVIF", "Missing virtual I/F" )
  endfunction
...
```

Could be connect or build phase

Field can be arbitrary but must match `set`

Error severity allows multiple `get` fails to be detected

The interface `set` requires an explicit `get` in the driver/monitor components

- `get` returns a bit value 1 if successful – should be checked!
- `get` has the same arguments as `set`
  - If the context is `this`, the instance name can be an empty string

183 © Cadence Design Systems, Inc. All rights reserved.

cadence

A top level (global) set requires an explicit `get` call anywhere the virtual interface is required.

```
static function bit get( uvm_component  cntxt, string  inst_name,
  string  field_name, ref  T  value )
```

`get` has the same arguments as `set`.

`cntxt` – component context from which the set is being done. Usually mapped to `this`. `get_full_name`() of the context is implicitly added to the instance name.

`inst_name` – both the context and instance name must provide a path to the instance containing the field to be updated. If get is called from the instance containing the field, and context is this, then the instance name is not required and can be set to the empty string ""

`field_name` – target field string name in the target instance. A consistent virtual interface name will make assignment easier.

`value` – virtual interface name. A consistent virtual interface name make assignment easier.

Note that when we use an explicit `get` for a corresponding `set`, the field name argument does not have to match the configuration property name. The field name can be any arbitrary identifier (since the explicit `get` does the assignment to the config property) but *must* match the field name used in the `set`.

`get` returns a bit value 1 if successful. You should explicitly check this return value to detect a get failure before the `run_phase` begins. Otherwise you will get null pointer run-time errors when the monitor or driver tries to access the virtual interface.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Convenience Types for Interface Assignment

yapp_pkg.sv

```
package yapp_pkg;
...
  typedef uvm_config_db#(virtual interface yapp_if) yapp_vif_cfg;

`include "yapp_packet.sv"
...
```

Declare convenience type

`uvm_config_db` is very type-dependent.

Declare a `typedef` for the interface configuration

- Declare in UVC package for visibility in both module and UVC
- Need to know the type name to connect UVC

```
module top;
  ...
  import yapp_pkg::*;
  yapp_if in0(clock, reset);

 initial begin
   yapp_vif_cfg::set( null, "*.tb.yapp.*", "vif", in0);
...
```

Use type in `set` and `get`

```
class yapp_tx_driver extends uvm_driver#(yapp_packet);
  ...
  if ( ! yapp_vif_cfg::get(this, "", "vif", vif ) )
...
```

184 © Cadence Design Systems, Inc. All rights reserved.

cadence

The issue with `uvm_config_db` is type sensitivity. If the type parameters do not match, the config setting will fail.

To help in readability and debug, it is strongly recommended that you declare a `typedef` for every interface type. This also makes the `set` and `get` calls simpler. The declaration must be placed where it is visible to both the top level module (for `set`) and UVC driver and monitor (for `get`). Declaring the `typedef` in the UVC package file is an ideal location.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Virtual Interface Limitations

Virtual interfaces cannot *write* to nets

- As procedural assignment to a net (`wire`) is illegal
- Bidirectional connections must be declared as `wire`

Nets *and* variables can be read via a virtual interface

Solution is to define two representations for an interface bidirectional connection

1. `wire` (`hdata_w`) for mapping to DUT `inout` port *and* for reading via virtual interface
2. `logic` (`hdata`) for writing from virtual interface

An `assign` statement is needed to drive `logic` writes onto `wire`

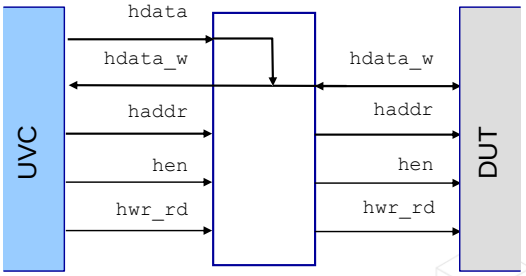
```
interface hbus_if (...);
  logic            hen;
  logic            hwr_rd;
  logic      [7:0] haddr;
  // for inout port on DUT
  wire       [7:0] hdata_w;

  logic      [7:0] hdata;

  assign hdata_w = hdata;
...
```

`wire` for DUT port and UVC read

`logic` for UVC write



185 © Cadence Design Systems, Inc. All rights reserved.

cadence

Interface variables can only be driven via a virtual interface using procedural assignment. This is a problem for bi-directional connections. They must be declared as nets (`wire`) which precludes driving them via procedural assignment. One solution is for the interface to provide a way of driving the net procedurally. To do this, we need two representations of the net connection in the interface:

A `wire` (`hdata_w` in the HBUS interface). This is for mapping to the `inout` port of the DUT in the module instantiation and for reading the bi-directional connection, via the virtual interface, in the UVC.

A `logic` (`hdata` in the HBUS interface). This is for driving the bi-directional connection, via the virtual interface, from the UVC.

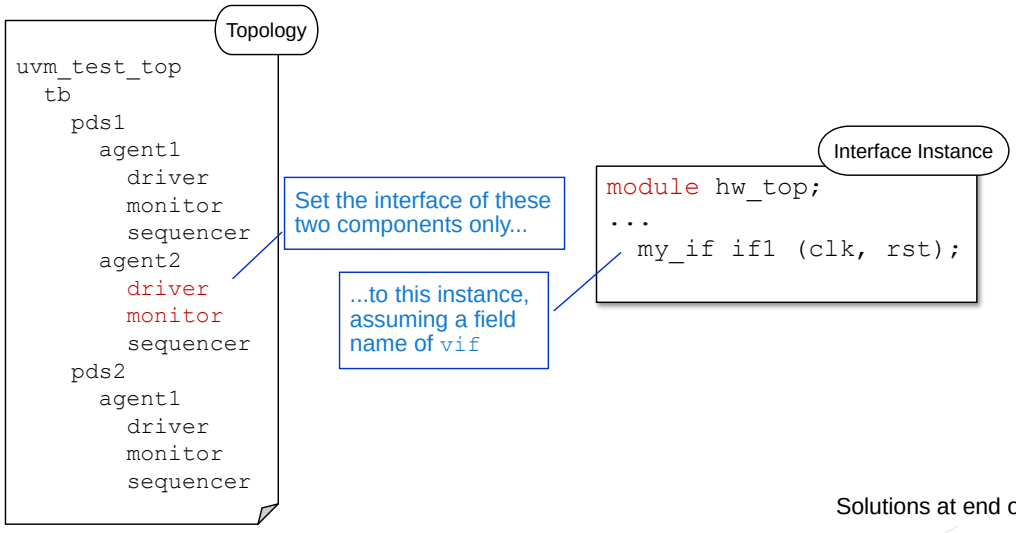
The interface must assign the wire variable from the logic variable in order to allow driven values from the UVC to pass into the DUT.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Connecting to a DUT Quiz

Write the code to set the interface of the highlighted components above to the interface instance, assuming a field name of `vif`.



Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 6 Connecting to the DUT Using Virtual Interfaces

- Modify the driver to use a virtual interface
  - Add protocol specific driver code to handle data injection
- Modify the monitor to collect packets from the signals
  - Using virtual interface
- Instantiate the interface in the hardware top module
- Connect the virtual interface to a physical interface
- Instantiate the DUT in the top testbench

 © Cadence Design Systems, Inc. All rights reserved.

cadence

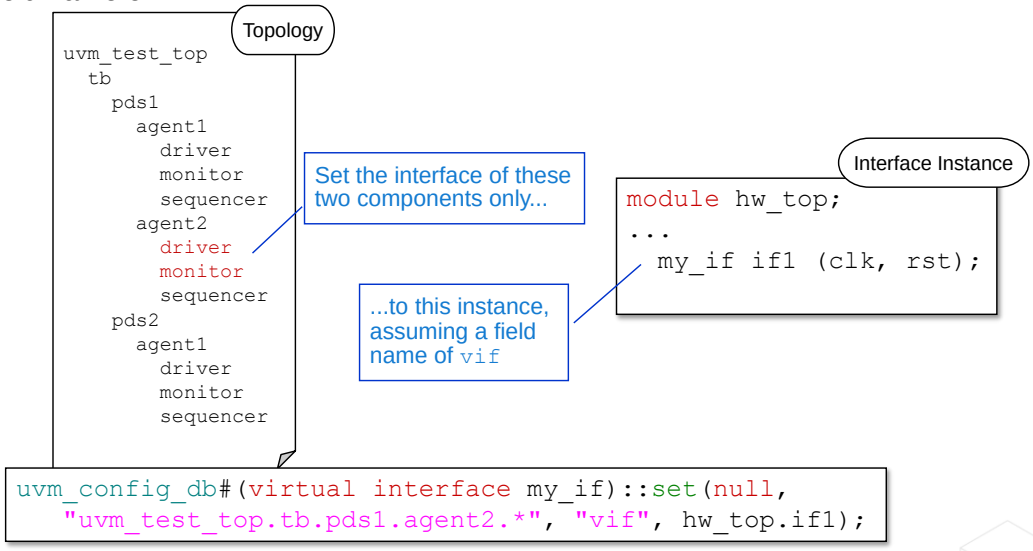
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Connecting to a DUT Quiz

Write the code to set the interface of the highlighted components above to the interface instance, assuming a field name of `vif`.



188 © Cadence Design Systems, Inc. All rights reserved.

cadence

Note the following:

- `uvm_config_db` is type parameterized for the interface type `my_if`.
- Context for set is `null` assuming we are calling this from the `uvm_top` module.
- Instance name is a string from `uvm_test_top`, through the hierarchy to `agent2` of `pds1`, and ends with a wildcard `*` to hit both `driver` and `monitor` in one set call. This assumes the `sequencer` does not access the interface.
- Field name is `vif` according to the comment.
- The value is a SystemVerilog hierarchical pathname through the `hw_top` module to the `if1` instance.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 12

Multiple UVCs

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multiple UVCs Objectives

In this module, you

- Explore the need for multiple testbenches
- Define applications for UVCs with multiple agents
- Instantiate and configure pre-built UVCs into the testbench
  - Channel
  - HBUS
  - Clock and Reset
- Investigate UVC configuration objects

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Environment Integration

So far, our YAPP verification environment has used a single, simple YAPP interface UVC.

Now we will integrate Channel, HBUS and Clock/Reset interface UVCs:

- These are slightly more sophisticated than the YAPP
- Illustrate common instantiation, configuration and compilation issues for reusable UVCs

Later we will integrate module UVCs:

- Multichannel (virtual) sequencer
  - Control stimulus over multiple interface UVCs
- Scoreboard
  - Check input packet comes out on right channel

Remember module UVCs are specific to a certain DUT or DUT configuration

- Not so easily reusable

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Once you begin integrating multiple UVCs in a single environment, you may notice the need for code that spans beyond a single UVC. UVM has envisioned this need and there really are two types of UVCs:

- Interface UVCs
- Module UVCs

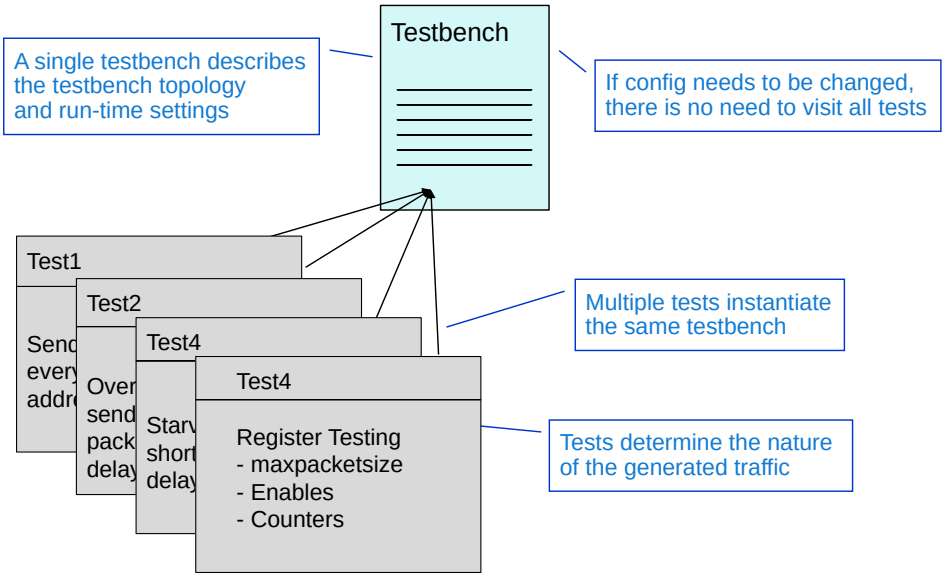
Thus far, you have been working with an interface UVC in your labs. Once you have multiple UVCs to integrate, it's the job of a module UVC to model the behavior across multiple interfaces. We will discuss the difference on the next few slides.

Also, in your next lab, you will be integrating all the UVCs and will get to see this first hand.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

YAPP Router Uses a Single Testbench



 © Cadence Design Systems, Inc. All rights reserved.

Typically there will be one testbench for the DUT, specifying topology and configuration settings to match the DUT.

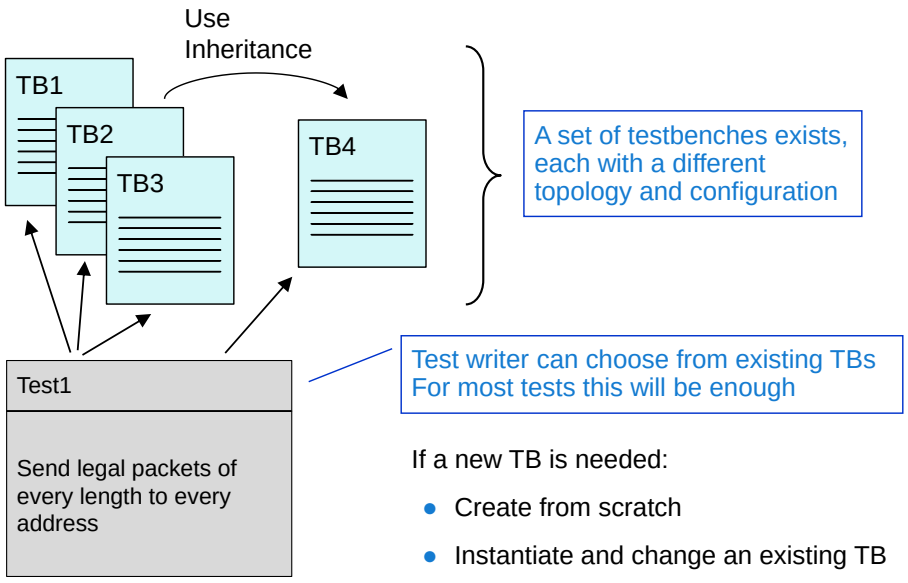
Multiple test classes then run on the single testbench, each carrying out a different verification goal according to the verification plan.

If the DUT changes, or the configuration of the testbench has to change for any other reason, then only the testbench needs to be updated, and the test could still be valid.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Some DUTs Require a Library of Testbenches



193 © Cadence Design Systems, Inc. All rights reserved.

If a DUT comes in many different flavors or configurations, then we will need a library of testbenches with a separate testbench for every different configuration.

For example, if our router came in two variants with three or four output channels, then we would have two separate testbenches, one with three channel UVC's and the other with four.

The test writer can then choose which testbench(es) from the library are suitable for their test.

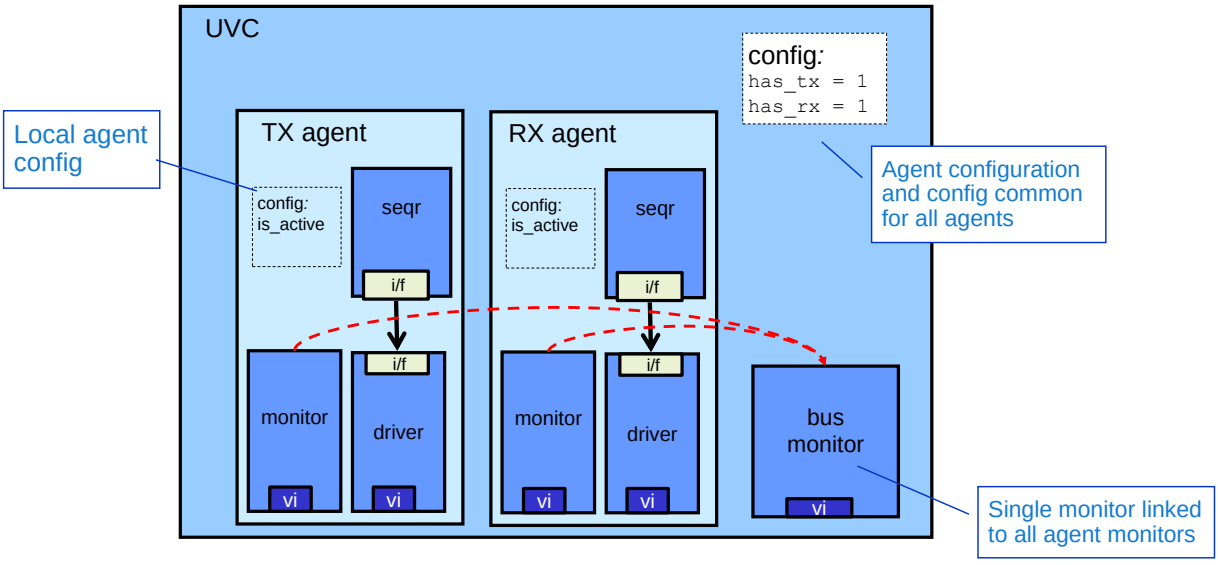
There are several options for developing a new testbench configuration, if one is required:

- A new testbench can be written from scratch.
- We could instantiate an existing testbench and change or reconfigure it s required.
- We could derive a new testbench subclass from an existing testbench and modify it accordingly.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multi-Agent Interface UVCs



Why have multiple agents in an environment?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

It is common to have multiple agents in an interface UVC environment. Agents will have local configuration information (e.g., `is_active`), but the environment will also contain configuration information common to all agents and configuration information for controlling individual agents (e.g., the `has_tx` and `has_rx` properties). Over the next few slides we'll explore these features in more detail.

When you have multiple agents in a UVC, monitors of different agents may be identical. In such cases it is more efficient to avoid duplication by only have one monitor - a bus-level monitor - which can be used by all agents.

The individual agents would declare monitor handles, but not create the instances. The UVC environment would create an instance of the bus-level monitor and then in a `connect_phase()` method, assign the bus-level monitor instance to the agent monitor handles.

© Cadence Design Systems, Inc. All rights reserved.

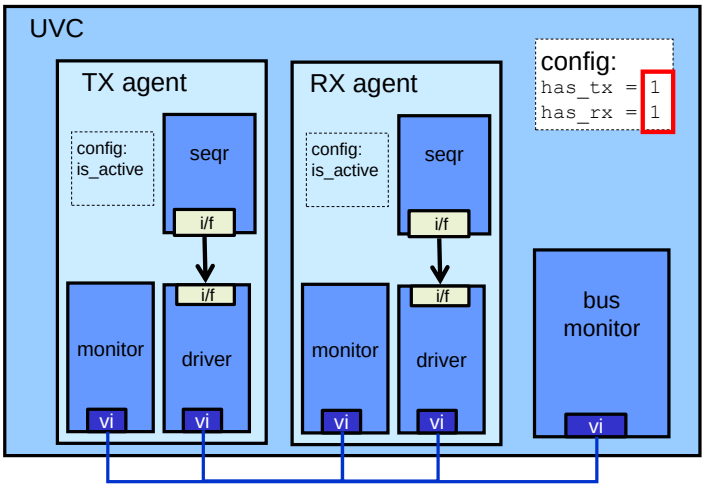
This document is for the sole use of Kun Wang of Seas

Interface UVC Development

With separate TX and RX agents, UVC can be:

- Developed
- Tested
- Packaged for reuse
- Supplied as a self-contained block with built-in verification

All without the need for a working DUT



 © Cadence Design Systems, Inc. All rights reserved.

An interface UVC may be developed long before working RTL code is available for the DUT. Therefore to help in development and testing , both RX and TX agents can be implemented in the UVC. The TX agent sends data onto the interface, and the RX agent reads the same data off the interface. Now, by instantiating both agents, the UVC can be developed and tested standalone, without the requirement for a DUT.

As well as test and development, this UVC configuration is useful for reuse. A reusable UVC can be supplied with test code in the tb directory which instantiates both agents and runs a standalone test. This can be used to verify the UVC is working correctly and demonstrate different configuration and usage options for the UVC.

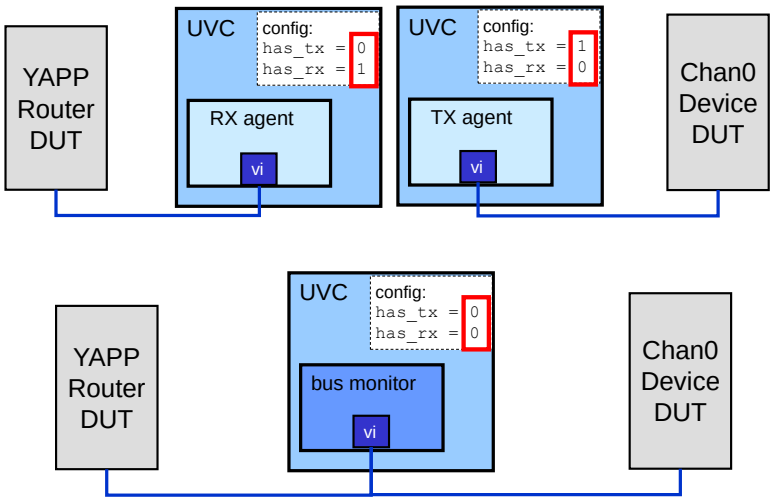
If the TX agent writes transactions to a bus, and the RX agent reads transactions off the same bus, using an identical protocol, then the monitors for both TX and RX agent will be identical. In this case, duplication of the agent monitors can be avoided by instantiating a single bus-level monitor in the UVC, and linking this instance into both agents.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface UVC Reuse in System Development

- Same interface UVC used to develop and test adjacent design blocks
- Same interface UVC used in passive mode during integration



196 © Cadence Design Systems, Inc. All rights reserved.

Since the YAPP and Channel interfaces use the same data item and the same protocol, we could have developed a single UVC with both an RX agent (which receives YAPP packets) and a TX agent (which sends YAPP packets). We could then use the TX agent for the YAPP interface and the RX agent for the Channel interface.

However even if the Channel interface used different data and protocol, a multi-agent TX/RX UVC architecture could still be beneficial. We probably have a system of multiple blocks where the router channels connect to another development block (here called the Chan0 device). The team developing the YAPP router can use a Channel UVC with an RX agent enabled to verify their block outputs. At the same time, the team developing the Chan0 Device can use the same Channel UVC with the TX agent enabled to generate their block inputs.

When the separate blocks are integrated into a system simulation, the Channel UVC can be used in passive mode to monitor the activity between the router and Channel 0 device.

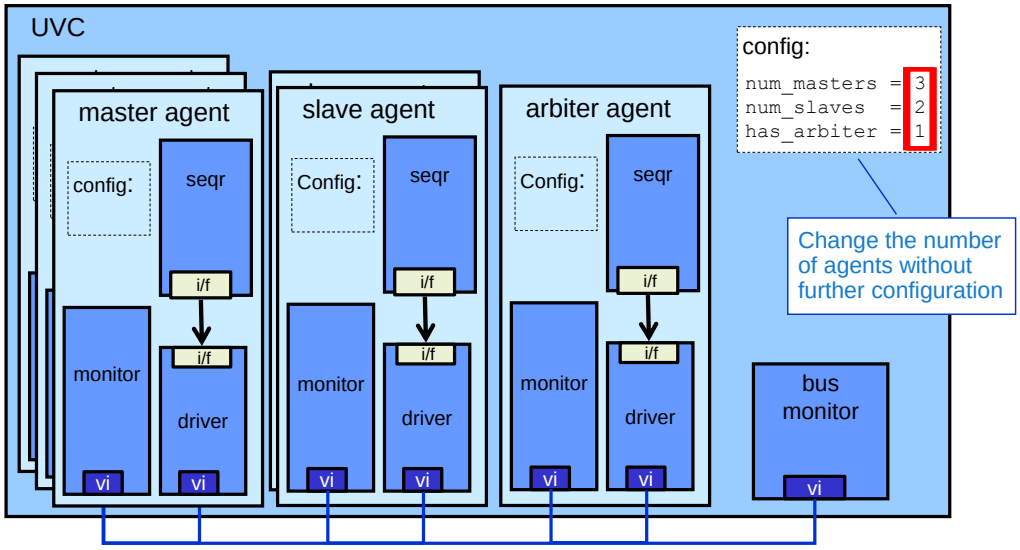
The theory is that since a consistent model of the Channel interface was used to develop and verify both blocks, integration issues in the system level simulation should be minimized.

For simplicity, the Channel UVC used in your lab exercises does NOT have multiple agents.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Complex Interface UVC Flexibility



- Flexibility for reuse

 © Cadence Design Systems, Inc. All rights reserved.

cadence

In some cases, the DUT may be one device on a bus with many other devices. In these cases, you may want to verify the DUT with other devices on the bus, for example to model slave to slave transfers, or the arbitration of many master devices on a bus. In these cases, additional agents can be created in the UVC to mimic the behavior of the other bus devices. Simple configuration settings in the UVC allow the number of created slave, master and arbitration agents to be easily changed, allowing the DUT to be verified under many different bus topographies.

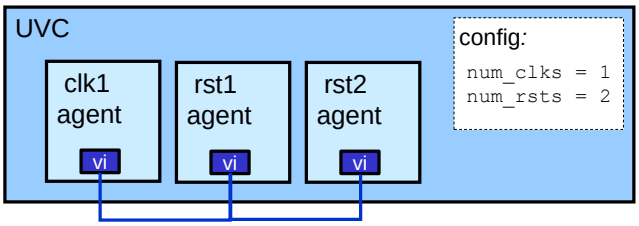
The HBUS UVC for the YAPP router demonstrates this kind of multiple agent UVC. The number of slave and master agents is easily configurable and is implemented using dynamic arrays of agent instances. See the HBUS code for more details.

With a potentially large number of agents, it is more efficient to avoid duplication of identical monitors in multiple agents. Duplication can be avoided by instantiating a single bus-level monitor in the UVC, and linking this instance into all created agents.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Clock and Reset UVC



Clocks and resets could also be controlled through a UVC:

- Configurable generation of multiple clocks through agents
- Generation of multiple resets (soft, hard, Power-On-Reset)
- UVC sequences can now control clock and reset characteristics
  - Period, duty cycle, start/stop timing for clocks
  - Level, delay, duration, timing for example, mid-packet) for resets

A Clock and Reset UVC is essential for acceleration efficiency

198 © Cadence Design Systems, Inc. All rights reserved.

cadence

Generating clocks and reset from a UVC gives improved result with hardware acceleration (see new slide) but there are many reasons for using a UVC to generate clocks and resets beyond acceleration.

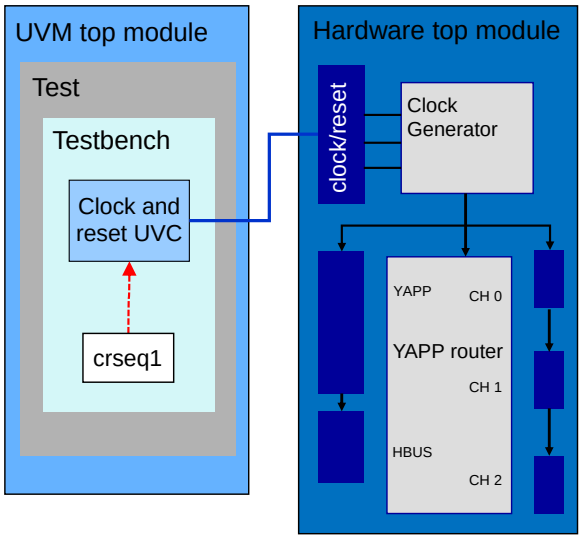
- Agents can generate multiple harmonically-related clock signals under configuration control.
- Agents can generate multiple resets (soft, hard and POR) under configuration control.
- UVC sequences can control clock behaviour, such as period and start/stop times.
- UVC sequences can control reset generation, for example to verify behaviour by generating resets during the middle of a packet transmission.

Reducing hardware/software interaction during simulation is key for the acceleration efficiency, so clock generation should occur in the hardware top module, but clocks can be controlled using UVC sequences.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Clock and Reset UVC for Acceleration



Structure of acceleration-compatible Clock/Reset UVC:

- Sequence controls Clock and Reset UVC
- UVC calls interface methods driving interface signals
- Signals mapped to input ports of clock generator module
- Clock generator outputs mapped to:
  - DUT clock reset ports
  - Other UVC interface ports

Other UVCs access clock and reset through their interfaces

199 © Cadence Design Systems, Inc. All rights reserved.

cadence

The Clock Reset UVC drives the clock and reset signals for the DUT and other UVC interfaces. For compatibility with hardware acceleration, there is recommended structure for the UVC:

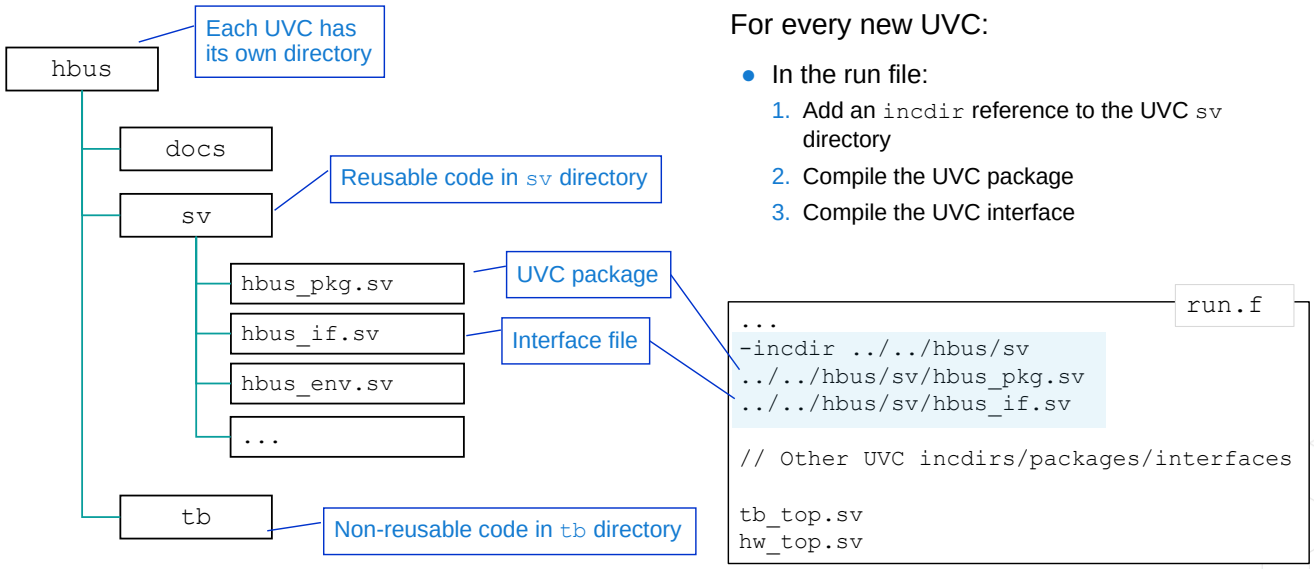
- UVC sequences control the generation of clocks and resets.
- The sequences trigger the UVC to make calls to methods defined in the Clock and Reset interface declared in the hardware top module.
- The interface methods drive interface signals which are mapped to input control ports of a clock generator module, also instantiated in the hardware top module.
- The clock generator drives clock and reset output ports according to values on the input control ports.
- The clock and reset output signals of the generator are mapped to the DUT and to the input ports of other UVC interface instantiations.

Other UVCs access the clock and reset signals from their interface instantiations.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Integrating Multiple UVCs: Compilation



For every new UVC:

- In the run file:
  1. Add an `incdir` reference to the UVC `sv` directory
  2. Compile the UVC package
  3. Compile the UVC interface

run.f

```
...
-incdir ../../hbus/sv
../../hbus/sv/hbus_pkg.sv
../../hbus/sv/hbus_if.sv

// Other UVC incdirs/packages/interfaces

tb_top.sv
hw_top.sv
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Every UVC should have it's own directory from where you can find the information to integrate the UVC into your verification environment. The reusable code for the UVC should be in the sv directory, from where you can find the UVC package and interface files. These need to be added to your run file for compilation. Also you should add an incdir to the UVC sv directory to resolve any includes used in the package. The compilation of every new UVC requires these three lines, as a minimum, in your run file.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Integrating Multiple UVCs: Import and Interface

```
module hw_top();
...
  yapp_if in0(clock, reset);
  hbus_if hb0(clock, reset);
...
```

Instantiate and connect interface

```
module tb_top();
  ...
  import yapp_pkg::*;
  import hbus_pkg::*;
  ...
  `include "router_tb.sv"
  `include "router_test_lib.sv"

  initial begin
    yapp_vif_config::set(null, ...);
    hbus_vif_config::set(null, ...);
    ...
    run_test();
  end
...
```

Import UVC package...

...before testbench and test includes

Set interface config – check UVC package for typedef

```
package hbus_pkg;
...
typedef uvm_config_db#(virtual hbus_if) hbus_vif_config;
...
```

For every new UVC:

- In the hardware top module `hw_top`:
  1. Instantiate the interface and connect to the DUT
- In the UVM top module `tb_top`:
  1. Import the UVC package *before* the testbench
  2. Add a configuration set to connect the interface

201 © Cadence Design Systems, Inc. All rights reserved.

cadence

For every new UVC, in the hardware top module `hw_top`, you need to instantiate the UVC interface and connect it to the DUT ports. In the UVM top module, `tb_top`, you need to import the UVC package, before the testbench as the testbench references the UVC. You also need to add a configuration set to connect the UVC to the required interface instantiation. You should check the UVC package to see if the developer has defined a typedef which you can use in the set.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Integrating Multiple UVCs: Configuration and Instantiation

```
class router_tb extends uvm_env;
...
  hbus_env hbus;
...
  function void build_phase(uvm_phase phase);
...
   uvm_config_int::set(this, "hbus", "num_masters", 1);
   uvm_config_int::set(this, "hbus", "num_slaves", 0);

   hbus = hbus_env::type_id::create("hbus", this);
...
```

Configure before creation

```
class hbus_cfg_obj extends uvm_object;

  int unsigned num_masters = 0;
  int unsigned num_slaves = 0;

  `uvm_object_utils_begin(hbus_cfg_obj)
  ... // property automation
  // data constructor
...
```

Configuration object for all config properties

Configure UVC *before* creating instance

Complex UVCs can have many config properties

Properties could be declared in a separate container

- Configuration object

Advantages

- All configuration is in one place
  - Good for readability and reuse
- Properties can be randomized
  - With dependency constraints
- Can inherit configuration object
  - Easily layer new policies and properties onto existing code

202 © Cadence Design Systems, Inc. All rights reserved.

cadence

Remember to set any config properties of the UVC before instantiation.

Complex UVCs may have a large number of config properties, the values of which may be dependent. For example, if there is more than one master agent, then an arbiter agent is required.

In such cases, it may be more convenient to use a separate container (a configuration object) for the UVC config properties. The configuration object is a data item so it has a data constructor and object utility macro. You should also automate (declare field macros for) the properties to help debug.

The configuration object class file should be added to the UVC package.

Having all the configuration properties for a UVC declared within one object is good for readability and reuse. The user can read a single file to understand all the available switches and options for the UVC, rather than relying upon documentation or searching through the code.

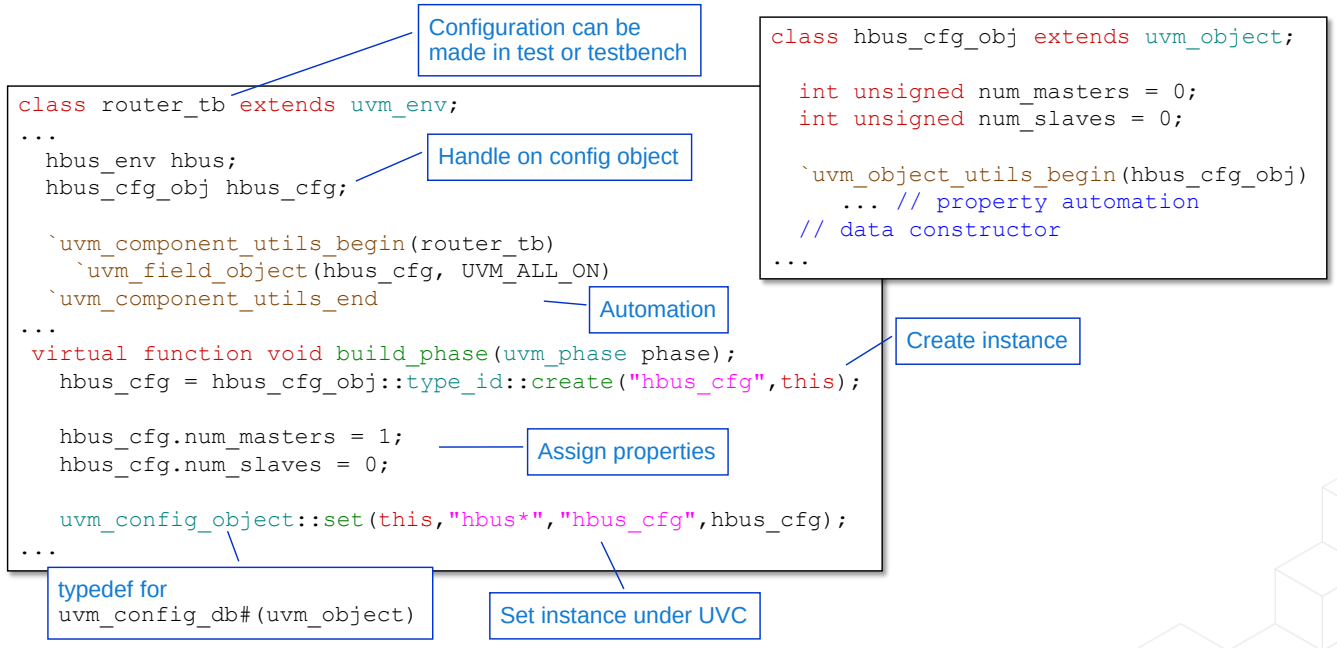
A configuration object can allow configuration properties to be randomized (by declaring them as rand). Constraint blocks can be added to the object to maintain inter-dependencies between the random properties.

If a user needs to add further policies or properties to a UVC, perhaps in support of a new driver subclass, then a configuration object subclass can be declared and used within the UVC by applying a factory type override.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Configuration with a UVC Configuration Object



203 © Cadence Design Systems, Inc. All rights reserved.

To configure a UVC with a configuration object, create an instance of the configuration object; assign values to the object properties and then set the object instance into the configuration database. This is usually done in the testbench, but can also be done in a test class to overwrite the testbench settings (see next slide).

This example uses automatic matching for the configuration object, where the `hbus_cfg` handle is automated in every hierarchical level it is accessed. Automatic matching means `hbus_cfg` is updated automatically by `apply_config_settings` called by `super.build_phase`. However this technique requires:

- The set call to use `uvm_config_object::set`.
- The set call to be made from a UVM class, not a module. Therefore the configuration object cannot contain a virtual interface.
- The configuration object handle `hbus_cfg` to be automated in every level where it is accessed.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Automatically Reading a UVC Configuration Object

```
class hbus_env extends uvm_env;

  hbus_cfg_obj hbus_cfg;

 `uvm_component_utils_begin(hbus_env)
    `uvm_field_object(hbus_cfg, UVM_ALL_ON)
 `uvm_component_utils_end
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    if (hbus_cfg == null) begin
      `uvm_warning("HBUS_CFG", "No config object found for HBUS - using default")
      hbus_cfg = hbus_cfg_obj::type_id::create("hbus_cfg",this);
    end
...
```

Top level of UVC

Handle on config object

Automation

Automatic matching: Config db read here

If no instance found, create a default instance

UVC should check config object exists

- Create a default instance if it doesn't

The same technique can be used in the testbench

- Allows test class to overwrite testbench configuration

Config object could also contain a virtual interface

- Must be set in `tb_top` module
- Needs explicit `get` to read
  - Cannot be read automatically

 © Cadence Design Systems, Inc. All rights reserved.

cadence

As we are following the conditions for automatic matching (see previous slide), `hbus_cfg` is updated automatically when we execute `super.build_phase`.

With automatic matching the UVC should check to see if the object exists. If the UVC is not configured, the handle will be `null`. The UVC can then create a default config object or raise an error.

A configuration object may be accessed at multiple hierarchical levels (test, testbench, UVC, agent..). Therefore this technique can be used at each level. If the handle is `null`, then the local class creates the config object. If the handle is not `null`, the class assumes a higher scope has already defined the UVC configuration and so does not create it's own instance. This allows the normal configuration precedence order for writes (highest component has highest precedence).

The configuration database stores a pointer to the configuration object. Multi-level access is therefore to the same object reference. If a configuration object is accessed in multiple levels, this can cause race conditions (incorrect order of read/writes) and side-effects (a write from one layer causes problems with another). However by only including static configuration data in the object (i.e. not updated during run-time) and restricting access to `build_phase` only (with a predictable top-down execution) these can be avoided.

In some cases, the virtual interface is also declared in the config object. However then the config instance must be created and set in the `tb_top` module which then requires an explicit `get` to read the config object. See the Connecting to a DUT module for more details.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multiple UVCs Quiz

When integrating a UVC, what are the:

- Three additions to be made in the top modules?
- Three additions to be made in the run file?

Solutions at the end of the module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

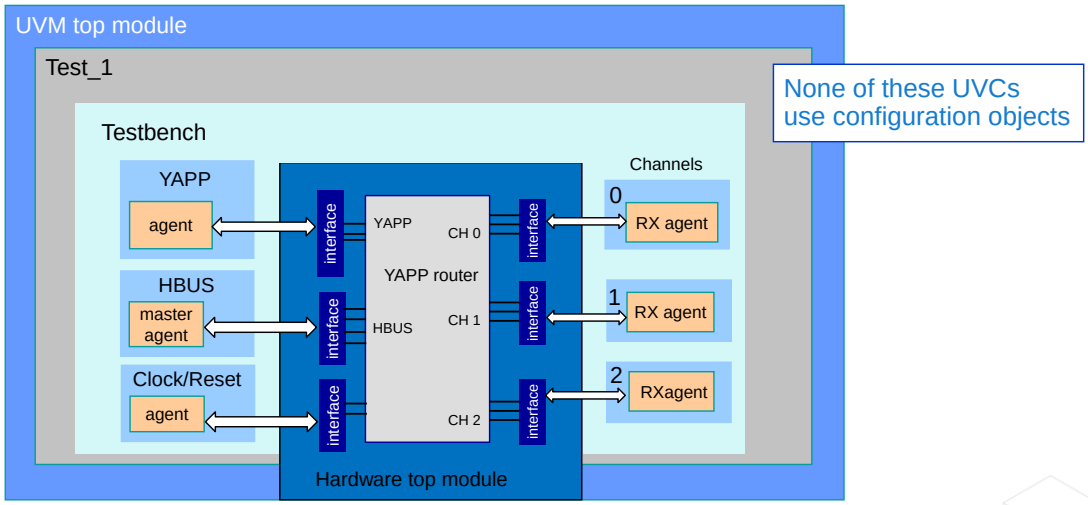
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 7 Multiple UVCs

- Connect, configure and test HBUS, Channel and Clock/Reset UVCs



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Multiple UVCs Quiz

When integrating a UVC, what are the:

- Three additions to be made in the top modules?
  - Instantiate the interface in `hw_top`
  - Import the UVC package in `tb_top`
  - Add a configuration set to connect the interface in `tb_top`
- Three additions to be made in the run file?
  - Add an `incdir` reference to the UVC `sv` directory
  - Compile the UVC package
  - Compile the UVC interface

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 13

Multichannel (Virtual) Sequences

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multichannel (Virtual) Sequences Objective

In this module, you

- Control stimulus in multiple UVCs through a multichannel sequencer by:
  - Creating and connecting multichannel sequencers
  - Defining multichannel sequences

Multichannel sequencers and sequences are also known as virtual sequencers and sequences.

 © Cadence Design Systems, Inc. All rights reserved.

The original term for a sequencer and sequence that controls multiple UVCs was virtual. However, this term was considered ambiguous, given the widespread use of the virtual term in classes.

Therefore, the official term used in the IEEE UVM standard is multichannel.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multichannel Sequences

The most typical sequences interact with a single DUT interface via the UVC driver

- We call these UVC sequences

Why would we need a sequence to coordinate more than one UVC?

- You might need to coordinate activity across several UVCs
- You might distribute a data stream over several input ports

Sequences that can control multiple UVCs are called multichannel sequences

- Also sometimes called Virtual Sequences

A multichannel sequence tells UVCs which sequence to run and when

 © Cadence Design Systems, Inc. All rights reserved.

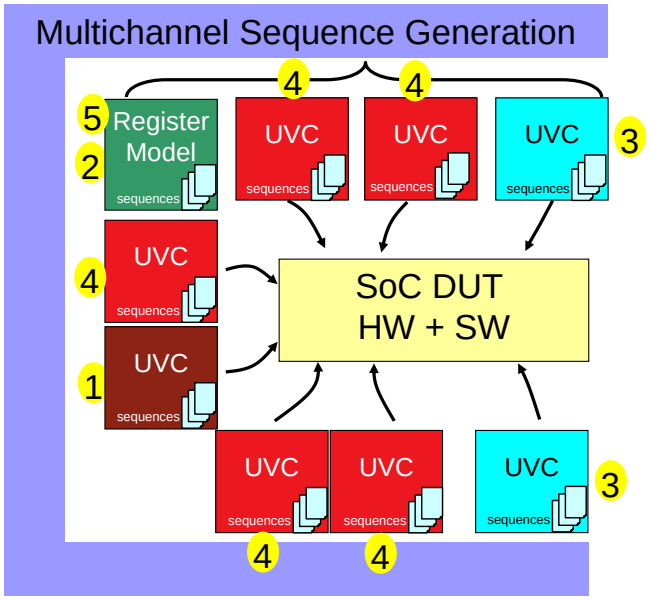
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why You Might Need Multichannel Sequences



Example usage (test flow):

1. Reset the device
2. Configure register space
3. Drive simultaneous traffic on two channels
4. When a particular condition is met, drive the remaining interfaces
5. Check status registers

Same actions are done for EVERY testcase

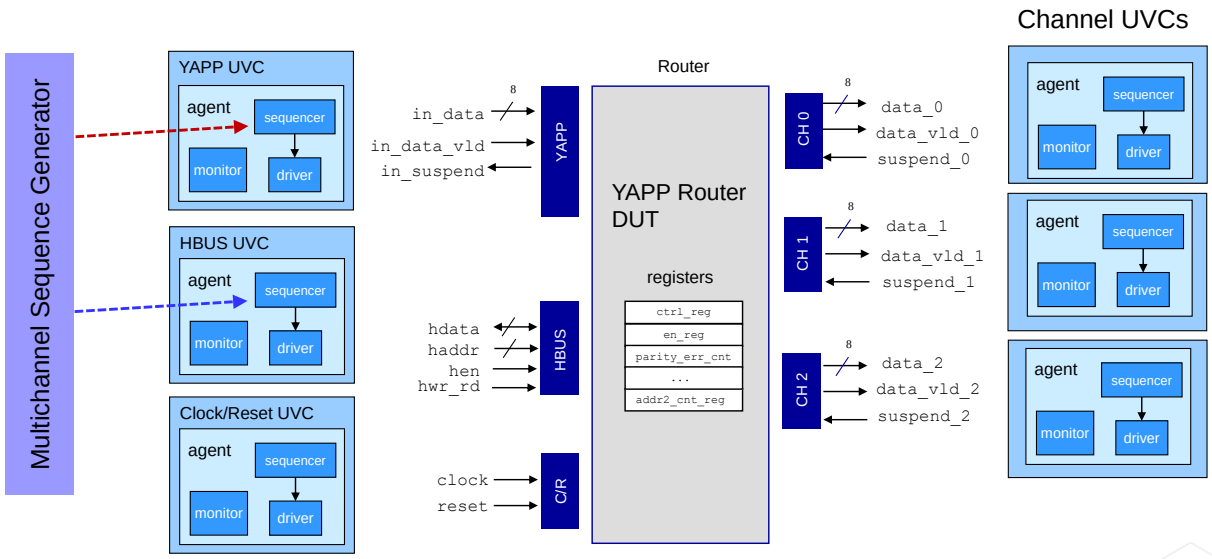
211 © Cadence Design Systems, Inc. All rights reserved.

This a high-level view showing how an SoC DUT can be simultaneously bombarded at all interfaces in an automated and coordinated way, and visualized. This methodology has been proven to take 1/10 the time to compose and deploy SoC verification environments.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Adding a Router Multichannel Sequencer



 © Cadence Design Systems, Inc. All rights reserved.

cadence

For our router, we need to coordinate the stimulus from the YAPP and the HBUS UVCs. For example, in a test we may want to do the following (in order):

- Send HBUS transactions to enable the router and set the maximum packet size to 64.
- Send ten packets from the YAPP UVC to check basic operation.
- Set the router maximum packet size register to 20 via the HBUS UVC.
- Use the YAPP UVC to send packets of sizes less than, equal to and greater than 20 to check oversized packets are correctly dropped.

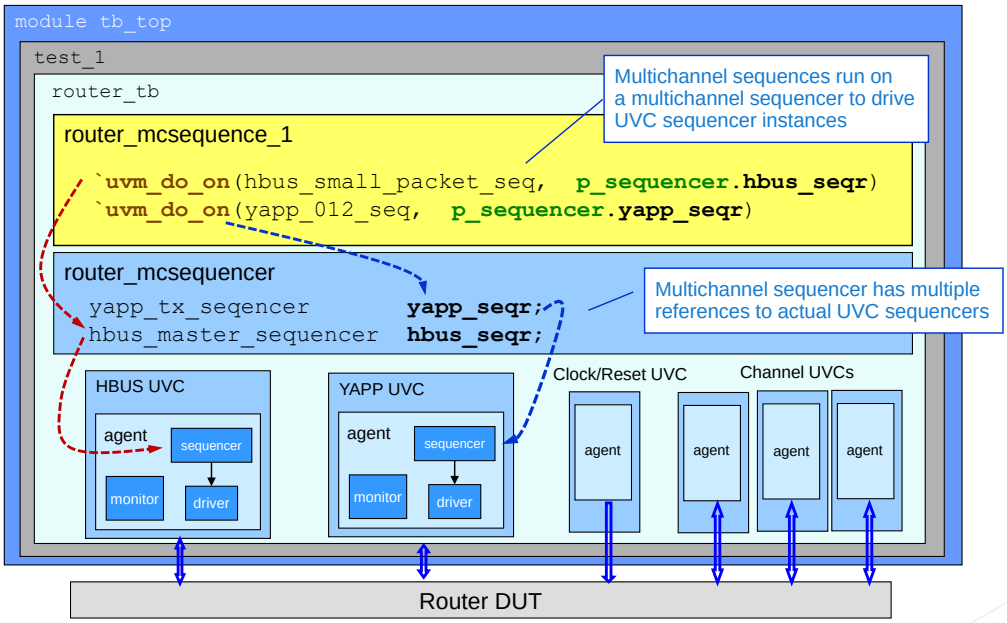
The Channel UVCs always run a single response sequence, so do not need to be controlled via the multichannel sequencer.

In our tests, the Clock and Reset UVC also only runs a single sequence, so this does not need to be controlled by the multichannel sequencer either. However there may be situations where you do want to include clock and reset generation in a multichannel sequence, for example to verify reset behavior during stimulus; check soft resets or switch clocks on or off.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Router Multichannel Sequencer Overview



213 © Cadence Design Systems, Inc. All rights reserved.

cadence

A multichannel sequencer is instantiated as a component of the testbench. The sequencer contains handles for the UVC sequencers which will be controlled by multichannel sequences.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Procedure for Adding a Multichannel Sequencer

1. Declare a multichannel sequencer component:
   - Derive from `uvm_sequencer` like a normal sequencer
   - Contain one or more references to UVC sequencer instances
2. Declare multichannel sequences:
   - Contain references for UVC interface sequences (that will be invoked)
   - Sequences are activated using `` `uvm_do_on() ``, and selecting a UVC sequencer reference
3. In the testbench:
   - Create the multichannel sequencer instance
   - Connect the sequencer references to actual UVC sequencer instances
4. In the test class:
   - Set the sequence of the multichannel sequencer to the required choice
   - Remove any `default_sequence` setting for the UVC
     - Control is no longer local

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Declaring a Multichannel Sequencer

UVC sequencer references are used as pointers to actual UVC sequencers.

- Do **not** create these in build

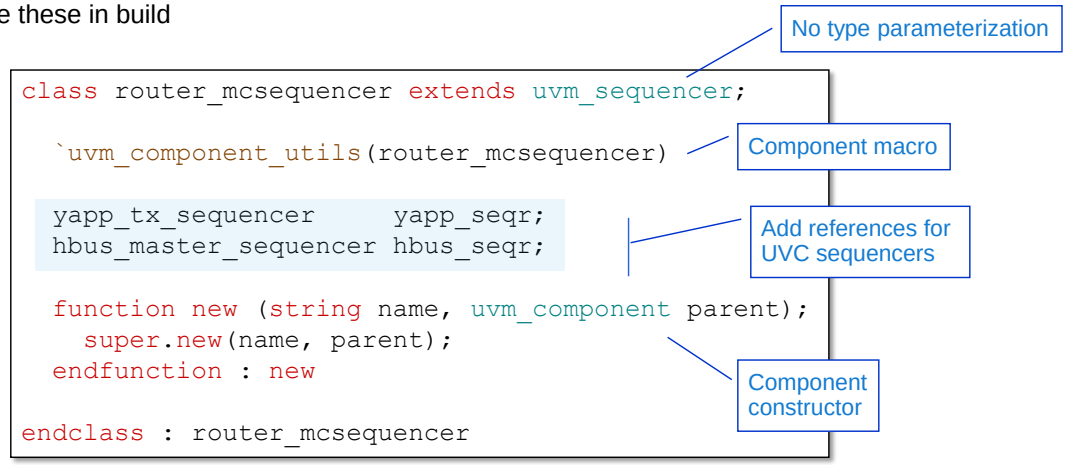
```
class router_mcsequencer extends uvm_sequencer;

  `uvm_component_utils(router_mcsequencer)

  yapp_tx_sequencer      yapp_seqr;
  hbus_master_sequencer hbus_seqr;

  function new (string name, uvm_component parent);
    super.new(name, parent);
  endfunction : new

endclass : router_mcsequencer
```



 © Cadence Design Systems, Inc. All rights reserved.

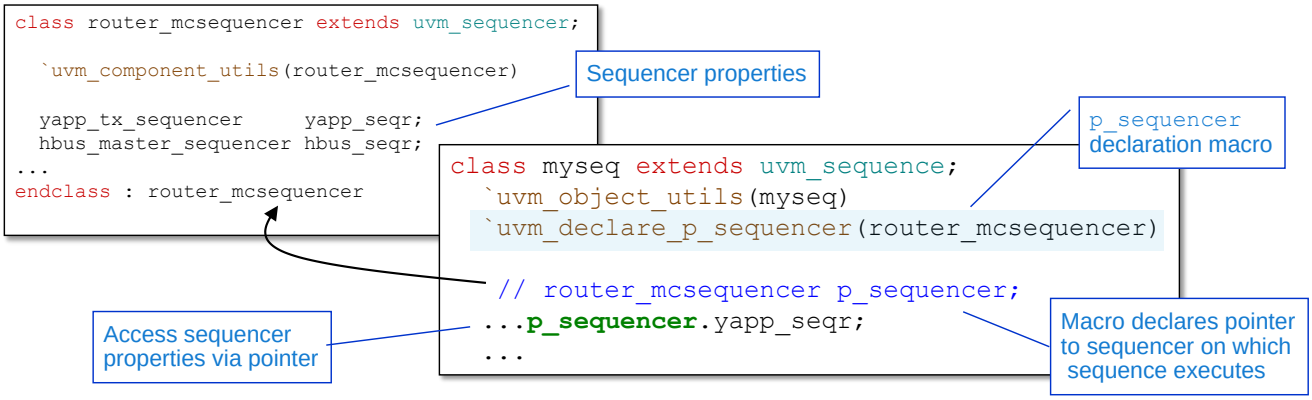
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

p_sequencer Variable



`p_sequencer` variable allows a sequence to access properties of its sequencer

- Must be declared via a macro:
  - Argument to macro is class name of sequencer
- Link is made when the sequence is executed on the sequencer

Occasionally used in normal sequences, but essential for multichannel sequences

216 © Cadence Design Systems, Inc. All rights reserved.

`p_sequencer` is a sequence variable which allows the sequence to access properties declared in the sequencer on which the sequence will run.

This is useful in normal sequences for certain applications. For example the sequence stimulus may be dependent upon a configuration property of a UVC, such as a source address. The address can be declared as a property of the sequencer and set in configuration of the UVC. The sequence can then access it's sequencer address using the `p_sequencer` variable.

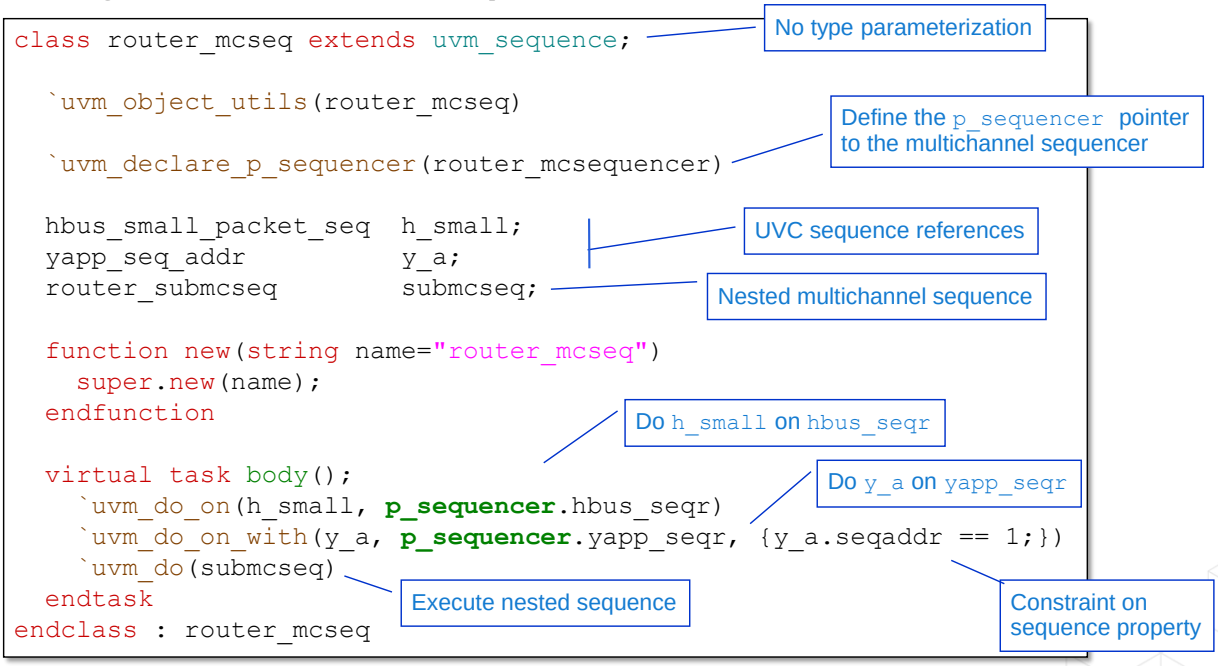
In multichannel sequences, the `p_sequencer` is essential, as it allows the multichannel sequence to access the UVC sequencer references declared in the multichannel sequencer, and therefore define which UVC sequence to run on which UVC sequencer.

The variable can be declared using the `` `uvm_declare_p_sequencer `` macro. The argument to the macro is the name of the sequencer class on which the sequence will run. The link is made when the sequence is executed on a sequencer. Therefore the same sequence can be run on multiple instances of a UVC, all with different sequencer property values.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Declaring a Multichannel Sequence



217 © Cadence Design Systems, Inc. All rights reserved.
cadence

A multichannel sequence extends from `uvm_sequence`, but without any type parameter, as it will be creating sequences of many different types.

The multichannel sequence is an object, like a normal sequence, and so has an object utility macro and data constructor.

The multichannel sequence should include the `p_sequencer` declaration macro to define a built-in pointer to the sequencer on which the sequence is executed. This allows the sequence to see the UVC sequencer handles declared in the multichannel sequencer.

Handles must be declared for all UVC sequences which will be called.

In the body of the multichannel sequence, we use a `` `uvm_do_on `` sequence macro to select a UVC sequence handle and execute it on the appropriate UVC sequencer pointer, accessed from the multichannel sequencer by using the `p_sequencer` pointer.

Additional constraints for UVC sequence random properties can be defined by using the `` `uvm_do_on_with `` option.

Nested multichannel sequences can be called by simply using the `` `uvm_do `` macro, just like for a normal nested sequence call.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multichannel Sequence Objections

```
class base_mcseq extends uvm_sequence;
...
  virtual task pre_body();
    if (starting_phase != null)
      starting_phase.raise_objection(this, get_type_name());
  endtask
   ...
  virtual task post_body();
     if (starting_phase != null)
       starting_phase.drop_objection(this, get_type_name());
  endtask
...
```

Can declare base multichannel sequence for inheritance

Raise objection in `pre_body`

Drop in `post_body`

Remember to add objection control to multichannel sequences:

- Raise and drop objections
  - Either in the sequence `body()`
  - Or define a base multichannel sequence with objection handling in `pre/post_body()`
- Check you have a drain time specified in the testbench

218 © Cadence Design Systems, Inc. All rights reserved.

cadence

Multichannel sequences need objection handling to control the end of simulation.

If a sequence is call via a `` `uvm_do `` variant, then it is defined as a subsequence and it's `pre/post_body()` methods are not executed. If objection handling is managed in these methods, then there will be no objections raised for the subsequence, and the multichannel sequence must handle objections instead.

Objections can be raised and dropped for a multichannel sequence at the beginning and end of `body()`, or, as in this example, a base multichannel sequence class can be declared which raises objections in `pre_body()` and drops the objection in `post_body()`. All multichannel sequences can then be extended from the base to inherit the objection control.

Remember the default drain time is zero. Therefore a drain time should be set for the simulation, either via a `set_drain_time()` call or an `all_dropped()` definition. See the chapter on sequences for more information.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Instantiating a Multichannel Sequencer

```
class router_tb extends uvm_env;
  ...
  hbus_env hbus;
  yapp_env yapp;
  router_mcsequencer mcseqr;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    hbus = hbus_env::type_id::create("hbus",this);
    ...
    mcseqr = router_mcsequencer::type_id::create("mcseqr",this);
  endfunction: build_phase

  virtual function void connect_phase(uvm_phase phase);
    mcseqr.hbus_seqr = hbus.masters[0].sequencer;
    mcseqr.yapp_seqr = yapp.tx_agent.sequencer;
    ...
  endfunction: connect_phase
...
```

Testbench

Multichannel sequencer handle

Create instance

Connect multichannel sequencer references to UVC sequencer instances

219 © Cadence Design Systems, Inc. All rights reserved.

cadence

The multichannel sequencer is created in the testbench, local to the UVCs it will control.

It is a component, so it needs to be created in the build phase of the testbench.

In the connect phase, the sequencer references of the multichannel sequencer must be assigned to the actual UVC sequencer instances using hierarchical pathnames. This assignment is a reference copy – the pointer for the UVC sequencer instance must be copied to the sequencer handle declared in the multichannel sequencer. The pathname uses handle names (not instance names) and is not a string, (unlike a configuration pathname) and so cannot contain wildcard characters.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Setting the Multichannel Sequence

```
class router_mctest extends base_test;
  // component macro
  // component constructor
  router_tb tb;

  function void build_phase(uvm_phase phase);
    ...
    uvm_config_wrapper::set(this, "tb.mcseqr.run_phase",
                            "default_sequence",
                            router_mcseq::get_type());

    uvm_config_wrapper::set(this,
                            "tb.chan?.*agent.sequencer.run_phase",
                            "default_sequence",
                            channel_rx_resp_seq::get_type());
   super.build_phase(phase);
    ...
  endfunction : build_phase
...
```

Test class

Set default sequence of multichannel sequencer

Set sequence of UVCs not controlled by multichannel sequencer

- Remember to remove any existing sequence settings for the UVCs now controlled by the multichannel sequencer

220 © Cadence Design Systems, Inc. All rights reserved.

cadence

You must specify which multichannel sequence to execute on the sequencer. As with a UVC sequencer, there are typically two ways to do this:

- Default sequence. A standard config call is used to set the default sequence of the multichannel sequencer.
- Test class start call. Use the start method to execute the sequence on the sequencer from a test class run phase.

Normally you would remove any default sequence setting for any UVCs which are now controlled by the multichannel sequencer (e.g., YAPP and HBUS). However this is not compulsory and some verification strategies (e.g., resets or interrupts) use multiple sequence settings for a given sequencer.

UVCs which are not controlled by the multichannel sequencer will still need a sequence setting (e.g., Channel). If a UVC always executes the same sequence, then the sequence setting could be placed in the testbench or a base test class so that it doesn't need to be set in each test.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multichannel Sequences Quiz

1. What is the purpose of a multichannel sequencer?
2. In which component is a multichannel sequencer instantiated?
3. What does the `p_sequencer` handle reference?
4. Name a UVM macro used to execute a UVC sequence from a multichannel sequencer.

Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 8 Writing Multichannel Sequences and System-Level Tests

- Modify Lab 7 to add a multichannel sequencer and execute HBUS and YAPP sequences

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Multichannel Sequences Quiz

1. What is the purpose of a multichannel sequencer?
   - To coordinate stimulus activity over multiple UVCs.
2. In which component is a multichannel sequencer instantiated?
   - Testbench
3. What does the `p_sequencer` handle reference?
   - The sequencer on which the sequence containing the handle is executed.
4. Name a UVM macro used to execute a UVC sequence from a multichannel sequencer.
   - The ones we looked at in the presentation are:
     - `` `uvm_do_on ``
     - `` `uvm_do_on_with ``

   There are others – please see the UVM class reference.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 14

Building a Scoreboard

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building a Scoreboard Objectives

In this module, you

- Implement a scoreboard
- Declare, instantiate and connect a TLM analysis interface, to pass packet information from UVCs to the scoreboard
- Declare clone and compare operations for the scoreboard

 © Cadence Design Systems, Inc. All rights reserved.

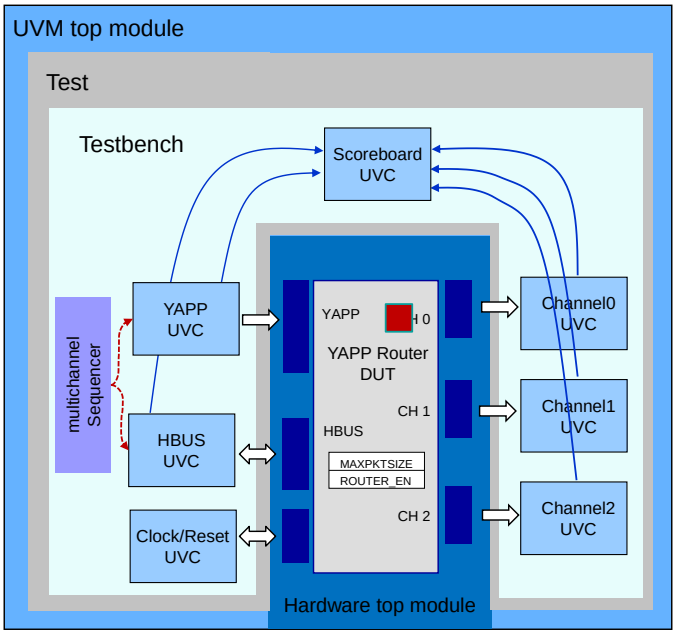
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Adding a Scoreboard



- Add a scoreboard component to the testbench
- Connect to the YAPP & Channel UVC monitors

- Input YAPP packets are sent to the scoreboard
- Output Channel packets are sent to the scoreboard
- Scoreboard compares data to check the router

HBUS transactions can also be captured

- Scoreboard must account for a disabled router or oversized packets

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

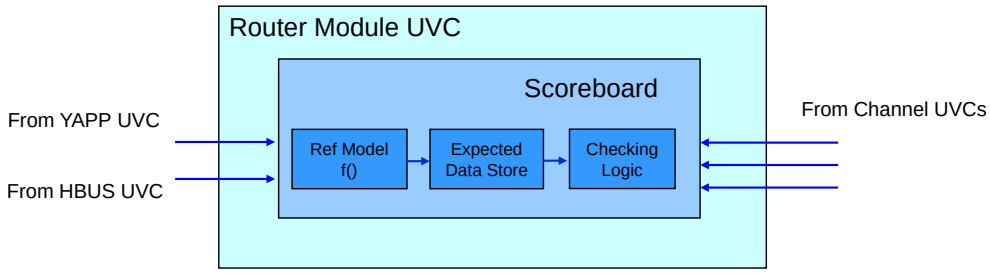
Scoreboard Architecture

There are three distinct functions in the scoreboard:

1. Transfer function (reference model)
2. Expected data storage mechanism
3. Checking logic

There may be other router-level checking and coverage functions:

- Scoreboard is one component of a router module UVC



 © Cadence Design Systems, Inc. All rights reserved.

cadence

The purpose of a scoreboard is to compare actual output data from the DUT against expected output data derived from the inputs to the DUT.

To achieve this, a scoreboard typically contains three main functions:

- A reference model or transfer function. This model mimics the behavior of the DUT in transforming the input data to obtain the expected output data. This is usually a high-level C++, SystemC or SystemVerilog model.
- Internal storage. The DUT may take hundreds of clock cycles to generate the output from a given input data item, whereas the reference model is typically untimed. Therefore some form of storage (and synchronisation) is required to hold the expected data until the corresponding actual data is available at the DUT outputs.
- Comparison logic. Checks that the actual output data matches the expected data. This may be as simple as a equivalence operator, or a complete functional model.

As well as these key functions, a scoreboard may also contain additional coverage and checking functionality or the scoreboard may be just one component of a complex hierarchical module UVC which contains other components for checking and coverage.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Scoreboard: Things to Consider

Transfer function:

- Reuse the architectural model from the C or SystemC® environment
  - Signal processing (for example, video/audio applications)
  - Encryption

Expected data store:

- Storage requirements have to be efficient: thousands of transactions
- Lookup efficiencies:
  - Queues are OK when output packets are same order as input packets
  - Associative array is much better when output packets can come out in a different order
    - Indexing key can be a issue...

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The checking logic of the scoreboard may be straightforward, but the transfer function and data store functions typically require some consideration and planning for the most effective implementation.

For an intensive data processing DUT, such as signal processing or encryption designs, the transfer function will be critically important for the scoreboard. You may be lucky enough to have access to a high level system model of the DUT function in C or SystemC which could be reused. It may be that the verification team has to create a reference model from the design specification. Obviously this should not be done in collaboration with the DUT design team to avoid duplication of errors.

Efficiency is important for the expected data store – the scoreboard may need to store thousands of transactions, so this implies the use of dynamic storage such queues or associative arrays. Queues are useful when the output data is in the same order as the input data, so the queue can be used as a FIFO. If the output data is not in the same order as the input, an associative array is better. However you need to find or generate a unique key (or address) for each packet. The key is used to write the input (or transformed input) packet into the associative array. When the corresponding packet is received at the output, the key must be known in order to read the packet out for comparison. Therefore the key must either be an untransformed part of the data (e.g., one byte of the payload) or be generated from the data (e.g., CRC check) so that the output data can be compared against the corresponding stored data.

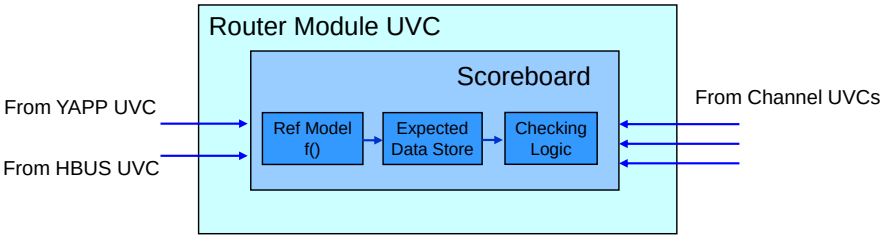
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

YAPP Router Scoreboard Implementation

How should we implement the router scoreboard?

- Transfer function (reference model):
  - Do we need this?
- Expected data storage mechanism:
  - Queues or associative arrays?
- Comparison function:
  - Built-in `compare()` or user-defined comparison?
- Communication with interface UVCs



229 © Cadence Design Systems, Inc. All rights reserved.

Do we need a transfer function?

The router does not alter packet data, therefore it may seem like a transfer function is not needed. However the router will drop over-sized and illegally addressed (address = 3) packets. Also all packets are dropped when the router is disabled. Dropped packets should not be written to the scoreboard. Therefore we will need to monitor HBUS transactions to keep track of maximum packet size and enable status, to filter out illegal and disabled packets.

Do we need data storage?

Packets are usually output in the same order as they are received, so a queue is the best choice. However if very short packets are sent to every router address and all of the output channels are suspended, packets can accumulate in the output buffers. Then if the channels are enabled in a different order than the packets were received, then it is possible for packets to come out in a different order. Therefore it is recommended to use three different queues – one for each channel.

What about comparison?

The data content of the YAPP and Channel packets may be the same, but for UVC encapsulation, channel packets are declared as a different type from the input YAPP packets,. Therefore we will need a user-defined compare method.

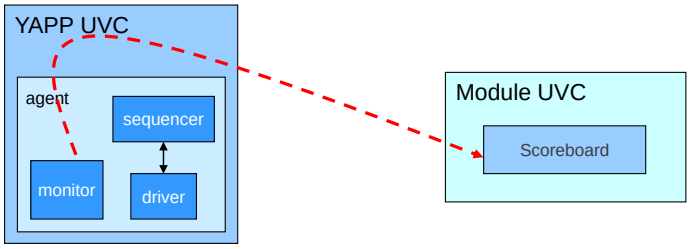
How do we send the packets from the YAPP, HBUS and Channel UVCs to the scoreboard?

We will spend the next few slides discovering how to implement this.

© Cadence Design Systems, Inc. All rights reserved.

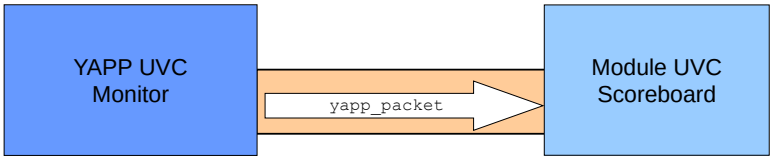
This document is for the sole use of Kun Wang of Seas

Communication Between Class Components



Needs equivalence of Verilog I/O ports for class instances!

- UVM provides TLM interfaces



230 © Cadence Design Systems, Inc. All rights reserved.

Packets collected in the monitors of the YAPP, HBUS and Channel UVC's must be sent to the scoreboard.

In traditional Verilog, we would use module ports to transfer data from one component to another. We cannot use ports to transfer data between class components. We need an equivalent to module ports for classes, and in UVM these are Transaction Level Modelling (TLM) interfaces.

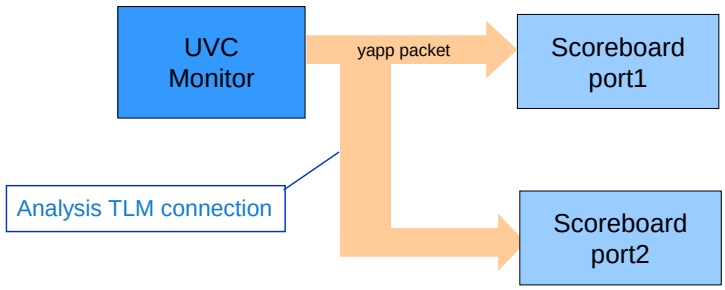
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Communication with UVCs: Analysis Interface

The analysis interface provides broadcast capability

- Transaction can be “broadcast” to zero, one, or multiple consumers
  - Producer (source) does not know how many consumers (destinations) are connected
- Intended for non-intrusive monitoring of transactions
  - For example, monitor broadcasting sent/received packets to multiple scoreboard components



 © Cadence Design Systems, Inc. All rights reserved.

There are many different forms of TLM interface, however we will concentrate on the Analysis interface. This is designed for communication between UVC monitors and scoreboards or other components.

Ideally you add an analysis port to every UVC monitor you create. If the analysis port is not required, you do not need to connect it and it is not an error to have an unconnected analysis port.

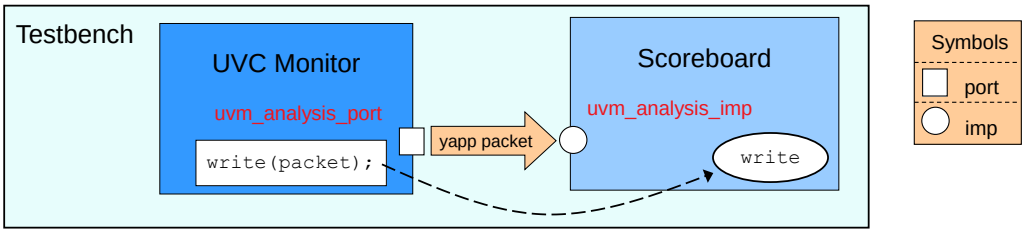
If, however, you later need five separate connections to other monitors and scoreboards, then connecting to the analysis port you previously defined is easy and straightforward.

For reusability, always add analysis ports to UVC monitors.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis Interface Implementation



Scoreboard (consumer):

- Instantiates an `uvm_analysis_imp` object, parameterized for data type
- Defines implementation of a `write` function to process the data

UVC monitor (producer):

- Instantiates an `uvm_analysis_port` object, parameterized for data type
- Initiates transfer by calling `write` method off port object

`imp` and `port` are connected together in higher hierarchical level (testbench)

- Maps port call to imp implementation

232 © Cadence Design Systems, Inc. All rights reserved.

cadence

For an analysis TLM connection passing YAPP packets, the scoreboard defines a `write()` function with the following prototype:

```
function void write(input yapp_packet packet);
```

Where the input argument packet is the YAPP packet to be processed.

The scoreboard also declares an instance of a `uvm_analysis_imp` object, parameterized for the data type which will be transferred across the connection.

The UVC monitor declares an instance of a `uvm_analysis_port` object, parameterized for data type.

The UVC monitor then calls the `write()` function off its port object.

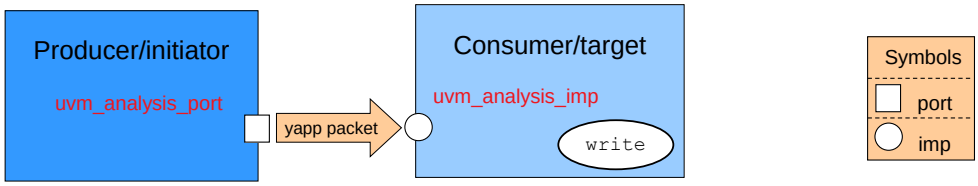
Finally the scoreboard `imp` and monitor `port` objects are connected together (or bound) at a higher level.

Now the monitor `write()` call is mapped to the implementation defined in the scoreboard. Specifically, when write is called off the port object, the simulator follows the connection to the component containing the terminating `imp` object and executes the write function defined there.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM Concepts: Port and Imp



Port and imp objects are the equivalents of Verilog module ports

Imp:

- Used inside *target* components
- Provides an implementation of the transaction method

Port:

- Used inside *initiator* components
- Provides access to an implementation of the transaction method

Also, export objects used in hierarchical connections – see later

 © Cadence Design Systems, Inc. All rights reserved.

cadence

There are several concepts to understand in transferring data via a TLM connection.

The first concept is data flow. Data is created by a producer and processed by a consumer. The producer is the source of the data transfer and the consumer is the destination.

The second concept is control. The transfer is started by the initiator which sends a request to the target.

If the producer is the initiator, then the transfer is a write operation, also called a push or put transfer. This is how analysis connections work.

If the producer is the target, then the transfer is a read operation, also called a pull or get transfer.

Imp stands for implementation and is the TLM connection object for the target component which implements the transaction method, in this case write.

Port is the connection object for the initiator component, which triggers the transaction by calling the transaction method. A port must be connected to a suitable imp to provides access to the implementation.

There is also an export connection object which is only required for hierarchical connections. This is covered in the TLM module later in this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simple Analysis Interface Example

```
class yapp_monitor extends uvm_monitor;
  ...
  uvm_analysis_port#(yapp_packet) yapp_out;

 function new (string name,
               uvm_component parent);
   super.new(name, parent);
   yapp_out = new("yapp_out", this);
 endfunction : new

  yapp_packet collected_packet;
...
  task collect_packets();
    // Collect Header {Length, Addr}
    // Collect the Payload
    // Collect Parity, set Parity Type
    yapp_out.write(collected_packet);
    ...
  endtask
...
```

YAPP UVC Monitor

Send packet to scoreboard

```
class router_sb extends uvm_scoreboard;
  ...
  uvm_analysis_imp
    #(yapp_packet, router_sb) yapp_in;

 function new (string name,
               uvm_component parent);
   super.new(name, parent);
   yapp_in = new("yapp_in", this);
 endfunction : new

...
  function void write
    (input yapp_packet packet);
    case (packet.addr)
       2'b00:q0.push_back(packet);
       ...
  endfunction
...
```

Module UVC Scoreboard

Process packet in `write` method

Analysis imp object:

- Type parameterized for transaction and component type
- Scope of object must have implementation of `write()`

Analysis port object:

- Type parameterized for transaction type
- Call `write()` from port with transaction argument

234 © Cadence Design Systems, Inc. All rights reserved.

cadence

In the YAPP UVC monitor, we declare a `um_analysis_port` object named `yapp_out`, and create the instance in the constructor of the monitor. The `uvm_analysis_port` object declaration has a single type parameter which is the type of the transaction (`yapp_packet`).

Note that we create the instance using a standard `new()` constructor rather than a `create()` call to the factory. TLM connection objects are tightly associated with specifically named communication methods (`write` for analysis objects). It's not possible to simply substitute one TLM connection type for another via type overrides as the required communication methods must also be defined. Therefore we prevent the use of type overrides on TLM objects by directly calling the constructor and not using the factory.

In the router scoreboard, we declare a `uvm_analysis_imp` object named `yapp_in`, and create the instance in the constructor of the scoreboard. The `uvm_analysis_imp` object has two parameters. Firstly the type of the transaction (`yapp_packet`), and secondly the class where the object is declared (`router_sb`).

As the scoreboard contains the analysis imp object declaration, it must also declare the `write()` method. The method has a single input argument of the transaction type (`yapp_packet`). In the `write()` function, we can place any code we need. Here we push the packet to a specific scoreboard queue based on the packet address.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simple Analysis Interface Connection

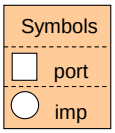
```
class router_testbench extends uvm_env;
  yapp_env    yapp;
  router_sb   scoreboard;
  ...
  virtual function void build_phase(uvm_phase phase);
    // create UVC instances
    scoreboard = router_sb::type_id::create("scoreboard", this);
  ...
  virtual function void connect_phase(uvm_phase phase);
    yapp.agent.monitor.yapp_out.connect(scoreboard.yapp_in);
    ...
```

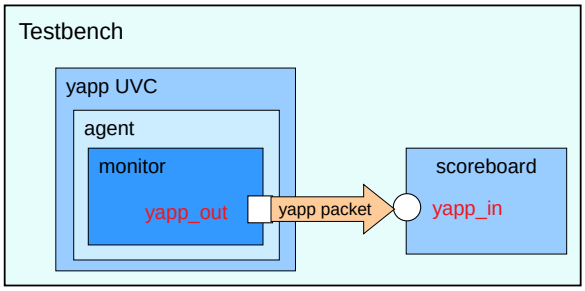
Instance name

Handle name

Connect called off port path and takes imp path as argument

Paths use handle names not instance names





235 © Cadence Design Systems, Inc. All rights reserved.

Connection direction is important and is the same as the data flow. The connect method is called off the "output" object and takes the "input" object as an argument. This means ports are always connected to implementations, i.e., the connect method is called from a port instance and takes an imp instance (or export) as an argument.

Connection is made in the testbench (typically) and uses hierarchical pathnames to find the analysis connection objects. The pathnames use handle names (not instance names) and are not strings, (unlike a configuration pathname) and so cannot contain wildcard characters.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multiple Analysis Imp Connections

```
class yapp_monitor extends uvm_monitor;
  ...
  uvm_analysis_port #(yapp_packet)
    yapp_out = new("yapp_out", this);
  ...
  yapp_packet collected_packet;
  ...
  task collect_packets();
    ...
    yapp_out.write(collected_packet);
  endtask
...
```

YAPP UVC Monitor

Only one `uvm_analysis_imp` object can be declared in a component.

Separate `imp` subclasses can be declared for multiple connections using `decl` macros.

The argument of the macro becomes the suffix for the imp object and `write` method.

Each new imp subclass must be instantiated and a separate `write` method defined.

```
class router_sb extends uvm_scoreboard;

  `uvm_analysis_imp_decl(_yapp)

  uvm_analysis_imp_yapp#(yapp_packet, router_sb)
    yapp_in = new("yapp_in", this);

  function void write_yapp(input yapp_packet packet);
       ...
  endfunction

  `uvm_analysis_imp_decl(_chan0)

   uvm_analysis_imp_chan0#(channel_packet, router_sb)
     chan0_in = new("chan0_in", this);

  function void write_chan0(input channel_packet packet);
       ...
  endfunction

  `uvm_analysis_imp_decl(_chan1)
  `uvm_analysis_imp_decl(_chan2)
  `uvm_analysis_imp_decl(_hbus)
  // object declarations/write methods
  // for _chan1, _chan2, & _hbus
...
```

Module UVC Scoreboard

236 © Cadence Design Systems, Inc. All rights reserved.

cadence

Unfortunately we don't have a simple analysis connection to our router scoreboard.

You can have many `port` objects in one component, but the default transaction implementation in the UVM library only allows *one* `imp` object in a single component. Obviously our scoreboard requires many `imp` connections, each with a separate `write` implementation.

You can create a new `imp` subclass type by inheritance, and this allows you to have as many `imp` objects as you need in a single component. UVM provides a simple macro to make the analysis `imp` subclass declaration - `` `uvm_analysis_imp_decl(<arg>) `` where `<arg>` is suffix name for both the new `imp` subclass object and its associated `write` method.

In the scoreboard, the `uvm_analysis_imp_decl` macro, with the argument `_yapp`, creates a new analysis imp subclass - `uvm_analysis_imp_yapp`. An instance of this class is then created, with a type parameter matching the transaction type. A separate `write` implementation must be defined for each new analysis `imp` subclass and this uses the same declaration macro argument as a suffix in its name, e.g. `write_yapp()` is the write communication method associated with the `uvm_analysis_imp_yapp` subclass.

You can use separate decl macros for the HBUS and channel connections.

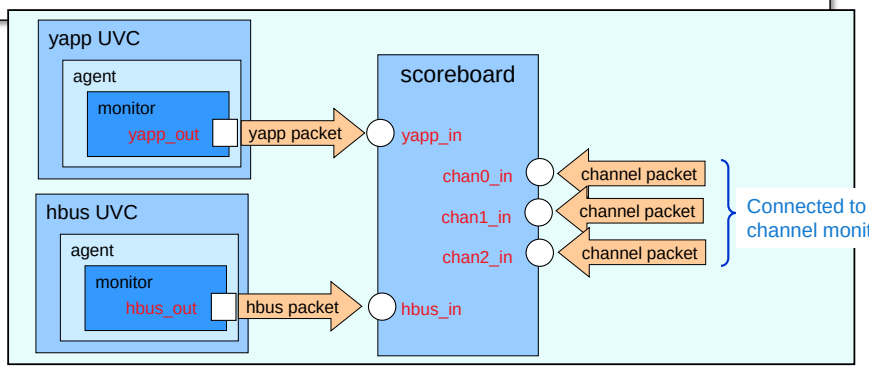
An alternative is to wrap the imp object in a component, together with the write implementation, and instantiate multiple imp components where required. There is a dedicated UVM class especially for this – `uvm_subscriber` (see UVM reference). However the downside of this approach is the extra pathnames introduced by having additional hierarchy.

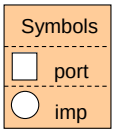
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Multiple Analysis Interface Connection

```
class router_testbench extends uvm_env;
  yapp_env    yapp;
  hbus_env    hbus;
  router_sb   scoreboard;
  ...
  virtual function void connect_phase(uvm_phase phase);
    yapp.agent.monitor.yapp_out.connect(scoreboard.yapp_in);
    hbus.masters[0].monitor.hbus_out.connect(scoreboard.hbus_in);
    // channel monitor connections
```





237 © Cadence Design Systems, Inc. All rights reserved.

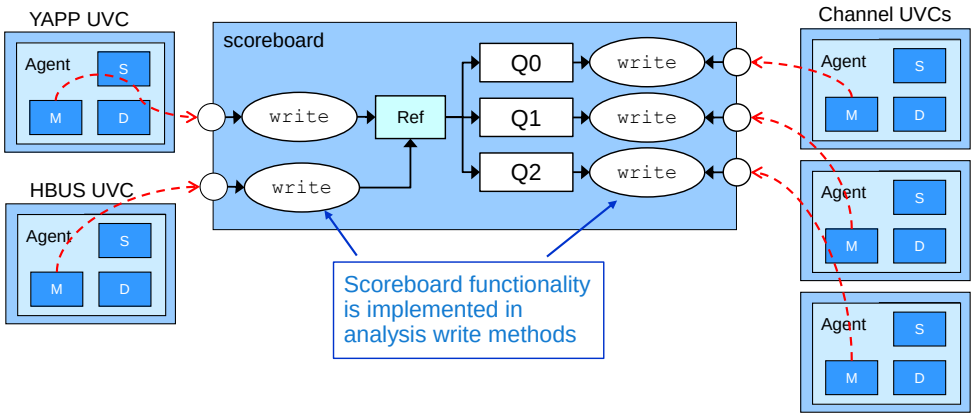
Connection direction is important and is the same as the data flow. The connect method is called off the "output" object and takes the "input" object as an argument. This means ports are always connected to implementations, i.e., the connect method is called from a port instance and takes an imp instance (or export) as an argument.

Connection is made in the testbench (typically) and uses hierarchical pathnames to find the analysis connection objects. The pathnames use handle names (not instance names) and are not strings, (unlike a configuration pathname) and so cannot contain wildcard characters.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

YAPP Scoreboard Structure



YAPP `write` checks reference model registers and pushes packets to queue.

HBUS `write` updates reference model `enable` and `maxpktsize` registers.

Channel `write` compares received packet with packet from queue.

Every `write` updates statistics which are summarized in the report phase.

238 © Cadence Design Systems, Inc. All rights reserved.

cadence

A scoreboard structure which uses `imp` connectors can define most of its functionality in the analysis `write` methods.

The scoreboard should contain a reference model which maintains a local copy of the router `enable_router` and `maxpktsize` registers.

The HBUS write method can be used to update these registers on HBUS write transactions, and possibly check the registers on read transactions.

The YAPP `write` method can check the registers and push the packet to a queue if the router is enabled and the packet length is less than the maximum size.

The channel `write` methods can pop the expected packet off a queue and compare the packet to the input packet argument of the method.

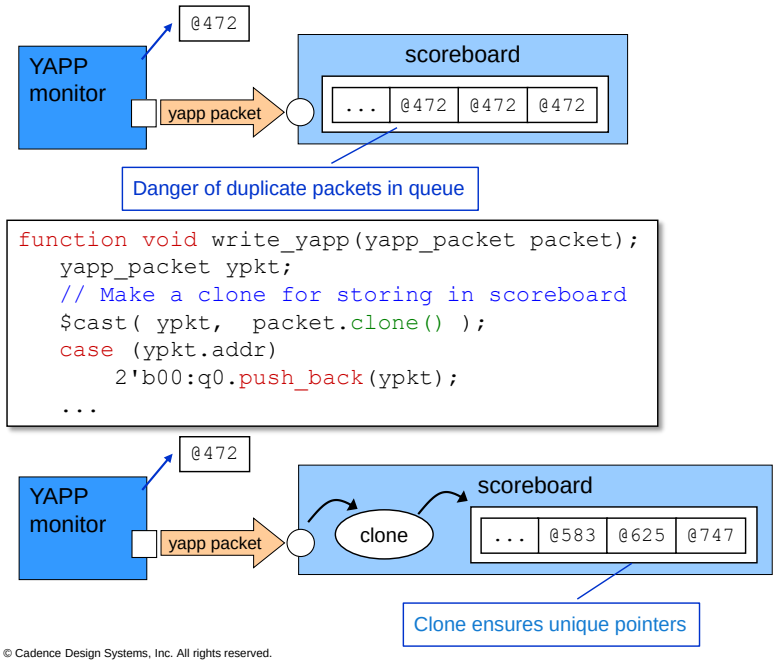
Every write should update counters in the scoreboard to keep track of received, dropped, matched and mismatched packets. A final summary of the statistics should be printed in a `report_phase()` method. It is also essential to check that the scoreboard queues are empty at the end of the simulation.

You may also have a connection to the monitor of the Clock and Reset UVC in order to detect a reset of the DUT to trigger a reset of the scoreboard. This depends on the functionality of the UVC, specifically whether it creates an analysis transaction on reset.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cloning Analysis Write Data



`write` only sends a reference:

- Pointer to packet instance

Monitor may use the same instance for every collected packet

This is a problem if the pointers are stored in queues or arrays

- Queue of identical pointers
- The next collected packet will overwrite all stored packets

Scoreboard should use clone

- Create a unique copy of every received packet before storage

239 © Cadence Design Systems, Inc. All rights reserved.

The `write()` TLM method just sends a reference pointer for the data item. This is a pointer to the object created in the UVC monitor. This causes problems if the monitor does not create separate instances for every captured data item, but instead reuses the same instance. So if we store the reference received from the `write()` method directly in the scoreboard, when subsequent data items are captured the stored data items will be overwritten.

For example, if the monitor uses the reference @472 to capture the first packet. This reference is sent to the scoreboard and directly written to the queue. So the scoreboard queue now contains the reference @472. If however the same instance is used by the monitor to capture the second packet, then, when it is written and stored in the scoreboard queue, we have two locations with the same reference @472, containing the data for the second packet.

As we cannot guarantee the behavior of the monitor, the scoreboard must take a clean copy of the received data item by cloning, before writing the cloned packet to the queue. Now every item received had a separate reference and overwriting should not occur.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simple Scoreboard Comparison

```
function bit pkt_compare(yapp_packet yp, channel_packet cp);
  if (yp.addr != cp.addr) begin
    `uvm_error("PKTCMP", "Address mismatch")
    return(0);
  end
  if (yp.length != cp.length) begin
    ...
```

User-defined comparer finds first mismatch

Debugging information

```
function void write_chan0(input channel_packet cp);
  yapp_packet yp;
  yp = q0.pop_front();
  // compare yp and cp
  if( pkt_compare(yp, cp) )
     ...
```

Comparison in channel `write()`

Channel `write` methods must compare input and output packets.

- Channel UVC sends a `channel_packet` type
- YAPP UVC sends a `yapp_packet` type (stored in the queue)

As these are different types, we require a user-defined compare

A simple compare implementation uses equality operators.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The scoreboard must compare the output packets of the router, as received from the Channel UVC, against the appropriate input packets as received from the YAPP UVC (and stored in the router queues). The best place is to do this in the Channel `write` methods.

However the Channel UVC uses a different type (`channel_packet`) to the YAPP UVC (`yapp_packet`). This is to enforce encapsulation and avoid declaration dependencies between different UVCs.

Therefore we need to declare our own user-defined comparison, to take two input packets of type `yapp_packet` and `channel` packet; compare their corresponding fields (omitting control fields like `packet_delay`) and then return 1 if all appropriate fields match. In a simple implementation, we can use inequality operators, although this means we must add our own debugging messages to highlight which field does not match. You should make your debugging messages more effective by printing the values of the mismatched fields.

In the example above, we exit the comparison on the first field mismatch. Obviously if the packet lengths do not match, comparing the payload data is irrelevant.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM Scoreboard Comparison

```
function bit ccomp (yapp_packet yp, channel_packet cp, uvm_comparer comparer = null);

  if (comparer == null)
    comparer = new();
  ccomp = comparer.compare_field("addr", yp.addr, cp.addr, 2);
  ccomp &= comparer.compare_field("length", yp.length, cp.length, 6);
  ...
function void write_chan0(input channel_packet cp);
  ...
  if( ccomp(yp, cp) )
     ...
```

Create comparer instance if not passed

Size

Use comparer methods

Use custom comparer in comparison

A more powerful comparer can use the methods of `uvm_comparer`:

- Automatic reporting of mismatches
- Allows user customization
  - Users can set up and pass in their own comparer instance

 © Cadence Design Systems, Inc. All rights reserved.

cadence

More powerful comparisons can be created using the built-in compare methods from the `uvm_comparer` class. This gives automatic reporting of mismatches and allows the user to modify the comparer switches and policies, and so customise the behavior of the user-defined comparison.

The `ccomp` method passes in `yapp_packet` and `channel_packet` instances, but also has a third argument of type `uvm_comparer` with a default value of `null`. If the user passes their own comparer instance, then this is used to access the comparison methods. Otherwise a comparer instance is created.

The simplest comparison method is `compare_field` which compares two integral values:

```
virtual function bit compare_field (string  name,
          uvm_bitstream_t  lhs,
          uvm_bitstream_t  rhs,
          int  size,
          uvm_radix_enum  radix  =  UVM_NORADIX )
```

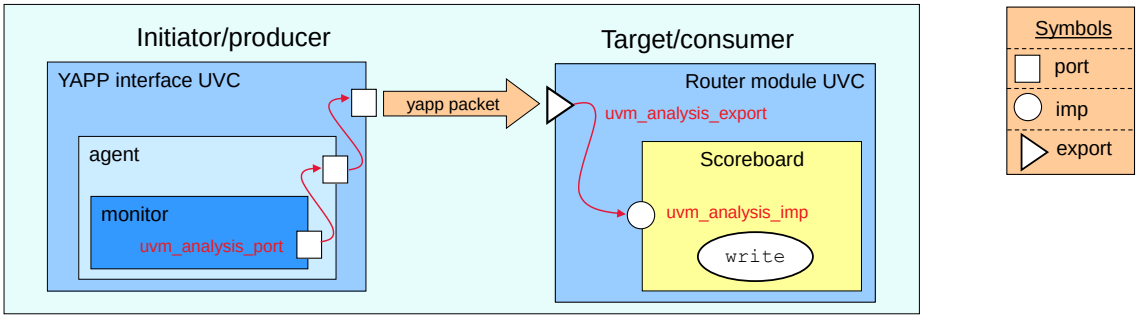
Where the string name is used in printing mismatches; lhs and rhs are the two integral values for comparison; size is the number of bits to compare (maximum 4096) and radix is the print radix used for reporting mismatches.

There are also `compare_string` and `compare_object` methods as well – see the reference material for further details.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Hierarchical Connections and Exports



Connections can be hierarchical

`port` is used to route `port` connectors to higher hierarchical levels

- All connections on the initiator side will be `port`

`export` is required to route `imp` connectors to higher hierarchical levels

- Only the component providing the implementation has an `imp` connector
- All other connections on target side are `export`

Path must be connected at each level

242 © Cadence Design Systems, Inc. All rights reserved.

`imp` and `port` connectors are sufficient for modelling most connections, but there is a third connector, `export`, which is used exclusively in hierarchical connections.

Normally hierarchical routing is not required. A `port` on an UVC monitor can be connected directly to an `imp` in a scoreboard by specifying full hierarchical pathname (e.g., `env.agent.monitor.analysis_port`). The structure of an UVC is fixed and so the user knows to look in the monitor component for the analysis ports.

However the internal hierarchy of a module UVC is more arbitrary, and it may be convenient to route all the module UVC connectors to the top level to allow use without knowledge of the internal structure.

On the initiator side, ports are routed up the hierarchy via other `port` instances.

On target side, only the component which defines the communication method is allowed to have an `imp` instance. So we need a third object to route connections up the target side hierarchy – `export`.

The hierarchical route must be connected at each level in the direction of control flow:

```
initiator.connect(target)
```

Connection rules are as follows:

- `Port` initiators can be connected to `port`, `export` or `imp` targets.
- `Export` initiators can be connected to `export` or `imp` targets.
- `Imp` cannot be a connection initiator. `Imp` is a target only and is always the last connection object on a route.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Building a Scoreboard Quiz

1. What are the three distinct functions of a scoreboard?

2. To how many consumer components can an analysis port be connected?

3. What are the names of the two declarations made available by the following macro:

```
`uvm_analysis_imp_decl(_possible)
```

4. Why should a scoreboard clone the data received by a `write` method?

Solutions at end of module

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

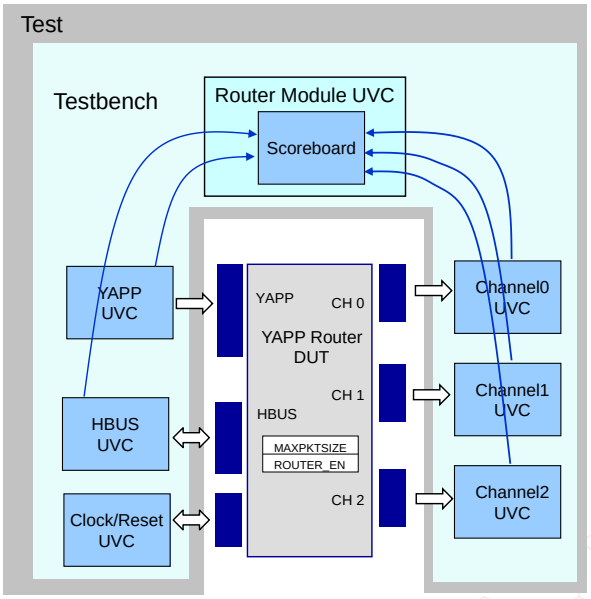
Labs

Lab 9A Creating a Scoreboard Using TLM

- Build scoreboard with expected data store and checking logic
- Scoreboard has Analysis ports for:
  - Adding packets that are sent to DUT
  - Checking packets coming out of the DUT
- Tip: Use `decl` macros for each connection

Lab 9B Router Module UVC

- Modify the scoreboard to cater for the router dropping oversized or disabled packets



244 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Solutions: Building a Scoreboard Quiz

1. What are the three distinct functions of a scoreboard?
   - Reference model, expected data storage, and comparison.
2. To how many consumer components can an analysis port be connected?
   - Any number, including zero.
3. What are the names of the two declarations made available by the following macro:

   ```
   `uvm_analysis_imp_decl(_possible)
   ```

   - Subclass `uvm_analysis_imp_possible`
   - Method `write_possible()`
4. Why should a scoreboard clone the data received by a write method?
   - The write method only sends a reference; therefore, if the sending component uses the same reference for every data item, overwriting of data in the storage function of the scoreboard is possible without cloning.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 15

Transaction Level Modeling (TLM)

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module Objectives

In this module, you

- Use key TLM concepts
  - Data and control flow
  - Blocking and nonblocking communication
- Select connector type based on required communication methods
- Instantiate and connect simple uni-directional TLM interfaces
- Create a scoreboard using TLM analysis FIFOs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM Application Programming Interface (API)

TLM API is a standard for communication between classes

- Developed by Open SystemC Initiative® (OSCI)
- Allow plug-and-play of UVM components

UVM provides an implementation of the TLM API

There are many different connector types in TLM:

- Uni-directional
- Bi-directional
- FIFO

Your requirements determine which connector type to use:

- Data flow
- Control flow
- Time-consuming or instantaneous

 © Cadence Design Systems, Inc. All rights reserved.

cadence

TLM was originally part of the SystemC specification. It defines an standard interface for connecting class components, consisting of a predefined set of communication building blocks and a methodology defining how these blocks are used.

There are many different types of TLM connection.

- Simple uni-directional connectors where data flows in one direction only.
- Bi-directional connectors where data flows in both directions.
- FIFO connectors, where the connector provides internal storage for data.

The characteristics of your data transfer determine which of the connector types you need to use, specifically the direction of data flow; control of the transfer and whether the transfer is can consume time or must be instantaneous.

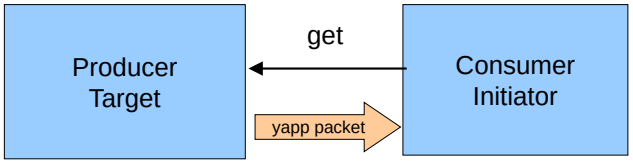
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Concepts: Data and Control Flow

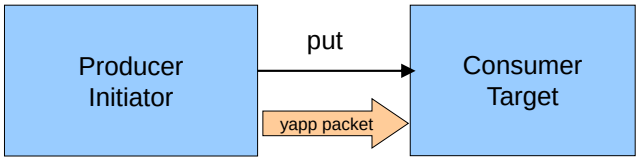
Data flow: producer to consumer

Control flow: initiator triggers transaction, target responds



Producer is target and consumer is initiator

- "Pull mode"
- Implemented with get operations



Producer is initiator and consumer is target

- "Push mode"
- Implemented with put operations

249 © Cadence Design Systems, Inc. All rights reserved.

Before we examine TLM objects, it is useful to explore some of the concepts.

Communication in TLM has two parts:

- Data flow. A producer creates data items and a consumer receives items.
- Control flow. The initiator requests the transaction and the target must respond to the request.

If the consumer is also the initiator, then this is a TLM get operation. The consumer requests the producer to send a data item. This is also known as "pull mode," as the consumer pulls or reads data from the producer

If the producer is also the initiator, this is a TLM put operation. The producer request the consumer to receive a data item. This is also known as "push mode" as the producer pushes, or writes, data to the consumer.

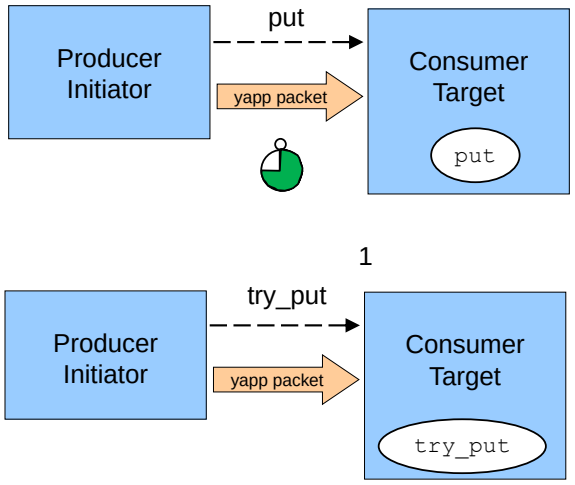
There are other options, but the data/control flow relationship between your components will determine the type of the connection, and therefore which TLM connection to use.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Concepts: Blocking and Nonblocking

What happens if the transaction cannot be completed yet?



Initiator can suspend execution until target is ready:

- Blocking (time consuming)
- Implemented with `get` or `put` tasks
- May block forever!

Initiator can register failure and continue:

- Nonblocking (instantaneous)
- Implemented with `try_get` and `try_put` functions
- These return 0 on failure, 1 on success

250 © Cadence Design Systems, Inc. All rights reserved.

What happens if the transaction cannot yet complete?

With get operations (pull mode), this can occur when the producer is not ready to deliver data.

With put operations (push mode), this can occur when the consumer not ready to receive data.

There are two options:

1. The initiator waits until the data or the consumer is ready . This is called blocking. Blocking methods are defined with `get()` or `put()` tasks to allow them to consume time.
2. The initiator registers that the transaction cannot complete and continues performing other tasks. This is called nonblocking. The initiator can come back later and see if the data is ready. Non-blocking methods are defined with `try_get()` or `try_put()` functions as they execute in zero time.

Most TLM connector types support both blocking and non-blocking methods, allowing an initiator to call whichever method is required.

Specialist connector subtypes can also be used which allow only blocking or only non-blocking access.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Uni-Directional TLM Methods Reference

Method	Description	Syntax
`put()`	Blocking put	`virtual task put(input TR t);`
`try_put()`	Nonblocking put – returns 1 if successful – returns 0 if not	`virtual function bit try_put(input TR t);`
`can_put()`	Nonblocking test put † – returns 1 if put would be successful – returns 0 if not	`virtual function bit can_put();`
`get()`	Blocking get	`virtual task get(output TR t);`
`try_get()`	Nonblocking get – returns 1 if successful – returns 0 if not	`virtual function bit try_get(output TR t);`
`can_get()`	Nonblocking test get † – returns 1 if get would be successful – returns 0 if not	`virtual function bit can_get();`
`peek()`	Blocking peek	`virtual task peek(output TR t);`
`try_peek()`	Nonblocking peek – returns 1 if successful – returns 0 if not	`virtual function bit try_peek(output TR t);`
`can_peek()`	Nonblocking test peek † – returns 1 if put would be successful – returns 0 if not	`virtual function bit can_peek();`

251 © Cadence Design Systems, Inc. All rights reserved.

† for use with conditional statements where `TR` is type of transaction

cadence

Data flow, control flow and blocking operation determine the choice of communication methods and hence the TLM connection to be used.

The blocking `put()` task attempts to put a transaction into the consumer and blocks until the consumer can accept the transaction.

The non-blocking `try_put()` function attempts to put a transaction into the consumer and returns 0 if the consumer cannot accept the transaction, or 1 if the transaction completes.

The non-blocking `can_put()` function checks if the consumer can accept a transaction and returns 0 if the consumer is not ready, or 1 if the transaction can be accepted. `can_*` methods are intended for use in conditional statements, e.g., `if (txn.can_put()) ...`

The blocking `get()` task attempts to get a transaction from the producer and blocks until the producer can generate the transaction.

Non-blocking `try_get()`, and `can_get()` are similar to their `put` equivalents.

The `peek()` methods are similarly to the `get()` methods, but *copy* the transaction instead of removing it. The transaction is not consumed, and a subsequent get or peek operation will return the same transaction.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selected Connector and Method Options

Method choice determines connector type

Connector	put()	try_put()	can_put()	get()	try_get()	can_get()	peek()	try_peek()	can_peek()
uvm_put_*	☑	☑	☑						
uvm_blocking_put_*	☑								
uvm_nonblocking_put_*		☑	☑						
uvm_get_*				☑	☑	☑			
uvm_blocking_get_*				☑					
uvm_nonblocking_get_*					☑	☑			
uvm_get_peek_*				☑	☑	☑	☑	☑	☑
uvm_blocking_get_peek_*				☑			☑		
uvm_nonblocking_get_peek_*					☑	☑		☑	☑

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The communication methods we need are determined by data flow, control flow and choice of blocking or non-blocking operation. Once we know the methods required, we can select the connection type.

In the connectors above, `*` can be replaced by `port`, `imp`, or `export`.

Peek connectors allow a data item to be copied from a producer, but not consumed, so that the data item remains on the interface, and can be copied by a peek operation from another component, or consumed in a get operation.

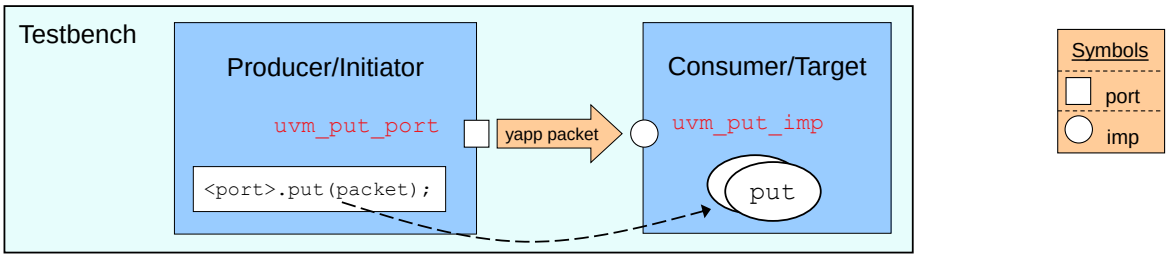
Note that all the methods for a specific connector type *must* be implemented. If you define an `uvm_put` connection between two components, then the component with the `uvm_put_imp` object must provide implementations of all three put methods, `put`, `try_put`, and `can_put`, even if these methods are not explicitly called.

Note there is also a `uvm_peek_*` connector type option, also in blocking and non-blocking forms, which allows only `peek` operations to be performed.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM Implementation Review



Target declares an `imp` connector, parameterized for data type

- For example, `uvm_put_imp` when the target is also a consumer

Target defines the implementation of transaction using specifically named methods

- Varients of `put` when the target is also the consumer

Initiator declares a `port` connector, parameterized for data type

- For example, `uvm_put_port` when the initiator is also the producer

`imp` and `port` are connected in a higher hierarchical level

Initiator starts transfer by calling the method from port

- Mapped to target implementation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The implementation of a TLM connection is the same, regardless of which type we use. This example illustrates a `put` operation.

The target component declares an instance of a TLM `imp` connector, parameterized for the data type which will be transferred across the connection. We use a predefined object type depending on the TLM operation, `uvm_get_imp()` for our example where the target is also the consumer.

The target component processes the data item via a series of communication methods. These methods must have specific names depending on the type of TLM operation. If the target is also the consumer, these methods are named as some variant of `put` (`try_put`, `can_put`, etc.).

The initiator component declares an instance of a TLM port connector, parameterized for data type. Again the connection type depends on the TLM operation and must be compatible with the target connector. Here we use `uvm_put_port` as the producer is also the initiator.

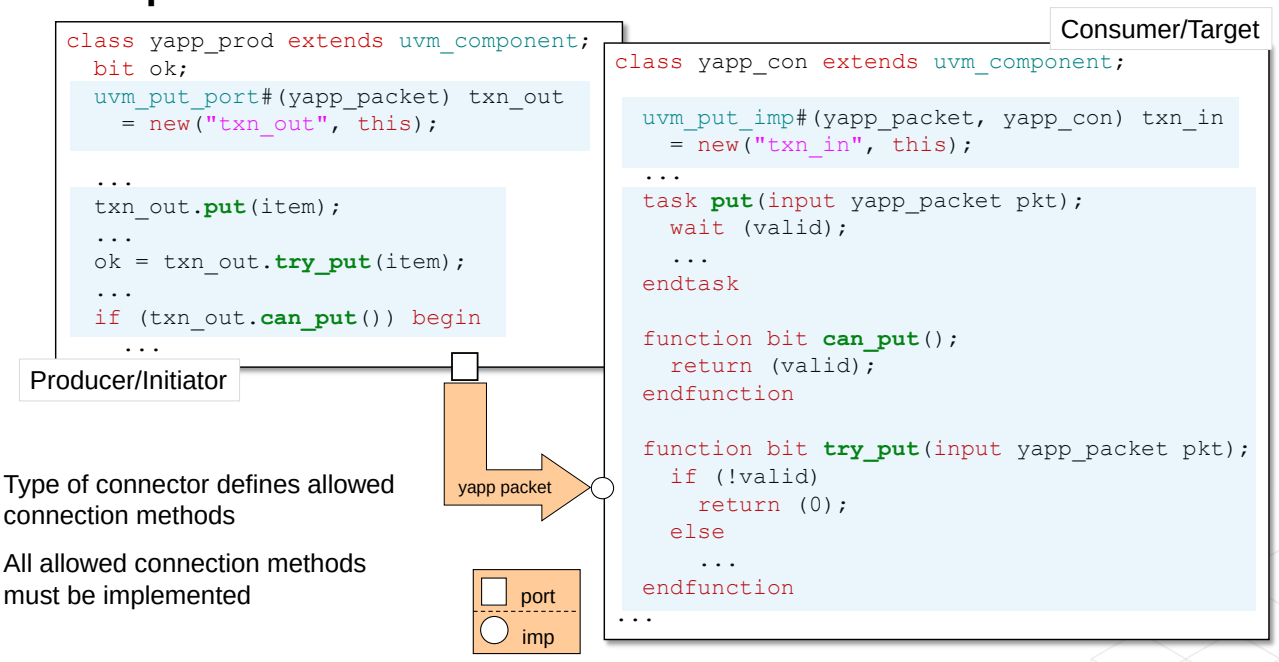
Finally, the target `imp` and initiator port objects are connected together (or bound) at a higher level.

Now the initiator can call the target methods directly via its port instance, and these are mapped to the implementations of the methods defined in the target.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Put Example



- Type of connector defines allowed connection methods
- All allowed connection methods must be implemented

254 © Cadence Design Systems, Inc. All rights reserved.
cadence

Here the consumer is the target. The consumer defines an instance of `uvm_put_imp` named `txn_in`. The instance must have a type parameter for the data (`yapp_packet`) and a type parameter for the component class (`yapp_con`). The instance is effectively a hierarchical component, therefore it uses a component constructor with two arguments, `name` and `parent` (`this`).

`yapp_con` has the `uvm_put_imp` object and so must provide implementations of the communication methods. All the methods for a specific connector type must be implemented, therefore `yapp_con` must define all three `put` methods – `put`, `try_put`, and `can_put` – even if these methods are not explicitly called.

The producer defines an instance of `uvm_put_port`, named `txn_out`, with a single type parameter for data (`yapp_packet`).

To put a `yapp_packet` data item, the producer calls one of the `put` methods off the `txn_out` port instance.

Note that both instances are created using `new`, not the factory, and are created in-line with the TLM object declaration. It is rare (and possibly dangerous) to use factory calls for creating TLM instances. The method names are specific to the TLM object type, therefore if the type is changed in an override, the method call may be illegal for the new type. However it may be more readable to construct the TLM instances in the constructor of the component (or even the `build_phase()`), rather than in-line with the declaration as above.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

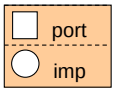
Put Connection

```
class tlm_tb extends uvm_env;
  yapp_prod producer;
  yapp_con  consumer;

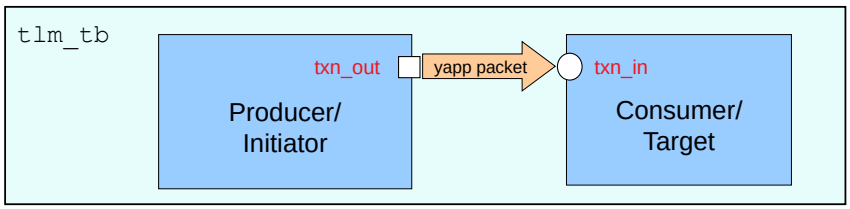
  `uvm_component_utils(tlm_tb)

  virtual function void build_phase(uvm_phase phase);
    producer = yapp_prod::type_id::create("producer", this);
    consumer = yapp_con::type_id::create("consumer", this);
    ...

  virtual function void connect_phase(uvm_phase phase);
    producer.txn_out.connect(consumer.txn_in);
    ...
```



Connect called off *initiator* with *target* as argument



255 © Cadence Design Systems, Inc. All rights reserved.

The final step in the TLM connection is to connect the `imp` and `port` connectors at a higher level. This may be done in an `agent` or `env` for connections within an UVC, or in the testbench for connections between UVC.

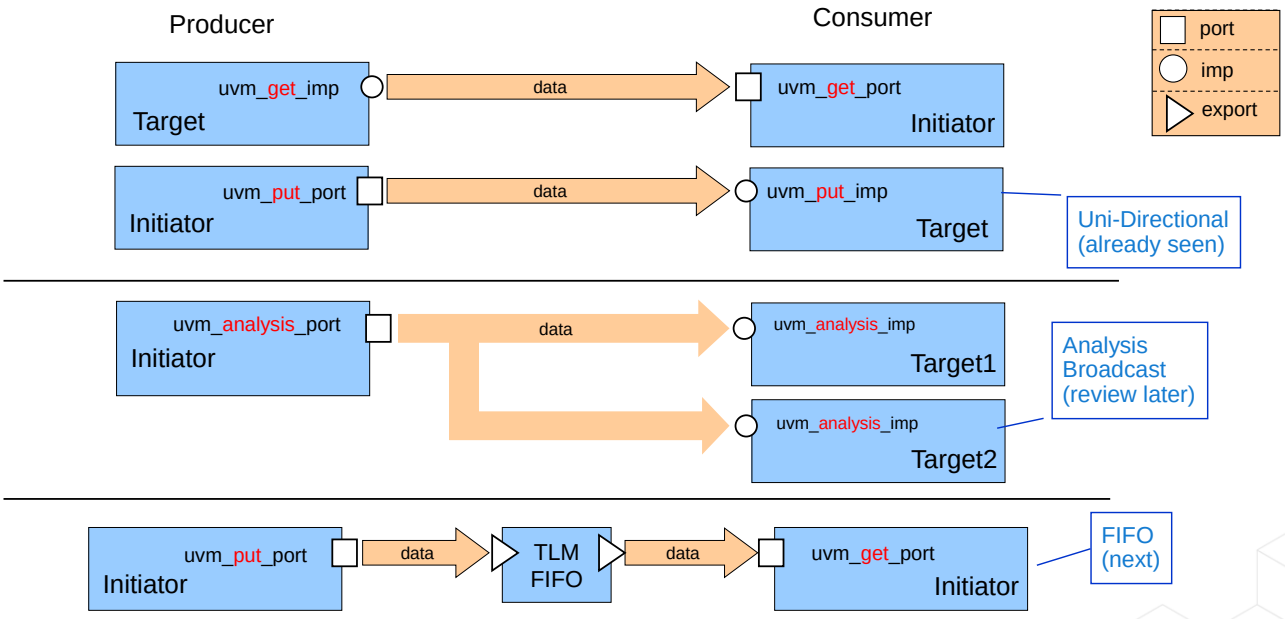
Connections are made by calling the `connect` method in the connect phase. There are a few simple connection rules to follow:

1. Connections are made in the direction of *control* flow, therefore `connect` is always called off the `port` connector instance (initiator) and takes the `imp` connector as an argument (target).
2. It is an error if a TLM instance is left unconnected. The sole exception to this rule is the TLM analysis connection.
3. The connect method of a TLM instance can only be called once, i.e., it can only be connected *to* a single TLM object instance. However a TLM object can *receive* multiple connections. In other words, TLM connections can be "many-to-one," with several different component `port` connected to one component `imp`. However connections cannot be “one-to-many” (one `port` connected to several `imp`), again with the exception of the analysis connection.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Uni-Directional TLM Communication Models



256 © Cadence Design Systems, Inc. All rights reserved.

As well as the simple, uni-directional connections get and put, there are also more complex uni-directional connections.

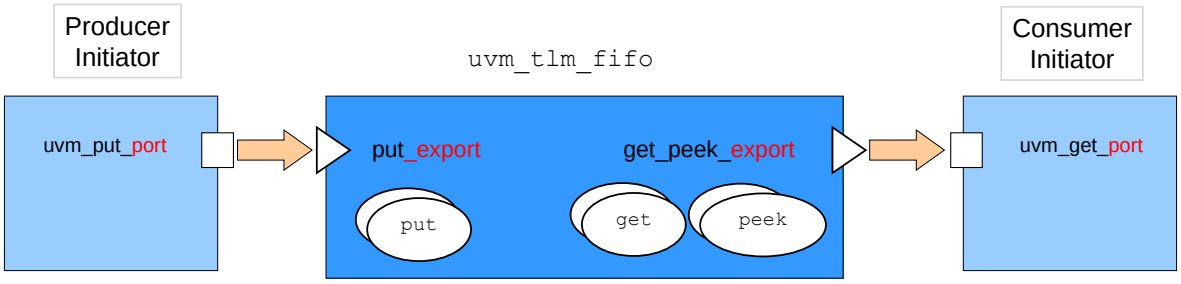
The analysis connection we have seen already, but one feature we haven't explored is the broadcast capability. One analysis port can be connected to multiple analysis imps. allowing a single data item to be sent to multiple targets (scoreboards, monitors, coverage and register models etc.)

The TLM FIFO is a FIFO component wrapped in `get` and `put imp` connectors. This has the benefit of data storage as well as providing implementations of the communication methods. Components connected to the TLM FIFO are in control of data transfer and can simply defined port connectors to initiate read and write operations on the FIFO.

© Cadence Design Systems, Inc. All rights reserved.

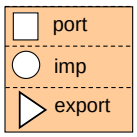
This document is for the sole use of Kun Wang of Seas

TLM FIFO Component



`uvm_tlm_fifo`:

- Used for buffering transactions between a producer and a consumer
- Has the following connectors:
  - `put_export`
    - `uvm_put_imp` for input (all put methods)
  - `get_peek_export`
    - `uvm_get_peek_imp` for output (all get and peek methods)



257 © Cadence Design Systems, Inc. All rights reserved.

cadence

The TLM FIFO object is effectively a FIFO component instantiated between and connected to two components. The FIFO contains `imp` connectors for the standard TLM `put` and `get/peek` interfaces, therefore the user does not have to define `imp` ports or communication methods, and the FIFO takes care of data storage.

The advantages are:

- The user does not need to define communication methods or `imp` connectors.
- The FIFO provides data storage between the write (`put`) and read (`get/peek`) components.
- There are a number of built-in methods for checking FIFO status.

The disadvantages are:

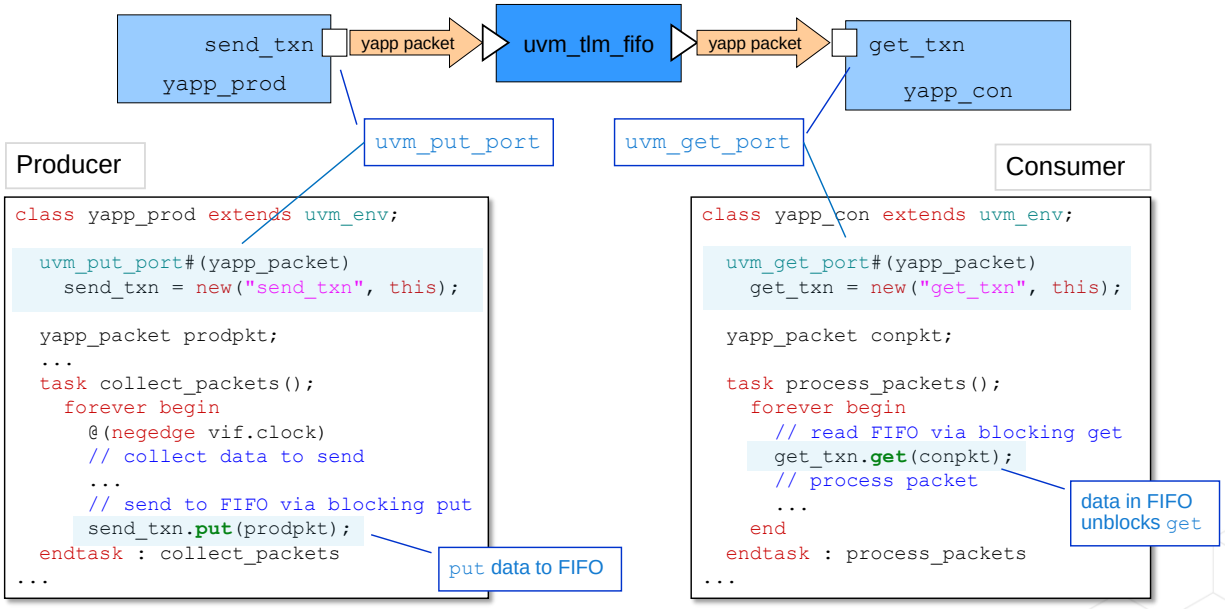
- The user must now initiate both sides of the transfer (both `get/peek` and `put`) to complete the transaction.
- Two connections must be made (both sides of the FIFO) rather than one.

The `put_export` and `get_peek_export` connection objects have alternatives which provide subsets of the full connector. For example, `blocking_put_export` and `nonblocking_put_export` can replace `put_export. blocking_get_export`, `nonblocking_get_export` and `get_export` (as well as others) can replace `get_peek_export`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM FIFO: Setting Up Components



258 © Cadence Design Systems, Inc. All rights reserved.
cadence

In the two components which will be connected to the FIFO object, we declare standard TLM port connectors – `put` to write to the FIFO and `get` (or `peek`) to read. We then call `put` and `get` methods off the ports to write and read data from the FIFO. Which variant of `put/get` we call depends how we wish to access the FIFO – with blocking, non-blocking or a combination of blocking/non-blocking access. All are available from the FIFO. Here we use blocking `get` and `put` to access the FIFO.

In this example, the producer component which sends data to the FIFO contains timing, delays and event expressions, which are used to collect the data to be sent.

In this example, the consumer component does not contain timing. It simply makes `get` calls in a forever loop and processes the packet data in zero time. As the consumer calls a blocking `get`, then consumer is always waiting for data to be sent to the FIFO. Therefore a `put` from the producer automatically triggers the `get` from the consumer and the processing of the packet.

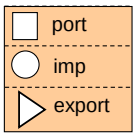
This is only one possible timing model for the FIFO communication. Both producer and consumer components could have completely different timing behavior, depending on your application.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM FIFO Connection






```
class tlm_test extends uvm_test;
  yapp_prod producer;
  yapp_con consumer;
  uvm_tlm_fifo #(yapp_packet) fifo;

  `uvm_component_utils(tlm_test)

  virtual function void build_phase(uvm_phase phase);
    producer = yapp_prod::type_id::create("producer", this);
    consumer = yapp_con::type_id::create("consumer", this);

    fifo = new("fifo", this, 5);

    ...
  virtual function void connect_phase(uvm_phase phase);
    producer.send_txn.connect(fifo.put_export);
    consumer.get_txn.connect(fifo.get_peek_export);
    ...
```

FIFO parameterized for data type

FIFO constructor includes size argument (default 1)

Connect both sides!

FIFO has export connections

259 © Cadence Design Systems, Inc. All rights reserved.

cadence

The `uvm_tlm_fifo` component needs to be declared and constructed in a hierarchical block which has visibility to both the consumer and producer components for the FIFO.

The `uvm_tlm_fifo` component declaration takes a single type parameter corresponding to the data item it will contain.

The `uvm_tlm_fifo` constructor is exactly like a normal component constructor except it has a third argument, `size`, which sets the maximum FIFO size. Default size is 1. A size of 0 is an unbounded FIFO.

If a FIFO is full, then subsequent `put` operations are blocked until a `get` operations occurs and the FIFO is no longer full.

Both sides of the FIFO must be connected in a `connect_phase` method. Note that connections on the FIFO are `export` objects, implying the FIFO component has internal hierarchy.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM FIFO Methods

The TLM FIFO has built-in methods for checking the FIFO status; creating or emptying the FIFO.

Method	Description	Syntax

`new`	Standard component constructor with an additional third argument, `size`, which sets the maximum FIFO size. Default size is 1.
	A size of 0 is an unbounded FIFO.
		`function new(string name,`
		`  uvm_component parent = null,`
		`  int size = 1);`

`size`	Returns size of FIFO. 0 indicates unbounded FIFO.	`virtual function int size();`
`used`	Returns number of entries written to the FIFO.	`virtual function int used();`
`is_empty`	Returns 1 if `used()` is 0, otherwise 0.	`virtual function bit is_empty();`
`is_full`	Returns 1 if `used()` is equal to `size`, otherwise 0.	`virtual function bit is_full();`
`flush`	Deletes all entries from the FIFO, upon which `used()` is 0 and `is_empty()` is 1.	`virtual function void flush();`

 © Cadence Design Systems, Inc. All rights reserved.

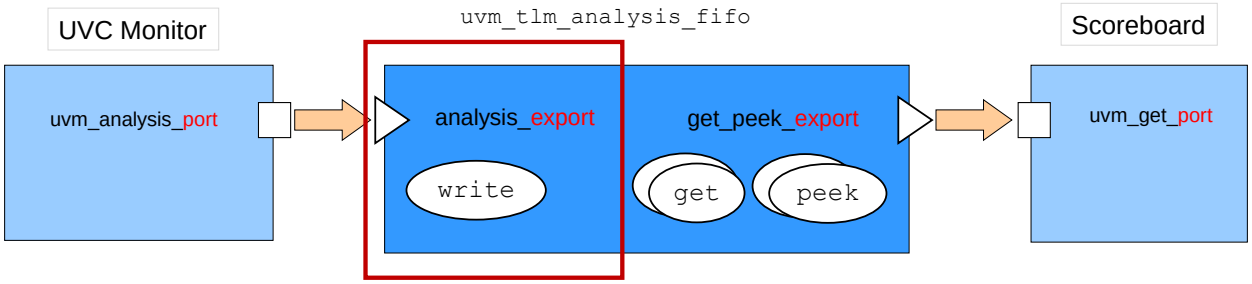
cadence

Every `uvm_tlm_fifo` component has a number of methods to construct the FIFO, extract information about the status of the FIFO, or delete the FIFO contents.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis FIFO

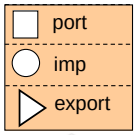


`uvm_tlm_analysis_fifo` is a specialization of `uvm_tlm_fifo`

- Intended to buffer write transactions between the UVC monitor analysis port and a scoreboard

It has the following characteristics:

- Unbounded (size = 0)
- `analysis_export` connector replaces `put_export`
  - Supports the analysis `write` method



 © Cadence Design Systems, Inc. All rights reserved.

cadence

`uvm_tlm_analysis_fifo` is a specific FIFO subclass for use with analysis ports. This `uvm_tlm_fifo` subclass is an unbounded FIFO object supporting a `write()` interface to accept data items.

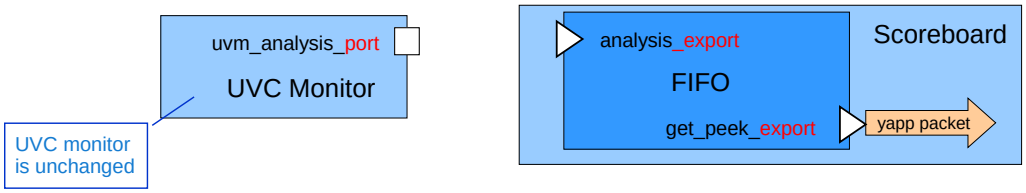
In effect, the `analysis_export` connector is a wrapper around the `put_export` connector of the `uvm_tlm_fifo` component.

We can use the analysis FIFO as a buffer between the analysis port of a UVC monitor, and the scoreboard component, and hence access all the advantages of the `uvm_tlm_fifo` – built-in storage and built-in implementation of communication methods.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis FIFO Scoreboard




```
class router_sb extends uvm_scoreboard;
   ...
   yapp_packet pkt;

   uvm_tlm_analysis_fifo#(yapp_packet) yapp_fifo = new("yapp_fifo", this);

   task run_phase(uvm_phase phase);
     forever begin
        yapp_fifo.get_peek_export.get(pkt);
        ... // process packet
     end
   endtask
   ...
```

Analysis FIFO is a scoreboard component

Call `get` directly from FIFO output

- By declaring the FIFO in the scoreboard, we can `get` directly from the FIFO output

262 © Cadence Design Systems, Inc. All rights reserved.

If we replace an analysis port-imp connection with an analysis FIFO connection, then only the scoreboard side changes. The UVC monitor component is exactly the same in both cases.

It is easier to use the analysis FIFO if it is declared as a instance within the scoreboard.

In this example we have created the scoreboard instance in-line with the declaration to save space on the slide. In real life you would create the FIFO instance in the scoreboard constructor or build phase method. Remember the analysis FIFO constructor does not allow a size to be defined - it is unbounded.

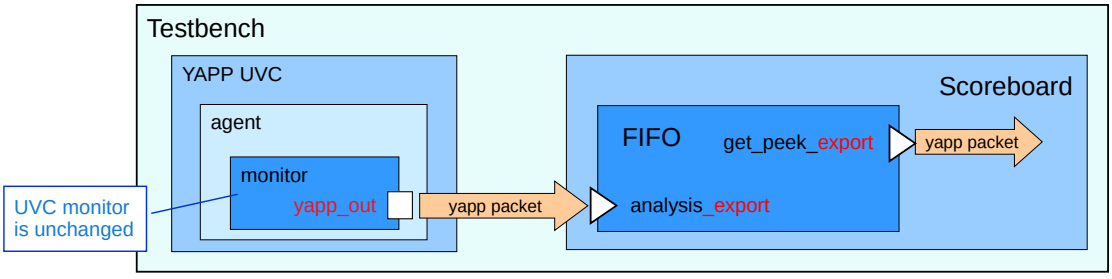
The scoreboard must implement an explicit `get` call to read the packet from the FIFO, for example from inside a `run_phase` task.

By declaring the FIFO in the scoreboard we can call `get` (and other methods) directly from the output port of the FIFO. Remember there are several alternatives to the `get_peek_export` connection which provide subsets of the full connector. For example, `blocking_get_export`, `nonblocking_get_export` and `get_export`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis FIFO Testbench Connection




```
class router_tb extends uvm_env;
  yapp_env    yapp;
  router_sb   scoreboard;
  ...
  virtual function void connect_phase(uvm_phase phase);
    yapp.agent.monitor.yapp_out.connect(scoreboard.yapp_fifo.analysis_export);
    ...
```

Scoreboard is a testbench component

Connect UVC monitor analysis port to FIFO analysis export

263 © Cadence Design Systems, Inc. All rights reserved.

Both sides of the analysis FIFO must be connected for the transaction to complete.

If the analysis FIFO is a subcomponent of the scoreboard, then the scoreboard can read directly from the FIFO port, as shown on the previous slide, and no explicit connection is required.

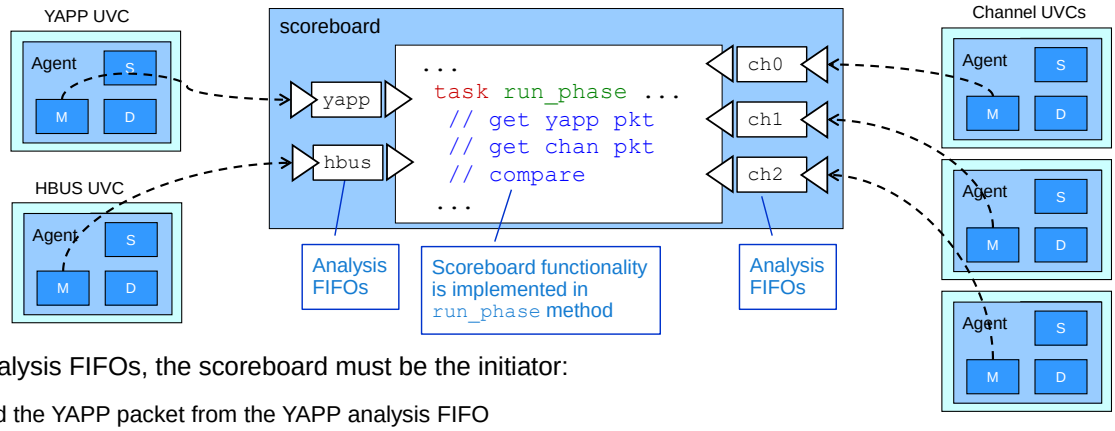
However the write side connection of the FIFO to the interface UVV monitor analysis port must be made (usually) in the testbench which has visibility of both UVC and scoreboard components.

The connection is made using a connect method call inside the connect phase method.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

YAPP Scoreboard Using Analysis FIFOs



With analysis FIFOs, the scoreboard must be the initiator:

- Read the YAPP packet from the YAPP analysis FIFO
- Read the corresponding channel packet from a channel FIFO
- Compare packets and update statistics
- Use a forked block to handle HBUS transactions

Timing of multiple reads can be challenging, depending on the design

264 © Cadence Design Systems, Inc. All rights reserved.

cadence

A scoreboard structure which uses analysis FIFOs to connect to UVC monitors, must implement most of it's functionality via a run phase task. When using analysis imp connectors, the checking is executed by the write of transactions to the scoreboard (push operation). With analysis FIFOs, checking must be executed by initiating reads on the FIFOs (pull operation).

The scoreboard should contain internal variables which maintain a local copy of the router `enable_router` and `maxpktsize` registers.

Using a forked block, the scoreboard should initiate reads on the HBUS FIFO to update these variables on HBUS write transactions, and possibly check the registers on read transactions.

Via the run phase method, the scoreboard must read the YAPP FIFO and check the registers to see if the router is enabled and the packet length is less than the maximum size.

If the YAPP packet is valid, the scoreboard must read the appropriate Channel FIFO and compare the YAPP packet to the Channel packet.

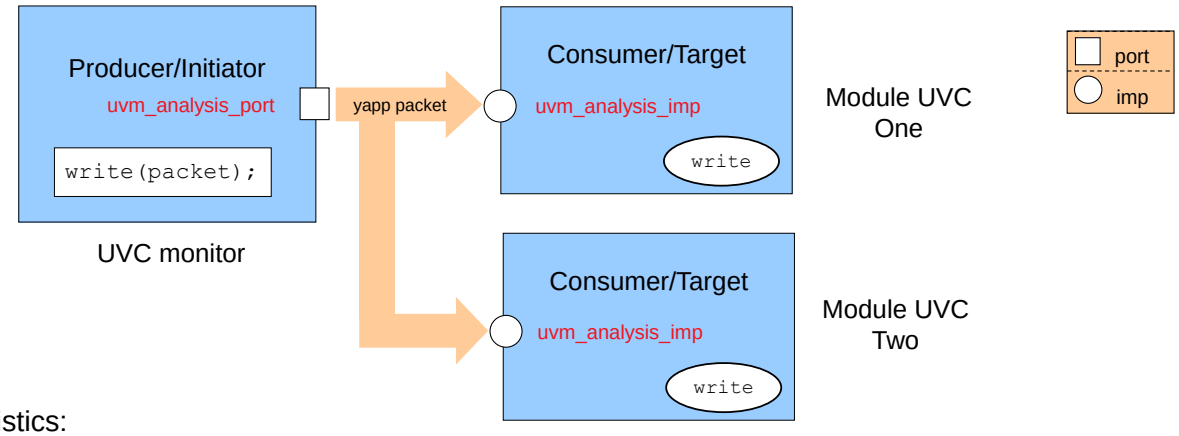
Every comparison should update counters in the scoreboard to keep track of received, dropped, matched and mismatched packets, for printing in a `report_phase()` method.

For the YAPP router, the order of reads from the FIFO's is simple and predictable. For a more complex DUT, the timing and order of reads from multiple analysis FIFOs can be challenging.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Analysis Broadcast



Characteristics:

- Any number of targets (including zero)
  - Only analysis connections can do this
- Write implementations can differ (must have same signature)
- Greatest flexibility for use

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We described analysis connections in the scoreboard module. Here is a brief review.

Analysis ports can (uniquely) be connected to any number of imp connectors, including zero. This allows us to declare analysis ports in UVC monitors; write collected data to the port and leave the port unconnected until we are ready to use it. This also allows one port write to execute multiple write implementations in separate target components, simply by connecting the port to multiple imp connectors. This a broadcast operation, and the write methods can be completely different, as long as they have the same prototype signature.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Broadcast Analysis Example

One write call can be connected to multiple, different implementations

```
class yapp_monitor extends uvm_monitor;

  uvm_analysis_port #(yapp_packet)
    send_yapp = new("send_yapp", this);

  yapp_packet packet;

...
  task xmit_packet();
    ...
    send_yapp.write(packet);
    ...
  endtask
...
```

UVC Monitor

```
class monitor_one extends uvm_monitor;

  uvm_analysis_imp#(yapp_packet, monitor_one)
    mone_in = new("mone_in", this);

  function void write
      (input yapp_packet packet);
    ...
  endfunction
...
```

Monitor One

```
class monitor_two extends uvm_monitor;

  uvm_analysis_imp#(yapp_packet, monitor_two)
    mtwo_in = new("mtwo_in", this);

  function void write
      (input yapp_packet packet);
    ...
  endfunction
...
```

Monitor Two

266 © Cadence Design Systems, Inc. All rights reserved.

cadence

The analysis `port` in component `yapp_monitor` will be connected to both `monitor_one` and `monitor_two` components. Each of the receiving components has an analysis `imp` object and a `write` communication method declared. The write method must have the same signature – i.e., they must be `void` functions called `write` with a single input argument of type `yapp_packet`, but their implementations can be completely different.

When the `send_yapp` port is connected to both `mone_in` and `mtwo_in` imps, then a single `write` call from `yapp_monitor` executes both write implementations in `monitor_one` and `monitor_two`.

© Cadence Design Systems, Inc. All rights reserved.

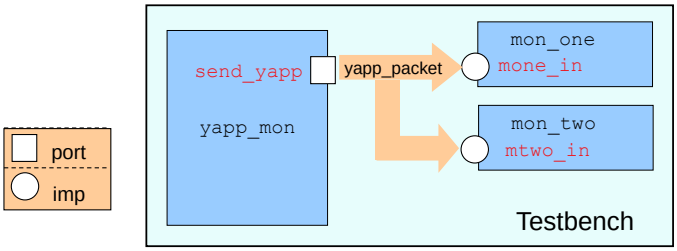
This document is for the sole use of Kun Wang of Seas

Broadcast Analysis Connection

```
class yapp_testbench extends uvm_env;

  yapp_monitor   yapp_mon;
  monitor_one    mon_one;
  monitor_two    mon_two;
  ...
  function void build_phase(uvm_phase phase);
    // create monitor instances
    ...
  virtual function void connect_phase(uvm_phase phase);
    yapp_mon.send_yapp.connect(mon_one.mone_in);
    yapp_mon.send_yapp.connect(mon_two.mtwo_in);
    ...
```

"One-to-many" connection only allowed for analysis



TB: Rules for UVM TLM Topology Connections

267 © Cadence Design Systems, Inc. All rights reserved.

cadence

In a higher level component, for example the testbench, the analysis `port` of the `yapp_monitor` instance must be connected to all `imp` connectors to which the data must be sent.

The connect method is always called off the source TLM connector and takes the destination object as the argument, for example:

```
myport.connect(myimp)
```

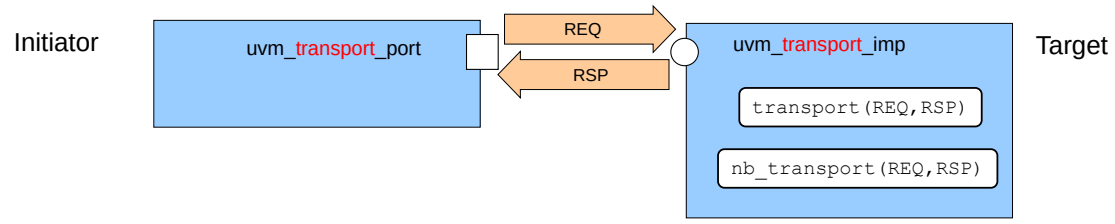
For non-analysis connections, the TLM object must be connected to a single destination only, i.e., each non-analysis object has exactly **one** connect call. An object may be used multiple times as the *argument* to a connect, but exactly once as the caller. This is called "many-to-one" connection. For example, many `ports` can be connected to a one `imp` connector.

For analysis connections, a TLM object can be connected to any number of destinations i.e., one analysis object can call `connect` many times. This is called "one-to-many" connection. For example, one `port` can be connected to many `imp` connectors. One-to-many is only allowed for analysis connections. An analysis connection can also *not* call connect. An unconnected TLM object is also only allowed for analysis connections.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Bi-Directional TLM Transport Connection



Transport connections put a request (REQ) and get a response (RSP) in one method

- Request and response can be different types

Two methods supported:

- Blocking `transport` task
  - `task transport(REQ request, output RSP response);`
- Nonblocking `nb_transport` function
  - `function bit nb_transport(REQ request, output RSP response);`
  - Returns 0 if transaction exchange cannot complete

TB: UVM TLM

 © Cadence Design Systems, Inc. All rights reserved.

Connector syntax

```
uvm_blocking_transport_xxx #(type REQ, type RSP)
uvm_nonblocking_transport_xxx #(type REQ, type RSP)
uvm_transport_xxx #(type REQ, type RSP)
```

Syntax for communication methods:

```
task transport(REQ request, output RSP response);
function bit nb_transport(REQ request, output RSP response);
```

For more information on bi-directional TLM connections, including Master and Slave TLM connections, search for UVM TLM on support.cadence.com.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM2 for UVM

A flavor of the SystemC® TLM2 standard is implemented in UVM:

- Support sockets
- Bidirectional Ports: `b_transport` and `nb_transport`
- Generic payload and base protocol
  - Allows description of stimuli in abstract systems before a protocol is decided
  - Initial release supports payload extensions

The main motivation for UVM TLM2 is connecting to SystemC ® models which use TLM2

- Multi-language solution is simulator dependent at this time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sockets are extensions to ports. You can pass different data types on the same socket.

The main use for TLM2 in UVM1.x is for connections to SystemC components already using TLM2 interfaces. However current UVM multi-language scenarios require vendor-specific solutions. There are expected to be standard solutions defined in later releases of UVM.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

TLM Summary

Most component communication should be via TLM interfaces

- Exception – multichannel sequencer
  - Reference pointers to UVC sequencers

Communication between UVC subcomponents:

- Built-in sequencer/driver TLM connection
  - `seq_item_port/export`
- Other connections must be user-defined
  - Monitor analysis port as a minimum

Communication between UVCs and higher-level components:

- User-defined TLM connections
  - Also, for communication between multi-language UVCs

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

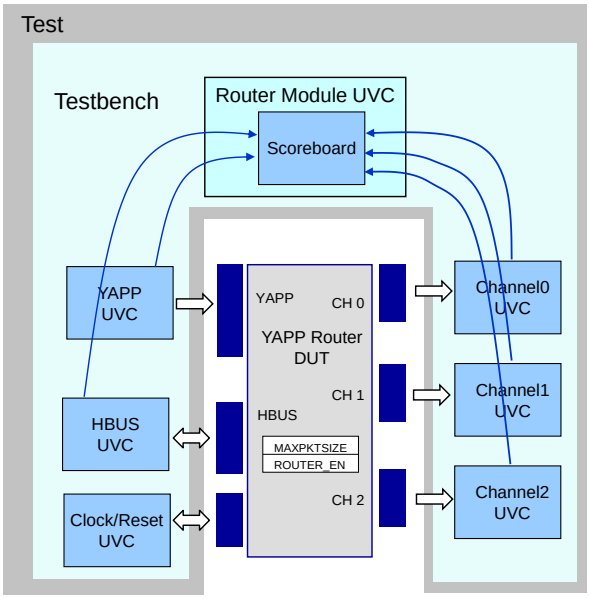
Labs (Optional)

Lab 9C Using TLM Export Connectors (Optional)

- Modifying the scoreboard to use TLM export connectors

Lab 9D Using TLM Analysis FIFOs (Optional)

- Develop an alternative scoreboard architecture using TLM analysis FIFOs to connect to UVC monitors



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 16

Functional Coverage Modeling (Optional)

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Functional Coverage Objective

In this module, you

- Use the Metric-Driven Verification (MDV) methodology to add coverage to:
  - An interface UVC monitor
  - A module UVC scoreboard

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

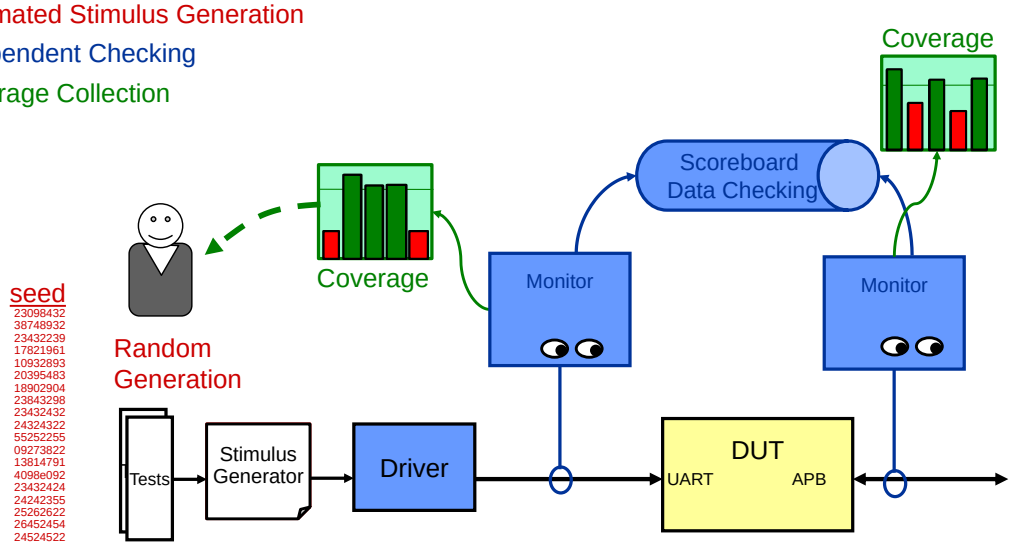
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Metric-Driven Verification Review

Components of a MDV environment

- Automated Stimulus Generation
- Independent Checking
- Coverage Collection



 © Cadence Design Systems, Inc. All rights reserved.

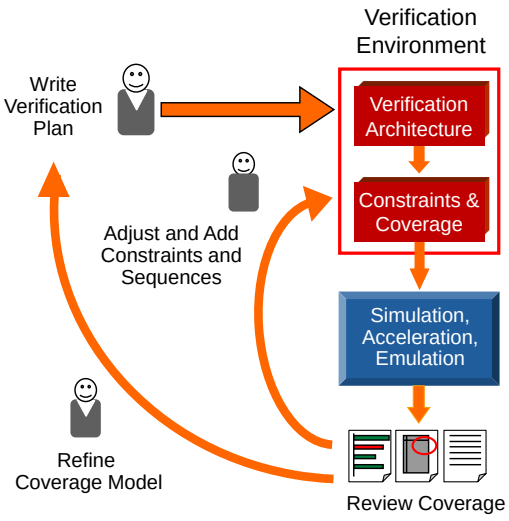
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Metric-Driven Verification Flow



Create a verification plan:

- Based on specification requirements
- Defines what to test and how
- Defines coverage model

Create smart testbench that:

- Generates random legal stimulus
- Includes monitors and coverage to measure progress against the goals
- Checks for undesired behavior (bugs)

Iteratively run simulations and analyze coverage

Adjust constraints and add new scenarios targeting coverage holes

Refine and enhance the coverage model

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Start with creation of verification goals using a planning process.

Create a smart testbench that generates random legal stimulus and sends it to a DUT.

Monitors and coverage are added to measure progress against the goals.

Checkers identify undesired behavior (bugs).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Why Metric Driven?

Shortens implementation time

- Random generator covers much of the “easy cases”
- New test scenarios added only to target remaining holes

Improves quality

- Visibility ensures that verification plan goals are met.
- Allows further exploration by crossing multiple dimensions, and revealing interesting (uncovered) combinations

Accelerates verification closure

- Eliminates redundant simulation cycles
- Facilitates test ranking

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Typical Coverage Questions

- Have all permutations of packets or transactions been input to the DUT?
- Have all CPU opcodes and operand combinations been input to the DUT?
- Have all legal state transitions occurred?
- Have all instruction types been interrupted?
- Have all cases of resource contention been tested?
- Have all queue limits been stressed?
- Have all modes been tested with all permutations of input stimuli?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Just a few samples. You can think of more.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Coverage Options

Multiple coverage metrics are usually required:

- Explicit (user defined)
  - Fully and clearly expressed within sources
    - Data-oriented coverage using covergroups
    - Sequence and temporal coverage using assertions
  - Planned and defined by the verification team
- Implicit
  - Derived or computed from the source
    - Code coverage measured by the simulator
  - May be defined outside the verification team
  - May be implied by the verification interface
    - For example, an industry-standard protocol

 © Cadence Design Systems, Inc. All rights reserved.

cadence

The purpose of this (and the following) slides is to emphasize that while during the verification planning process we focus on the explicit coverage (both spec-based and implementation-based), we shouldn't forget about the implicit coverage that can and should be collected and taken into account.

The metric of explicit coverage is defined by the engineer. Implicit coverage metrics are inherent characteristics of what we are measuring, such as the device RTL.

Specification coverage requires metrics derived from the device functional specification. Implementation coverage uses metrics derived from the device RTL source. Explicit implementation: for example, RTL-visible detail such as a four-stage fully-pipelined encryption engine. The metrics themselves may be data or temporal.

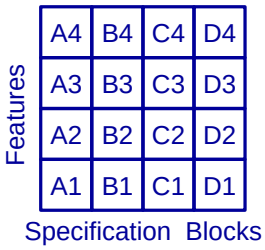
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Metric-Driven Verification Example

This table represents the attributes we want to verify:

- Goals
- Functional requirements
- Checks
- Block-level coverage



Multiple coverage metrics should be used:

- Explicit functional coverage
- Explicit assertions (static formal verification and simulation)
- Directed tests
- Implicit code coverage

For some attributes, multiple metrics are required to determine coverage

For others, a single metric is sufficient

 © Cadence Design Systems, Inc. All rights reserved.

cadence

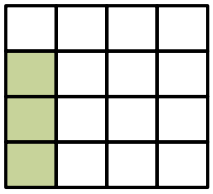
This page does not contain notes.

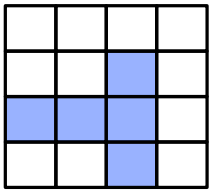
© Cadence Design Systems, Inc. All rights reserved.

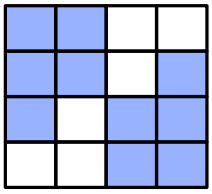
This document is for the sole use of Kun Wang of Seas

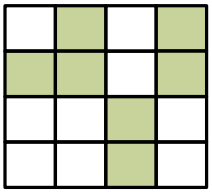
Integrating Coverage Techniques

Initial Coverage Results


Directed Tests


Assertions

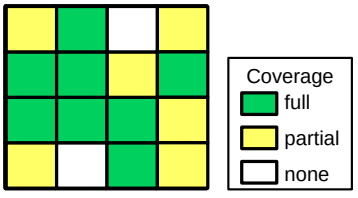

Functional Coverage


Code Coverage

Recommendations:

- Start with explicit coverage to represent verification goals
- Balance with implicit coverage to ensure thoroughness

Good tool required to integrate coverage from different sources!



Total Coverage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Explicit Coverage in SystemVerilog

Two types of explicit functional coverage:

- Assertions for control-oriented coverage
  - Defined as procedural statements
  - Cannot be defined in a class
  - For example, did `gnt` follow `req` after three, four, and five cycles

Assertion functional coverage

```
property req_gnt (cyc);
 @(posedge clk)
   $rose(req) ##0 (req && !gnt)[cyc] ##1 gnt;
endproperty

cover property (req_gnt(3));
cover property (req_gnt(4));
cover property (req_gnt(5));
```

- Covergroups for data-oriented coverage
  - Can be declared as a class member and created in class constructor
  - Used in interface and module UVCs
  - For example, did we send all size packets to all destinations

Covergroup functional coverage

```
covergroup cg @(posedge clk);
  len : coverpoint pkt.length {
    illegal_bins zero  = {0};
    bins sml   = {[1:10]};
    ...
  }
  addrxlen : cross pkt.addr, len;
endgroup

cg cg1 = new();
```

A module covergroup instance has a separate name

281 © Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog offers both data-oriented functional coverage and control-oriented functional coverage.

Control-oriented coverage uses SystemVerilog Assertion (SVA) syntax and the cover directive. It is used to cover sequences of signal values over time. Assertions cannot be declared or “covered” in a class declaration, so their use in an UVM verification environment is restricted to interfaces only.

Data-oriented coverage uses the covergroup construct. This samples variable values on a clock or signal event, and increments a counter (bin) corresponding to the sampled value. Sampled values can be cross-correlated to count, for example, how many packets of a certain length were sent to a certain address. Covergroups can be declared and created in classes, therefore they are essential for explicit coverage in an UVM verification environment.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Where to Place Coverage

Usually, coverage comes from monitors

- Can still obtain coverage when agent is passive

Interface and module UVC monitors used for difference coverage goals, for example:

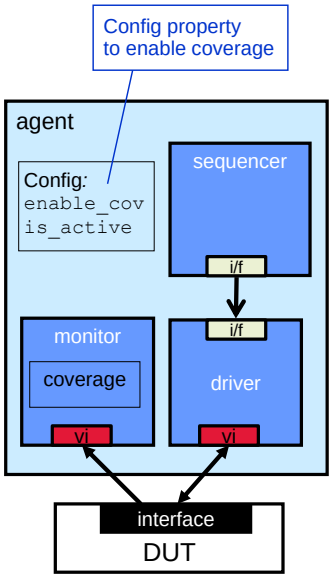
- Interface may cover data permutations
- Module may cover routing, latency ...

Coverage may also be placed in UVCs directly

- For example, tracking architectural properties over several simulation runs
  - Number of slaves on a bus

Coverage could be placed in data item class

- Need policy/control knob to control creation
- Customised coverage for different goals is difficult



282 © Cadence Design Systems, Inc. All rights reserved.

Most coverage constructs are placed in monitors. This allows coverage to be captured even if the agent or UVC is passive. In fact there is an argument that most debug code such as packet printing should also be placed in monitors. In this way both the driver's conversion of packet to RTL signals, and the monitor's reconstruction of the packet from RTL signals are tested.

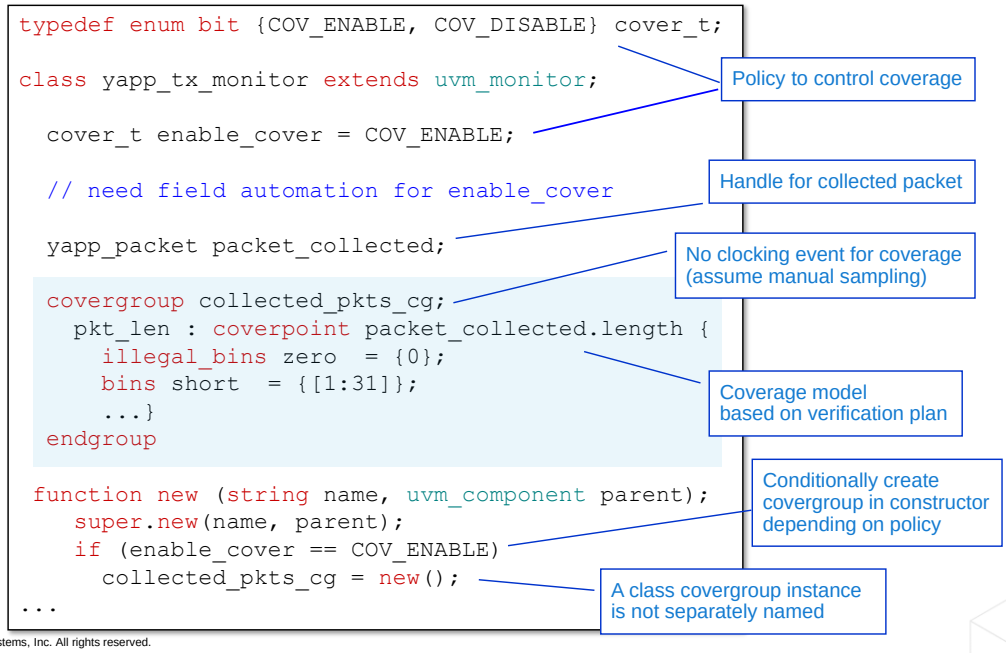
For covering some architectural properties over several simulation runs, e.g., number of slaves on a bus, coverage may occasionally be added to directly to UVCs or agents.

Coverage may also be placed in simple data class declarations. However it would be inefficient to create a coverage model for every instance of the data class, in every monitor, driver, sequence etc. Therefore at the minimum, a policy should be defined to enable/disable coverage. A “one size fits all” coverage model hardwired into the data class may be inefficient and inflexible for meeting a variety of coverage goals. Also managing the coverage model over a complex data class hierarchy may be difficult. For example there is no way to add new coverpoints to an existing covergroup, so for adding extra subclass properties to the coverage model would be difficult.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface Monitor Coverage Declaration



283 © Cadence Design Systems, Inc. All rights reserved.

The covergroup declaration is written according to the requirements of the coverage model defined in the verification plan.

Remember a covergroup instance must be created using new in order to track values. The covergroup is created in the monitor constructor.

When a covergroup instance is created in a class, it does not have a separate instance name. The new constructor is called directly off the declaration name of the covergroup.

The covergroup instance must be created in the class constructor, not a UVM phase method (such as build_phase)

A policy control knob is used to enable or disable coverage by conditionally creating the covergroup depending on the policy value. By default, the coverage is enabled in this example. The property enable_cover could be set from a test or testbench using a configuration set method. Multiple or alternate covergroups could conditionally created by expanding the policy.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Interface Monitor Coverage Trigger

Typical interface monitor coverage:

- Packets of all lengths are sent
- Packets sent to all addresses
  - Legal and illegal
- Different packets sent with zero delay to a single address
- Parity errors sent to all addresses
- Cross-correlation of the above
  - Number of bad parity packets sent to each legal address
  - Number of long packets sent to each legal address
  - Number of consecutive packets sent to each address with zero delay

```
class yapp_tx_monitor extends uvm_monitor;

  // constructor

 covergroup collected_pkts_cg;
   ...
 endgroup

  yapp_packet packet_collected;

  task run_phase(..)
    collect_packet();
    ...

 task collect_packet();
   // read RTL signals via interface
   // reconstruct packet
   collected_pkts_cg.sample();
   ...
```

Task to collect packets from interface

Trigger coverage sample when packet collected

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Module UVC Coverage

Typical module UVC coverage:

- Routing – packets flow from input ports to all legal outputs
- Latency – all packets received within specified delay

Scoreboard may also contain coverage

```
class router_monitor extends uvm_monitor;

  int chan0_rec;
   ...

  covergroup chan_cov;
    ch0: coverpoint chan0_rec {
       ignore_bins ch0_uncov = {0};
       bins ch0_cov = {[1:$]}; }
    ...
  endgroup: chan_cov

  function void write_chan0(yapp_packet pkt);
    ...
    chan0_rec++;
    chan_cov.sample();
  endfunction


  function new (string name, uvm_component parent);
    super.new(name, parent);
    chan_cov = new();
...
```

Coverage model based on verification plan

Analysis port write from UVC monitor

Coverage trigger

Create covergroup

285 © Cadence Design Systems, Inc. All rights reserved.

cadence

The module UVC monitor receives data from the interface monitors via analysis ports, and so the implementation of the analysis write function is a convenient place to put coverage code.

In this simple model, every time channel0 receives a packet, a channel counter is incremented and the coverage model triggered. In the coverage model, a zero count is ignored and a single bin records the number of packets received. In this example there is no distinction between bad packets (e.g., bad parity) and good. You may need to avoid triggering coverage on bad packets.

A scoreboard may be an additional or alternative place to implement coverage.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

When to Cover

When

- As soon as info is available?
- As soon as info is sent to the DUT?
- As soon as info is processed by the DUT?

Ideally, coverage sampling is delayed until DUT responds to the scenario

- Avoids false optimism of covered but not checked

Simplify coverage by relying on a checker

- Sample coverage when stimulus is driven into the DUT
- Ensure that the checker delays sim until response is checked
- Only include coverage results from passing simulations

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary: With and Without MDV

Without MDV

- A lot of time is spent writing tests and constraint sets
- High effort in creating and maintaining the hundreds of tests required for signoff
- Guesswork is required to know if verification is done
- There can be many redundant simulations; there is no way to know

With MDV

- Test writing and constraints are only written for corner cases
- Only a small number of tests need to be written and maintained
- Verification “doneness” can be tracked according to the coverage goals and metrics indicated by the verification plan

 © Cadence Design Systems, Inc. All rights reserved.

cadence

MDV principle: Rely on a few dozen constraint sets, each using multiple seeds, to reach most scenarios. Rely on coverage to show which scenarios remain unreached and need new constraint sets.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 10 Creating a Simple Functional Coverage Model (Optional)

- Create a coverage model for the input packet traffic to collect the following info:
  - Ensure all lengths of packets are sent into DUT. All lengths are bucketized.
  - Ensure all addresses received a packet, including illegal address.
  - Ensure all size packets were sent to all addresses with parity errors, except for address 3.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Coverage in Xcelium

`xrun` coverage options

`-coverage <options>`

`Block|Expression|Toggle|Fsm|fUnctional|All`

`-covworkdir <cov_work_dir>`

default is `./cov_work`

`-covdut <design_top>`

Select DUT top for instrumenting coverage

`-covdesign <design name>`

Identifies the coverage design

`-covtest <test_name>`

pass `$TESTNAME` to write separate `.dat` files

`-covoverwrite`

Enables overwrite of coverage data files (useful during development)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 17

Register Modeling in UVM

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Register Modeling in UVM Objectives

In this module, you

- Apply a simple approach to UVM Register Verification

This module is in four parts:

- Introduction
  - Describe the purpose and aim of register modeling
- Generation
  - Create and test a register reference model
- Integration
  - Add the model to an environment and run built-in tests
- Simulation
  - Create user-defined stimulus

Note in this context, register modeling also covers memory verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

What Do We Need to Verify?

Address	Register	Reset	Field	Field Name	Policy	Description
0x1000	ctrl_reg	0xff	5:0	maxpktsize	RW	Maximum packet length
			7:6		RW	Unused
0x1001	en_reg	0x01	0	router_en	RW	Router enable
			1	parity_err_cnt_en	RW	Parity error count enable
			2	oversized_pkt_cnt_en	RW	Oversized packet count enable
			3	[reserved]	RW	Not implemented
			4	addr0_cnt_en	RW	Address 0 packet count enable
			5	addr1_cnt_en	RW	Address 1 packet count enable
			6	addr2_cnt_en	RW	Address 2 packet count enable
			7	addr3_cnt_en	RW	Illegal address 3 packet count enable
0x1004	parity_err_cnt_reg	0x00	7:0		RO	Packet parity error count
0x1005	oversized_pkt_cnt_reg	0x00	7:0		RO	Oversized (dropped) packet count
0x1006	addr3_cnt_reg	0x00	7:0		RO	Illegal address packet count
0x1009	addr0_cnt_reg	0x00	7:0		RO	Address 0 packet count
0x100a	addr1_cnt_reg	0x00	7:0		RO	Address 1 packet count
0x100b	addr2_cnt_reg	0x00	7:0		RO	Address 2 packet count
0x100d	mem_size_reg	0x00	7:0		RO	Length of last packet received

Check functionality

Check register and memory addressing

Check reset

Check field offsets

Check access

Check memory width

Start Address	Memory Name	Size	Policy	Description
0x1010	yapp_pkt_mem	0:63	RO	Stores last packet received
0x1100	yapp_mem	0:255	RW	"Scratch" memory

292 © Cadence Design Systems, Inc. All rights reserved.

cadence

These are the registers for the extended YAPP router design.

We need to verify these registers and memory blocks as part of the overall router verification. Specifically we need to check that:

- Registers and memories are accessed by the specified address and only the specified address (i.e., we don't have aliasing – multiple registers mapped to same address – unless explicitly required).
- Registers are reset to the correct value.
- Fields are mapped to the correct location with no aliasing.
- Access policies are correctly implemented.
- Memory widths are correctly sized.
- Functionality of registers and memories are as required. For example, the address 0 packet counter increments when address 0 packets are received, if the counter is enabled and the packet has not been dropped due to length exceeding maximum packet size (oversized packet).

We can also check possible ambiguities in the specification. For example:

- If an address 0 packet is dropped due to oversize, is the address 0 counter still incremented if enabled?
- If an illegal address 3 packet with bad parity is received, is the parity error counter still incremented if enabled?

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Would You Verify the Router Registers?

Create a reference model of the registers

- Address, size, reset value, access policy (RW, RO..) etc.
- Holds "shadow" copies of DUT registers
  - But not for memory

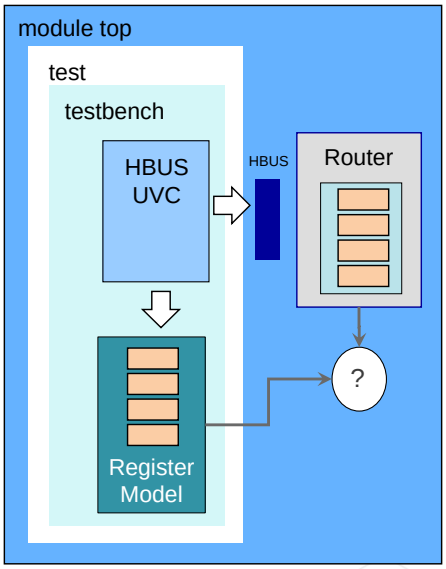
Create stimulus to read/write DUT registers

- Using sequences of HBUS UVC

Update register model with expected results

Compare DUT registers against register model

How can UVM help?



293 © Cadence Design Systems, Inc. All rights reserved.
cadence

How would you verify the YAPP Router registers with UVM? Here's one plan:

1. Create a reference model for the registers to model their characteristics (address, size, reset, access policy). This should also include a model of the memory blocks in the router so we can check simple memory access operations.
2. Create stimulus for the HBUS UVC to read and write the router registers. We should be able to use or modify the supplied sequences in the HBUS UVC sequence library.
3. Connect the register reference model to the analysis port of the HBUS UVC monitor to capture the HBUS transactions on the router. The register model must update the shadow registers according to the router specification, for example:
   a. Increment `addr0_cnt_reg` when an address 0 packet is received and the counter is enabled..
   b. Store the last received packet in `yapp_pkt_mem` and the length of the received packet in `mem_size_reg`.
4. Check the register values in the DUT against the values in the reference model after each set of stimulus to make sure the two match.

This is quite a significant amount of work, particularly in creating the reference model and stimulus sequences. How can UVM help us?

© Cadence Design Systems, Inc. All rights reserved.

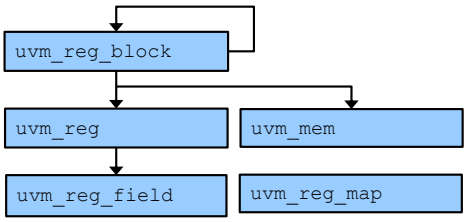
This document is for the sole use of Kun Wang of Seas

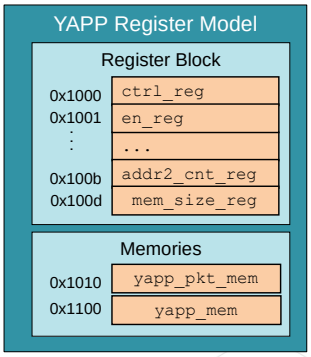
Register Model

UVM provides base classes for modeling registers and memories

Used to create register models:

- Hierarchical architecture
  - Define fields, registers, blocks and memories for the DUT
- Capture register attributes
  - Address, size, endianess, reset value, access policy (RW, RO, etc.)
- Link a name to address
  - Write to `en_reg` rather than `0x1001`
- Hold expected values for checking
- Supports multiple address maps
  - Where registers/memories are accessible via multiple physical interfaces
- Can include register coverage
- Reusable within and between projects





294 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM provides classes specifically for modeling register and memory blocks. The classes support key functionality such as:

1. Hierarchical architecture. You can define fields within registers. You can define registers, memories and/or register blocks within other register blocks.
2. Definition of key register attributes, such as address, size, reset, access and endianess.
3. Linking a name to a register address, so that in stimulus you can refer to the register name (write `en_reg`) rather than the address, aiding readability.
4. Several copies of register values (e.g., expected, intended) to help with verification.
5. Separate address mapping to allow multiple maps where individual registers are accessed via different address from different interfaces of the DUT.
6. Support for adding coverage to register access and values.

UVM register models are intended to be reusable blocks of verification IP to be portable within and between projects.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Register API Access Methods

Methods for register/memory access:

- To read/write/randomize register models and DUT
- Frontdoor (interface) and backdoor (pathname) access
- Allows register operations to be independent of the bus protocol
  - Don't need to know protocol
  - Register sequences can be reused with different protocols
- Reusable within and between projects

Built-in sequences for common register operations also provided

- Read/write testing, bit-bashing, memory test patterns

```
hbus_small_packet_seq  small_pkt;
hbus_read_ctrl_seq     rd_ctrl;

...
`uvm_do(small_pkt)
`uvm_do(rd_ctrl)
```

Instead of UVC-specific sequences...

... call standard register access methods

```
yapp_regs.ctrl_reg.write(status, 8'h14);
yapp_regs.en_reg.write(status, 8'h01);
yapp_regs.ctrl_reg.read(status, rdata);
```

Register block

Register

295 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM also provides an API for accessing registers. These are a set of standard method calls to:

- Read and write the register model and the DUT.
- Set, get or randomize values in the register model only.
- Synchronize the DUT to the register model or vice-versa.

The API calls are made using a hierarchical pathname through the register model to the register being accessed. Each register model must contain at least one register block, so the minimum path is `<model>.<block>.<register>`. This means you need a handle on the register model to make an API call, which can be an issue (see later).

Access operations on the DUT can either be front-door or back-door. Front door operations are via transactions of the interface UVC (e.g., HBUS read or write operations). Backdoor operations are via a hierarchical pathname (Out-Of-Module Reference) directly to the variables in the DUT representing the registers.

Frontdoor API calls are converted to interface UVC transactions by an adapter (see later).

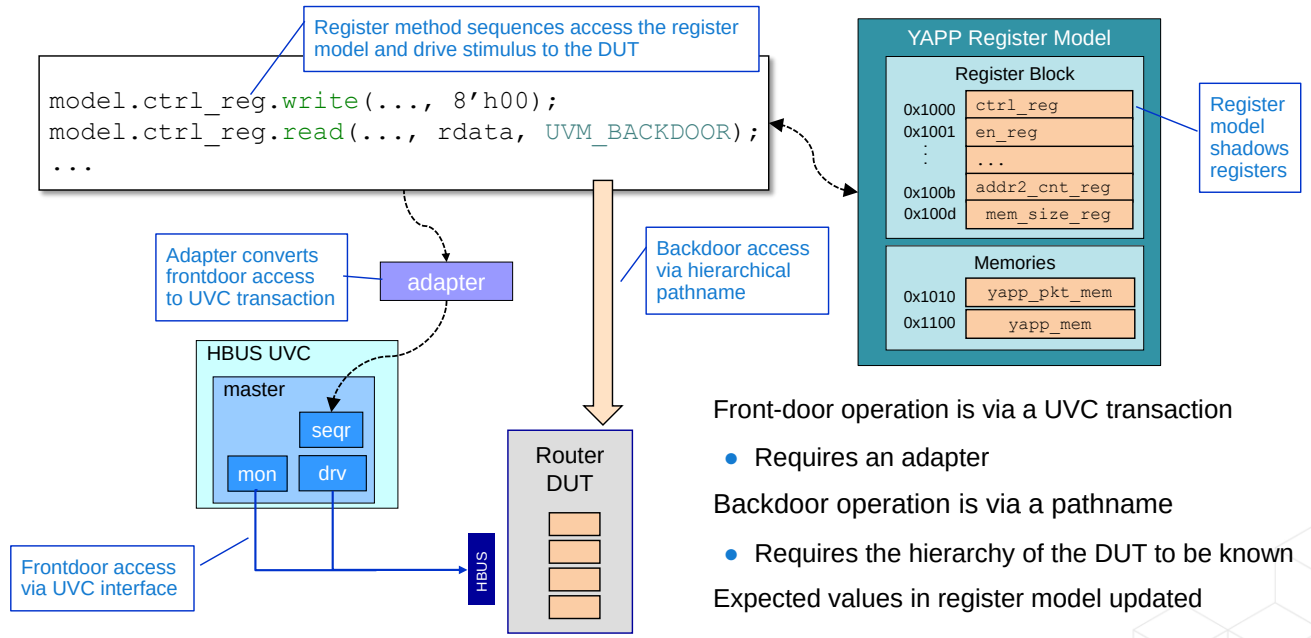
The advantage of the API is that register operations can be independent of the interface protocol and can be used with different interfaces simply by changing the adapter. This makes them reusable across projects.

UVM also provides a number of useful, built-in register sequences (e.g., reset test and simple memory tests) can be simply instantiated and executed.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

How Does It All Fit Together?



We write register sequences using register access method calls. These calls can drive stimulus into the DUT and update the register model automatically. The aim is to maintain the register model with the expected contents of the DUT registers and check frequently for mismatches between the two.

Front-door operations on the DUT are executed from an interface UVC. This requires an adapter which converts between the generic properties of the access call and the UVC specific transaction. The adapter should be provided as part of the UVC package.

Front-door operations involve signal transitions on the interface and consume clock cycles.

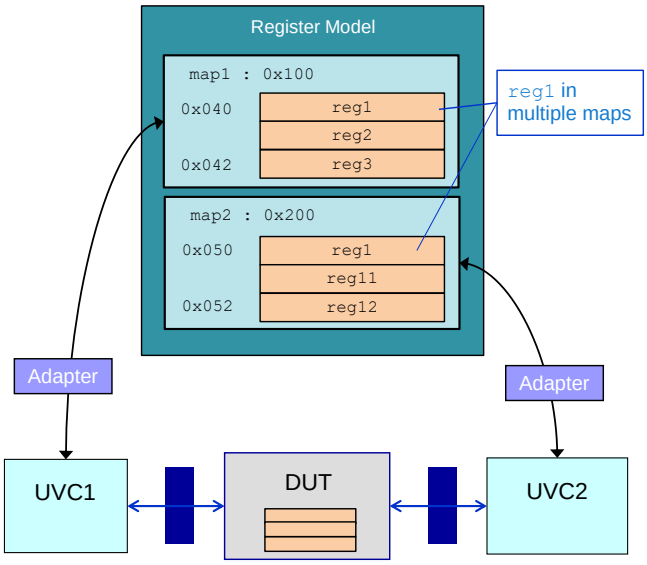
Back-door operations use a hierarchical pathname to the register variables in the DUT. The pathname to each register is contained in the register model, Changing the DUT hierarchy may require updating the register model.

Backdoor operations are instantaneous and do not change interface signals.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Address Maps



Multiple interfaces may access the DUT registers

- There is a separate register block for each interface
- A register may be accessed from multiple interfaces
  - With a different address and access policy in each

Each register-accessing interface has an address map

- Defines the registers visible from that interface
- Specifies register address and access policy for that interface

If there is only one map, it is named `default_map`

Every register access is via a map:

- Implicit if the register is singly mapped
- Must be explicit if the register is multiply-mapped
  - As an argument to the register method call

```
model.reg1.write(status, 8'h14, UVM_FRONTDOOR, map1);
```

297 © Cadence Design Systems, Inc. All rights reserved.

Registers must be defined in address maps, which specify their access policy and address (as an off-set from a base address).

We only have one physical interface on the YAPP router, so we only need address map and this is named `default_map`. However, UVM allows you to define multiple address maps if you have multiple physical interfaces to DUT registers or memories. Registers must be added to a map for direct access. When added, you define the address and the policy of the register as seen through that map. Therefore, you can have multiple views of a single register through different maps (with different address and policy). For different access policies from different interfaces, the register is defined in the model with an overall access policy (e.g. Read-Write) and this policy can be refined in each map. For example, the register may be Read-Only in `map1` and Write-Only in `map2`.

If a register is declared in more than one map, you must specify the map you wish to use as an argument to the API register access method. If the register is declared in a single map, the map argument is not required.

Remapping is possible when (for example) transitioning from block level to system level testing.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Generation

Creating the register model can be difficult:

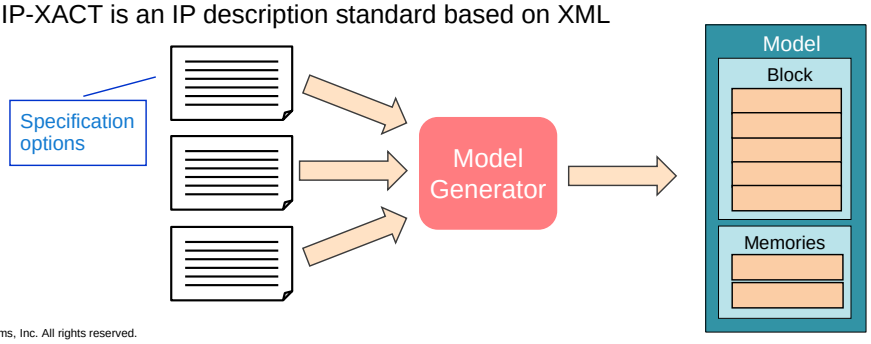
- There may be thousands of registers
- Error-prone with many code dependencies

You can generate the register model automatically from a register specification

- Easier to update model if specification changes

Several standard, proprietary or ad-hoc formats supported for register specification

- IEEE 1685 IP-XACT is an IP description standard based on XML



 © Cadence Design Systems, Inc. All rights reserved.

Hand-coding the register model from basic UVM register classes can be difficult. There are many code dependencies to take into consideration, and register characteristics may be defined in multiple locations. Also the register model hierarchy must be built by hand, and this gives further opportunity for errors and omissions.

A far better solution is to generate the register model automatically. Model generator tools can take register specifications and create the register model UVM code ready for inclusion in your test environment. Also if the register specification changes, the register model can be simply regenerated for quick and easy update.

A commonly used standard for the register specification is IP-XACT (IEEE1685) which is based on XML. This is a standard for describing IP characteristics which can be modified to define register behavior.

Other forms of register specification can be used such as proprietary formats or spreadsheets.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

IP-XACT XML for addr0_cnt_reg Register

```
<!- addr0 count register -->
<spirit:register>
  <spirit:name>addr0_cnt_reg</spirit:name>
  <spirit:addressOffset>0x9</spirit:addressOffset>
  <spirit:typeIdentifier>addr0_cnt_reg_c</spirit:typeIdentifier>
  <spirit:size>8</spirit:size>
  <spirit:access>read-only</spirit:access>

  <spirit:reset>
    <spirit:value>0x00</spirit:value>
    <spirit:mask>0xff</spirit:mask>
  </spirit:reset>

  <spirit:vendorExtensions>
    <cadence:extensions>
      <cadence:hdlAccess cadence:path="mode0_reg"/>
    </cadence:extensions>
    <vendorExtensions:collect_coverage>cov_all</vendorExtensions:collect_coverage>
  </spirit:vendorExtensions>

</spirit:register>
```

addr0_cnt_reg name

Offset from a base address

Identical registers can be given same type name for efficiency

Access policy

Reset value and mask

addr0_cnt_reg
0x1009 addr0_cnt_reg_fld
7 0

Cadence vendor extensions define (e.g.) backdoor path

Generic vendor extensions define coverage

XML is typically created by third-party tools or editors

299 © Cadence Design Systems, Inc. All rights reserved.

cadence

This is the IP-XACT 1.5 description of `addr0_cnt_reg`. It defines all the key characteristics of the register. It also defines that the register contains a single, full-width field which is given the derived name `addr0_cnt_reg_fld` by reg_verifier. Note also the access policy and reset value.

The IP-XACT schema is currently not sufficient to describe everything we need for a register in a UVM model. However the schema supports arbitrary vendor extensions for this information. There are two types of vendor extensions.

1. Generic extensions which are supported by most model generators (here `collect_coverage`).
2. Vendor specific extensions which are supported by one vendors generators only (here Cadence's backdoor pathname definition using hdlAccess).

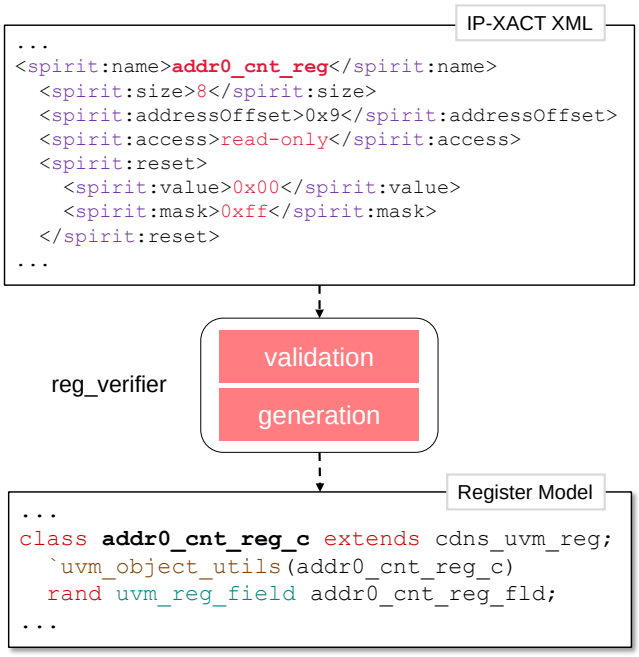
The IP-XACT specification is gradually adding some of the more common vendor extensions to the list of supported elements, but it is unlikely to support all possible options, therefore vendor extensions will always be an important part of a register specification.

There are many third party tools that support different input formats and output IP-XACT XML. However to be useful, these tools must output IP-XACT 1.5 which allows vendor extensions, otherwise you will be unable to use fundamental UVM register features like backdoor API access.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence's Model Generator – reg_verifier



Generates register model

Input:

- IP-XACT XML
  - IEEE 1685 standard
  - Optional Cadence extensions

Schema validation

- Checks XML complies with IP-XACT

Output:

- UVM Register Model

Can create and run a quick test on the generated code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

XML is a markup language, like, for example, HTML. XML code that conforms to the language rules is described as “well-formed.” A schema is a set of constraints on the XML structure and content, defining, for example, allowed tags and their order of definition. XML code which conforms to a specific schema is described as “validated.”

IP-XACT is an XML schema for design IP documentation created by Spirit Consortium – which is now part of Accellera.

The Cadence® Register Model Generator, `reg_verifier`, generates a UVM register model from a valid IP-XACT definition. `reg_verifier` first validates the XML before generating the model. As such, your IP-XACT XML needs references in the header to the schema being validated. See `reg_verifier` documentation for more details.

As IP_XACT is intended for IP documentation and not register modeling, some UVM features require additional markups beyond those defined in IP-XACT. The schema allows optional vendor-specific markups, called extensions. It is expected that key vendor extensions for specific UVM features will eventually become part of the IP-XACT standard.

A useful feature of `reg_verifier` is that it can create a testbench to allow immediate compilation and simulation of the register model, so the user can check the model meets their requirements.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Using reg_verifier

Part of the simulator installation

```
reg_verifier -domain uvmreg -top <IP-XACT file> -dut_name <model name> [options]
```

Key options

`-target_dir <name>`	Output directory where the generated files will be created
`-out_file <name>`	Output SV filename
`-pkg <name>`	Name of the SV package containing the model
`-quicktest`	Generates a quick test to check the register model
`-cov`	Enable coverage in model

Note: Use the `-h[elp]` option to see the full list

301 © Cadence Design Systems, Inc. All rights reserved.

cadence

Other selected options:

`-incdir`	Comma separated list of include directories for XML files
`-init_xml`	Initialization XML files to import before importing the design (top)
`-old_vendor_schema`	Enable old style vendor extensions conversion

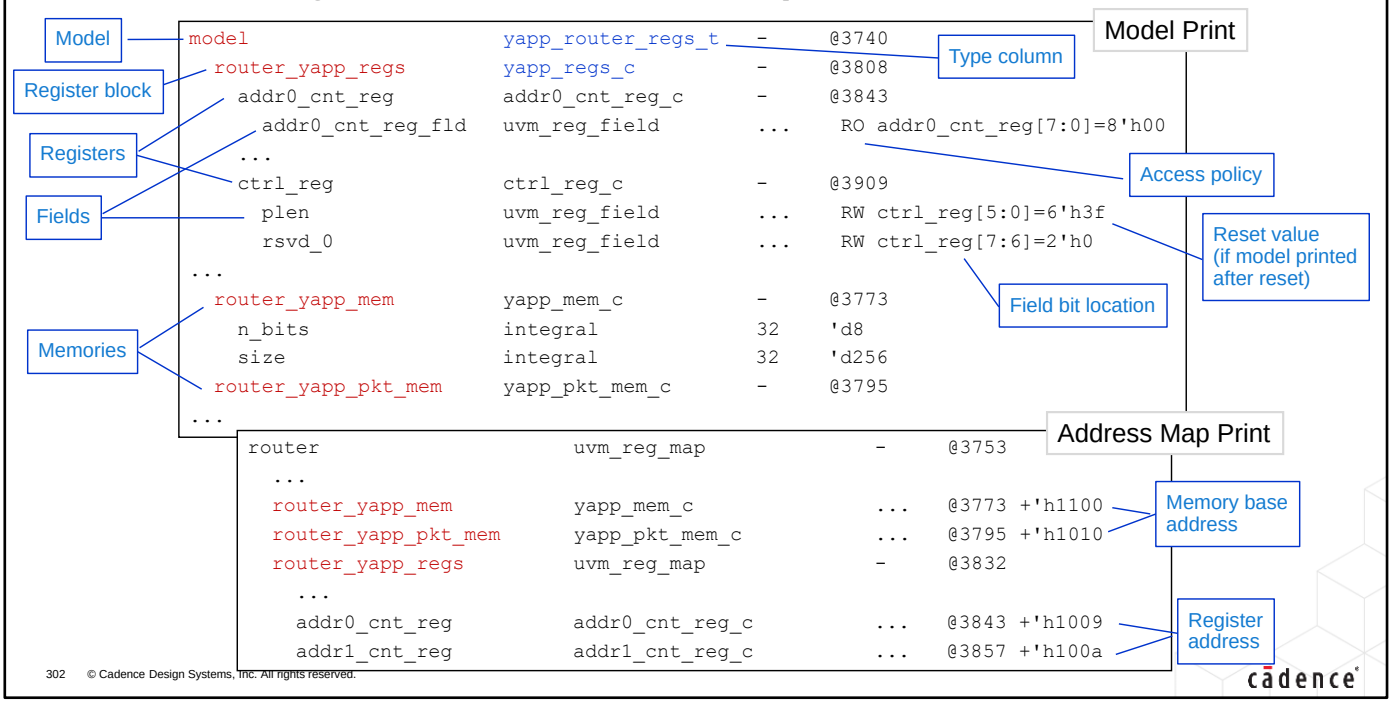
Domain Options (uvmreg)

`-no_uvm_factory`	Disable factory (performance improvement for huge models)

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Understanding Model and Address Map Prints



Printing the model will show you:

- The model hierarchy, of model containing one or more register blocks which contains registers with individual fields. Also memory is shown, usually outside of the register block.
- The types for the model, block, register, field and memory elements. Note the field types are UVM types, whereas everything else is a user-defined type. We need these types for instantiating the register model in pour testbench and for creating convenience handles to make register stimulus easier (see later).
- The access policy of the each field.
- The reset value of each field, but only if we reset the model before we print. Some version of the quicktest in reg_verifier print the model first, so modifying the test to print after reset is useful.
- Field bit locations in the register.

Printing the address map shows us the base address of memories and the addresses of registers as seen through that map. We need one map for every physical interface to the DUT registers, and this allows us to have registers visible from different DUT interfaces via different addresses and possibly with different access policies. For example, a register may be RO via one interface and WO via another.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 11A Generation

Creating a register model and performing simple tests

- Create a register model for the YAPP router
- Run the basic QuickTest on the register model
- Instructions are in the Lab document

Time: 15 minutes

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Integration

Next stage is to integrate the register model into the testbench

Some assumptions:

- Keeping the model in sync with the DUT (prediction)
  - Use Implicit Prediction:
    - Easiest prediction model to use
    - Will only capture HBUS transactions generated by register methods, not HBUS sequences
- Instantiate register model directly in testbench class
- Call register methods from test class
  - Need access to register model
    - From test, access model easily via a hierarchal path

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Once we have generated the register model, we need to integrate it into the test environment

For simplicity, we are making the following assumptions:

Prediction – keeping the register model and DUT in sync.

We could manually update the register model for every DUT registers access, but it would be easier if this happened automatically. UVM has a default Implicit Prediction mode, where an API access to a DUT register also tries to keep the register model up to date. Implicit prediction is the easiest mode to use, but it only updates the register model for API DUT registers access. If DUT registers are also accessed by interface UVC sequences not generated from an API call, then Explicit Prediction is required, which is beyond this module.

Register Model Instantiation

We will instantiate the register model directly in the testbench class. The register model may be specific to a particular variant of the DUT, so this is the best place for the model.

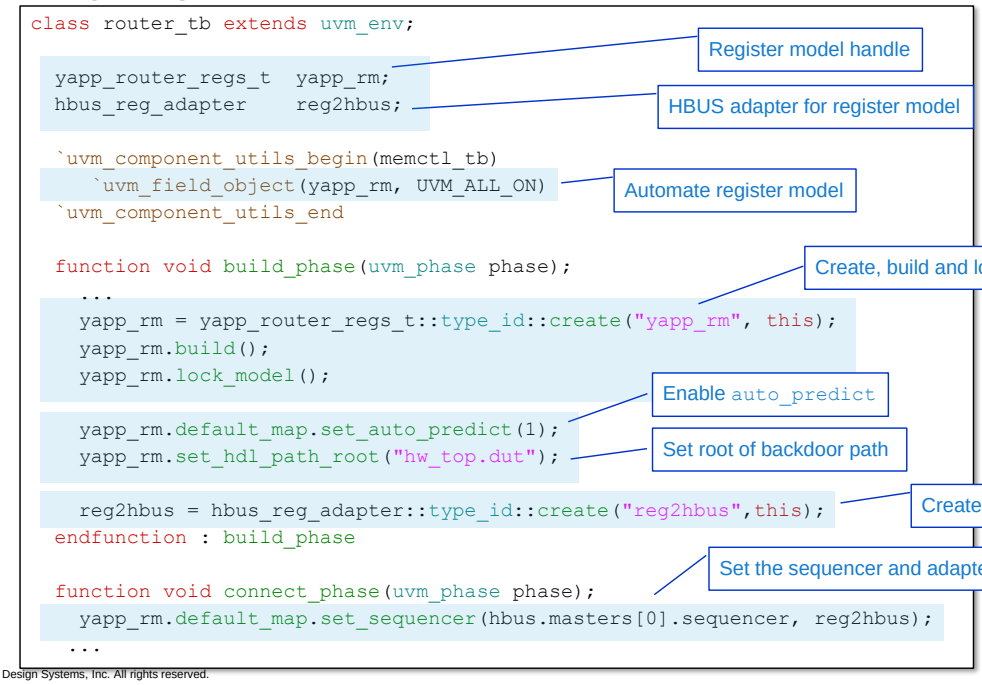
Executing Register Sequences

To execute an API register call, we need access to the register model. By executing register sequences directly from the test class, we can reference the register model with a simple hierarchical pathname. You can obviously execute API register sequences in a multichannel sequence; as the subsequence of another sequence or as a default sequence of the UVC, but this requires reference of the register model via the configuration database, which is not as straightforward.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Integrating Register Model into Testbench



Here are the updates required to the testbench class to integrate the register model:

1. Declare a handle for the top level of the generated register model.
2. Declare a handle for the UVC adapter (should be supplied with the UVC package).
3. Automate the register model handle as an object for debug.
4. Create, build and lock the register model. As the model is a data item, you must explicitly build the hierarchy. It is not built by the build phase (which is for components only). Locking the model creates the address map and prevents any further changes.
5. Turn implicit auto-prediction on for the address map. A model with a single, unnamed address map (because it has only one physical interface) uses the name `default_map`.
6. Create an instance of the adapter.
7. Finally in the connect phase, set the sequencer for the address map. The first argument is the interface UVC sequencer for the map, the second is the adapter instance. This makes the connection between register API calls and interface UVC transactions.

Prediction tries to keep the shadow copies of the register fields in the model up to date with what we expect to find in the DUT. This is for checking. So, for example, a write to a DUT register will also update the register model. Auto-prediction is one of 3 built-in UVM prediction modes. With auto-predict, only a register method access will update the model. If a register is written by a UVC sequence executed directly on the UVC, then auto-predict does not update the model and a different prediction mode is required.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selected UVM Built-In Sequences for Registers

Built-In-Sequence	Description	Skip Attribute
uvm_reg_hw_reset_seq	Reads all of the register in a block and check their POR value.	**NO_HW_RESET_TEST**
uvm_reg_bit_bash_seq	Sequentially walks '0' and '1', checking that it is appropriately set or cleared, based on the field access policy specified for the field containing the target bit.	**NO_BIT_BASH_TEST**
uvm_[reg|mem]_access_seq	Write frontdoor and read (checks) backdoor, later writes backdoor and reads (checks) frontdoor.	**NO_[REG|MEM]_ACCESS_TEST**
uvm_[reg|mem]_shared_access_seq	For each address map in which the register or memory is accessible, writes the register or memory via one map then confirms that the value was written by reading it from all other address maps.	**NO_SHARED_ACCESS_TEST**
uvm_reg_mem_hdl_paths_seq	This sequence can be used to check that the specified backdoor paths are indeed accessible by the simulator.	
uvm_mem_walk_seq	Write a walking pattern into the memory then checks that it can be read back with the expected value.	**NO_MEM_WALK_TEST**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

There are built-in sequences provided in the UVM Library. If you do not want these sequences available to your register model, you can enable individual skip attributes for specific sequences through the configuration database. This disables the `uvm_reg_bit_bash_seq` for all registers:

```
uvm_resource_db#(bit)::set(
   {"REG::",all_blks.get_full_name(),".*"},
   "NO_BIT_BASH_TEST", 1);
```

The skip attribute can also be specified in the IP-XACT register specification, which is easier for a register whose behavior is unsuitable for these tests. For example, the bit bash test could not be used for a register which is incremented on every write to another register. A skip attribute specified in the register specification will be added to the register model by the generator.

```
<vendorExtensions:attributes>
  <vendorExtensions:attribute type="bit">
  <vendorExtensions:name>NO_BIT_BASH_TEST</vendorExtensions:name>
  <vendorExtensions:value>1</vendorExtensions:value>
</vendorExtensions:attribute>
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selected Cadence Built-In Sequences

Built-In-Sequence	Description	Skip Attribute
uvm_reg_built_in_read_all_regs_seq	Reads all of the register in a block. It is assumed that explicit monitor does register value checking.	NO_RD_REG_TESTS
uvm_reg_built_in_write_all_regs_seq	Writes all registers with random value. Sequence has a flag to direct the value to be specific value, otherwise it is random.	NO_WR_REG_TESTS
uvm_reg_built_in_wr_follow_rd_seq	Writes and then immediately reads every register. For writing, sequence has a flag to direct the value to be specific value, otherwise it is random.	NO_RD_REG_TESTS or NO_WR_REG_TESTS
uvm_reg_built_in_aliasing_seq	Writes a random value to a register and reads all the other registers to make sure the write did not affect them. It does this for each and every register in the block.	NO_RD_REG_TESTS or NO_WR_REG_TESTS
uvm_reg_generate_access_file_seq	This sequence can be used to generate access file for backdoor access optimization.	
uvm_reg_built_in_all_seq	This sequence is going to call all built-in-sequences one-by-one for a container.	Individual skip attributes

 © Cadence Design Systems, Inc. All rights reserved.

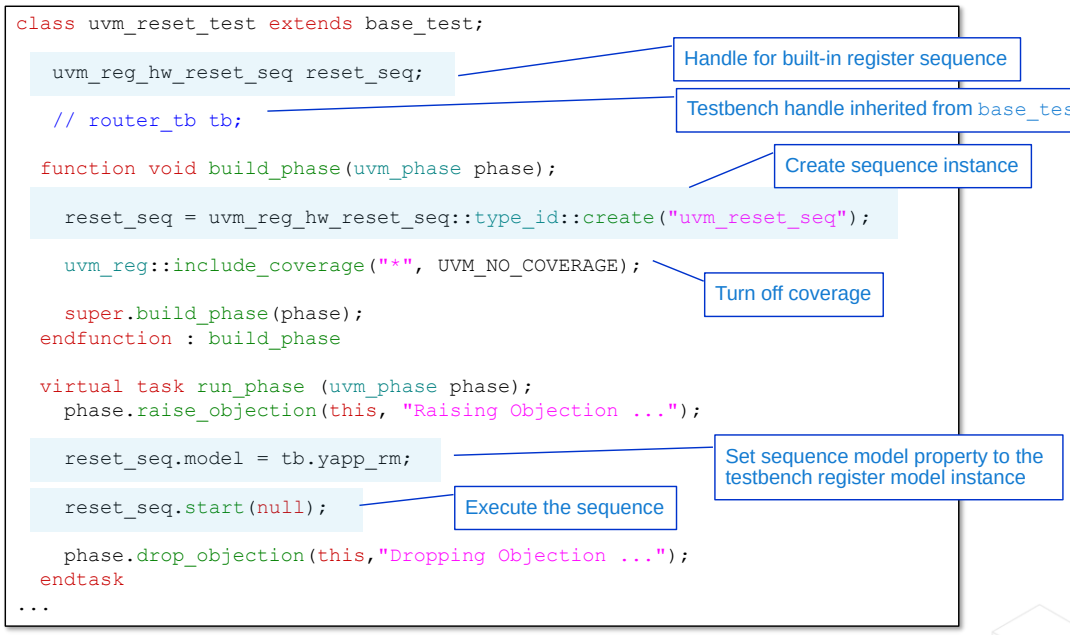
cadence

Cadence also provides several useful built-in sequences beyond that supplied in UVM. Again there are opt-out attributes to disable the sequence for as many registers are required.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Executing a Built-In Register Sequence from Test



308 © Cadence Design Systems, Inc. All rights reserved.

This code executes the built-in sequence `uvm_reg_hw_reset_seq` on the register model.

This sequence is executed from the test class for easy access to the register model. You can also execute register sequences from multichannel sequencers, as a nested sequence or as a default sequence. However in these cases access to the register model is via the configuration database.

To execute the test:

1. Declare a handle for the sequence.
2. Create an instance of the sequence.
3. In the connect or run phase, assign the `model` property of the sequence instance to the register model using a hierarchical pathname.
4. Execute the sequence using `start`. The start method uses the sequencer of the register model which was assigned using the `set_sequencer()` method in the testbench. Therefore the start argument is `null`.

We can also turn coverage off for this test via `include_coverage()`. This removes a number of warning messages from the log file.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 11B Integration

Integrating a register model and executing built-in sequences

- Integrate the register model into your testbench class
- Execute the built-in reset register test
- Run a memory test (Optional)

Instructions are in the Lab document

Time: 30 minutes

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We will execute the built-in reset register test which frontdoor reads every DUT register and compares it against the rest value stored in the register model.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Simulation

Built-in register sequences can be useful, but you will need user-defined stimulus

Using the UVM register API, we can:

- Read and write DUT registers
- Load the register model with expected results
- Randomize register values
- Update the DUT registers to match the model and vice-versa

Operations can be:

- Frontdoor via the interface DUT and consuming time
- Backdoor via a hierarchical pathname, instantaneously

 © Cadence Design Systems, Inc. All rights reserved.

Built-in sequences are useful, but rarely sufficient to verify a design. User-defined register stimulus is often required and we use the register API to build this.

The API enables us to:

- Read and write the registers of the DUT.
- Read and write the shadow register values in the register model. We do this to update the register model with expected values (where possible) for registers written by the DUT, or to load the register model with a particular configuration before updating the DUT to match the model (see below).
- Randomize the registers in the model, under constraints, before updating the DUT to match the model (see below).
- Update the DUT to match the register model by initiating write operations on DUT registers which do not match their model counterparts. We can also update the model to mirror the DUT registers by reading every register.

Some operations allow automated checking – for example DUT register read will check the read value against the expected value in the model.

Operations

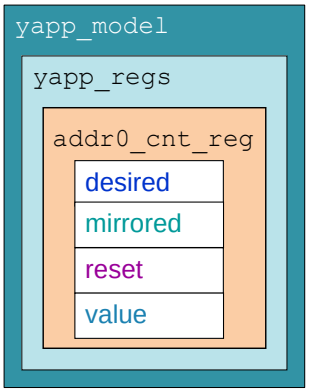
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Data Modeling and the Register Model

The register model stores multiple values for each field:

- Desired
  - What we want in the DUT
  - Allows us to build up a model configuration for later update to the DUT
- Mirrored
  - What we think is in the DUT
  - Prediction keeps mirrored up-to-date
  - Used in scoreboarding and coverage
- Reset
  - Hard reset value
  - Allows other reset values (POR, soft etc.)
- Value
  - Used only for randomization constraints
- desired is identical to mirrored, *except* in an update process



 © Cadence Design Systems, Inc. All rights reserved.

There are also other properties for holding field values, but these are the key four.

- Desired. For efficiency, we can setup the register model with a set of required values, and then update the DUT registers with a single API call. Desired is used for holding the required value.
- Mirrored holds the expected value of the DUT register. We try to keep this up to date with what we expect for the DUT register, although this is not always possible.
- The reset property is modelled as an associative array indexed by a string. By default, a single reset value is written to the array at the index "HARD". The user can specify alternative reset kinds (e.g., "SOFT") using vendor extensions to add additional entries to the reset array.
- Value is the only public property, i.e., the only property which should be directly accessed and then only in randomization constraints. All other properties should only be accessed using the Register API methods.

Each field variable has an access policy, extracted from the register specification. The IP-XACT1.5 standard supports several common policies, and vendor extensions allow many more. Custom policies can be defined for unique or unusual policy behavior.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Convenience handles

Test

```
class api_test extends base_test;
 ...
  virtual task run_phase (uvm_phase phase);
    tb.yapp_rm.router_yapp_regs.ctrl_reg.write(...);
...
```

Testbench | Register Model | Register Block | Register | Method

Test

```
class api_test extends base_test;
 ...
  yapp_regs_c yapp_regs;

  function void connect_phase(uvm_phase phase);
    yapp_regs = tb.yapp_rm.router_yapp_regs;
  endfunction

  virtual task run_phase (uvm_phase phase);
   yapp_regs.ctrl_reg.write(...);
...
```

Register block handle

Assign handle

Simpler to use handle in call

API methods are called off the register

You need a pathname to the register:

- From current scope to register model instance...
- ...through model hierarchy to register instance

Convenience handles can simplify pathname

- Declare register block handles in test
- Assign handles via full pathname
- Use handles in register method calls

Benefits:

- Easier interface for test writers
- Better reuse for register sequences
- Useful if model hierarchy can change
  - For example, from block to system-level migration

312 © Cadence Design Systems, Inc. All rights reserved.

cadence

It is beneficial to additional add local instance of the register blocks and memories within the model. It makes use of the registers and memories within the register model easier to use for a test writer. They do not need to understand the hierarchical path to the register blocks and memories within the register model.

This works well when executing register methods from the test class.

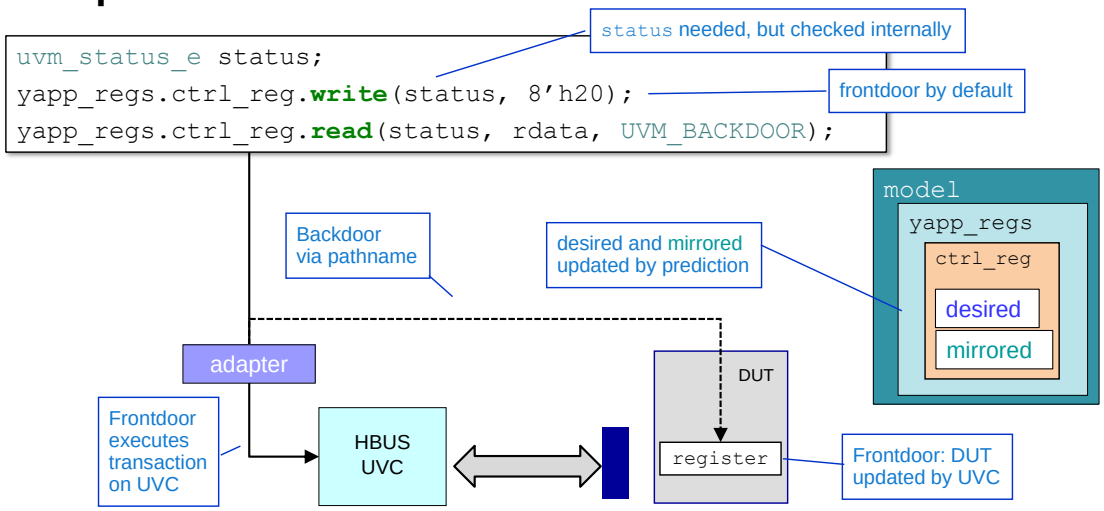
When calling methods from sequences, we typically pass the path information via the configuration database. The test or testbench write a reference to the register model into the database, from where it can be read by a sequence which calls register methods. However, it is still easier to set the register block into the config database to abstract away the internal hierarchy of the register model.

In system-level integration, it is common to build a system-level register model by creating hierarchies of multiple register blocks. Therefore by abstracting away this internal hierarchy, we can reuse register stimulus in system-integration tests simply by updating the convenience handle.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Basic Operations: Read/Write



- Read can automatically compare read DUT value and model mirrored value:
  - `<model>.<map>.set_check_on_read(1)`
  - Error issued if value read from DUT does not match mirrored

313 © Cadence Design Systems, Inc. All rights reserved.

cadence

The normal access API are the `read()` and `write()` methods.

When using front-door access, the API call is translated to DUT signal transactions by the interface UVC. For back-door access, the API call directly reads or writes the register variable in the DUT via a hierarchical pathname.

For both front and backdoor access, both mirrored and desired are updated to reflect the expected value in the DUT after the transaction. For example. clear-on-read bits will be cleared for a read operation.

The status argument must be mapped to a variable of type `uvm_status_e`. On a successful operation, status will be `UVM_IS_OK`. In practice, it is rare to explicitly check this value since any problems will generate `uvm_error` messages from within the method call.

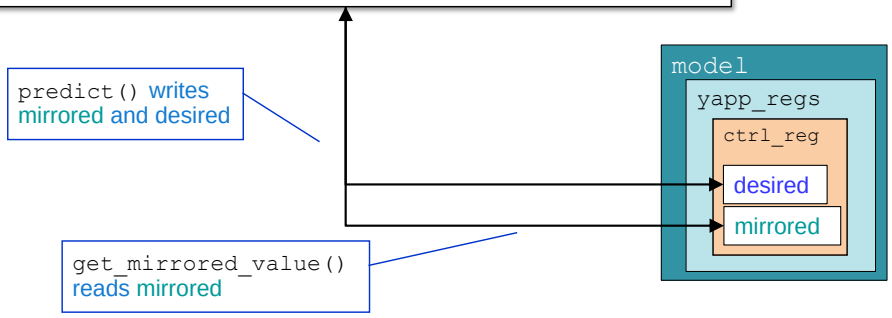
If you call `set_check_on_read(1)` from the address map of the register model, then a read operation will automatically compare the read value from the DUT against the mirrored value in the register model. If they differ, an error report will be generated. However the mirrored value of the model will still be updated with the value read of the DUT by prediction.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Accessing Mirrored: predict/get_mirrored_value

```
ok = yapp_regs.ctrl_reg.predict(8'h20);
rdata = yapp_regs.ctrl_reg.get_mirrored_value();
```



- Used in the manual prediction of registers set by DUT
  - To set **mirrored** (and **desired**) to expected values prior to a "set_check" read
- If called directly, both methods ignore access policies by default

314 © Cadence Design Systems, Inc. All rights reserved.

cadence

`predict()` allows you to change mirrored (and desired) in the register model,. For example, to set mirrored with the expected value before a read() operation with check-on-read set, for a register which is written internally by the DUT.

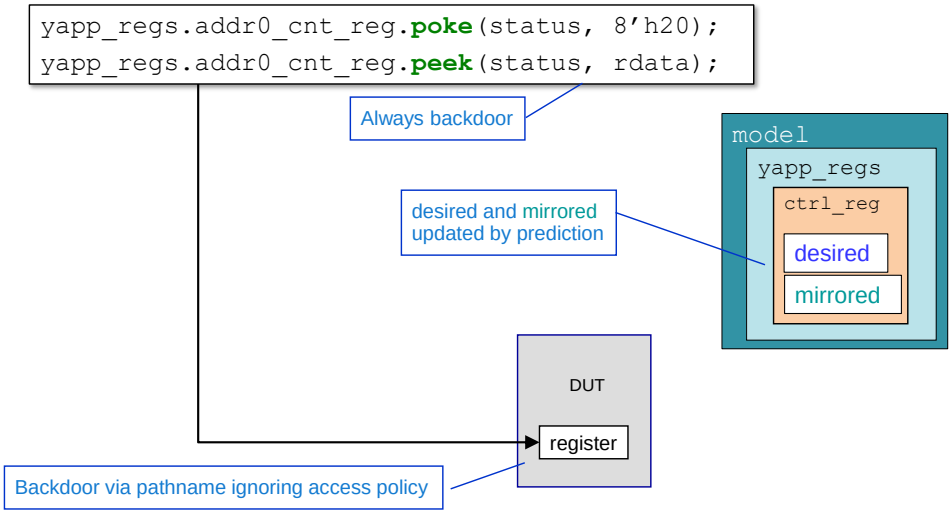
`get_mirrored_value()` allows you to read mirrored, for example to perform a read, modify, write operation.

Both API methods ignore register access policies by default. For example, `predict()` can write to a read-only register.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Backdoor DUT Access: Peek/Poke



`write()` obeys access policies (e.g., RO will fail) whereas `poke()` ignores policy

315 © Cadence Design Systems, Inc. All rights reserved.

`peek()` and `poke()` methods execute backdoor reads or writes directly to the DUT register via a hierarchical pathname. They are always backdoor and do not initiate transactions from an interface UVC.

Unlike backdoor reads and writes, peek and poke ignore access policies. For example, you can poke a read-only register.

Mirrored is updated to reflect the actual sampled or deposited value in the register.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Methods Example: Standard Accessibility Checks

RW Register Check

```
uvm_status_e status;
yapp_regs.en_reg.write(status, 8'hA5);
yapp_regs.en_reg.peek(status, rdata);
// check rdata = 8'hA5
yapp_regs.en_reg.poke(status, 8'h5A);
yapp_regs.en_reg.read(status, rdata);
// check rdata = 8'h5A
```

RW register:

- Frontdoor write a unique value
- Peek and check the value matches
- Poke a new value
- Frontdoor read this new value

RO Register Check

```
uvm_status_e status;
yapp_regs.addr0_cnt_reg.poke(status, 8'hA5);
yapp_regs.addr0_cnt_reg.read(status, rdata);
// check rdata = 8'hA5
yapp_regs.addr0_cnt_reg.write(status, 8'h5A);
yapp_regs.addr0_cnt_reg.peek(status, rdata);
// check rdata unchanged = 8'hA5
```

RO register:

- Poke a unique value
- Frontdoor read and check the value matches
- Frontdoor write a new value
- Peek and check the register is unchanged

316 © Cadence Design Systems, Inc. All rights reserved.

cadence

As an example of the method use, here are the standard accessibility checks for RO/RW registers.

RW register:

- Frontdoor write a unique value to the register
- Backdoor peek the register to make sure the value appeared in the correct DUT variable
- Backdoor poke a new value directly into the DUT register variable
- Frontdoor read and check the read value matches the poked value

RO register

- Backdoor poke a unique value into the DUT
- Frontdoor read the register and check the read value matches the poked value
- Frontdoor write a new value – this should have no effect as the register is RO
- Peek the DUT register and confirm the write had no effect

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Model Access Only: get/set/randomize

```
yapp_regs.en_reg.set(8'hff);
rdata = yapp_regs.en_reg.get();
yapp_regs.addr0_cnt_reg.set(8'h00);
ok = yapp_regs.randomize();
ok = yapp_regs.ctrl_reg.randomize() with {plen.value == 6'h1A;};
rdata = yapp_regs.ctrl_reg.get();
```

Only accesses the register model

RO registers cannot be set

Fields, registers or blocks can be randomized

Constraints are applied to the `value` property of a register or field

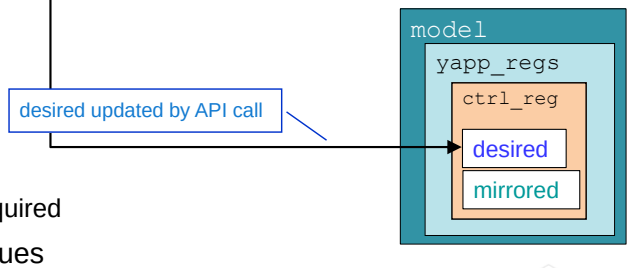
Only accesses **desired**

- Does not affect the DUT
- Respect the access policy
  - RO registers or fields cannot be set

Randomize allows optional constraints

- The whole register model can be randomized, if required

Used with `update()` to sync model and DUT values



317 © Cadence Design Systems, Inc. All rights reserved.

cadence

These access desired in the register model only. They have no interaction with the DUT or an interface UVC. They allow us to setup the register model with a set of required values, and then update the DUT registers to match the register model with a single API call (`update`).

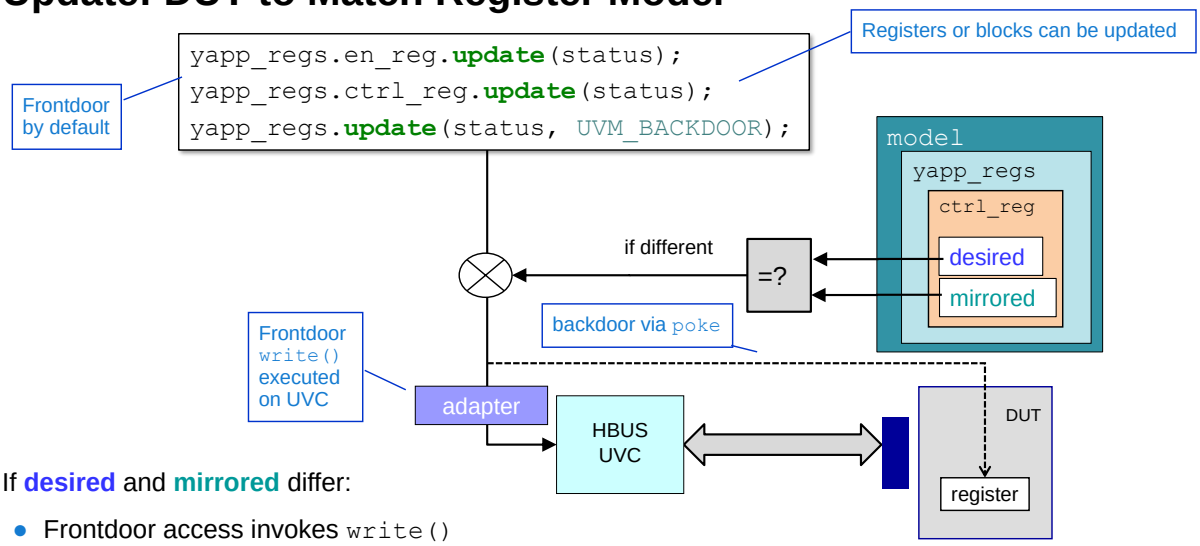
`set()` respects access policies when assigning desired, so, for example, read-only registers are not affected. Mirrored is used as the current register value if required to calculate desired. For example, for an 8-bit Write-1-Toggle access register, setting 8'hFF will result in desired being the inverse of mirrored.

`randomize()` can apply a random set to a single field, register or the entire register model. Constraints can be defined inline with the call, or as a property of the register using a vendor extension in the IP-XACT specification.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Update: DUT to Match Register Model



If **desired** and **mirrored** differ:

- Frontdoor access invokes `write()`
- Backdoor access invokes `poke()`

mirrored updated by predictor

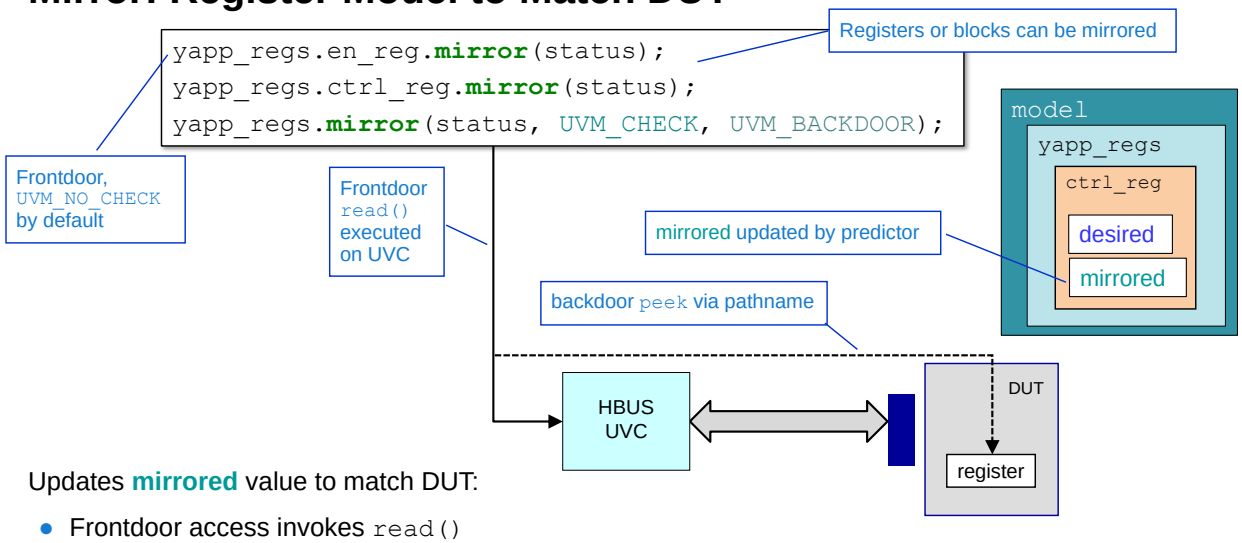
 © Cadence Design Systems, Inc. All rights reserved.

`update()` is used after `set()` or `randomize()` to perform a "bulk" write to the DUT in a single API call. Where desired and mirrored differ, update will execute a frontdoor write or backdoor poke on the DUT register. Mirrored will be updated by the predictor at the end of the cycle. Registers are checked in the order of declaration in the register model.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Mirror: Register Model to Match DUT



Updates **mirrored** value to match DUT:

- Frontdoor access invokes `read()`
- Backdoor access invokes `peek()`

Will compare read value with mirrored value if `UVM_CHECK` is set

319 © Cadence Design Systems, Inc. All rights reserved.

Frontdoor `mirror()` updates mirrored to match the DUT register. It does this by executing a frontdoor `read()` on the DUT register, but discarding the read value. Mirrored is updated by the predictor.

Frontdoor `mirror()` differs from `read()` by having a `check` argument. The default value is `UVM_NO_CHECK`, but if the argument is set to `UVM_CHECK`, the read value from the DUT is compared against mirrored before it is updated. An error message is generated if the values do not match. The address map method `set_check_on_read()` can set the `UVM_CHECK` flag for all registers.

Fields can be excluded from the comparison check using the `set_compare()` method. This can be done via a subclass and a type override.

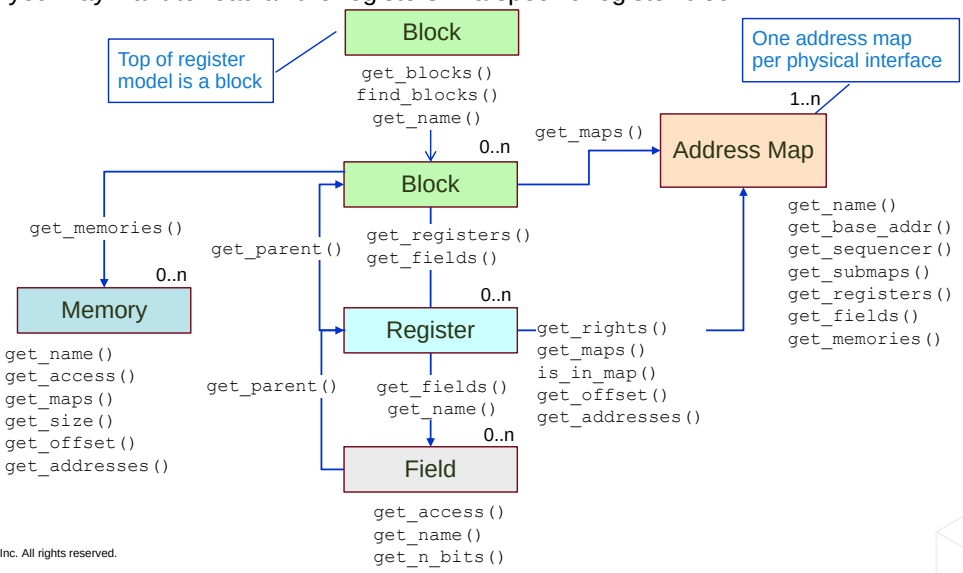
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Selected Register Model Introspection Methods

The register API provides navigation and selection methods

- For example, you may want to read all the registers in a specific register block



320 © Cadence Design Systems, Inc. All rights reserved.

These are just some of the properties you can obtain from the API building blocks. For a detailed description, use the reference manual.

Selected methods:

get_registers()

- Returns a queue of all the registers in a block

get_name()

- Returns the name of the block, register, field, memory of file as a string

get_rights()

- Returns the accessibility (RW, RO, or WO) of a register in a given address map

get_access()

- Returns the current access policy of a field when accessed through a given address map

get_n_bits()

- Returns the size in bits of a field, register or memory location

get_addresses()

- Returns the external physical address(es) of a register or memory location

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Introspection with Register API

```
...
 yapp_regs_c yapp_regs;
 uvm_reg qreg[$], rwregs[$], roregs[$];

virtual task run_phase (uvm_phase phase);
   ...
   yapp_regs.get_registers(qreg);

   foreach (qreg[i])
     if (qreg[i].get_rights() == "RO")
       roregs.push_back(qreg[i]);

     rwregs = qreg.find(i) with (i.get_rights() == "RW");

     foreach (rwregs[i]) begin
       rwregs[i].write(status, 8'h5A);
       rwregs[i].read(status, rdata);
       ...
```

Queues of registers

Method is hierarchical and returns a queue of registers

Extract (by access rights) into new queue

Extraction using array locator methods

Iterate over queue

Operations on individual registers is inefficient

Use introspection to create multiple queues of registers

- With same characteristics, e.g.,
  - Access rights
  - Offsets
  - Any register with "`cnt_reg`" in name

Then operate on every queue element

321 © Cadence Design Systems, Inc. All rights reserved.

cadence

`get_registers()` is a method defined for a register block or an address map. It is hierarchical by default and returns a queue of `uvm_reg` type.

```
virtual function void get_registers ( ref uvm_reg regs[$],  input
uvm_hier_e hier = UVM_HIER )
```

You can make the method non-hierarchical by passing the value `UVM_NO_HIER` to the argument `hier`.

We then filter the register queue to extract the mode registers into a different queue. There are many ways to do this – you could filter by access policy, offset or address, but here we filter by access rights.

Filtering can be done by reading each elements characteristics and conditionally popping the result to other queues, or the SystemVerilog array locator methods can be used.

Then we iterate over the Read/Write register queue to write every register by the frontdoor and read every register by the backdoor. We assume the check-on-read is set for the mode registers to verify the write/read.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Lab

Lab 11C Simulation

Creating user-defined register sequences with API calls

- Write register sequences using API calls
- Execute sequences on the DUT and check the results

Instructions are in the Lab document

Time: (open-ended)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

SystemVerilog Register Verification Using UVM

New, separate course on Register Verification using UVM

Objectives

- Review register modelling, abstraction, and the UVM register layer
- Generate a register model
- Create a register adapter to connect to a design
- Configure and use pre-built register sequence
- Explore the different modes of prediction to keep the register model in sync with the DUT
- Create reusable register sequences using the register access methods
- Model quirky and unusual register behavior with callbacks
- Update a scoreboard to use the register model to validate the DUT

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This module is a very quick overview of UVM register modeling. Cadence has a separate, in-depth, 2-day class just on UVM register modeling, covering the topics listed above.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 18

Top Five Things that Break in UVM-IEEE (And How to Fix Them)

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Top Five Things that Break in UVM-IEEE Objectives

- Should you migrate to IEEE1800.2-2017 (UVM-IEEE)?
  - No essential new content
  - Easy transition.....
  - ....but some things will break
- Our aim is to show Top 5 common issues when migrating from UVM1.x to UVM-IEEE...
  - .. and how to fix them
  - So you can decide if its worth it..
- Most issues are fairly trivial
- Caveat:
  - This is our experience of migrating UVM code – your experience may differ
    - Particularly if you use internal library features, implementation code, deep library tricks....

 © Cadence Design Systems, Inc. All rights reserved.

cadence

UVM-IEEE has been around for a while, but there seems little interest in migrating to the standard. Perhaps because UVM-IEEE does not contain any essential new content. The good news is that migration is relatively easy, although some things will break.

The aim of this modules is to show you the top 5 common issues when migrating to UVM-IEEE, and how to fix them. So, you can decide if migration is worth it. As additional encouragement, most of these issues are fairly trivial.

Some caveats before we start. This is our experience in migrating code. Your experience may differ. Particularly if your UVM environment uses internal library features or deep library tricks. Historically UVM has been bad at distinguishing between the user API and implementation code which is liable to change between releases. UVM-IEEE attempts to resolve this by clearly documenting the user API. If your UVM testbenches use implementation code, migration may be more painful than described here..

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

UVM-IEEE – Standard, Contributions and Implementation

For UVM-IEEE, we use the Cadence® UVM-IEEE library supplied with Xcelium®

- Accellera reference library with Cadence extensions

Strictly speaking, the Accellera library is not UVM-IEEE

- Contains 3 content groups:
  1. @uvm-ieee
     - Documented in IEEE1800.2 standard
  2. @uvm-contrib
     - May be considered for future standardization
  3. @uvm-accellera
     - Implementation API not considered for standardization

Our migration allows the full Accellera library

- @uvm-ieee, @uvm-contrib and @uvm-accellera content

This is the easiest migration path but may cause issues in the future

 © Cadence Design Systems, Inc. All rights reserved.

cadence

For migration, we will use the UVM-IEEE library supplied with Cadence's Xcelium Simulator. This is the Accellera UVM-IEEE reference library with the usual Cadence vendor extensions.

However the Accellera library contains 3 groups of content. As well as the standard UVM-IEEE API, the library also contains contribution content, which may be considered for future UVM releases, and implementation content which will not be considered for standardization. All three groups are documented in the library source code with @uvm-ieee, @uvm-contrib and @uvm-accellera tags. We will allow all three content groups from the library. This makes our migration dependent on the Accellera library and may be storing up issues for the future, but if we restricted ourselves only to @uvm-ieee content, there would far more than five things which break.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

5: Deprecated Constructs

Deprecated features in UVM1.2 are removed

```
set_config_int("hbus", "num_masters", 1);
```
UVM1.2

```
uvm_config_int::set(this, "hbus", "num_masters", 1);
```
UVM-IEEE

```
if (starting_phase != null)
   starting_phase.raise_objection(this, get_type_name());
```
UVM1.2

```
uvm_phase sp = get_starting_phase();
if (sp != null)
  sp.raise_objection(this, get_type_name());
```
UVM-IEEE

```
uvm_test_done.set_drain_time(this, 200ns);
```
UVM1.2

```
uvm_objection obj = phase.get_objection();
obj.set_drain_time(this, 200ns);
```
UVM-IEEE

327 © Cadence Design Systems, Inc. All rights reserved.

cadence

At number 5 is deprecated constructs. Any construct marked as deprecated in UVM1.2 is removed in UVM-IEEE.

For example:

- `set_config_*` methods are removed in favour of the `uvm_config_db set` methods for writing values into the configuration database.
- Direct access to the `starting_phase` property when handling objections in sequences is no longer possible. You must use the `get_starting_phase()` method to read the property.
- The `uvm_test_done` handle on the objection mechanism has been removed. The objection handle can be accessed by calling `get_objection()` from a `phase` handle, e.g. the `phase` argument to any UVM phase method.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

4: Automation Policies (UVM1.x)

UVM1.x allows direct access to:

- Policies to define operation of automation methods
- Properties to customize policies

For example, Default printer policies and properties

```
uvm_table_printer mytp = new();
uvm_comparer mycomp = new();

initial begin
 uvm_default_printer = uvm_default_line_printer;

 mytp.knobs.begin_elements = -1;

 packet.print(uvm_default_tree_printer);
 packet.print(mytp);

 mycomp.sev = UVM_WARNING;
 ok = packet.compare(packet2, mycomp);
...
```

UVM1.2

User policies

Direct access to defaults

Direct access to printer properties under `knobs`

Direct access to comparer properties

Class Declaration

`class uvm_printer_knobs`

Variables

header	Indicates whether the uvm_printer::format_header function should be called when printing an object.
footer	Indicates whether the uvm_printer::format_footer function should be called when printing an object.
full_name	Indicates whether uvm_printer::adjust_name should print the full name of an identifier or just the leaf name.
identifier	Indicates whether uvm_printer::adjust_name should print the identifier.
type_name	Controls whether to print a field's type name.
size	Controls whether to print a field's size.
depth	Indicates how deep to recurse when printing objects.
reference	Controls whether to print a unique reference ID for object handles.
begin_elements	Defines the number of elements at the head of a list to print.
end_elements	This defines the number of elements at the end of a list that should be printed.
	Specifies the string prepended to each output line
	This knob specifies the number of spaces to use for level indentation.
	This setting indicates whether or not the initial object that is printed (when current depth is 0) prints the full path name.
mcd	This is a file descriptor, or multi-channel descriptor, that specifies where the print output should be directed.

Printer policy properties

328 © Cadence Design Systems, Inc. All rights reserved.

cadence

In UVM versions prior to UVM-IEEE, direct access to library properties is generally allowed. We will use the automation police as an example. In UVM1.x, you have direct access to default policy properties, such as printers, and to printer properties, such as the number of array elements (`begin_elements`). All the properties (variables) listed under the `uvm_printer_knobs` class can be directly read and written.

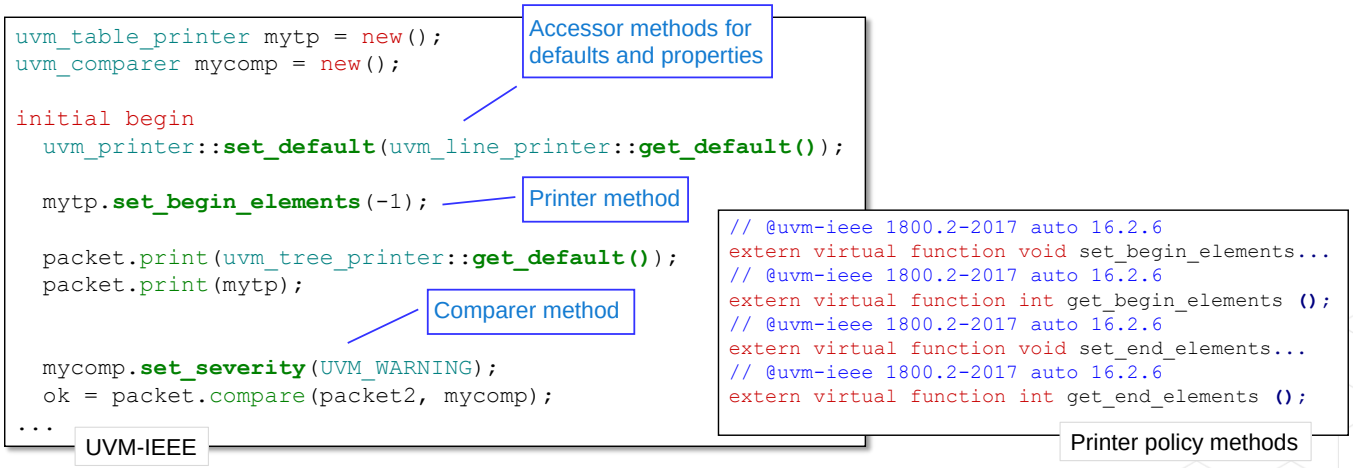
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

4: Automation Policies (UVM-IEEE)

Direct access to automation policy properties removed

- Policy and property access is by methods



329 © Cadence Design Systems, Inc. All rights reserved.

cadence

In UVM-IEEE, many UVM properties such as automation policies and printer controls, are declared as `local` or `protected`. Therefore direct access is prohibited. Instead, UVM-IEEE declares accessor methods to set or get the property values. This allows the library to control read and write access by, for example, declaring a get method but not a set method for a read-only property.

There are several other changes to policies in UVM-IEEE, but many are for advanced usage models that the average user is unlikely to encounter.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

3: uvm_top Accessor Methods

uvm_top and its properties are not directly accessible

- uvm_top required for print_topology, find, set_timeout etc.

uvm_top accessor method

```
class base_test extends uvm_test;
...

  function void end_of_elaboration_phase(...);

    comp = uvm_top.find("*.agent.seqr");

    uvm_top.print_topology();
...
```

UVM1.2

Direct access uvm_top

```
class base_test extends uvm_test;
...
   uvm_root uvm_top = uvm_root::get();

 function void end_of_elaboration_phase(...);

    comp = uvm_top.find("*.agent.seqr");

    uvm_top.print_topology();
...
```

UVM-IEEE

330 © Cadence Design Systems, Inc. All rights reserved.

cadence

Another consequence of UVM-IEEE's adoption of accessor methods in preference to direct library property access affects uvm_top.

uvm_top is the handle on the top-most uvm_root class instance of a UVM hierarchy. uvm_top instantiates the uvm_test_top test class instance and starts the phasing to build the UVM hierarchy and run the simulation. In UVM1.2 and earlier, the uvm_top handle is directly accessible. uvm_top has many useful methods such as print_topology() for printing the UVM hierarchy, find for searches and timeout setting. In UVM-IEEE, direct access to the uvm_top handle is forbidden, and you must access the handle using the static method get() of the uvm_root class.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

2: Debugging methods

Debugging methods removed from UVM-IEEE

- For example, `check_config_usage()`
  - Workaround is to copy/paste UVM1.2 implementation
  - Also opportunity to make report more meaningful

```
function void check_phase(uvm_phase phase);
  check_config_usage();
endfunction
```

UVM1.2

User defined implementation based on UVM1.2

```
function void check_config_usage(bit recurse=1);
  uvm_resource_pool rp = uvm_resource_pool::get();
  uvm_queue#(uvm_resource_base) rq;
  rq = rp.find_unused_resources();
  if(rq.size() == 0)
    return;
  `uvm_warning("CFGNRD"," Configuration Mismatch");
  rp.print_resources(rq, 1);
endfunction
```

Modified report

UVM-IEEE

331 © Cadence Design Systems, Inc. All rights reserved.

UVM-IEEE removes several debugging methods from the library. The intention is that debugging is now the responsibility of the user, through user-defined methods or EDA Vendor tools. One example of a removed method is `check_config_usage()` which reports unmatched configuration settings in UVM1.2. If you still need this functionality in your testbench, you can simply copy the UVM1.2 implementation and add it to your testbench as a user-defined method. This also gives the opportunity to modify the original method report, for example to make it a warning rather than an info severity.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

1: Sequence Macro Rationalization

Huge number of sequence macros in UVM1.2

- "`with`" macros for constraint blocks
- "`pri`" macros for sequence/item priority
- "`on`" macros for multichannel (virtual) sequences

UVM-IEEE rationalizes the macro set:

- All macros containing "`with`", "`pri`" or "`on`" are removed
- Signature of `` `uvm_do `` is changed

i.e. UVM-IEEE only supports 4 macros:

- `` `uvm_do ``
- `` `uvm_create ``
- `` `uvm_rand_send ``
- `` `uvm_send ``

UVM1.2

```
`uvm_create
`uvm_do
`uvm_do_pri
`uvm_do_with
`uvm_do_pri_with
`uvm_create_on
`uvm_do_on
`uvm_do_on_pri
`uvm_do_on_with
`uvm_do_on_pri_with
`uvm_send
`uvm_send_pri
`uvm_rand_send
`uvm_rand_send_pri
`uvm_rand_send_with
`uvm_rand_send_pri_with
```

332 © Cadence Design Systems, Inc. All rights reserved.

This is the most impactful change. In UVM1.2, there are a huge number of sequence macros. The basic `uvm_do` macro has the following variants:

- `uvm_do_with` for adding constraint blocks
- `uvm_do_pri` for defining sequence item priority in concurrent sequence arbitration
- `uvm_do_pri_with` for both priority and constraint specification
- `uvm_do_on` for multichannel or virtual sequencers
- `with`, `pri` and `pri_with` options on the `uvm_do_on` macro.

Also the breakdown of the do macro into create/send/rand_send steps also come with priority and constraint variants.

UVM-IEEE rationalizes the macro set by adding on, priority and constraint arguments to the basic `uvm_do` macro, and removing the need for the on/pri/with variants. As such, only the 4 sequence macros listed are supported in UVM-IEEE:

- `uvm_do`
- `uvm_create`
- `uvm_rand_send`
- `uvm_send`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Missing Sequence Macros in UVM-IEEE: Option 1

`uvm_do_with and `uvm_do_on macros (and variants) break in UVM-IEEE

Option 1

1. Use the new `uvm_do signature
   - Intended as a generic, multipurpose sequence macro

Defaults for arguments

UVM1.x default 100

Constraint block at END of new macro

```
uvm_do(SEQ_OR_ITEM, SEQR=get_sequencer(), PRIORITY=-1, CONSTRAINTS={})
```

Consecutive commas to skip arguments

```
`uvm_do_with(req, {addr == 2;})
`uvm_do_on(hseq, p_sequencer.hbus_seqr)
`uvm_do_pri(yseq, 200)
```

UVM1.2

```
`uvm_do(req,,,{addr == 2;})
`uvm_do(hseq, p_sequencer.hbus_seqr)
`uvm_do(yseq,,200)
```

Skip SEQR argument

UVM-IEEE

333 © Cadence Design Systems, Inc. All rights reserved.

cadence

This is the big issue in UVM-IEEE migration - if you use sequence macros, these will break with UVM-IEEE and will need to be modified. There are a couple of options.

Option 1 is to rewrite sequence code to modify with, pri and on macro calls to use the new UVM-IEEE `uvm_do` macro signature. For on macro variants, we use the `SEQR` argument, pri macros `PRIORITY` and with macros `CONSTRAINTS`. The macro arguments have default values (introduced in SystemVerilog2009), so you can skip the arguments you don't use, but named association for macro arguments is not allowed in SystemVerilog, so we skip arguments by using consecutive commas. This is similar to parameter mapping in Verilog95, before named association for parameter assignment. Consecutive commas are not good for readability and maintainability. You could repeat the default values in the macro calls for clarity, but this is very verbose.

For current users of sequence priority, be aware that the default priority value in the UVM-IEEE macro call is -1 compared to 100 in UVM1.x.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Missing Sequence Macros in UVM-IEEE: Option 2

`uvm_do_with and `uvm_do_on macros (and variants) break in UVM-IEEE

Option 2

2. Compile the original macros
   - From library under deprecated directory..
   - ..or copy into a project directory
   - Use original code

```
$ ls $UVMHOME/sv/src/deprecated/macros
uvm_object_defines.svh  uvm_sequence_defines.svh
```

```
import uvm_pkg::*;
`include "uvm_macros.svh"
`include "deprecated/macros/uvm_sequence_defines.svh"
...
`uvm_do_with(req, {addr == 2;})

`uvm_do_on(hseq, p_sequencer.hbus_seqr)

`uvm_do_pri(yseq, 200)
```

UVM-IEEE

334 © Cadence Design Systems, Inc. All rights reserved.

cadence

Option 2 is to compile the original sequence macro declarations. These are in the `uvm_sequence_define.svh` file under the deprecated directory of the UVM-IEEE library. This is probably not a long term solution, as files in the deprecated directory are liable to be removed at a future date, but it is the simplest and easiest option with the minimum of changes to current code. You could even copy the sequence defines file into a local project directory for protection against removal. Or write your own macro wrapper file which maps the UVM1.x macros to the UVM-IEEE macros.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Summary

Three groups of "breaks"

- Deprecated constructs
  - UVM1.2 deprecated constructs removed
- Accessor methods
  - Direct access to properties removed
  - Replaced by methods
- Code rationalization
  - Sequence macros
    - `` `uvm_do_with `` is the biggest break for most users

Migration to UVM-IEEE is relatively easy...

- ... little reason to upgrade beyond conformance to standards

 © Cadence Design Systems, Inc. All rights reserved.

cadence

So, in conclusion, there are basically three groups of issues which will break your code in migration. Deprecated constructs. Accessor methods which replace direct access to library properties, and sequence macro rationalization. The removal of `uvm_do_with` will probably cause the biggest issue for most users.

Our experience is that migration to UVM-IEEE is relatively easy. However, apart from conformance to standards, there are few compelling reasons to migrate.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



Module 19

Next Steps

cadence

This page does not contain notes.

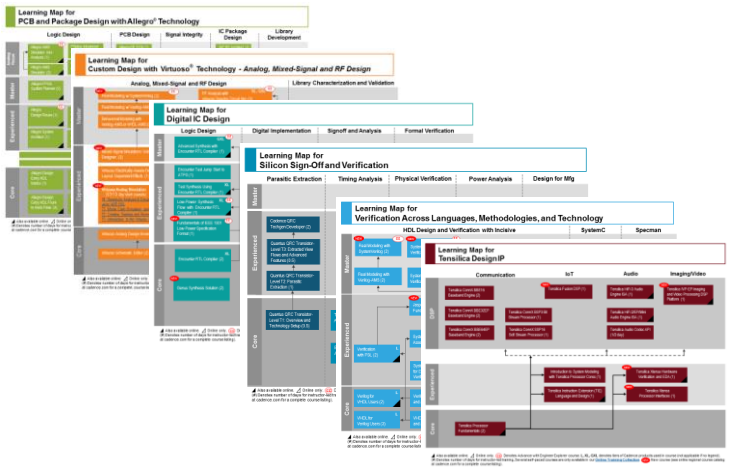
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click here to see all our courses in each technology area and the recommended order in which to take them.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

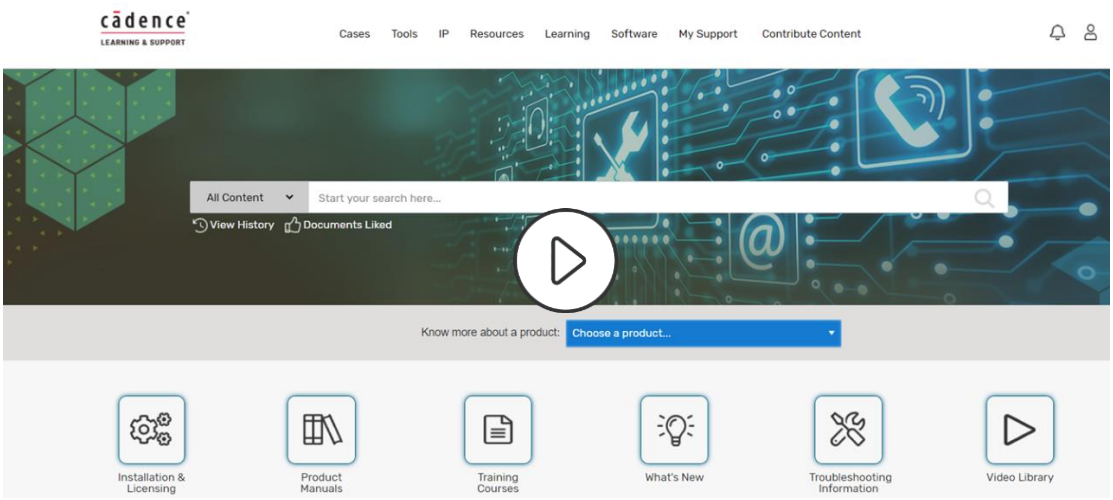
Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Cadence Learning and Support



Cadence Support now includes over 2000 product/language/methodology videos ("Training Bytes")!

338 © Cadence Design Systems, Inc. All rights reserved.

cadence

Click here to view the demo of Cadence Learning and Support.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion

Thank you!

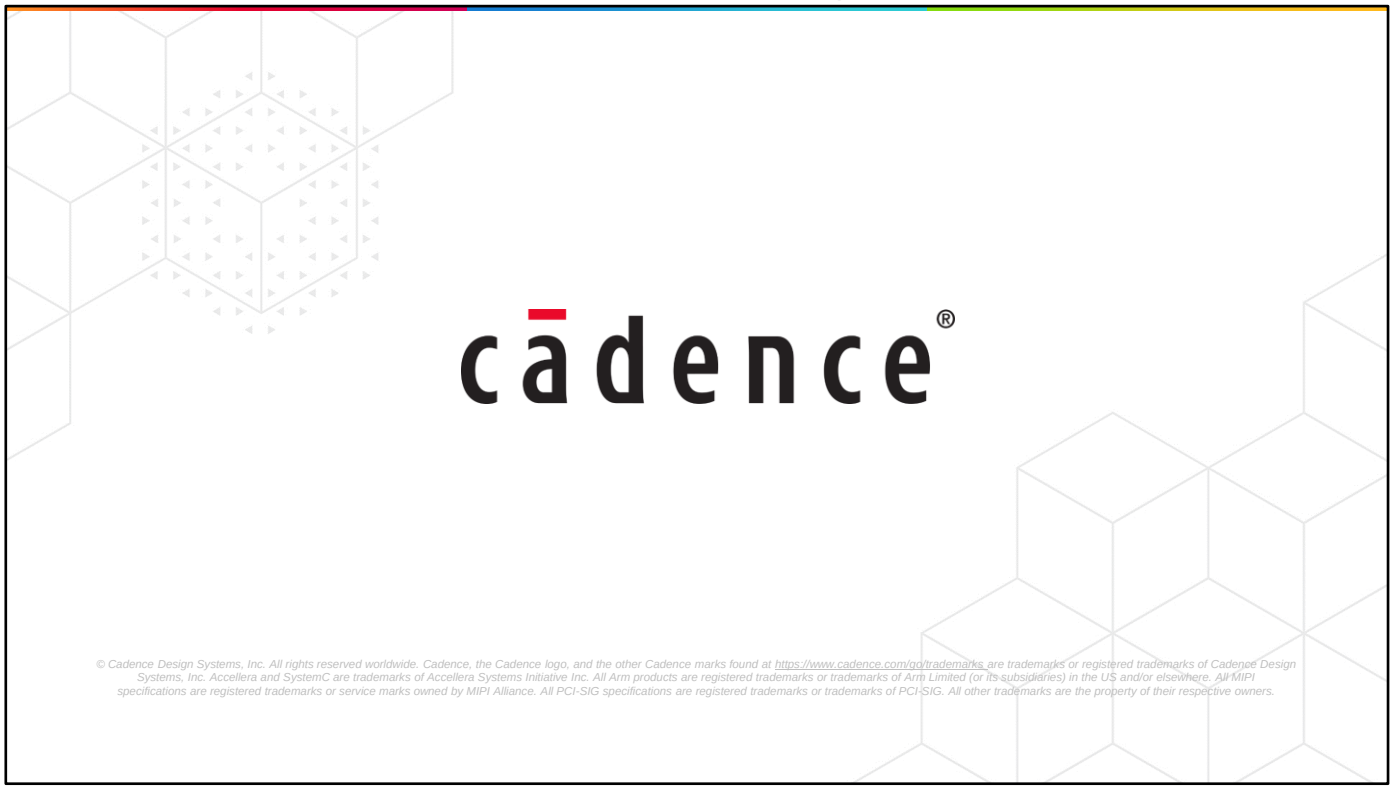
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kun Wang of Seas



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.